



Oracle Database 19c: PL/SQL Workshop

Student Guide – Volume II
D108650GC10

Copyright © 2021, Oracle and/or its affiliates.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Trademark Notice

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

1009022021

For Instructor Use Only.
This document should not be distributed.

Contents

1 Introduction

Lesson Objectives	1-2
Lesson Agenda	1-3
Course Objectives	1-4
Course Road Map	1-5
Lesson Agenda	1-11
Human Resources (HR) Schema for This Course	1-12
Course Agenda	1-13
Class Account Information	1-16
Appendixes and Practices Used in This Course	1-17
Lesson Agenda	1-18
Oracle Database 19c: Focus Areas	1-19
Oracle Database 19c	1-20
Lesson Agenda	1-22
PL/SQL Development Environments	1-23
Oracle SQL Developer	1-24
Specifications of SQL Developer	1-25
SQL Developer Interface	1-26
Coding PL/SQL in SQL*Plus	1-27
SQL Developer Web	1-28
Lesson Agenda	1-33
Oracle SQL and PL/SQL Documentation	1-34
Additional Resources	1-35
Summary	1-36
Practice 1 Overview: Getting Started	1-37

2 Introduction to PL/SQL

Course Road Map	2-2
Objectives	2-3
Agenda	2-4
Limitations of SQL	2-5
Why PL/SQL?	2-6
Why PL/SQL	2-7
About PL/SQL	2-8
Benefits of PL/SQL	2-9

PL/SQL Runtime Architecture 2-11
PL/SQL Block Structure 2-12
Agenda 2-13
Block Types 2-14
Examining an Anonymous Block 2-16
Executing an Anonymous Block 2-17
Agenda 2-18
Enabling Output of a PL/SQL Block 2-19
Viewing the Output of a PL/SQL Block 2-20
Quiz 2-21
Summary 2-22
Practice 2: Overview 2-23

3 Declaring PL/SQL Variables

Course Road Map 3-2
Objectives 3-3
Agenda 3-4
Variables 3-5
Variables in PL/SQL 3-6
Requirements for Variable Names 3-8
Using Variables in PL/SQL 3-9
Declaring and Initializing PL/SQL Variables 3-10
Agenda 3-11
Declaring and Initializing PL/SQL Variables 3-12
Initializing Variables Through a SELECT Statement 3-13
Types of Variables 3-15
Declaring Variables 3-16
Guidelines for Declaring and Initializing PL/SQL Variables 3-17
Guidelines for Declaring PL/SQL Variables 3-18
Naming Conventions of the PL/SQL Structures Used in This Course 3-19
Data Types for Strings 3-20
Delimiters in String Literals 3-21
Data Types for Numeric values 3-22
Data Types for Date and Time values 3-23
Data Type Conversion 3-25
Agenda 3-28
The %TYPE Attribute 3-29
Declaring Variables with the %TYPE Attribute 3-30
Declaring Boolean Variables 3-31
LOB Data Type Variables 3-32
Composite Data Types: Records and Collections 3-33

Agenda 3-34
Bind Variables 3-35
Bind Variables: Examples 3-36
Using AUTOPRINT with Bind Variables 3-37
Quiz 3-38
Summary 3-39
Practice 3: Overview 3-40

4 Writing Executable Statements

Course Road Map 4-2
Objectives 4-3
Agenda 4-4
Lexical Units in a PL/SQL Block 4-5
PL/SQL Block Syntax and Guidelines 4-7
Commenting Code 4-8
SQL Functions in PL/SQL 4-9
SQL Functions in PL/SQL: Examples 4-10
Using Sequences in PL/SQL blocks 4-11
Using Sequences in PL/SQL Blocks 4-14
Agenda 4-15
Nested blocks 4-16
Nested Blocks: Example 4-17
Variable Scope and Visibility 4-18
Using a Qualifier with Nested Blocks 4-20
Challenge: Determining the Variable Scope 4-21
Agenda 4-23
Operators in PL/SQL 4-24
Operators in PL/SQL: Examples 4-25
Programming Guidelines 4-26
Indenting Code 4-27
Quiz 4-28
Summary 4-29
Practice 4: Overview 4-30

5 Using SQL Statements Within a PL/SQL Block

Course Road Map 5-2
Objectives 5-3
Agenda 5-4
SQL Statements in PL/SQL 5-5
SELECT Statements in PL/SQL 5-7
Retrieving Data in PL/SQL: Example 5-9

Retrieving Data in PL/SQL	5-10
Naming Ambiguities	5-11
Avoiding Naming Ambiguities	5-12
Agenda	5-13
Using PL/SQL to Manipulate Data	5-14
Insert Data: Example	5-15
Update Data: Example	5-16
Delete Data: Example	5-17
Merging Rows	5-18
Agenda	5-20
SQL Cursor	5-21
SQL Cursor Attributes for Implicit Cursors	5-23
Quiz	5-25
Summary	5-26
Practice 5: Overview	5-27

6 Writing Control Structures

Course Road Map	6-2
Objectives	6-3
PL/SQL Control Structures	6-4
Agenda	6-5
IF Statement	6-6
IF-ELSIF Statements	6-7
Simple IF Statement	6-8
IF THEN ELSE Statement	6-9
IF ELSIF ELSE Clause	6-10
NULL Value an in IF Statement	6-11
Agenda	6-12
CASE Expressions	6-13
Searched CASE Expressions	6-14
CASE Statement	6-16
Handling Nulls	6-17
Logic Tables	6-18
Boolean Expression or Logical Expression?	6-19
Agenda	6-20
Iterative Control: LOOP Statements	6-21
Basic Loops	6-22
Basic Loop: Example	6-23
WHILE Loops	6-24
WHILE Loops: Example	6-25
FOR Loops	6-26

FOR Loops: Example 6-28
FOR Loop Rules 6-29
Suggested Use of Loops 6-30
Nested Loops and Labels 6-31
Nested Loops and Labels: Example 6-32
PL/SQL CONTINUE Statement 6-33
PL/SQL CONTINUE Statement: Example 1 6-34
PL/SQL CONTINUE Statement: Example 2 6-35
Quiz 6-36
Summary 6-37
Practice 6: Overview 6-38

7 Working with Composite Data Types

Course Road Map 7-2
Objectives 7-3
Agenda 7-4
Composite Data Types 7-5
PL/SQL Records Versus Collections 7-7
Agenda 7-8
PL/SQL Records 7-9
Creating a PL/SQL Record 7-10
Creating a PL/SQL Record: Example 7-11
PL/SQL Record Structure 7-12
%ROWTYPE Attribute 7-14
Creating a PL/SQL Record: Example 7-16
Advantages of Using the %ROWTYPE Attribute 7-17
Another %ROWTYPE Attribute: Example 7-18
Inserting a Record by Using %ROWTYPE 7-19
Updating a Row in a Table by Using a Record 7-20
Agenda 7-21
Associative Arrays (INDEX BY Tables) 7-22
Associative Array Structure 7-24
Steps to Create an Associative Array 7-25
Creating and Accessing Associative Arrays 7-26
Associative Arrays with Record values 7-27
Using Collection Methods 7-28
Using Collection Methods with Associative Arrays 7-29
Nested Tables 7-30
Nested Tables: Syntax and Usage 7-31
Variable-Sized Arrays (Varrays) 7-33
VARRAYs: Syntax and Usage 7-34

Summarizing Collections 7-35

Quiz 7-36

Summary 7-37

Practice 7: Overview 7-38

8 Using Explicit Cursors

Course Road Map 8-2

Objectives 8-3

Agenda 8-4

Cursors 8-5

Implicit Cursors 8-6

Explicit Cursor 8-7

Controlling Explicit Cursors 8-8

Agenda 8-9

Declaring the Cursor 8-10

Opening the Cursor 8-12

Fetching Data from the Cursor 8-13

Closing the Cursor 8-15

Cursors and Records 8-16

Cursor FOR Loops 8-17

Explicit Cursor Attributes 8-19

%ISOPEN Attribute 8-20

%ROWCOUNT and %NOTFOUND: Example 8-21

Cursor FOR Loops Using Subqueries 8-22

Agenda 8-23

Cursors with Parameters 8-24

Agenda 8-26

FOR UPDATE Clause 8-27

WHERE CURRENT OF Clause 8-29

WHERE CURRENT OF Clause: Example 8-30

Quiz 8-32

Summary 8-33

Practice 8: Overview 8-34

9 Handling Exceptions

Course Road Map 9-2

Objectives 9-3

Agenda 9-4

What Is an Exception? 9-5

Handling an Exception: Example 9-6

Understanding Exceptions with PL/SQL 9-7

Handling Exceptions 9-8
Exception Types 9-9
Agenda 9-10
Syntax to Trap Exceptions 9-11
Guidelines for Trapping Exceptions 9-13
Trapping Internally Predefined Exceptions 9-14
Internally Defined Exception Trapping: Example 9-15
Trapping Predefined Exceptions 9-16
Functions for Trapping Exceptions 9-19
Trapping User-Defined Exceptions 9-21
RAISE Statement 9-22
Trapping User-Defined Exceptions 9-23
Propagating Exceptions in a Sub-Block 9-24
The RAISE_APPLICATION_ERROR Procedure 9-25
Quiz 9-28
Summary 9-29
Practice 9: Overview 9-30

10 Introducing Stored Procedures and Functions

Course Road Map 10-2
Objectives 10-3
Agenda 10-4
What Are PL/SQL Subprograms? 10-5
Differences Between Anonymous Blocks and Subprograms 10-7
Agenda 10-8
Procedure: Syntax 10-9
Creating a Procedure 10-10
Invoking a Procedure 10-12
Agenda 10-13
Function: Syntax 10-14
Creating a Function 10-15
Invoking a Function 10-16
Passing a Parameter to the Function 10-17
Invoking the Function with a Parameter 10-18
Quiz 10-19
Summary 10-20
Practice 10: Overview 10-21

11 Creating Procedures

Course Road Map 11-2
Objectives 11-3

Lesson Agenda 11-4
Modularized Program Design 11-5
Modularizing Code with PL/SQL 11-6
Benefits of Modularization 11-7
What Are PL/SQL Subprograms? 11-8
Lesson Agenda 11-9
Procedures 11-10
What Are Procedures? 11-11
Creating Procedures: Overview 11-12
Creating Procedures 11-13
Creating Procedures Using SQL Developer 11-14
Compiling Procedures 11-15
Calling Procedures 11-16
Calling Procedures Using SQL Developer 11-17
Procedures 11-18
What Are Parameters and Parameter Modes? 11-19
Formal and Actual Parameters 11-20
Procedural Parameter Modes 11-21
Comparing the Parameter Modes 11-22
Using the IN Parameter Mode: Example 11-23
Using the OUT Parameter Mode: Example 11-24
Using the IN OUT Parameter Mode: Example 11-25
Passing Parameters to Procedures 11-26
Passing Actual Parameters: Creating the raise_sal Procedure 11-27
Passing Actual Parameters: Examples 11-28
Using the DEFAULT Option for the Parameters 11-29
Lesson Agenda 11-30
Handled Exceptions 11-31
Handled Exceptions: Example 11-32
Exceptions Not Handled 11-33
Exceptions Not Handled: Example 11-34
Removing Procedures: Using the DROP SQL Statement or SQL Developer 11-35
Viewing Procedure Information Using the Data Dictionary Views 11-36
Viewing Procedures Information Using SQL Developer 11-37
Quiz 11-38
Summary 11-39
Practice 11-1 Overview: Creating a New Database Connection 11-40
Practice 11-2 Overview: Creating, Compiling, and Calling Procedures 11-41

12 Creating Functions

Course Road Map 12-2
Objectives 12-3
Lesson Agenda 12-4
Functions 12-5
Creating Functions syntax 12-6
Tax Calculation 12-8
The Difference Between Procedures and Functions 12-9
Creating Functions: Overview 12-10
Invoking a Stored Function: Example 12-11
Using Different Methods for Executing Functions 12-12
Creating and Compiling Functions Using SQL Developer 12-14
Lesson Agenda 12-15
Using a Function in a SQL Expression: Example 12-16
Calling User-Defined Functions in SQL Statements 12-17
Restrictions When Calling Functions from SQL Expressions 12-18
Side Effects of Function Execution 12-19
Controlling Side Effects 12-20
Guidelines to Control Side Effects 12-21
Lesson Agenda 12-22
Passing Parameters to Functions 12-23
Named and Mixed Notation from SQL: Example 12-24
Viewing Functions Using Data Dictionary Views 12-25
Viewing Functions Information Using SQL Developer 12-26
Lesson Agenda 12-27
Removing Functions: Using the DROP SQL Statement or SQL Developer 12-28
Quiz 12-29
Summary 12-30
Practice 12: Overview 12-31

13 Debugging Subprograms

Course Road Map 13-2
Objectives 13-3
Lesson Agenda 13-4
Before Debugging PL/SQL Subprograms 13-5
Lesson Agenda 13-9
Debugging a Subprogram: Overview 13-10
Lesson Agenda 13-12
The Debugging – Log Tab Toolbar 13-13
Tracking Data and Execution 13-15
Lesson Agenda 13-16

Debugging a Procedure Example: Creating a New emp_list Procedure 13-17
Debugging a Procedure Example: Creating a New get_location Function 13-18
Setting Breakpoints and Compiling emp_list for Debug Mode 13-19
Compiling the get_location Function for Debug Mode 13-20
Debugging emp_list and Entering Values for the PMAXROWS Parameter 13-21
Debugging emp_list: Step Into (F7) the Code 13-22
Viewing the Data 13-23
Modifying the Variables While Debugging the Code 13-24
Debugging emp_list: Step Over Versus Step Into 13-25
Debugging emp_list: Step Out of the Code (Shift + F7) 13-26
Debugging emp_list: Step to End of Method 13-27
Debugging a Subprogram Remotely: Overview 13-28
Summary 13-29
Practice 13 Overview: Introduction to the SQL Developer Debugger 13-30

14 Creating Packages

Course Road Map 14-2
Objectives 14-3
Lesson Agenda 14-4
DBMS_OUTPUT.PUT_LINE 14-5
What Is a Package? 14-6
Advantages of Packages 14-7
How Do You Create PL/SQL Packages? 14-8
Components of a PL/SQL Package 14-9
Application Program Interface 14-10
Lesson Agenda 14-11
Creating the Package Specification: Using the CREATE PACKAGE Statement 14-12
Creating Package Specification: Using SQL Developer 14-13
Creating the Package Body: Using SQL Developer 14-14
Example of a Package Specification: comm_pkg 14-15
Creating the Package Body 14-16
Example of a Package Body: comm_pkg 14-17
Invoking the Package Subprograms: Examples 14-18
Invoking Package Subprograms: Using SQL Developer 14-19
Creating and Using Bodiless Packages 14-20
Viewing Packages by Using the Data Dictionary 14-21
Viewing Packages by Using SQL Developer 14-22
Removing Packages 14-23
Removing Package Bodies 14-24
Guidelines for Writing Packages 14-25

Quiz 14-26
Summary 14-27
Practice 14 Overview: Creating and Using Packages 14-28

15 Working with Packages

Course Road Map 15-2
Objectives 15-3
Lesson Agenda 15-4
Why Overload Subprograms? 15-5
Overloading Subprograms in PL/SQL 15-6
Overloading Procedures Example: Creating the Package Specification 15-8
Overloading Procedures Example: Creating the Package Body 15-9
Restrictions on Overloading 15-10
STANDARD package 15-11
Overloading and the STANDARD Package 15-12
Lesson Agenda 15-13
Package Instantiation and Initialization 15-14
Initializing Packages in Package Body 15-15
Using User-Defined Package Functions in SQL 15-16
User-Defined Package Function in SQL: Example 15-17
Lesson Agenda 15-18
Package State 15-19
Serially Reusable Packages 15-20
Memory Architecture 15-21
Serially Reusable Packages 15-23
Persistent State of Packages 15-24
Persistent State of Package Variables: Example 15-25
Persistent State of a Package Cursor: Example 15-26
Executing the CURS_PKG Package 15-28
Quiz 15-29
Summary 15-30
Practice 15 Overview: Working with Packages 15-31

16 Using Oracle-Supplied Packages in Application Development

Course Road Map 16-2
Objectives 16-3
Lesson Agenda 16-4
Using Oracle-Supplied Packages 16-5
Examples of Some Oracle-Supplied Packages 16-6
Lesson Agenda 16-7
How the DBMS_OUTPUT Package Works 16-8

Using the UTL_FILE Package	16-9
Some of the UTL_FILE Procedures and Functions	16-10
File Processing Using the UTL_FILE Package: Overview	16-11
Using the Available Declared Exceptions in the UTL_FILE Package	16-12
FOPEN and IS_OPEN Functions: Example	16-13
Using UTL_FILE: Example	16-16
What Is the UTL_MAIL Package?	16-18
Setting Up and Using the UTL_MAIL: Overview	16-19
Summary of UTL_MAIL Subprograms	16-20
Installing and Using UTL_MAIL	16-21
The SEND Procedure Syntax	16-22
The SEND_ATTACH_RAW Procedure	16-23
Sending Email with a Binary Attachment: Example	16-24
The SEND_ATTACH_VARCHAR2 Procedure	16-26
Sending Email with a Text Attachment: Example	16-27
Quiz	16-32
Summary	16-33
Practice 16 Overview: Using Oracle-Supplied Packages in Application Development	16-34

17 Using Dynamic SQL

Course Road Map	17-2
Objectives	17-3
Lesson Agenda	17-4
What is Dynamic SQL?	17-5
When do you use Dynamic SQL?	17-6
Using Dynamic SQL	17-7
Execution Flow of SQL Statements	17-8
Dynamic SQL implementation	17-9
Lesson Agenda	17-10
Native Dynamic SQL (NDS)	17-11
Using the EXECUTE IMMEDIATE Statement	17-12
Dynamic SQL with a DDL Statement: Examples	17-13
Dynamic SQL with DML Statements	17-14
Dynamic SQL with a Single-Row Query: Example	17-15
Executing a PL/SQL Anonymous Block Dynamically	17-16
BULK COLLECT INTO clause	17-17
OPEN FOR clause	17-18
Using BULK COLLECT and OPEN FOR clause	17-19
Summarizing Methods for Using Dynamic SQL	17-20
Lesson Agenda	17-22

Using the DBMS_SQL Package 17-25
Using the DBMS_SQL Package Subprograms 17-26
Using DBMS_SQL with a DML Statement: Deleting Rows 17-27
Using DBMS_SQL with a Parameterized DML Statement 17-29
Quiz 17-30
Summary 17-31
Practice 17 Overview: Using Dynamic SQL 17-32

18 Creating Triggers

Course Road Map 18-2
Objectives 18-3
Lesson Agenda 18-4
What are Triggers? 18-6
Defining Triggers 18-7
Why do you use Triggers? 18-8
Trigger Event Types 18-9
Available Trigger Types 18-10
Trigger Event Types and Body 18-11
Lesson Agenda 18-12
Creating DML Triggers by Using the CREATE TRIGGER Statement 18-13
Creating DML Triggers by Using SQL Developer 18-14
Specifying the Trigger Execution Time 18-15
Creating a DML Statement Trigger Example: SECURE_EMP 18-16
Testing Trigger SECURE_EMP 18-17
Using Conditional Predicates 18-18
Multiple Triggers of the Same Type 18-19
CALL Statements in Triggers 18-20
Lesson Agenda 18-21
Statement-Level Triggers Versus Row-Level Triggers 18-22
Creating a DML Row Trigger 18-24
Correlation names and Pseudorecords 18-25
Correlation Names and Pseudorecords 18-26
Using OLD and NEW Qualifiers 18-27
Using OLD and NEW Qualifiers: Example 18-28
Using the WHEN Clause to Fire a Row Trigger Based on a Condition 18-30
Trigger-Firing Sequence: Single-Row Manipulation 18-31
Trigger-Firing Sequence: Multirow Manipulation 18-32
Summary of the Trigger Execution Model 18-33
Lesson Agenda 18-34
INSTEAD OF Triggers 18-35
Creating an INSTEAD OF Trigger: Example 18-36

Creating an INSTEAD OF Trigger to Perform DML on Complex Views 18-37
Lesson Agenda 18-39
The Status of a Trigger 18-40
System Privileges Required to Manage Triggers 18-41
Managing Triggers by Using the ALTER and DROP SQL Statements 18-42
Managing Triggers by Using SQL Developer 18-43
Viewing Trigger Information 18-44
Using USER_TRIGGERS 18-45
Testing Triggers 18-46
Quiz 18-47
Summary 18-48
Practice 18 Overview: Creating Statement and Row Triggers 18-49

19 Creating Compound, DDL, and Event Database Triggers

Course Road Map 19-2
Objectives 19-3
Lesson Agenda 19-4
What is a Compound Trigger? 19-5
Working with Compound Triggers 19-6
Why Compound Triggers? 19-7
Compound Trigger Structure 19-8
Compound Trigger Structure for Views 19-9
Compound Trigger Restrictions 19-10
Lesson Agenda 19-11
Mutating Tables 19-12
Mutating Table: Example 19-13
Using a Compound Trigger to Resolve the Mutating Table Error 19-15
Lesson Agenda 19-18
Creating Triggers on DDL Statements 19-19
Creating Triggers on DDL Statements -Example 19-20
Lesson Agenda 19-21
Creating Database Triggers 19-22
Creating Triggers on System Events 19-23
LOGON and LOGOFF Triggers: Example 19-24
Lesson Agenda 19-25
Guidelines for Designing Triggers 19-26
Quiz 19-27
Summary 19-28
Practice 19 Overview: Creating Compound, DDL, and Event Database Triggers 19-29

20 Design Considerations for the PL/SQL Code

Course Road Map	20-2
Objectives	20-3
Lesson Agenda	20-4
Standardizing Constants and Exceptions	20-5
Standardizing Exceptions	20-6
Standardizing Exception Handling	20-7
Standardizing Constants	20-8
Local Subprograms	20-9
Lesson Agenda	20-10
Definer's and Invoker's Rights	20-11
Specifying Invoker's Rights: Setting AUTHID to CURRENT_USER	20-12
Granting Privileges to Invoker's Rights Unit	20-13
Lesson Agenda	20-14
Autonomous Transactions	20-15
Features of Autonomous Transactions	20-16
Using Autonomous Transactions: Example	20-17
Lesson Agenda	20-19
Using the NOCOPY Hint	20-20
Effects of the NOCOPY Hint	20-21
When Does the PL/SQL Compiler Ignore the NOCOPY Hint?	20-22
Using the PARALLEL_ENABLE Hint	20-23
Using the Cross-Session PL/SQL Function Result Cache	20-24
Declaring and Defining a Result-Cached Function: Example	20-25
Using the DETERMINISTIC Clause with Functions	20-26
Using the RETURNING Clause	20-27
Lesson Agenda	20-28
Using Bulk Binding	20-29
Bulk Binding: Syntax and Keywords	20-30
Bulk Binding FORALL: Example	20-31
Using BULK COLLECT INTO with Queries	20-34
Using BULK COLLECT INTO with Cursors	20-35
Using BULK COLLECT INTO with a RETURNING Clause	20-36
Quiz	20-37
Summary	20-38
Practice 20 Overview: Design Considerations for PL/SQL Code	20-39

21 Tuning the PL/SQL Compiler

Course Road Map	21-2
Objectives	21-3
Lesson Agenda	21-4

Optimizing PL/SQL Compiler Performance	21-5
Initialization Parameters for PL/SQL Compilation	21-6
Using the Initialization Parameters for PL/SQL Compilation	21-8
Displaying the PL/SQL Initialization Parameters	21-9
Displaying and Setting PL/SQL Initialization Parameters	21-10
Changing PL/SQL Initialization Parameters: Example	21-11
Lesson Agenda	21-12
PL/SQL Compile-Time Warnings	21-13
Benefits of Compiler Warnings	21-14
Categories of PL/SQL Compile-Time Warning Messages	21-15
Enabling Warning Messages	21-16
Setting Compiler Warning Levels: Using PLSQL_WARNINGS, Examples	21-17
Enabling Compiler Warnings: Using PLSQL_WARNINGS in SQL Developer	21-18
Viewing the Current Setting of PLSQL_WARNINGS	21-19
Viewing Compiler Warnings	21-20
SQL*Plus Warning Messages: Example	21-21
Defining PLSQL_WARNINGS for Program Units	21-22
Lesson Agenda	21-23
Using the DBMS_WARNINGS Package	21-24
Using the DBMS_WARNING Package Subprograms	21-25
The DBMS_WARNING Procedures: Syntax, Parameters, and Allowed Values	21-26
The DBMS_WARNING Procedures: Example	21-27
The DBMS_WARNING Functions: Syntax, Parameters, and Allowed Values	21-28
The DBMS_WARNING Functions: Example	21-29
Using DBMS_WARNING: Example	21-30
Quiz	21-32
Summary	21-33
Practice 21 Overview: Tuning PL/SQL Compiler	21-34

22 Managing Dependencies

Course Road Map	22-2
Objectives	22-3
Lesson Agenda	22-4
What are Dependencies in a Schema?	22-5
How Dependencies Work?	22-6
Dependent and Referenced Objects	22-8
Querying Object Dependencies: Using the USER_DEPENDENCIES View	22-10
Querying an Object's Status	22-11
Categorizing Dependencies	22-12
Lesson Agenda	22-13

Direct Dependencies	22-14
Indirect Dependencies	22-15
Displaying Direct and Indirect Dependencies	22-16
Lesson Agenda	22-18
Fine-Grained Dependency Management	22-19
Fine-Grained Dependency Management: Example 1	22-21
Fine-Grained Dependency Management: Example 2	22-23
Guidelines for Reducing Invalidation	22-24
Object Revalidation	22-25
Lesson Agenda	22-26
Remote Dependencies	22-27
Managing Remote Procedure Dependencies	22-28
Setting the REMOTE_DEPENDENCIES_MODE Parameter	22-29
Timestamp Checking	22-30
Signature Checking	22-35
Lesson Agenda	22-36
Revalidating PL/SQL Program Units	22-37
Unsuccessful Recompilation	22-38
Successful Recompilation	22-39
Recompiling Procedures	22-40
Lesson Agenda	22-41
Packages and Dependencies: Subprogram References the Package	22-42
Packages and Dependencies: Package Subprogram References Procedure	22-43
Quiz	22-44
Summary	22-45
Practice 22 Overview: Managing Dependencies in Your Schema	22-46

A Using SQL Developer

Objectives	A-2
What Is Oracle SQL Developer?	A-3
Specifications of SQL Developer	A-4
SQL Developer Interface	A-5
Creating a Database Connection	A-7
Browsing Database Objects	A-10
Displaying the Table Structure	A-11
Browsing Files	A-12
Creating a Schema Object	A-13
Creating a New Table: Example	A-14
Using the SQL Worksheet	A-15
Executing SQL Statements	A-19
Saving SQL Scripts	A-20

Executing Saved Script Files: Method 1 A-21
Executing Saved Script Files: Method 2 A-22
Formatting the SQL Code A-23
Using Snippets A-24
Using Snippets: Example A-25
Debugging Procedures and Functions A-26
Database Reporting A-27
Creating a User-Defined Report A-28
Search Engines and External Tools A-29
Setting Preferences A-30
Resetting the SQL Developer Layout A-32
Data Modeler in SQL Developer A-33
Summary A-34

B Using SQL*Plus

Objectives B-2
SQL and SQL*Plus Interaction B-3
SQL Statements Versus SQL*Plus Commands B-4
Overview of SQL*Plus B-5
Logging In to SQL*Plus B-6
Displaying the Table Structure B-7
SQL*Plus Editing Commands B-9
Using LIST, n, and APPEND B-11
Using the CHANGE Command B-12
SQL*Plus File Commands B-13
Using the SAVE, START Commands B-14
SERVEROUTPUT Command B-15
Using the SQL*Plus SPOOL Command B-16
Using the AUTOTRACE Command B-17
Summary B-18

C Commonly Used SQL Commands

Objectives C-2
Basic SELECT Statement C-3
SELECT Statement C-4
WHERE Clause C-5
ORDER BY Clause C-6
GROUP BY Clause C-7
Data Definition Language C-8
CREATE TABLE Statement C-9
ALTER TABLE Statement C-10

DROP TABLE Statement C-11
GRANT Statement C-12
Privilege Types C-13
REVOKE Statement C-14
TRUNCATE TABLE Statement C-15
Data Manipulation Language C-16
INSERT Statement C-17
UPDATE Statement Syntax C-18
DELETE Statement C-19
Transaction Control Statements C-20
COMMIT Statement C-21
ROLLBACK Statement C-22
SAVEPOINT Statement C-23
Joins C-24
Types of Joins C-25
Qualifying Ambiguous Column Names C-26
Natural Join C-27
Equijoins C-28
Retrieving Records with Equijoins C-29
Additional Search Conditions Using the AND and WHERE Operators C-30
Retrieving Records with Nonequijoins C-31
Retrieving Records by Using the USING Clause C-32
Retrieving Records by Using the ON Clause C-33
Left Outer Join C-34
Right Outer Join C-35
Full Outer Join C-36
Self-Join: Example C-37
Cross Join C-38
Summary C-39

D REF Cursors

Cursor Variables D-2
Using Cursor Variables D-3
Defining REF CURSOR Types D-4
Using the OPEN-FOR, FETCH, and CLOSE Statements D-7
Example of Fetching D-10

E Managing PL/SQL Code

Objectives E-2
Agenda E-3
Conditional Compilation E-4

How Does Conditional Compilation Work?	E-5
Using Selection Directives	E-6
Using Predefined and User-Defined Inquiry Directives	E-7
The PLSQL_CCFLAGS Parameter and the Inquiry Directive	E-8
Displaying the PLSQL_CCFLAGS Initialization Parameter Setting	E-9
The PLSQL_CCFLAGS Parameter and the Inquiry Directive: Example	E-10
Using Conditional Compilation Error Directives to Raise User-Defined Errors	E-11
Using Static Expressions with Conditional Compilation	E-12
DBMS_DB_VERSION Package: Boolean Constants	E-13
DBMS_DB_VERSION Package Constants	E-14
Using Conditional Compilation with Database Versions: Example	E-15
Using DBMS_PREPROCESSOR Procedures to Print or Retrieve Source Text	E-17
Agenda	E-18
Obfuscation	E-19
Benefits of Obfuscating	E-20
What's New in Dynamic Obfuscating Since Oracle 10g?	E-21
Nonobfuscated PL/SQL Code: Example	E-22
Obfuscated PL/SQL Code: Example	E-23
Dynamic Obfuscation: Example	E-24
PL/SQL Wrapper Utility	E-25
Running the PL/SQL Wrapper Utility	E-26
Results of Wrapping	E-27
Guidelines for Wrapping	E-28
DBMS_DDL Package Versus wrap Utility	E-29
Summary	E-30

F Implementing Triggers

Objectives	F-2
Controlling Security Within the Server	F-3
Controlling Security with a Database Trigger	F-4
Enforcing Data Integrity Within the Server	F-5
Protecting Data Integrity with a Trigger	F-6
Enforcing Referential Integrity Within the Server	F-7
Protecting Referential Integrity with a Trigger	F-8
Replicating a Table Within the Server	F-9
Replicating a Table with a Trigger	F-10
Computing Derived Data Within the Server	F-11
Computing Derived Values with a Trigger	F-12
Logging Events with a Trigger	F-13
Summary	F-15

G Using the DBMS_SCHEDULER and HTP Packages

Objectives G-2

Generating Webpages with the HTP Package G-3

Using the HTP Package Procedures G-4

Creating an HTML File with SQL*Plus G-5

The DBMS_SCHEDULER Package G-6

Creating a Job G-8

Creating a Job with Inline Parameters G-9

Creating a Job Using a Program G-10

Creating a Job for a Program with Arguments G-11

Creating a Job Using a Schedule G-12

Setting the Repeat Interval for a Job G-13

Creating a Job Using a Named Program and Schedule G-14

Managing Jobs G-15

Data Dictionary Views G-16

Summary G-17

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Working with Packages

Course Road Map



2

In the fourth unit, we have seven lessons – Creating Stored Procedures, Creating Functions, Debugging Subprograms, Creating Packages, Working with Packages, Using Oracle Supplied Packages in Application Development, and Using Dynamic SQL. You will learn to write PL/SQL subprograms, packages, and use them in Application Development.

0

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Overload package procedures and functions
- Use forward declarations
- Create an initialization block in a package body
- Manage persistent package data states for the life of a session



0

3

This lesson introduces the more advanced features of PL/SQL, including overloading, forward referencing, one-time-only procedures, and the persistency of variables, constants, exceptions, and cursors. It also explains the effect of packaging functions that are used in SQL statements.

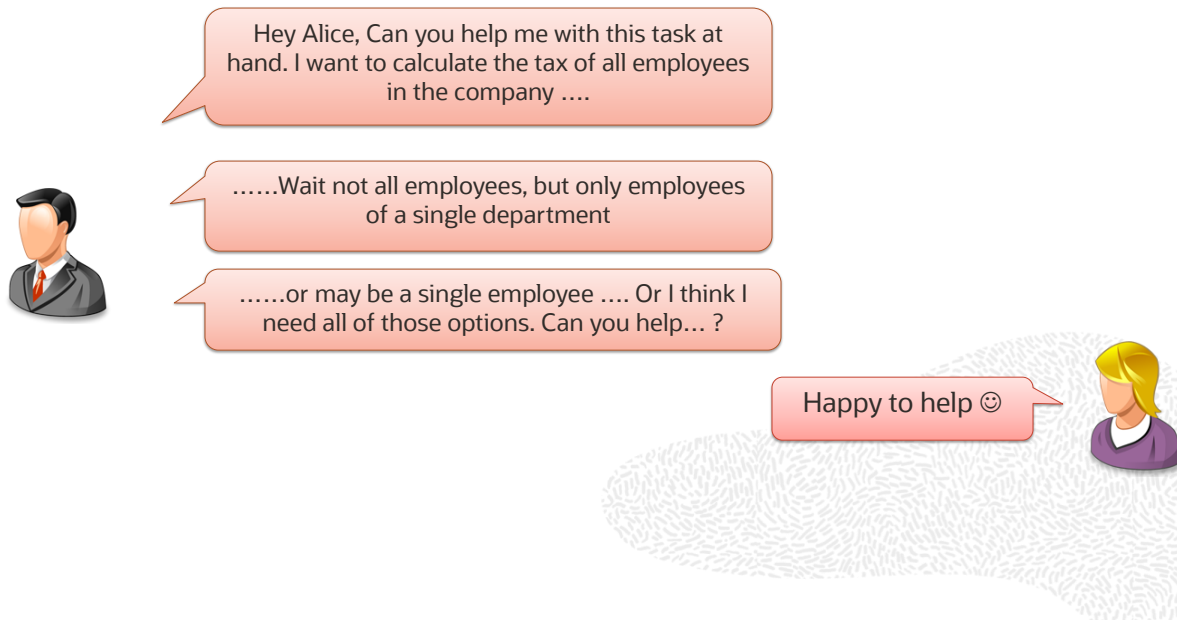
For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Overloading package subprograms
- Initializing packages
- Managing persistent package state in a session



Why Overload Subprograms?



5

Here is a situation, where the HR manager has to calculate the tax of the employees. It is not just calculating the tax applicable to all the employees always. Sometimes he might want to calculate tax for all the employees in a particular department . Sometimes the HR manager may also want to calculate tax for a single employee. All these are variants of the same operation tax calculation with different inputs.

Let's look at this problem from Alice's perspective. Alice may have to write a procedure to calculate tax for the employees. However the procedure may have different set of input parameters each time. In order to accommodate this varying set of inputs, Alice can write an overloaded subprogram.

An overloaded subprogram is one with the same name, but different set of input parameters.

Overloading Subprograms in PL/SQL

- Enables you to create two or more subprograms with the same name
- Requires that the subprogram's formal parameters differ in number, order, or data type family
- Enables you to build flexible ways for invoking subprograms with different data
- Provides a way to extend functionality without loss of existing code; that is, adding new parameters to existing subprograms
- Provides a way to overload local subprograms, package subprograms, and type methods, but not stand-alone subprograms

6

The overloading feature in PL/SQL enables you to develop two or more packaged subprograms with the same name. Overloading is useful when you want a subprogram to accept similar sets of parameters that have different data types. For example, the `TO_CHAR` function has more than one way to be called, enabling you to convert a number or a date to a character string.

PL/SQL allows overloading of package subprogram names and object type methods.

The key rule is that you can use the same name for different subprograms as long as their formal parameters differ in *number, order, or data type* family.

Consider using overloading when you are:

- Processing rules for two or more subprograms are similar, but the type or number of parameters used varies
- Providing alternative ways for finding different data with varying search criteria. For example, you may want to find employees by their employee ID and also provide a way to find employees by their last name. The logic is intrinsically the same, but the parameters or search criteria differ.
- Extending functionality when you do not want to replace existing code

Note: Stand-alone subprograms cannot be overloaded. Writing local subprograms in object type methods is not discussed in this course.

Restrictions

You cannot overload:

- Two subprograms if their formal parameters differ only in data type and the different data types are in the same family (`NUMBER` and `DECIMAL` belong to the same family.)
- Two subprograms if their formal parameters differ only in subtype and the different subtypes are based on types in the same family (`VARCHAR` and `STRING` are PL/SQL subtypes of `VARCHAR2`.)
- Two functions that differ only in return type, even if the types are in different families

Resolving Calls

The compiler tries to find a declaration that matches the call. It searches first in the current scope and then, if necessary, in successive enclosing scopes. The compiler stops searching if it finds one or more subprogram declarations in which the name matches the name of the called subprogram. For similarly named subprograms at the same level of scope, the compiler needs an exact match in number, order, and data type between the actual and formal parameters.

For Instructor Use Only.
This document should not be distributed.

Overloading Procedures Example: Creating the Package Specification

```
CREATE OR REPLACE PACKAGE dept_pkg IS
  PROCEDURE add_department
    (p_deptno departments.department_id%TYPE,
     p_name departments.department_name%TYPE := 'unknown',
     p_loc departments.location_id%TYPE := 1700);

  PROCEDURE add_department
    (p_name departments.department_name%TYPE := 'unknown',
     p_loc departments.location_id%TYPE := 1700);
END dept_pkg;
/
```

8

The slide shows the `dept_pkg` package specification with an overloaded procedure called `add_department`. The first declaration takes three parameters that are used to provide data for a new department record inserted into the department table. The second declaration takes only two parameters because this version internally generates the department ID through an Oracle sequence.

It is better to specify data types using the `%TYPE` attribute for variables that are used to populate columns in database tables, as shown in the example in the slide; however, you can also specify the data types as follows:

```
CREATE OR REPLACE PACKAGE dept_pkg_method2 IS

  PROCEDURE add_department(p_deptno NUMBER,
    p_name VARCHAR2 := 'unknown', p_loc NUMBER := 1700);

  ...
```

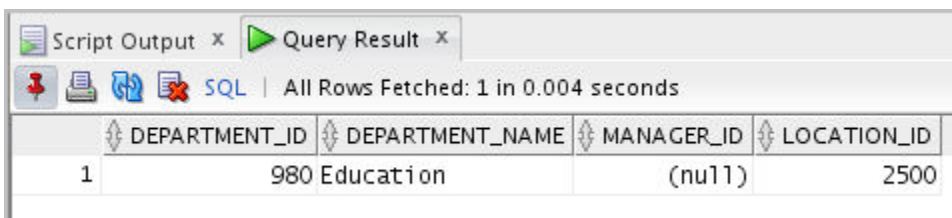
Overloading Procedures Example: Creating the Package Body

```
-- Package body of package defined on previous slide.
CREATE OR REPLACE PACKAGE BODY dept_pkg IS
PROCEDURE add_department -- First procedure's definition
(p_deptno departments.department_id%TYPE,
 p_name    departments.department_name%TYPE := 'unknown',
 p_loc     departments.location_id%TYPE := 1700) IS
BEGIN
    INSERT INTO departments (department_id,
                             department_name, location_id)
    VALUES (p_deptno, p_name, p_loc);
END add_department;
PROCEDURE add_department -- Second procedure's definition
(p_name    departments.department_name%TYPE := 'unknown',
 p_loc     departments.location_id%TYPE := 1700) IS
BEGIN
    INSERT INTO departments (department_id,
                             department_name, location_id)
    VALUES (departments_seq.NEXTVAL, p_name, p_loc);
END add_department;
END dept_pkg; /
```

9

If you call `add_department` with an explicitly provided department ID, then PL/SQL uses the first version of the procedure. Consider the following example:

```
EXECUTE dept_pkg.add_department(980, 'Education', 2500)
SELECT * FROM departments
WHERE department_id = 980;
```

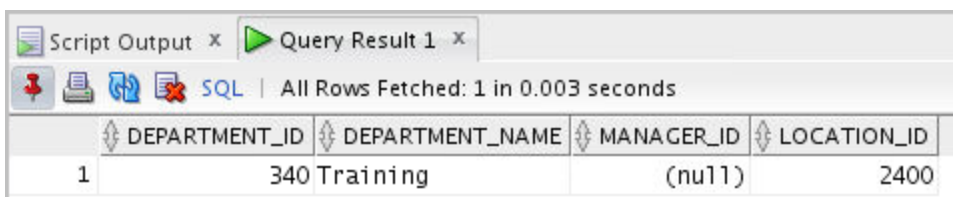


The screenshot shows the 'Query Result' window in SQL Developer. It displays a single row of data from the 'departments' table. The columns are DEPARTMENT_ID, DEPARTMENT_NAME, MANAGER_ID, and LOCATION_ID. The values in the row are 1, 980 Education, (null), and 2500.

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	980	Education	(null)	2500

If you call `add_department` with no department ID, PL/SQL uses the second version:

```
EXECUTE dept_pkg.add_department ('Training', 2400)
SELECT * FROM departments
WHERE department_name = 'Training';
```



The screenshot shows the 'Query Result' window in SQL Developer. It displays a single row of data from the 'departments' table. The columns are DEPARTMENT_ID, DEPARTMENT_NAME, MANAGER_ID, and LOCATION_ID. The values in the row are 1, 340 Training, (null), and 2400.

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	340	Training	(null)	2400

Restrictions on Overloading

- You cannot overload:
 - Stand-alone subprograms
 - Subprograms whose parameters differ only in the mode (IN, OUT)
 - Subprograms whose parameters belong to the same family respectively
 - Functions which return only in the return type but not the formal parameters

10

You can't overload the subprograms in the scenarios mentioned in the slide.

You can't overload stand-alone subprograms because they are independent units and it will result in a naming conflict.

In a package, you can't overload subprograms which differ only in the parameter mode type. Consider

```
procedure A(p1 IN NUMBER) and procedure A(p2 OUT NUMBER)
```

These subprograms cannot be overloaded and will result in naming conflict.

Two subprograms can't be overloaded, if their parameters correspond to same family of data types.

For instance,

```
PROCEDURE s (p INTEGER) and PROCEDURE s (p REAL)
```

can't be overloaded, the parameters belong to the same family of data types.

Similarly two functions which only differ in the return type can't be overloaded.

For instance,

```
FUNCTION f (p INTEGER) RETURN BOOLEAN and
```

```
FUNCTION f (p INTEGER) RETURN INTEGER
```

can't be overloaded.

You get a run-time error when you overload subprograms with the preceding features.

Note: The preceding restrictions apply if the names of the parameters are also the same.

If you use different names for the parameters, you can invoke the subprograms by using named notation for the parameters.

STANDARD package

- Is an Oracle-supplied package
- Defines the PL/SQL environment
- Declares types, exceptions, and subprograms, which are available automatically to every PL/SQL program

11

The PL/SQL environment is defined by the `STANDARD` package which is an Oracle-supplied package.

The package specification declares public types, variables, exceptions, subprograms, which are available automatically to PL/SQL programs. For example, the `STANDARD` package declares function `ABS`, which returns the absolute value of its argument.

The contents of the `STANDARD` package are directly visible to applications. You need not qualify references to its contents by prefixing the package name.

For instance to invoke the `ABS` function of the `STANDARD` package, you need not write `STANDARD.ABS`; writing `ABS` will automatically refer to the `STANDARD` package.

Overloading and the STANDARD Package

- Most built-in functions in the STANDARD package are overloaded.
- An example is the TO_CHAR function:

```
FUNCTION TO_CHAR (p1 DATE) RETURN VARCHAR2;  
FUNCTION TO_CHAR (p2 NUMBER) RETURN VARCHAR2;  
FUNCTION TO_CHAR (p1 DATE, P2 VARCHAR2) RETURN VARCHAR2;  
FUNCTION TO_CHAR (p1 NUMBER, P2 VARCHAR2) RETURN VARCHAR2;  
. . .
```

- A PL/SQL subprogram with the same name as a built-in subprogram overrides the standard declaration in the local context, unless qualified by the package name.

12

Most of the built-in functions that are found in the STANDARD package are overloaded. For example, the TO_CHAR function has four different declarations, as shown in the slide. The TO_CHAR function can take either the DATE or the NUMBER data type and convert it to the character data type. The format to which the date or number has to be converted can also be specified in the function call.

If you redeclare a built-in subprogram in another PL/SQL program, then your local declaration overrides the standard or built-in subprogram. To be able to access the built-in subprogram, you must qualify it with its package name. For example, if you redeclare the TO_CHAR function to access the built-in function, you refer to it as STANDARD.TO_CHAR.

If you redeclare a built-in subprogram as a stand-alone subprogram, then, to access your subprogram, you must qualify it with your schema name, for example, SCOTT.TO_CHAR.

In the example in the slide, PL/SQL resolves a call to TO_CHAR by matching the number and data types of the formal and actual parameters.

Lesson Agenda

- Overloading package subprograms
- Initializing packages
- Managing persistent package state in a session



Package Instantiation and Initialization

- Packages are stored as schema objects.
- Packages are instantiated when referenced.
- Each application which references the package has its own instance.
- The package is initialized whenever instantiated.

14

When you create and compile packages, they are stored as schema objects in the database. Applications can access the members of the package through the package name as `package_name.procedure_name` like `DBMS_OUTPUT.PUT_LINE` where `DBMS_OUTPUT` is the package name and `PUT_LINE` is the procedure name.

Whenever an application references a package to use its functionality, an instance of the package is created for the application.

Various package members are initialized on instantiation. Following steps are executed as part of initialization:

- Assign initial values to public constants.
- Assign initial values to public variables whose declarations specify them.
- Execute the initialization part of the package body.

Initializing Packages in Package Body

In package body, you can initialize package members in the initialization block at the end of the package body.

```
CREATE OR REPLACE PACKAGE taxes IS
  v_tax  NUMBER;
  ... -- declare all public procedures/functions
END taxes;
/
CREATE OR REPLACE PACKAGE BODY taxes IS
  ... -- declare all private variables
  ... -- define public/private procedures/functions
BEGIN
  SELECT  rate_value INTO v_tax
  FROM    tax_rates
  WHERE   rate_name = 'TAX';
END taxes;
/
```

15

The first time a component in a package is referenced, the entire package is loaded into memory for the user session. By default, the initial value of variables is `NULL` (if not explicitly initialized). To initialize package variables, you can:

- Use assignment operations in their declarations for simple initialization tasks
- Add code block to the end of a package body for more complex initialization tasks

Consider the block of code at the end of a package body, it is the package initialization block that executes once, when the package is first invoked within the user session.

The example in the slide shows the `v_tax` public variable being initialized to the value in the `tax_rates` table the first time the `taxes` package is referenced.

Note: If you initialize the variable in the declaration by using an assignment operation, it is overwritten by the code in the initialization block at the end of the package body. The initialization block is terminated by the `END` keyword for the package body.

Using User-Defined Package Functions in SQL

- Functions in user-defined packages can be accessed like those in Oracle-supplied packages.
- The user-defined package function usage is similar to that of functions such as MAX, MIN and other predefined functions.
- The package specification and body should be compiled before using them in the SQL statements in the current schema.
- You have to identify the functions with the package name.

16

After compiling both the package specification and package body, user-defined packages are as good as Oracle-supplied packages. You can access the package functions in SQL statements by referencing the function through the package name.

The function execution should be free of any side effects, which we discussed in the lesson titled “Creating Functions.”

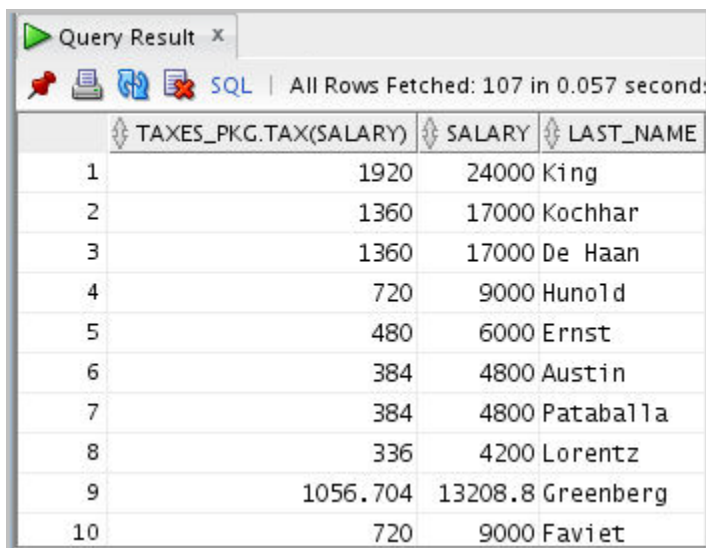
User-Defined Package Function in SQL: Example

```
CREATE OR REPLACE PACKAGE taxes_pkg IS
    FUNCTION tax (p_value IN NUMBER) RETURN NUMBER;
END taxes_pkg;
/
CREATE OR REPLACE PACKAGE BODY taxes_pkg IS
    FUNCTION tax (p_value IN NUMBER) RETURN NUMBER IS
        v_rate NUMBER := 0.08;
    BEGIN
        RETURN (p_value * v_rate);
    END tax;
END taxes_pkg;
/
```

```
SELECT taxes_pkg.tax(salary), salary, last_name
FROM employees;
```

17

The first code example in the slide shows how to create the package specification and the body encapsulating the tax function in the `taxes_pkg` package. The second code example shows how to call the packaged tax function in the `SELECT` statement. The results are as follows:



The screenshot shows a 'Query Result' window with a table of 10 rows. The columns are TAXES_PKG.TAX(SALARY), SALARY, and LAST_NAME. The data represents the tax calculation for each employee in the 'employees' table.

	TAXES_PKG.TAX(SALARY)	SALARY	LAST_NAME
1	1920	24000	King
2	1360	17000	Kochhar
3	1360	17000	De Haan
4	720	9000	Hunold
5	480	6000	Ernst
6	384	4800	Austin
7	384	4800	Pataballa
8	336	4200	Lorentz
9	1056.704	13208.8	Greenberg
10	720	9000	Faviet

....

Lesson Agenda

- Overloading package subprograms
- Initializing packages
- Managing persistent package state in a session



Package State

- Package State refers to the values of the variables, constants, and cursors which are part of the package.
- When a package is instantiated, its state is initialized.
- The state of the package may change during application execution.
- A package without any members that hold a value is a stateless package.

Serially Reusable Packages

`SERIALLY_REUSABLE` packages let you design applications that manage memory better for scalability.



How does the memory management actually work in an Oracle database for packages?

20

Serially Reusable Packages are no different from packages in usage perspective. They are efficient in managing memory in scalable applications.

To understand how are they different from packages, we first need to understand how packages use memory.

For Instructor Use Only.
This document should not be distributed.

Memory Architecture

There are three important data structures defined according to the Memory Architecture of the database instance:

System Global Area

- Group of shared memory structures, which has information of one database instance. All server and background processes use it.

User Global Area

- Memory area associated with a user session connecting with the database instance

Program Global Area

- It is the unshared memory region that contains data and control information exclusively for use by an Oracle process.
- One PGA exists for each server process and background process.

21

When an instance is started, Oracle Database allocates a memory area and starts background processes.

The memory area stores information such as the following:

- Program code
- Information about each connected session, even if it is not currently active
- Information needed during program execution, for example, the current state of a query from which rows are being fetched
- Information such as lock data that is shared and communicated among processes
- Cached data, such as data blocks and redo records, that also exists on disk

There are three primary memory areas defined according to the memory architecture of the database instance:

1. System global area
2. Program global area
3. User global area

System global area has all shared memory structures associated with a database instance. All server processes and background processes of the database instance are in the System global area.

Program Global Area is created by the database whenever an Oracle process is initiated. It is used only by the process and not shared with other processes.

User Global Area is the memory area which has all the data pertaining to the communication between an application and database.

When a user connects with the database, all the data used by the user is stored in the User Global Area.

If a session loads a **PL/SQL package** into memory, then the UGA contains the package state, which is the set of values stored in all the package variables at a specific time. The package state changes when a package subprogram changes the variables. By default, the package variables are unique to and persist for the life of the session.

For Instructor Use Only.
This document should not be distributed.

Serially Reusable Packages

- Package state of a package is stored in the user global area (UGA) for each user.
- Package state of a `SERIALLY REUSABLE PACKAGE` is stored in a work area in a small pool in the system global area (SGA).
- Storing the package state in the UGA limits the scalability of the application.

23

When the package state is stored in the UGA, the amount of UGA memory needed increases linearly with the number of users. The package state can persist for the life of a session, locking UGA memory until the session ends. In some applications, such as Oracle Office, a typical session lasts several days. In such scenarios, the UGA is occupied for an extended period of time limiting the scalability.

When a package is declared to be `SERIALLY_REUSABLE`, its package state is stored in a work area in a small pool in the system global area (SGA). The package state persists only for the life of a server call. After the server call, the work area returns to the pool. Because the memory is returned to the pool, the scalability of the application is better.

However, If a subsequent server call references the package, then Oracle Database reuses an instantiation from the pool. Reusing an instantiation reinitializes it; therefore, changes made to the package state in previous server calls are invisible. We have to persist the package state after returning the server call, so that the user can reference to the earlier package state.

Persistent State of Packages

The collection of package variables and the values define the package state. The package state is:

- Initialized when the package is first loaded
- Persistent (by default) for the life of the session:
 - Stored in the User Global Area (UGA)
 - Unique to each session
 - Subject to change when package subprograms are called or public variables are modified
- Not persistent for the session but persistent for the life of a subprogram call when using `PRAGMA SERIALLY_REUSABLE` in the package specification

24

The collection of public and private package variables represents the package state for the user session. That is, the package state is the set of values stored in all the package variables at a given point in time. In general, the package state exists for the life of the user session.

Package variables are initialized the first time a package is loaded into memory for a user session. The package variables are, by default, unique to each session and hold their values until the user session is terminated. In other words, the variables are stored in the User Global Area (UGA) memory allocated by the database for each user session. The package state changes when a package subprogram is invoked and its logic modifies the variable state. Public package state can be directly modified by operations appropriate to its type.

`PRAGMA` signifies that the statement is a compiler directive. `PRAGMAS` are processed at compile time, not at run time. They do not affect the meaning of a program; they simply convey information to the compiler. If you add `PRAGMA SERIALLY_REUSABLE` to the package specification, then the database stores package variables in the System Global Area (SGA) shared across user sessions. In this case, the package state is maintained for the life of a subprogram call or a single reference to a package construct. The `SERIALLY_REUSABLE` directive is useful if you want to conserve memory and if the package state does not need to persist for each user session.

Persistent State of Package Variables: Example

Time	Events	State for Scott		State for Jones	
		v_std_comm [variable]	MAX (commission_pct) [column]	v_std_comm [variable]	MAX (commission_pct) [Column]
9:00	Scott> EXECUTE comm_pkg.reset_comm(0.25)	0.10 0.25	0.4	-	0.4
9:30	Jones> INSERT INTO employees(last_name, commission_pct) VALUES('Madonna', 0.8);	0.25	0.4		0.8
9:35	Jones> EXECUTE comm_pkg.reset_comm (0.5)	0.25	0.4	0.10 0.5	0.8
10:00	Scott> EXECUTE comm_pkg.reset_comm(0.6) Err -20210 'Bad Commission'	0.25	0.4	0.5	0.8
11:00	Jones> ROLLBACK;	0.25	0.4	0.5	0.4
11:01	EXIT ...	0.25	0.4	-	0.4
12:00	EXEC comm_pkg.reset_comm(0.2)	0.25	0.4	0.2	0.4

25

The slide sequence is based on two different users, Scott and Jones, executing `comm_pkg` (covered in the lesson titled “Creating Packages”), in which the `reset_comm` procedure invokes the `validate` function to check the new commission. The example shows how the persistent state of the `v_std_comm` package variable is maintained in each user session.

At 9:00: Scott calls `reset_comm` with a new commission value of 0.25, the package state for `v_std_comm` is initialized to 0.10 and then set to 0.25, which is validated because it is less than the database maximum value of 0.4.

At 9:30: Jones inserts a new row into the `EMPLOYEES` table with a new maximum `v_commission_pct` value of 0.8. This is not committed, so it is visible to Jones only. Scott’s state is unaffected.

At 9:35: Jones calls `reset_comm` with a new commission value of 0.5. The state for Jones’s `v_std_comm` is first initialized to 0.10 and then set to the new value 0.5 that is valid for his session with the database maximum value of 0.8.

At 10:00: Scott calls `reset_comm` with a new commission value of 0.6, which is greater than the maximum database commission visible to his session, that is, 0.4. (Jones did not commit the 0.8 value.)

Between 11:00 and 12:00: Jones rolls back the transaction (`INSERT` statement) and exits the session. Jones logs in at 11:45 and successfully executes the procedure, setting his state to 0.2.

Persistent State of a Package Cursor: Example

```
CREATE OR REPLACE PACKAGE curs_pkg IS -- Package spec
  PROCEDURE open;
  FUNCTION next(p_n NUMBER := 1) RETURN BOOLEAN;
  PROCEDURE close;
END curs_pkg;

CREATE OR REPLACE PACKAGE BODY curs_pkg IS
-- Package body
  CURSOR cur_c IS
    SELECT employee_id FROM employees;
  PROCEDURE open IS
  BEGIN
    IF NOT cur_c%ISOPEN THEN
      OPEN cur_c;
    END IF;
  END open;
  . . . -- code continued on next slide
```

26

The example in the slide shows the `CURS_PKG` package specification and body. The body declaration is continued in the next slide.

To use this package, perform the following steps to process the rows:

1. Call the `open` procedure to open the cursor.

Persistent State of a Package Cursor: Example

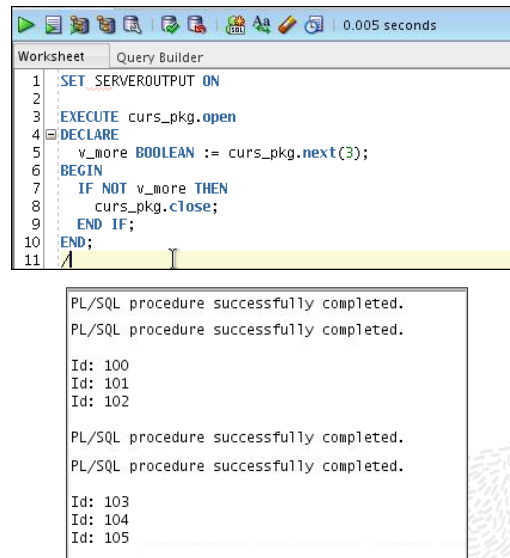
```
. . .  
FUNCTION next(p_n NUMBER := 1) RETURN BOOLEAN IS  
  v_emp_id employees.employee_id%TYPE;  
BEGIN  
  FOR count IN 1 .. p_n LOOP  
    FETCH cur_c INTO v_emp_id;  
    EXIT WHEN cur_c%NOTFOUND;  
    DBMS_OUTPUT.PUT_LINE('Id: ' || (v_emp_id));  
  END LOOP;  
  RETURN cur_c%FOUND;  
END next;  
PROCEDURE close IS  
BEGIN  
  IF cur_c%ISOPEN THEN  
    CLOSE cur_c;  
  END IF;  
END close;  
END curs_pkg;
```

27

2. Call the `next` procedure to fetch one or a specified number of rows. If you request more rows than actually exist, the procedure successfully handles termination. It returns `TRUE` if more rows need to be processed; otherwise it returns `FALSE`.
3. Call the `close` procedure to close the cursor, before or at the end of processing the rows.

Note: The cursor declaration is private to the package. Therefore, the cursor state can be influenced by invoking the package procedure and functions listed in the slide.

Executing the CURS_PKG Package



The screenshot shows the Oracle SQL Developer Query Builder interface. The top toolbar includes icons for running and saving scripts, with a timer showing 0.005 seconds. The 'Worksheet' tab is active, displaying a PL/SQL script with line numbers 1 through 11. The script is as follows:

```
1 SET SERVEROUTPUT ON
2
3 EXECUTE curs_pkg.open
4 DECLARE
5     v_more BOOLEAN := curs_pkg.next(3);
6 BEGIN
7     IF NOT v_more THEN
8         curs_pkg.close;
9     END IF;
10 END;
```

Below the script, the 'Output' window displays the results of the execution. It shows two successful completions of the PL/SQL procedure, each followed by three rows of data (Id: 100, Id: 101, Id: 102) and another two successful completions followed by three more rows (Id: 103, Id: 104, Id: 105).

28

Recall that the state of a package variable or cursor persists across transactions within a session. However, the state does not persist across different sessions for the same user. The database tables hold data that persists across sessions and users. The call to `curs_pkg.open` opens the cursor, which remains open until the session is terminated, or the cursor is explicitly closed. The anonymous block executes the `next` function in the `Declaration` section, initializing the `BOOLEAN` variable `b_more` to `TRUE`, because there are more than three rows in the `EMPLOYEES` table. The block checks for the end of the result set and closes the cursor, if appropriate. When the block executes, it displays the first three rows:

```
Id :100
Id :101
Id :102
```

If you click the Run Script icon (or press F5) again, the next three rows are displayed:

```
Id :103
Id :104
Id :105
```

To close the cursor, you can issue the following command to close the cursor in the package, or exit the session:

```
EXECUTE curs_pkg.close
```

Quiz



Overloading subprograms in PL/SQL:

- a. Enables you to create two or more subprograms with the same name
- b. Requires that the subprogram's formal parameters differ in number, order, or data type family
- c. Enables you to build flexible ways for invoking subprograms with different data
- d. Provides a way to extend functionality without loss of existing code; that is, adding new parameters to existing subprograms

29

Answer: a, b, c, d

The overloading feature in PL/SQL enables you to develop two or more packaged subprograms with the same name. Overloading is useful when you want a subprogram to accept similar sets of parameters that have different data types. For example, the `TO_CHAR` function has more than one way to be called, enabling you to convert a number or a date to a character string.

PL/SQL allows overloading of package subprogram names and object type methods.

The key rule is that you can use the same name for different subprograms as long as their formal parameters differ in *number, order, or data type* family.

Consider using overloading when:

- Processing rules for two or more subprograms are similar, but the type or number of parameters used varies
- Providing alternative ways for finding different data with varying search criteria. For example, you may want to find employees by their employee ID and also provide a way to find employees by their last name. The logic is intrinsically the same, but the parameters or search criteria differ.
- Extending functionality when you do not want to replace existing code

Summary

In this lesson, you should have learned how to:

- Overload package procedures and functions
- Use forward declarations
- Create an initialization block in a package body
- Manage persistent package data states for the life of a session



30

Overloading is a feature that enables you to define different subprograms with the same name. It is logical to give two subprograms the same name when the processing in both the subprograms is the same but the parameters passed to them vary.

PL/SQL permits a special subprogram declaration called a forward declaration. A forward declaration enables you to define subprograms in logical or alphabetical order, define mutually recursive subprograms, and group subprograms in a package.

A package initialization block is executed only when the package is first invoked within the other user session. You can use this feature to initialize variables only once per session.

You can keep track of the state of a package variable or cursor, which persists throughout the user session, from the time the user first references the variable or cursor to the time the user disconnects.

Using the PL/SQL wrapper, you can obscure the source code stored in the database to protect your intellectual property.

For Instructor Use Only.
This document should not be distributed.

Practice 15 Overview: Working with Packages

This practice covers the following topics:

- Using overloaded subprograms
- Creating a package initialization block
- Using a forward declaration



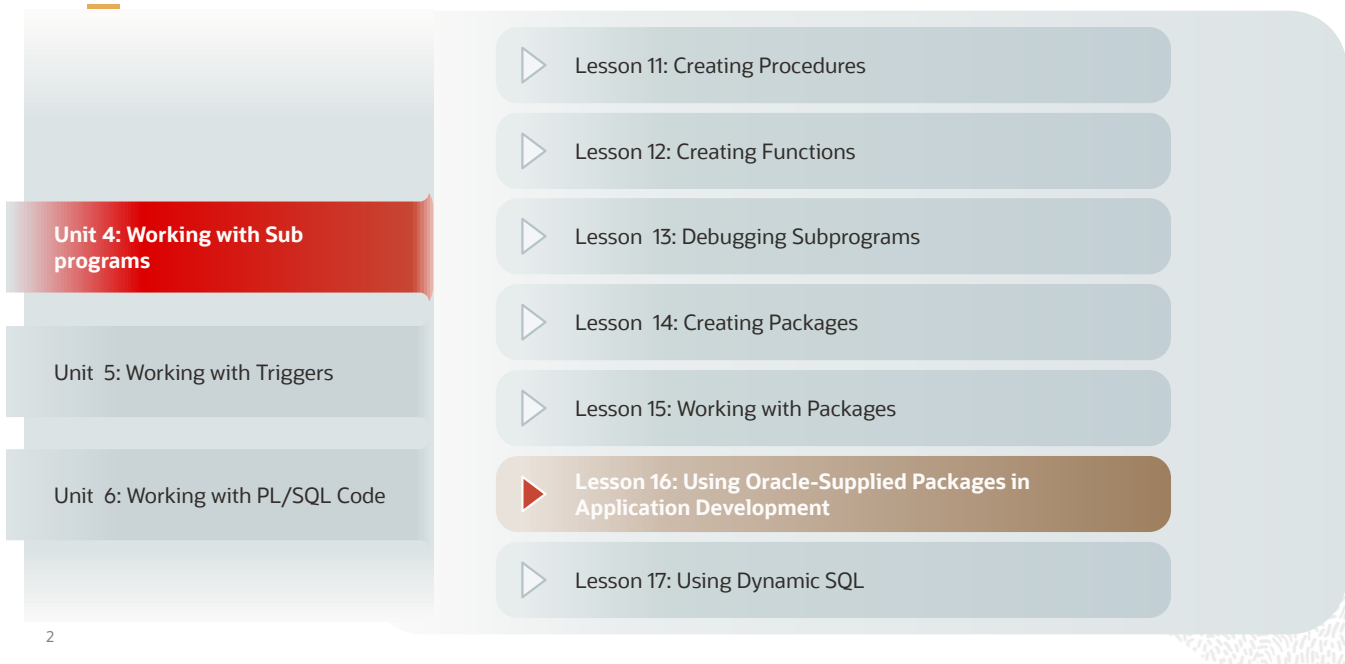
31

In this practice, you modify an existing package to contain overloaded subprograms and you use forward declarations. You also create a package initialization block within a package body to populate a PL/SQL table.

For Instructor Use Only.
This document should not be distributed.

Using Oracle-Supplied Packages in Application Development

Course Road Map



2

In the fourth unit, we have seven lessons – Creating Stored Procedures, Creating Functions, Debugging Subprograms, Creating Packages, Working with Packages, Using Oracle Supplied Packages in Application Development, and Using Dynamic SQL. You will learn to write PL/SQL subprograms, packages, and use them in Application Development.

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Describe how the `DBMS_OUTPUT` package works
- Use `UTL_FILE` to direct output to operating system files
- Describe the main features of `UTL_MAIL`
- Describe the JSON Data Structures



3

In this lesson, you learn how to use some of the Oracle-supplied packages and their capabilities.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Identifying the benefits of using the Oracle-supplied packages and listing some of those packages
- Using the following Oracle-supplied packages:
 - DBMS_OUTPUT
 - UTL_FILE
 - UTL_MAIL



Using Oracle-Supplied Packages

- The Oracle-supplied packages:
 - Are provided with the Oracle server
 - Extend the functionality of the database
 - Provide PL/SQL access to SQL statements
- Most of the Oracle-supplied packages are installed during database creation.



Modifying Oracle-supplied packages can cause internal errors and database security violations. Do not modify supplied packages.

5

Packages are provided with the Oracle server to allow either of the following:

- PL/SQL access to certain SQL features
- The extension of the functionality of the database

The PL/SQL functions defined in the Oracle-supplied packages can be invoked in SQL statements, thus SQL statements can access PL/SQL functions. You can use the functionality provided by these packages when creating your application, or you may simply want to use these packages as ideas when you create your own stored procedures.

Most of the standard packages are created while creating the database. Certain packages are not installed automatically; you have to install them if required by referring to appropriate Oracle documentation.

You can use Oracle-supplied packages while creating your own packages or you can refer to them for ideas in creating your own stored procedures.

Examples of Some Oracle-Supplied Packages

Here is an abbreviated list of some Oracle-supplied packages:

- `DBMS_OUTPUT`
- `UTL_FILE`
- `UTL_MAIL`
- `DBMS_ALERT`
- `DBMS_LOCK`
- `DBMS_SESSION`
- `DBMS_APPLICATION_INFO`
- `HTP`
- `DBMS_SCHEDULER`

6

The list of PL/SQL packages provided with an Oracle database grows with the release of new versions. This lesson covers the first three packages in the slide. For more information, refer to the *Oracle Database PL/SQL Packages and Types Reference*. The following is a brief description about the remaining listed packages in the slide:

- `DBMS_OUTPUT` provides debugging and buffering of text data.
- `UTL_FILE` enables your PL/SQL programs to read and write operating system text files and provides a restricted version of standard operating system stream file I/O.
- `UTL_MAIL` is a utility for managing email which includes commonly used email features, such as attachments, CC, BCC, and return receipt.
- `DBMS_ALERT` supports asynchronous notification of database events.
- `DBMS_LOCK` is used to request, convert, and release locks through Oracle Lock Management services.
- `DBMS_SESSION` provides access to `SQL ALTER SESSION` statements, and other session information, from stored procedures.
- `DBMS_APPLICATION_INFO` lets you register an application name with the database for auditing or performance-tracking purposes.
- `HTP` package has hypertext procedures to generate HTML tags.
- `DBMS_SCHEDULER` provides a collection of scheduling functions that are callable from any PL/SQL program.

Lesson Agenda

- Identifying the benefits of using the Oracle-supplied packages and listing some of those packages
- Using the following Oracle-supplied packages:
 - DBMS_OUTPUT
 - UTL_FILE
 - UTL_MAIL

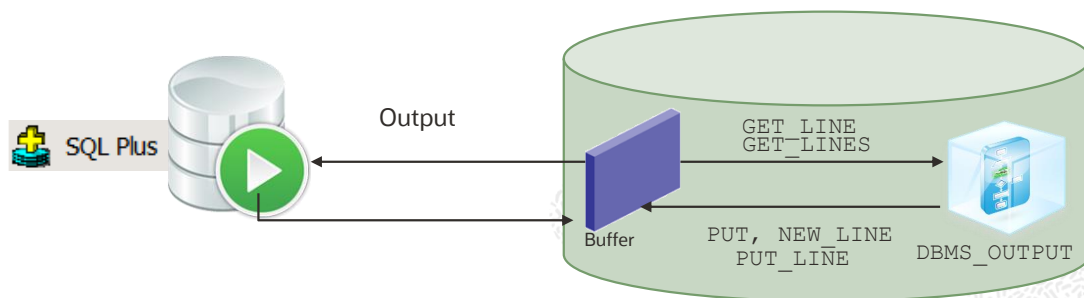


For Instructor Use Only.
This document should not be distributed.

How the DBMS_OUTPUT Package Works

The `DBMS_OUTPUT` package enables you to send messages from stored subprograms and triggers to other stored subprograms or buffer.

- `PUT` and `PUT_LINE` place text in the buffer.
- `GET_LINE` and `GET_LINES` read the buffer.
- Messages are not sent until the sending subprogram or trigger completes.



8

The `DBMS_OUTPUT` package sends textual messages from any PL/SQL block into a buffer in the database. You can output results to the window for debugging purposes. You can trace a code execution path for a function or procedure. You can send messages between subprograms and triggers.

Procedures provided by the package include the following:

- `PUT` appends text from the procedure to the current line of the line output buffer.
- `NEW_LINE` places an end-of-line marker in the output buffer.
- `PUT_LINE` combines the action of `PUT` and `NEW_LINE` (to trim leading spaces).
- `GET_LINE` retrieves the current line from the buffer into a procedure variable.
- `GET_LINES` retrieves an array of lines into a procedure-array variable.
- `ENABLE/DISABLE` enables and disables calls to `DBMS_OUTPUT` procedures.

The buffer size can be set by using:

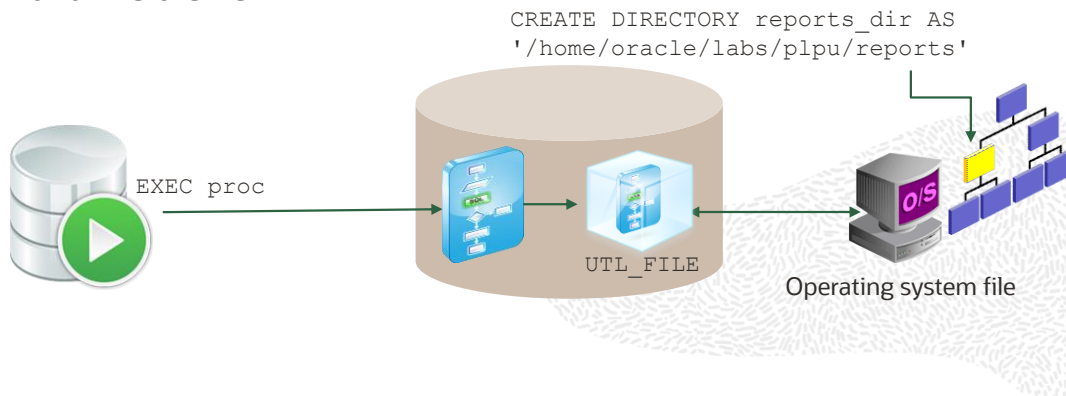
- The `SIZE n` option appended to the `SET SERVEROUTPUT ON` command where `n` is the number of characters. The minimum is 2,000 and the maximum is unlimited. The default is 20,000.
- An integer parameter between 2,000 and 1,000,000 in the `ENABLE` procedure

Note: There is no mechanism to flush output during the execution of a procedure.

Using the UTL_FILE Package

The UTL_FILE package extends PL/SQL programs to read and write operating system text files:

- Provides a restricted version of operating system stream file I/O for text files
- Enables creating files in operating system directories defined by a CREATE DIRECTORY statement



9

With the UTL_FILE package, you can write PL/SQL programs that can read and write operating system text files. The set of files and directories that are accessible through UTL_FILE is defined by the database parameters and other factors.

If certain database user say 'Scott' has read and write access to an operating system directory dir1, then the user can only access the files in that directory on the disk. The database user cannot access the files in its subdirectories or parent directories.

In the past, accessible directories for the UTL_FILE functions were specified in the initialization file using the UTL_FILE_DIR parameter. However this is no longer recommended. Oracle recommends you use directory object feature. This feature provides more granularity and flexibility with respect to security.

You can use the directory objects by using the CREATE DIRECTORY instead of UTL_FILE_DIR parameter. A CREATE DIRECTORY statement associates an alias with an operating system directory. The database directory alias can be granted the READ and WRITE privileges to control the type of access to files in the operating system.

For example:

```
CREATE DIRECTORY my_dir AS '/temp/my_files';  
GRANT READ, WRITE ON DIRECTORY my_dir TO Scott;
```

The database user must have the CREATE ANY DIRECTORY privilege granted to create a directory object in the operating system file system. The CREATE DIRECTORY privilege is granted only to SYS and SYSTEM by default.

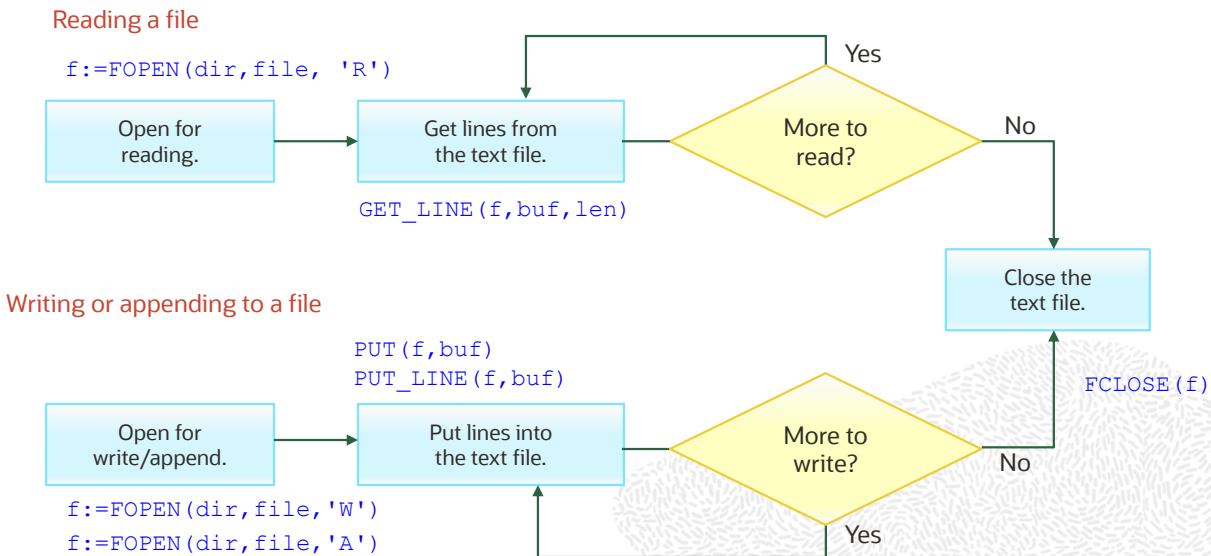
Some of the UTL_FILE Procedures and Functions

Subprogram	Description
ISOPEN function	Determines if a file handle refers to an open file
FOPEN function	Opens a file for input or output
FCLOSE function	Closes all open file handles
FCOPY procedure	Copies a contiguous portion of a file to a newly created file
FGETATTR procedure	Reads and returns the attributes of a disk file
GET_LINE procedure	Reads text from an open file
FREMOVE procedure	Deletes a disk file, if you have sufficient privileges
FRENAME procedure	Renames an existing file to a new name
PUT procedure	Writes a string to a file
PUT_LINE procedure	Writes a line to a file, and so appends an operating system–specific line terminator

10

The table in the slide lists some of the UTL_FILE Package subprograms. For a complete list of the package's subprograms, see *Oracle Database PL/SQL Packages and Types Reference*.

File Processing Using the UTL_FILE Package: Overview



11

You can use the procedures and functions in the `UTL_FILE` package to open files with the `FOPEN` function. You can then either read from or write or append to the file until processing is done. After you finish processing the file, close the file by using the `FCLOSE` procedure.

The slide demonstrates the process of reading from or writing to a file using the subprograms of `UTL_FILE` package.

To read from a file:

1. You open the file using the `FOPEN` procedure with the required parameters.
2. After the file is open, you read data from it using `GET_LINE`, `GET_LINE_NCHAR`, or `GET_RAW` procedures. You can check whether a file is open or not by using the `ISOPEN` procedure.
3. You invoke appropriate subprograms until you complete reading data from the file and close the file.

The process is similar for writing to a file using the `UTL_FILE` package. To write to a file:

1. You open the file using the `FOPEN` procedure, the difference being the mode in which you open the file. While reading the file you open the file in the read mode `'R'`, whereas while writing to the file you open the file in write `'W'` or append `'A'` mode.
2. Once the file is open, you can write to the file using `PUT`, `PUT_LINE`, `PUT_LINE_NCHAR`, `PUT_NCHAR`, `PUTF`, `PUTF_NCHAR`, `PUT_RAW` subprograms of the `UTL_FILE` package.
3. After you complete writing to the file, you close the file.

Using the Available Declared Exceptions in the UTL_FILE Package

Exception Name	Description
INVALID_PATH	File location invalid
INVALID_MODE	The open_mode parameter in FOPEN invalid
INVALID_FILEHANDLE	File handle invalid
INVALID_OPERATION	File could not be opened or operated on as requested
READ_ERROR	Destination buffer too small, or operating system error occurred during the read operation
WRITE_ERROR	Operating system error occurred during the write operation
INTERNAL_ERROR	Unspecified PL/SQL error

12

The UTL_FILE package declares 15 exceptions that indicate an error condition in the operating system file processing. You may have to handle one of these exceptions when using UTL_FILE subprograms.

A subset of the exceptions are displayed in the slide. For additional information about the remaining exceptions, refer to *Oracle Database PL/SQL Packages and Types Reference*.

Note: These exceptions must always be prefixed with the package name. UTL_FILE procedures can also raise predefined PL/SQL exceptions such as NO_DATA_FOUND or VALUE_ERROR.

The NO_DATA_FOUND exception is raised when reading past the end of a file by using UTL_FILE.GET_LINE or UTL_FILE.GET_LINES.

FOPEN and IS_OPEN Functions: Example

- This FOPEN function opens a file for input or output.

```
FUNCTION FOPEN (p_location IN VARCHAR2,  
               p_filename IN VARCHAR2,  
               p_open_mode IN VARCHAR2)  
RETURN UTL_FILE.FILE_TYPE;
```

- The IS_OPEN function determines whether a file handle refers to an open file.

```
FUNCTION IS_OPEN (p_file IN FILE_TYPE)  
RETURN BOOLEAN;
```

13

The parameters include the following:

- `p_location` parameter: Specifies the name of a directory alias defined by a `CREATE DIRECTORY` statement, or an operating system–specific path specified by using the `utl_file_dir` database parameter
- `p_filename` parameter: Specifies the name of the file, including the extension, without any path information
- `open_mode` string: Specifies how the file is to be opened. Values are:
 - 'R' for reading text (use `GET_LINE`)
 - 'W' for writing text (`PUT`, `PUT_LINE`, `NEW_LINE`, `PUTF`, `FFLUSH`)
 - 'A' for appending text (`PUT`, `PUT_LINE`, `NEW_LINE`, `PUTF`, `FFLUSH`)

The return value from `FOPEN` is a file handle whose type is `UTL_FILE.FILE_TYPE`. The handle must be used on subsequent calls to routines that operate on the opened file.

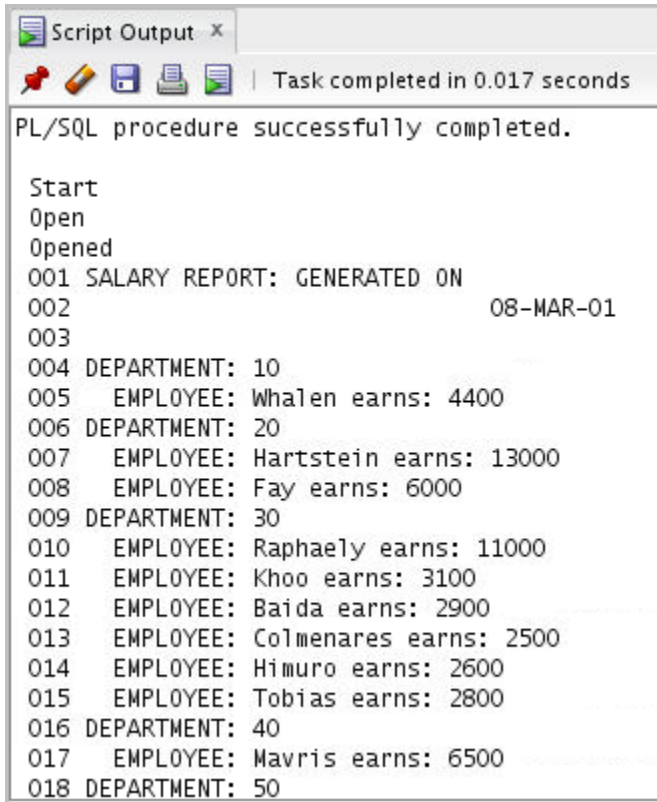
The `IS_OPEN` function parameter is the file handle. The `IS_OPEN` function tests a file handle to see whether it identifies an opened file. It returns a Boolean value of `TRUE` if the file has been opened; otherwise it returns a value of `FALSE` indicating that the file has not been opened. The example in the slide shows how to combine the use of the two subprograms. For the full syntax, refer to *Oracle Database PL/SQL Packages and Types Reference*.

Ensure that the external file and the database are on the same PC.

```
CREATE OR REPLACE PROCEDURE read_file(p_dir VARCHAR2, p_filename
    VARCHAR2) IS
    f_file UTL_FILE.FILE_TYPE;
    buffer VARCHAR2(200);
    lines PLS_INTEGER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE(' Start ');
    IF NOT UTL_FILE.IS_OPEN(f_file) THEN
        DBMS_OUTPUT.PUT_LINE(' Open ');
        f_file := UTL_FILE.FOPEN (p_dir, p_filename, 'R');
        DBMS_OUTPUT.PUT_LINE(' Opened ');
        BEGIN
            LOOP
                UTL_FILE.GET_LINE(f_file, buffer);
                lines := lines + 1;
                DBMS_OUTPUT.PUT_LINE(TO_CHAR(lines, '099')||' '||buffer);
            END LOOP;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                DBMS_OUTPUT.PUT_LINE(' ** End of File **');
        END;
        DBMS_OUTPUT.PUT_LINE(lines||' lines read from file');
        UTL_FILE.FCLOSE(f_file);
    END IF;
END read_file;
/
SHOW ERRORS
SET SERVEROUTPUT ON
EXECUTE read_file('REPORTS_DIR', 'instructor.txt')
```

For Instructor Use Only.
This document should not be distributed.

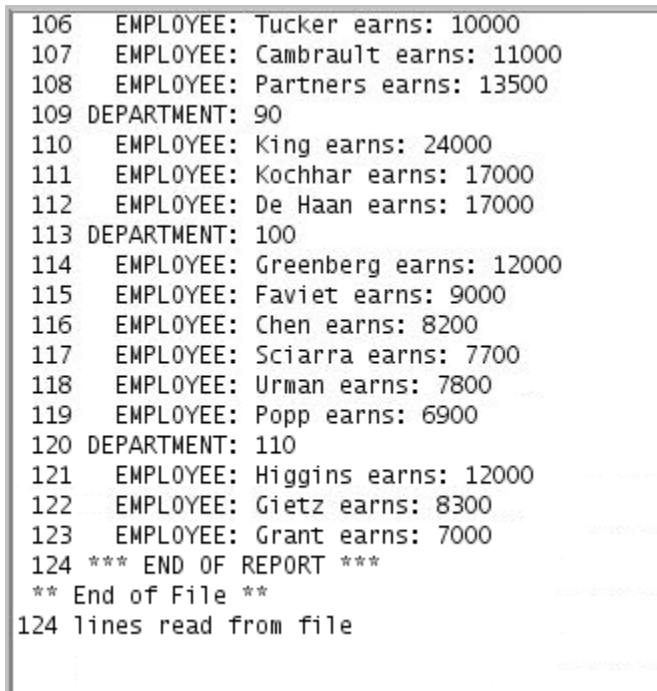
The partial output of the above code is as follows:



The screenshot shows a 'Script Output' window with a title bar 'Script Output x'. Below the title bar is a toolbar with icons for a red pin, a pencil, a save icon, a print icon, and a document icon. To the right of the toolbar, it says 'Task completed in 0.017 seconds'. The main text area contains the following output:

```
PL/SQL procedure successfully completed.

Start
Open
Opened
001 SALARY REPORT: GENERATED ON
002                                08-MAR-01
003
004 DEPARTMENT: 10
005   EMPLOYEE: Whalen earns: 4400
006 DEPARTMENT: 20
007   EMPLOYEE: Hartstein earns: 13000
008   EMPLOYEE: Fay earns: 6000
009 DEPARTMENT: 30
010   EMPLOYEE: Raphaely earns: 11000
011   EMPLOYEE: Khoo earns: 3100
012   EMPLOYEE: Baida earns: 2900
013   EMPLOYEE: Colmenares earns: 2500
014   EMPLOYEE: Himuro earns: 2600
015   EMPLOYEE: Tobias earns: 2800
016 DEPARTMENT: 40
017   EMPLOYEE: Mavris earns: 6500
018 DEPARTMENT: 50
```



This block shows the continuation of the PL/SQL output from the previous block. It contains the following text:

```
106   EMPLOYEE: Tucker earns: 10000
107   EMPLOYEE: Cambrault earns: 11000
108   EMPLOYEE: Partners earns: 13500
109 DEPARTMENT: 90
110   EMPLOYEE: King earns: 24000
111   EMPLOYEE: Kochhar earns: 17000
112   EMPLOYEE: De Haan earns: 17000
113 DEPARTMENT: 100
114   EMPLOYEE: Greenberg earns: 12000
115   EMPLOYEE: Faviet earns: 9000
116   EMPLOYEE: Chen earns: 8200
117   EMPLOYEE: Sciarra earns: 7700
118   EMPLOYEE: Urman earns: 7800
119   EMPLOYEE: Popp earns: 6900
120 DEPARTMENT: 110
121   EMPLOYEE: Higgins earns: 12000
122   EMPLOYEE: Gietz earns: 8300
123   EMPLOYEE: Grant earns: 7000
124 *** END OF REPORT ***
** End of File **
124 lines read from file
```

Using UTL_FILE: Example

```
CREATE OR REPLACE PROCEDURE sal_status(  
  p_dir IN VARCHAR2, p_filename IN VARCHAR2) IS  
  f_file UTL_FILE.FILE_TYPE;  
  CURSOR cur_emp IS  
    SELECT last_name, salary, department_id  
    FROM employees ORDER BY department_id;  
  v_newdeptno employees.department_id%TYPE;  
  v_olddeptno employees.department_id%TYPE := 0;  
BEGIN  
  f_file:= UTL_FILE.FOPEN (p_dir, p_filename, 'W');  
  UTL_FILE.PUT_LINE(f_file,  
    'REPORT: GENERATED ON ' || SYSDATE);  
  UTL_FILE.NEW_LINE (f_file);  
  . . .
```

16

In the slide example, the `sal_status` procedure creates a report of employees for each department, along with their salaries. The data is written to a text file by using the `UTL_FILE` package.

In the code example, the `file` variable is declared as `UTL_FILE.FILE_TYPE`, is a record type which is private to the package `UTL_FILE`. This record has three fields, `id`, `data_type` and `byte_mode`. A variable of `FILE_TYPE` serves a file handle; you can perform various operations on the file with the help of this handle.

The `sal_status` procedure accepts two parameters:

- The `p_dir` parameter for the name of the directory in which to write the text file
- The `p_filename` parameter to specify the name of the file

For example, to call the procedure, use the following after ensuring that the external file and the database are on the same PC:

```
EXECUTE sal_status('REPORTS_DIR', 'salreport2.txt')
```

Note: The directory location used (`REPORTS_DIR`) must be in uppercase characters if it is a directory alias created by a `CREATE DIRECTORY` statement. When reading a file in a loop, the loop should exit when it detects the `NO_DATA_FOUND` exception. The `UTL_FILE` output is sent synchronously. A `DBMS_OUTPUT` procedure does not produce an output until the procedure is completed.

Using UTL_FILE: Example

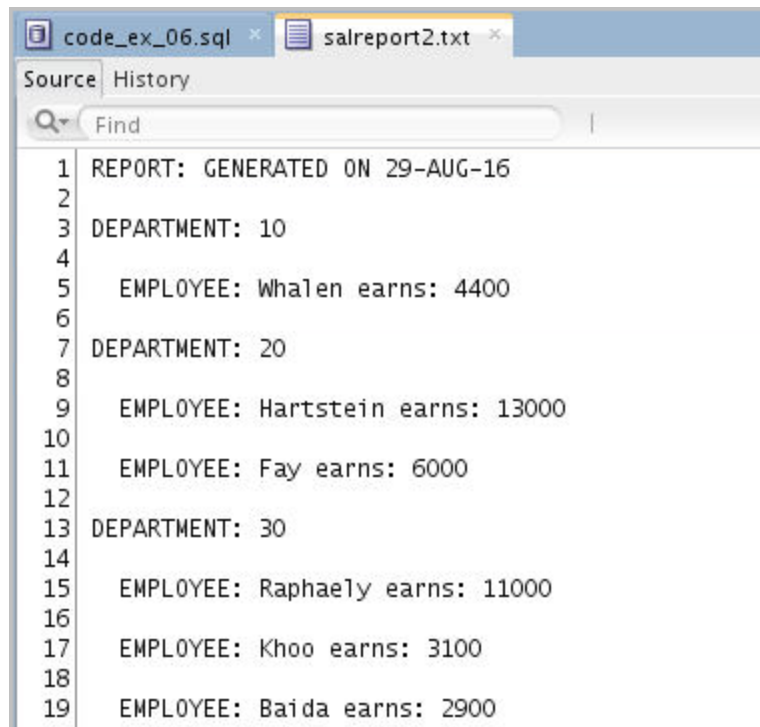
```
...
FOR emp_rec IN cur_emp LOOP
  IF emp_rec.department_id <> v_olddeptno THEN
    UTL_FILE.PUT_LINE (f_file,
      'DEPARTMENT: ' || emp_rec.department_id);
    UTL_FILE.NEW_LINE (f_file);
  END IF;
  UTL_FILE.PUT_LINE (f_file,
    '  EMPLOYEE: ' || emp_rec.last_name ||
    '  earns: ' || emp_rec.salary);
  v_olddeptno := emp_rec.department_id;
  UTL_FILE.NEW_LINE (f_file);
END LOOP;
UTL_FILE.PUT_LINE(f_file, '*** END OF REPORT ***');
UTL_FILE.FCLOSE (f_file);
EXCEPTION
  WHEN UTL_FILE.INVALID_FILEHANDLE THEN
    RAISE_APPLICATION_ERROR(-20001, 'Invalid File.');
```

17

Execute the sal_status procedure:

```
EXECUTE sal_status('REPORTS_DIR', 'salreport2.txt')
```

The following is a sample of the salreport2.txt output file:



```
code_ex_06.sql * salreport2.txt *
Source History
Find
1 REPORT: GENERATED ON 29-AUG-16
2
3 DEPARTMENT: 10
4
5   EMPLOYEE: Whalen earns: 4400
6
7 DEPARTMENT: 20
8
9   EMPLOYEE: Hartstein earns: 13000
10
11  EMPLOYEE: Fay earns: 6000
12
13 DEPARTMENT: 30
14
15  EMPLOYEE: Raphaely earns: 11000
16
17  EMPLOYEE: Khoo earns: 3100
18
19  EMPLOYEE: Baida earns: 2900
20
```

What Is the UTL_MAIL Package?

- Is a utility for managing email
- Requires the setting of the SMTP_OUT_SERVER database initialization parameter
- Provides the following procedures:
 - SEND for messages without attachments
 - SEND_ATTACH_RAW for messages with binary attachments
 - SEND_ATTACH_VARCHAR2 for messages with text attachments



18

The UTL_MAIL package is a utility for managing email that includes commonly used email features such as attachments, CC, BCC, and return receipt.

The UTL_MAIL package is not installed by default because of the SMTP_OUT_SERVER configuration requirement and the security exposure this involves. When installing UTL_MAIL, you should take steps to prevent the port defined by SMTP_OUT_SERVER being swamped by data transmissions. To install UTL_MAIL, log in as the sys user in SQL*Plus and execute the following scripts:

```
@$ORACLE_HOME/rdbms/admin/utlmail.sql
```

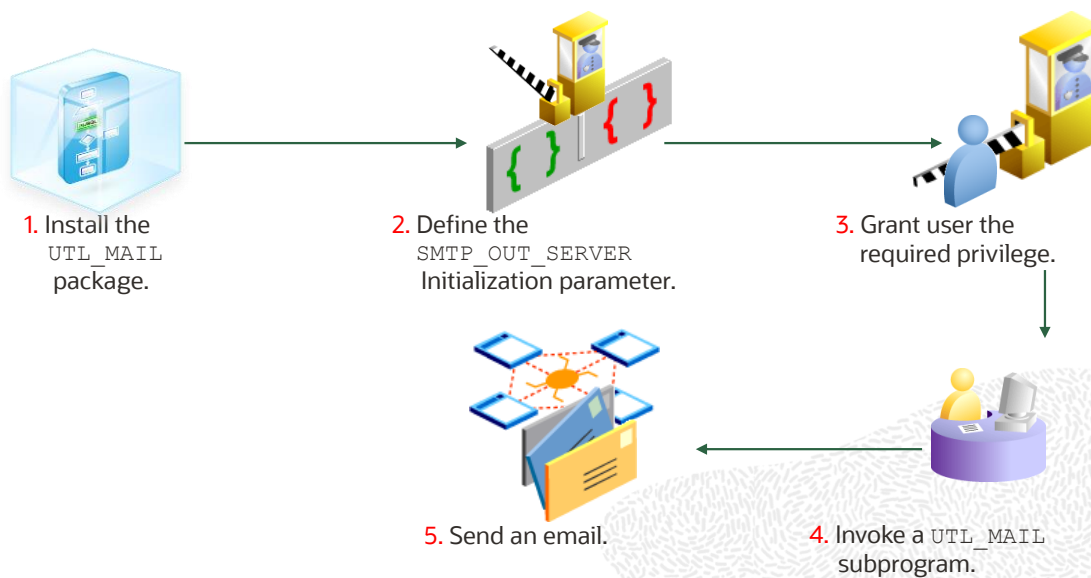
```
@$ORACLE_HOME/rdbms/admin/prvtmail.plb
```

You should define the SMTP_OUT_SERVER parameter in the init.ora initialization file. However, if SMTP_OUT_SERVER is not defined, this invokes a default of DB_DOMAIN, which is guaranteed to be defined to perform appropriately.

The SMTP_OUT_SERVER parameter specifies the SMTP host and port to which UTL_MAIL delivers outbound email. Multiple servers can be specified, separated by commas. If the first server in the list is unavailable, then UTL_MAIL tries the second server, and so on.

For Instructor Use Only.
This document should not be distributed.

Setting Up and Using the UTL_MAIL: Overview



19

In Oracle Database 11g, the UTL_MAIL package is now an invoker's rights package and the invoking user will need the CONNECT privilege granted in the access control list assigned to the remote network host to which he wants to connect. The Security Administrator performs this task.

You also require an EXECUTE privilege on UTL_MAIL to send a mail from the PL/SQL program.

Note

- For information about how a user with SYSDBA capabilities grants a user the required fine-grained privileges required for using this package, refer to the "Managing Fine-Grained Access to External Network Services" topic in *Oracle Database Security Guide* and the *Oracle Database Advanced PL/SQL* instructor-led training course.
- Because of firewall restrictions, the UTL_MAIL examples in this lesson cannot be demonstrated; therefore, no labs were designed to use UTL_MAIL.

Summary of UTL_MAIL Subprograms

Subprogram	Description
SEND procedure	Packages an email message, locates SMTP information, and delivers the message to the SMTP server for forwarding to the recipients
SEND_ATTACH_RAW Procedure	Represents the SEND procedure overloaded for RAW attachments
SEND_ATTACH_VARCHAR2 Procedure	Represents the SEND procedure overloaded for VARCHAR2 attachments

Installing and Using UTL_MAIL

- As SYSDBA, using SQL Developer or SQL*Plus:
 - Install the UTL_MAIL package

```
@?/rdbms/admin/utlmail.sql
@?/rdbms/admin/prvtmail.plb
```

- Set the SMTP_OUT_SERVER

```
ALTER SYSTEM SET SMTP_OUT_SERVER='smtp.server.com' SCOPE=SPFILE
```

- As a developer, invoke a UTL_MAIL procedure:

```
BEGIN
  UTL_MAIL.SEND('otn@oracle.com','user@oracle.com',
    message => 'For latest downloads visit OTN',
    subject => 'OTN Newsletter');
END;
```

21

The slide shows how to configure the SMTP_OUT_SERVER parameter to the name of the SMTP host in your network, and how to install the UTL_MAIL package that is not installed by default. Changing the SMTP_OUT_SERVER parameter requires restarting the database instance. These tasks are performed by a user with SYSDBA capabilities.

The last example in the slide shows the simplest way to send a text message by using the UTL_MAIL.SEND procedure with at least a subject and a message. The first two required parameters are the following :

- The sender email address (in this case, otn@oracle.com)
- The recipients email address (for example, user@oracle.com). The value can be a comma-separated list of addresses.

The UTL_MAIL.SEND procedure provides several other parameters, such as cc, bcc, and priority with default values, if not specified. In the example, the message parameter specifies the text for the email, and the subject parameter contains the text for the subject line. To send an HTML message with HTML tags, add the mime_type parameter (for example, mime_type=>'text/html').

Note: For details about all the UTL_MAIL procedure parameters, refer to *Oracle Database PL/SQL Packages and Types Reference*.

The SEND Procedure Syntax

Packages an email message into the appropriate format, locates SMTP information, and delivers the message to the SMTP server for forwarding to the recipients

```
UTL_MAIL.SEND (  
  sender      IN      VARCHAR2 CHARACTER SET ANY_CS,  
  recipients  IN      VARCHAR2 CHARACTER SET ANY_CS,  
  cc          IN      VARCHAR2 CHARACTER SET ANY_CS  
                  DEFAULT NULL,  
  bcc         IN      VARCHAR2 CHARACTER SET ANY_CS  
                  DEFAULT NULL,  
  subject     IN      VARCHAR2 CHARACTER SET ANY_CS  
                  DEFAULT NULL,  
  message     IN      VARCHAR2 CHARACTER SET ANY_CS,  
  mime_type   IN      VARCHAR2  
                  DEFAULT 'text/plain; charset=us-ascii',  
  priority    IN      PLS_INTEGER DEFAULT NULL  
  replyto     IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL);
```

22

This procedure packages an email message into the appropriate format, locates SMTP information, and delivers the message to the SMTP server for forwarding to the recipients. It hides the SMTP API and exposes a one-line email facility for ease of use.

The SEND Procedure Parameters

- **sender:** The email address of the sender
- **recipients:** The email addresses of the recipients, separated by commas
- **cc:** The email addresses of the CC recipients, separated by commas. The default is `NULL`.
- **bcc:** The email addresses of the BCC recipients, separated by commas. The default is `NULL`.
- **subject:** A string to be included as email subject string. The default is `NULL`.
- **message:** A text message body
- **mime_type:** The mime type of the message, default is `'text/plain; charset=us-ascii'`
- **priority:** The message priority. The default is `NULL`.
- **reply_to:** Defines to whom the reply email is to be sent.

The SEND_ATTACH_RAW Procedure

This procedure is the SEND procedure overloaded for RAW attachments.

```
UTL_MAIL.SEND_ATTACH_RAW (  
  sender      IN      VARCHAR2 CHARACTER SET ANY_CS,  
  recipients  IN      VARCHAR2 CHARACTER SET ANY_CS,  
  cc          IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  bcc         IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  subject     IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  message     IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  mime_type   IN      VARCHAR2 DEFAULT CHARACTER SET ANY_CS  
                DEFAULT 'text/plain; charset=us-ascii',  
  priority    IN      PLS_INTEGER DEFAULT 3,  
  attachment  IN      RAW,  
  att_inline  IN      BOOLEAN DEFAULT TRUE,  
  att_mime_type IN     VARCHAR2 CHARACTER SET ANY_CS  
                DEFAULT 'text/plain; charset=us-ascii',  
  att_filename IN     VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  replyto     IN      VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL);
```

23

- **sender:** The email address of the sender
- **recipients:** The email addresses of the recipients, separated by commas
- **cc:** The email addresses of the CC recipients, separated by commas. The default is NULL.
- **bcc:** The email addresses of the BCC recipients, separated by commas. The default is NULL.
- **subject:** A string to be included as email subject string. The default is NULL.
- **message:** A text message body
- **mime_type:** The mime type of the message, default is 'text/plain; charset=us-ascii'
- **priority:** The message priority. The default is NULL.
- **attachment:** A RAW attachment
- **att_inline:** Specifies whether the attachment is viewable inline with the message body. The default is TRUE.
- **reply_to:** Defines to whom the reply email is to be sent.

Sending Email with a Binary Attachment: Example

```
CREATE OR REPLACE PROCEDURE send_mail_logo IS
BEGIN
    UTL_MAIL.SEND_ATTACH_RAW(
        sender => 'me@oracle.com',
        recipients => 'you@somewhere.net',
        message =>
            '<HTML><BODY>See attachment</BODY></HTML>',
        subject => 'Oracle Logo',
        mime_type => 'text/html',
        attachment => get_image('oracle.gif'),
        att_inline => true,
        att_mime_type => 'image/gif',
        att_filename => 'oralogo.gif');
END;
```

24

The slide shows a procedure calling the `UTL_MAIL.SEND_ATTACH_RAW` procedure to send a textual or an HTML message with a binary attachment. In addition to the `sender`, `recipients`, `message`, `subject`, and `mime_type` parameters that provide values for the main part of the email message, the `SEND_ATTACH_RAW` procedure has the following highlighted parameters:

- The `attachment` parameter (required) accepts a `RAW` data type, with a maximum size of 32,767 binary characters.
- The `att_inline` parameter (optional) is Boolean (default `TRUE`) to indicate that the attachment is viewable with the message body.
- The `att_mime_type` parameter (optional) specifies the format of the attachment. If not provided, it is set to `application/octet`.
- The `att_filename` parameter (optional) assigns any file name to the attachment. It is `NULL` by default, in which case, the name is assigned a default name.

The `get_image` function in the example uses a `BFILE` to read the image data. Using a `BFILE` requires creating a logical directory name in the database by using the `CREATE DIRECTORY` statement. The code for `get_image` is shown on the following page.

The `get_image` function uses the `DBMS_LOB` package to read a binary file from the operating system:

```
CREATE OR REPLACE FUNCTION get_image(  
    filename VARCHAR2, dir VARCHAR2 := 'TEMP')  
RETURN RAW IS  
    image RAW(32767);  
    file BFILE := BFILENAME(dir, filename);  
BEGIN  
    DBMS_LOB.FILEOPEN(file, DBMS_LOB.FILE_READONLY);  
    image := DBMS_LOB.SUBSTR(file);  
    DBMS_LOB.CLOSE(file);  
    RETURN image;  
END;  
/
```

To create the directory called `TEMP`, execute the following statement in SQL Developer or SQL*Plus:

```
CREATE DIRECTORY temp AS 'd:\temp';
```

Note

- You need the `CREATE ANY DIRECTORY` system privilege to execute this statement.
- Because of firewall restrictions at the Oracle Education Center, the examples on this page and the previous page are not available for demonstration.

The SEND_ATTACH_VARCHAR2 Procedure

This procedure is the SEND procedure overloaded for VARCHAR2 attachments.

```
UTL_MAIL.SEND_ATTACH_VARCHAR2 (  
  sender          IN    VARCHAR2 CHARACTER SET ANY_CS,  
  recipients      IN    VARCHAR2 CHARACTER SET ANY_CS,  
  cc              IN    VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  bcc             IN    VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  subject         IN    VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  message         IN    VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  mime_type       IN    VARCHAR2 CHARACTER SET ANY_CS  
                  DEFAULT 'text/plain; charset=us-ascii',  
  priority        IN    PLS_INTEGER DEFAULT 3,  
  attachment      IN    VARCHAR2 CHARACTER SET ANY_CS,  
  att_inline      IN    BOOLEAN DEFAULT TRUE,  
  att_mime_type   IN    VARCHAR2 CHARACTER SET ANY_CS  
                  DEFAULT 'text/plain; charset=us-ascii',  
  att_filename    IN    VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL,  
  replyto        IN    VARCHAR2 CHARACTER SET ANY_CS DEFAULT NULL);
```

26

- **sender:** The email address of the sender
- **recipients:** The email addresses of the recipients, separated by commas
- **cc:** The email addresses of the CC recipients, separated by commas. The default is NULL.
- **bcc:** The email addresses of the BCC recipients, separated by commas. The default is NULL.
- **subject:** A string to be included as email subject string. The default is NULL.
- **Message:** A text message body
- **mime_type:** The mime type of the message, default is 'text/plain; charset=us-ascii'
- **priority:** The message priority. The default is NULL.
- **attachment:** A text attachment
- **att_inline:** Specifies whether the attachment is inline. The default is TRUE.
- **att_mime_type:** The mime type of the attachment, default is 'text/plain; charset=us-ascii'
- **att_filename:** The string specifying a file name containing the attachment. The default is NULL.

Sending Email with a Text Attachment: Example

```
CREATE OR REPLACE PROCEDURE send_mail_file IS
BEGIN
    UTL_MAIL.SEND_ATTACH_VARCHAR2(
        sender => 'me@oracle.com',
        recipients => 'you@somewhere.net',
        message =>
            '<HTML><BODY>See attachment</BODY></HTML>',
        subject => 'Oracle Notes',
        mime_type => 'text/html',
        attachment => get_file('notes.txt'),
        att_inline => false,
        att_mime_type => 'text/plain',
        att_filename => 'notes.txt');
END;
/
```

27

The slide shows a procedure that calls the `UTL_MAIL.SEND_ATTACH_VARCHAR2` procedure to send a textual or an HTML message with a text attachment. In addition to the `sender`, `recipients`, `message`, `subject`, and `mime_type` parameters that provide values for the main part of the e-mail message, the `SEND_ATTACH_VARCHAR2` procedure has the following parameters highlighted:

- The `attachment` parameter (required) accepts a `VARCHAR2` data type with a maximum size of 32,767 binary characters.
- The `att_inline` parameter (optional) is a Boolean (default `TRUE`) to indicate that the attachment is viewable with the message body.
- The `att_mime_type` parameter (optional) specifies the format of the attachment. If not provided, it is set to `application/octet`.
- The `att_filename` parameter (optional) assigns any file name to the attachment. It is `NULL` by default, in which case, the name is assigned a default name.

The `get_file` function in the example uses a `BFILE` to read a text file from the operating system directories for the value of the `attachment` parameter, which could simply be populated from a `VARCHAR2` variable. The code for `get_file` is shown on the following page.

What is JSON?

- **JSON** stands for JavaScript Object Notation
- **JSON** is a lightweight format for storing and transporting data
- **JSON** is often used when data is sent from a server to a web page
- **JSON** is easy to understand as it uses name-value pairs

28

Oracle Database supports JSON natively with relational database features, including transactions, indexing, declarative querying, and views. Unlike relational data, JSON data can be stored in the database, indexed, and queried without any need for a schema that defines the data.

0
For Instructor Use Only.
This document should not be distributed.

Using JSON

- Create a **JSON** column in a table
- Add an **IS JSON** check constraint
- Insert **JSON** data into the column
- Query the **JSON** data

29

In general, you will perform the following tasks when working with JSON data in Oracle Database: (1) create a JSON column with an **is json** check constraint, (2) insert JSON data into the column, and (3) query the JSON data.

PL/SQL and JSON

Oracle has several predefined PL/SQL Data Structures

- JSON_ELEMENT_T Object Type
- JSON_OBJECT_T Object Type
- JSON_ARRAY_T Object Type
- JSON_SCALAR_T Object Type
- JSON_KEY_LIST Type

30

Oracle Database supports JSON natively with relational database features, including transactions, indexing, declarative querying, and views. Unlike relational data, JSON data can be stored in the database, indexed, and queried without any need for a schema that defines the data.

For Instructor Use Only.
This document should not be distributed.

PL/SQL and JSON

Use PL/SQL With JSON Data:

You can manipulate JSON data within PL/SQL code using SQL code or PL/SQL object types for JSON. (You cannot use an empty JSON field name in any SQL code that you use in PL/SQL.)

Built-in PL/SQL functions:

`json_value`, `json_query`, `json_object`, `json_array`, `json_exists`, `is json`, `is not json`, and `json_equal`.

PL/SQL Boolean Functions:

`json_exists`, `is json`, `is not json`, and `json_equal`
`json_value` can return a BOOLEAN value.

31

There are also PL/SQL object types for JSON, which you can use for fine-grained construction and manipulation of In-Memory JSON data. You can introspect it, modify it, and serialize it back to textual JSON data. For more information, see the Oracle Database PL/SQL and Packages Types Reference and the Oracle JSON Developer's Guide

Quiz



The Oracle-supplied `UTL_FILE` package is used to access text files in the operating system of the database server.

The database provides functionality through directory objects to allow access to specific operating system directories.

- a. True
- b. False

32

Answer: a

The Oracle-supplied `UTL_FILE` package is used to access text files in the operating system of the database server. The database provides read and write access to specific operating system directories by using:

- A `CREATE DIRECTORY` statement that associates an alias with an operating system directory. The database directory alias can be granted the `READ` and `WRITE` privileges to control the type of access to files in the operating system.
- The paths specified in the `utl_file_dir` database initialization parameter

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned:

- How the `DBMS_OUTPUT` package works
- How to use `UTL_FILE` to direct output to operating system files
- About the main features of `UTL_MAIL`



33

This lesson covers a small subset of packages provided with the Oracle database. You have extensively used `DBMS_OUTPUT` for debugging purposes and displaying procedurally generated information on the screen in `SQL*Plus`.

In this lesson, you should have learned how to use the power features provided by the database to create text files in the operating system by using `UTL_FILE`. You also learned how to send email with or without binary or text attachments by using the `UTL_MAIL` package.

Note: For more information about all PL/SQL packages and types, refer to *PL/SQL Packages and Types Reference*.

Practice 16 Overview: Using Oracle-Supplied Packages in Application Development

This practice covers how to use `UTL_FILE` to generate a text report.

34

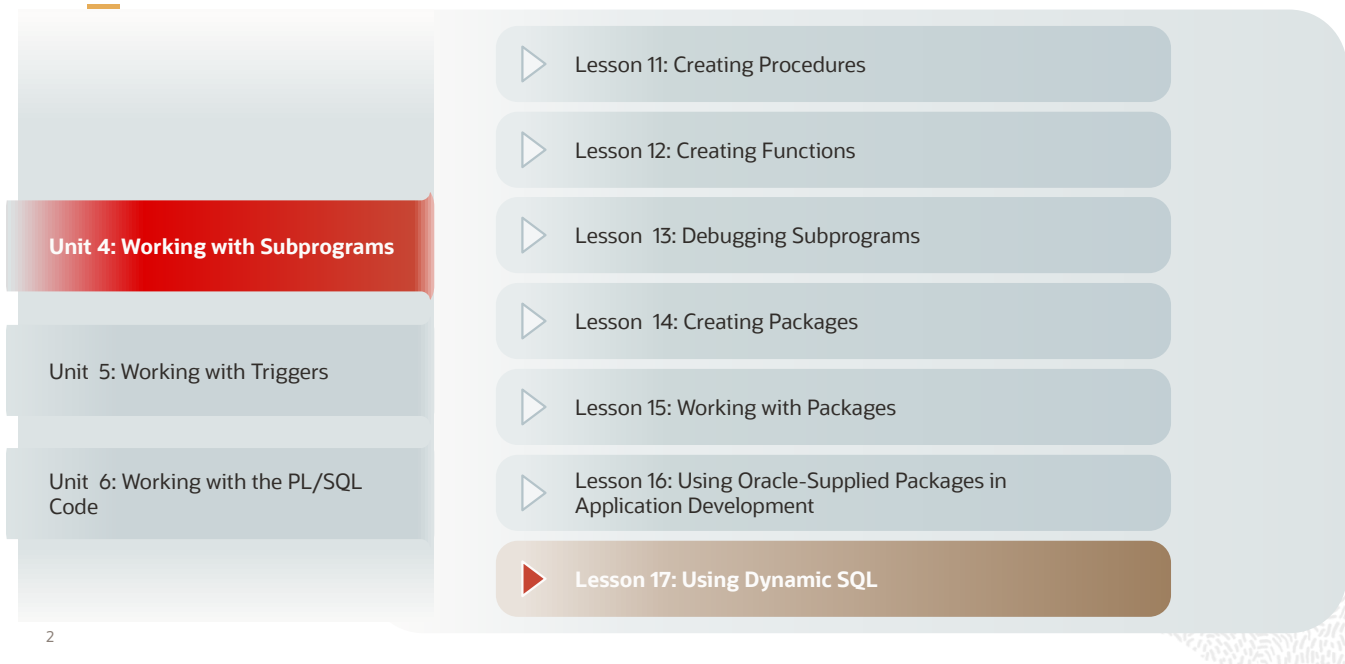
In this practice, you use `UTL_FILE` to generate a text file report of employees in each department.



0
For Instructor Use Only.
This document should not be distributed.

Using Dynamic SQL

Course Road Map



2

In the fourth unit, we have seven lessons – Creating Stored Procedures, Creating Functions, Debugging Subprograms, Creating Packages, Working with Packages, Using Oracle Supplied Packages in Application Development, and Using Dynamic SQL. You will learn to write PL/SQL subprograms, packages, and use them in Application Development.

Objectives

After completing this lesson, you should be able to:

- Describe the execution flow of SQL statements
- Build and execute SQL statements dynamically by using Native Dynamic SQL (NDS)
- Identify situations when you must use the DBMS_SQL package instead of NDS to build and execute SQL statements dynamically



3

In this lesson, you learn to construct and execute SQL statements dynamically, that is, at run time by using NDS statements in PL/SQL.

For Instructor Use Only.
This document should not be distributed.

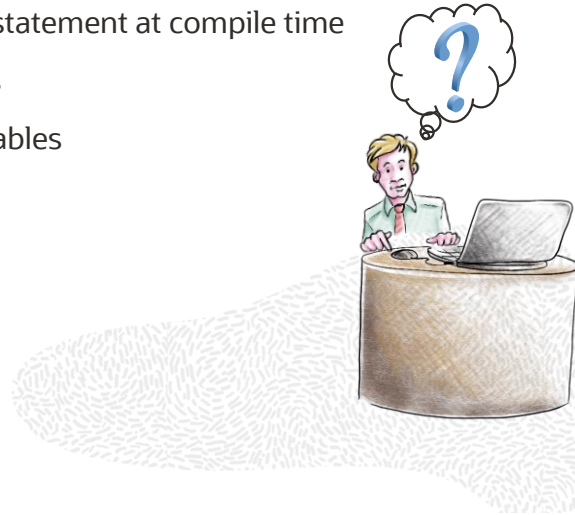
Lesson Agenda

- Dynamic SQL and its features
- Using NDS
- Using the DBMS_SQL package

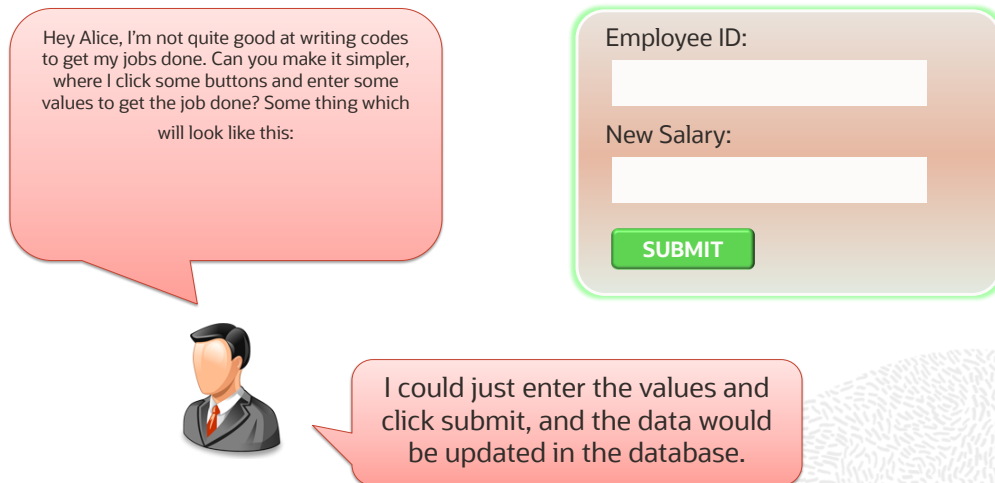


What is Dynamic SQL?

- Dynamic SQL enables you to generate and run SQL statements at run time.
- You may write a dynamic SQL statement when you :
 - Don't know the full text of the SQL statement at compile time
 - Don't know the number of variables
 - Don't know data types of input variables



When do you use Dynamic SQL?



6

Consider the requirement of the human resource (HR) manager in this scenario. The HR manager wants a simple user interface in which he/she could enter the values and the values would be updated in the database. As a PL/SQL developer, the user interface is not your focus here. You can create such a user interface by using frontend technologies like HTML.

As a PL/SQL developer, whose job is to reflect the changes onto the database, you have a challenge of handling unknown data. The data is known only at the run time, but not at compile time. When the HR manager enters the values from the front end, the data has to reach the PL/SQL block that updates the database. The values of the `employee_id` and `new salary` are passed onto the PL/SQL block that in turn runs the update.

You can use dynamic SQL in this case, i.e. where the developer has to handle unknown data. Apart from unknown data, you can use dynamic SQL when you don't know about the data types of the variables and so on. In the following slides, we will look at how you can use dynamic SQL for changing the requirements of SQL statements.

If the HR manager enters 100 for `employee_id` and 4000 for salary, then the query should be:

```
UPDATE employees
SET salary = 4000
WHERE employee_id = 100;
```

If the HR manager enters some other values, like 150 for `employee_id` and 3000 for salary, then the corresponding SQL statement will change. The actual query is known only at run time. It is ideal to use dynamic SQL in such situations.

Using Dynamic SQL

- Use dynamic SQL when the full text of the dynamic SQL statement is unknown until run time. Therefore, its syntax is checked at *run time* rather than at *compile time*.
- Use dynamic SQL when one of the following items is unknown at precompile time:
 - Text of the SQL statement such as commands, clauses, and so on
 - The number and data types of host variables
 - References to database objects such as tables, columns, indexes, sequences, usernames, and views
- Use dynamic SQL to make your PL/SQL programs more general and flexible.

7

In PL/SQL, you need dynamic SQL to execute the following SQL statements where the full text is unknown at compile time, such as:

- A `SELECT` statement that includes an identifier that is unknown at compile time (such as a table name)
- A `WHERE` clause in which the column name is unknown at compile time

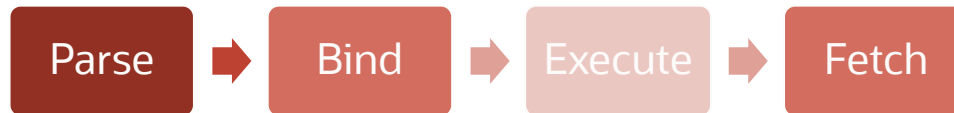
Note

For additional information about dynamic SQL, see the following resources:

- *Pro*C/C++ Programmer's Guide*
 - *Lesson 13, Oracle Dynamic SQL*, covers the four available methods that you can use to define dynamic SQL statements. It briefly describes the capabilities and limitations of each method and then offers guidelines for choosing the right method. Later sections in the guide show you how to use the methods and include example programs that you can study.
 - *Lesson 15, Oracle Dynamic SQL: Method 4*, contains detailed information about Method 4 (query with unknown number of select-list items or input host variables, more on slide 21) when defining dynamic SQL statements.
- *Oracle PL/SQL Programming* by Steven Feuerstein and Bill Pribyl
- *Lesson 16, Dynamic SQL and Dynamic PL/SQL*, contains additional information about dynamic SQL.

Execution Flow of SQL Statements

- All SQL statements go through some or all of the following stages:



- Some stages may not be relevant for all statements:
 - The fetch phase is applicable to queries.
 - For embedded SQL statements such as `SELECT`, `DML`, `MERGE`, `COMMIT`, `SAVEPOINT`, and `ROLLBACK`, the parse and bind phases are performed at compile time.
 - For dynamic SQL statements, all phases are performed at run time.

8

All SQL statements have to go through various stages. Nonetheless, some stages may not be relevant for all statements. The following are the key stages:

- **Parse:** Every SQL statement must be parsed. Parsing the statement includes checking the statement's syntax and validating the statement, and ensuring that all references to objects are correct and relevant privileges to those objects exist.
- **Bind:** After parsing, the Oracle server may need values from or for any bind variable in the statement. The process of obtaining these values is called binding variables. This stage may be skipped if the statement does not contain bind variables.
- **Execute:** At this point, the Oracle server has all necessary information and resources, and the statement is executed. For non-query statements, this is the last phase.
- **Fetch:** In the fetch stage, which is applicable to queries, the rows are selected and ordered (if requested by the query), and each successive fetch retrieves another row of the result until the last row has been fetched.

Dynamic SQL implementation

PL/SQL provides the following two methods to use Dynamic SQL in PL/SQL blocks:

- NDS
- DBMS_SQL package

9

There are two methods of implementing dynamic SQL in PL/SQL blocks:

1. **NDS** – It is a PL/SQL language feature for building and running dynamic SQL statements.
2. **DBMS_SQL package** – It is an API that enables you to build, run, and describe dynamic SQL statements.

To write an NDS code, you must know at compile time the number and data types of the input and output variables of the dynamic SQL statement. If you do not have this information at compile time, you must use the `DBMS_SQL` package.

Lesson Agenda

- Dynamic SQL and its features
- NDS
- Using the DBMS_SQL package



Native Dynamic SQL (NDS)

- Provides native support for dynamic SQL directly in the PL/SQL language
- Processes most dynamic SQL statements with the `EXECUTE IMMEDIATE` statement
- Gives the following choices if the dynamic SQL statement is a `SELECT` statement that returns multiple rows:
 - Use the `EXECUTE IMMEDIATE` statement with the `BULK COLLECT INTO` clause
 - Use the `OPEN-FOR`, `FETCH`, and `CLOSE` statements

11

NDS provides the ability to dynamically execute SQL statements whose structure is constructed at execution time. NDS supports it through the following statements:

- **EXECUTE IMMEDIATE:** Prepares a statement from a string, executes it, returns output of execution into variables, and then deallocates resources
- **OPEN-FOR:** Prepares and executes a statement using a cursor variable
- **FETCH:** Retrieves the results of an opened statement by using the cursor variable
- **CLOSE:** Closes the cursor used by the cursor variable and deallocates resources

Using the EXECUTE IMMEDIATE Statement

Use the EXECUTE IMMEDIATE statement for NDS or PL/SQL anonymous blocks:

```
EXECUTE IMMEDIATE dynamic_string
[INTO {define_variable
      [, define_variable] ... | record}]
[USING {IN|OUT|IN OUT} bind_argument
      [, {IN|OUT|IN OUT} bind_argument] ... ];
```

- INTO is used for single-row queries and specifies the variables or records into which column values are retrieved.
- USING is used to hold all bind arguments. The default parameter mode is IN.

12

The EXECUTE IMMEDIATE statement can be used to execute SQL statements or PL/SQL anonymous blocks. The syntactical elements include the following:

- dynamic_string is a string expression that represents a dynamic SQL statement (without a terminator) or a PL/SQL block (with a terminator).
- The INTO clause specifies the variables or record into which column values are retrieved. It is used only for single-row queries. For each value retrieved by the query, there must be a corresponding, type-compatible variable or field in the INTO clause.
- define_variable is a PL/SQL variable that stores the selected column value.
- record is a user-defined or %ROWTYPE record that stores a selected row.
- bind_argument is an expression whose value is passed to the dynamic SQL statement or PL/SQL block.
- The USING clause holds all bind arguments, which are input to the dynamic SQL statement to be executed. The default parameter mode is IN. You cannot pass a NULL literal value; instead you can pass an expression that might evaluate to NULL.

NDS supports all SQL data types. The input to the dynamic SQL statement through the bind arguments can be strings, numbers, dates, schema-level collections, LOBs, associative arrays, and so on. The INTO clause can have PL/SQL variables or records that correspond to the expected output of the dynamic SQL statement.

Note: Use OPEN-FOR, FETCH, and CLOSE for a multi row query.

Dynamic SQL with a DDL Statement: Examples

```
-- Create a table using dynamic SQL

CREATE OR REPLACE PROCEDURE create_table(
  p_table_name VARCHAR2, p_col_specs VARCHAR2) IS
BEGIN
  EXECUTE IMMEDIATE 'CREATE TABLE ' || p_table_name ||
    ' (' || p_col_specs || ')';
END;
/
```

```
-- Call the procedure

BEGIN
  create_table('EMPLOYEE_NAMES',
    'id NUMBER(4) PRIMARY KEY, name VARCHAR2(40)');
END;
/
```

13

The code examples show the creation of a `create_table` procedure that accepts the table name and column definitions (specifications) as parameters.

The procedure call shows the creation of a table called `EMPLOYEE_NAMES` with two columns:

- An ID column with a `NUMBER` data type used as a primary key
- A name column of up to 40 characters for the employee name

Any data definition language (DDL) statement can be executed by using the syntax shown in the slide, whether the statement is dynamically constructed or specified as a literal string. You can create and execute a statement that is stored in a PL/SQL string variable, as in the following example:

```
CREATE OR REPLACE PROCEDURE add_col(p_table_name VARCHAR2,
                                     p_col_spec   VARCHAR2) IS
  v_stmt VARCHAR2(100) := 'ALTER TABLE ' || p_table_name ||
    ' ADD ' || p_col_spec;
BEGIN
  EXECUTE IMMEDIATE v_stmt;
END;
/
```

To add a new column to a table, enter the following:

```
EXECUTE add_col('employee_names', 'salary number(8,2)')
```

Dynamic SQL with DML Statements

```
-- Insert a row into a table with two columns:
CREATE PROCEDURE add_row(p_table_name VARCHAR2,
    p_id NUMBER, p_name VARCHAR2) IS
BEGIN
    EXECUTE IMMEDIATE 'INSERT INTO ' || p_table_name ||
        ' VALUES (:1, :2)' USING p_id, p_name;
END;
```

```
-- Delete rows from any table:
CREATE FUNCTION del_rows(p_table_name VARCHAR2)
RETURN NUMBER IS
BEGIN
    EXECUTE IMMEDIATE 'DELETE FROM ' || p_table_name;
    RETURN SQL%ROWCOUNT;
END;
/
```

14

0

The first code example in the slide defines a dynamic SQL statement, which isn't a query and is without host variables. The examples in the slide demonstrate the following:

- The `add_row` procedure shows how to provide input values to a dynamic SQL statement with the `USING` clause. The bind variable names `:1` and `:2` are not important; however, the order of the parameter names (`p_id` and `p_name`) in the `USING` clause is associated with the bind variables by position, in the order of their respective appearance. Therefore, the PL/SQL parameter `p_id` is assigned to the `:1` placeholder, and the `p_name` parameter is assigned to the `:2` placeholder. Placeholder or bind variable names can be alphanumeric but must be preceded with a colon.
- The `del_rows` function deletes rows from a specified table and returns the number of rows deleted by using the implicit SQL cursor `%ROWCOUNT` attribute. Executing the function is shown below the example for creating a function.

Execute the following code to see the execution of both the procedures:

```
BEGIN
    add_row('EMPLOYEE_NAMES', 100, 'KING');
    DBMS_OUTPUT.PUT_LINE(
        del_rows('EMPLOYEE_NAMES') || ' rows deleted. ');
END;
/
```

Note: The `EXECUTE IMMEDIATE` statement prepares (pares) and immediately executes the dynamic SQL statement. Dynamic SQL statements are always parsed.

Also, note that a `COMMIT` operation is not performed in either example. Therefore, the operations can be undone with a `ROLLBACK` statement.

Dynamic SQL with a Single-Row Query: Example

```
CREATE FUNCTION get_emp( p_emp_id NUMBER )
RETURN employees%ROWTYPE IS
  v_stmt VARCHAR2(200);
  v_emprec employees%ROWTYPE;
BEGIN
  v_stmt := 'SELECT * FROM employees ' ||
            'WHERE employee_id = :p_emp_id';
  EXECUTE IMMEDIATE v_stmt INTO v_emprec USING p_emp_id ;
  RETURN v_emprec;
END;
/
DECLARE
  v_emprec employees%ROWTYPE := get_emp(100);
BEGIN
  DBMS_OUTPUT.PUT_LINE('Emp: ' || v_emprec.last_name);
END;
/
```

```
Function GET_EMP compiled
PL/SQL procedure successfully completed.
Emp: King
```

15

The code example in the slide is an example of defining a dynamic SQL statement with a single row queried—that is, query with a known number of select-list items and input host variables.

The single-row query example demonstrates the `get_emp` function that retrieves an `EMPLOYEES` record into a variable specified in the `INTO` clause. It also shows how to provide input values for the `WHERE` clause.

The anonymous block is used to execute the `get_emp` function and return the result into a local `EMPLOYEES` record variable.

The example could be enhanced to provide alternative `WHERE` clauses depending on input parameter values, making it more suitable for dynamic SQL processing.

Note

- For an example of “Dynamic SQL with a Multirow Query: Example” using `REF CURSORS`, see the `demo_07_13_a` in the `/home/oracle/labs/plpu/demo` folder.
- For an example of using `REF CURSORS`, see the `demo_07_13_b` in the `/home/oracle/labs/plpu/demo` folder.
- `REF CURSORS` are covered in the *Oracle Database: Advanced PL/SQL* course.

Executing a PL/SQL Anonymous Block Dynamically

```
CREATE FUNCTION annual_sal( p_emp_id NUMBER)
RETURN NUMBER IS
  v_plsql varchar2(200) :=
    'DECLARE '||
    '  rec_emp employees%ROWTYPE; '||
    'BEGIN '||
    '  rec_emp := get_emp(:empid); '||
    '  :res := rec_emp.salary * 12; '||
    'END;';
  v_result NUMBER;
BEGIN
  EXECUTE IMMEDIATE v_plsql
    USING IN p_emp_id, OUT v_result;
  RETURN v_result;
END;
/
EXECUTE DBMS_OUTPUT.PUT_LINE(annual_sal(100))
```

Anonymous PL/SQL block
initialized to variable
v_plsql

16

The `annual_sal` function dynamically constructs an anonymous PL/SQL block. The PL/SQL block contains bind variables for:

- The input of the employee ID using the `:empid` placeholder
- The output result computing the annual employees' salary using the placeholder called `:res`

Note: This example demonstrates how to use the `OUT` result syntax (in the `USING` clause of the `EXECUTE IMMEDIATE` statement) to obtain the result calculated by the PL/SQL block. The procedure output variables and function return values that can be obtained in a similar way from a dynamically executed anonymous PL/SQL block.

The output of the slide example is as follows:

```
Function ANNUAL_SAL compiled
PL/SQL procedure successfully completed.
288000
```

BULK COLLECT INTO clause

- It is used instead of `INTO` in SQL statements retrieving data into PL/SQL block.
- The variable followed by `BULK COLLECT INTO` should be capable of holding multiple rows such as a collection or a cursor.
- Improves the performance of a PL/SQL block with SQL statements retrieving huge volumes of data
- The presence of `BULK COLLECT INTO` in the `SELECT` statements returns the results from SQL to PL/SQL engine in batches instead of one at a time.

17

You use the `BULK COLLECT INTO` clause in `SELECT` statements or any other SQL statements that retrieve data in huge volumes.

While executing a PL/SQL block with SQL statements, the SQL engine and PL/SQL engine communicate with each other. The SQL statement execution in the PL/SQL block is handed over to the SQL engine. The SQL engine returns the result to PL/SQL statement as it processes the statement. This exchange of information between the SQL engine and PL/SQL engine results in context switches that further add to the performance overhead.

Executing a `SELECT` statement with `BULK COLLECT INTO` clause results in SQL returning results to PL/SQL in batches instead of one at a time, thus improving the performance of the PL/SQL block.

OPEN FOR clause

- The OPEN FOR clause is used to associate a cursor variable with a query.

```
OPEN { cursor_variable | :host_cursor_variable }  
FOR select_statement [ using_clause ] ;
```

- It allocates database resources to process the query.
- It identifies the result set based on the query.
- If the query has a FOR UPDATE clause, then the OPEN FOR statement locks the rows of the result set.

Using BULK COLLECT and OPEN FOR clause

```
DECLARE
  TYPE EmpCurTyp IS REF CURSOR;
  TYPE NumList IS TABLE OF NUMBER;
  TYPE NameList IS TABLE OF VARCHAR2(25);
  emp_cv EmpCurTyp;
  empids NumList;
  enames NameList;
  sals NumList;
BEGIN
  OPEN emp_cv FOR 'SELECT employee_id, last_name FROM employees';
  FETCH emp_cv BULK COLLECT INTO empids, enames;
  CLOSE emp_cv;
  DBMS_OUTPUT.PUT_LINE(empids.count);
END;
```

PL/SQL procedure successfully completed.

107

19

The code in the slide demonstrates the usage of `BULK COLLECT` and `OPEN FOR` statements.

The `OPEN FOR` statement in the code opens a `REF CURSOR` for the `SELECT` statement. To learn more about `REF CURSORS`, you can refer to appendix `REF CURSORS`.

Summarizing Methods for Using Dynamic SQL

Method #	SQL Statement Type	NDS SQL Statements Used
Method 1	Non-query without host variables	EXECUTE IMMEDIATE without the USING and INTO clauses
Method 2	Non-query with known number of input host variables	EXECUTE IMMEDIATE with a USING clause
Method 3 (returning a single row)	Query with known number of select-list items and input host variables returning a single row as result set	EXECUTE IMMEDIATE with the USING and INTO clauses
Method 3 (returning multiple rows)	Query with known number of select-list items and input host variables returning multiple rows as result set	EXECUTE IMMEDIATE with cursors using BULK COLLECT and OPEN FOR clauses
Method 4	Query with unknown number of select-list items or input host variables	Use the DBMS_SQL package

20

In the slide we discuss the four available methods for NDS. That is, Method 2 encompasses Method 1, Method 3 encompasses Methods 1 and 2, and Method 4 encompasses Methods 1, 2, and 3. However, each method is most useful for handling a certain kind of SQL statement, as follows:

Method 1 applies to those SQL statements that don't accept host variables as input and those that don't return any results. They are also known as non-query statements.

You execute them directly with `EXECUTE IMMEDIATE` as in this example

```
EXECUTE IMMEDIATE 'DELETE FROM EMPLOYEES WHERE DEPTNO = 20'
```

With Method 1, the SQL statement is parsed every time it is executed. Examples of non-queries include DDL statements, `UPDATES`, `INSERTS`, or `DELETES`.

Method 2 applies to those SQL statements that accept from the host environment in the form of host variables.

You execute such statements by using `EXECUTE IMMEDIATE` with the `USING` clause as shown below:

```
EXECUTE IMMEDIATE 'INSERT INTO EMPLOYEES (FIRST_NAME, LAST_NAME, JOB_ID)
VALUES (:emp_first_name, :emp_last_name, :job_id)' USING 'Stephen',
'King', 'AD_VP'
```

The number of placeholders for input host variables and the data types of the input host variables must be known at precompile time.

Method 3 applies to those state SQL statements that accept a known number of variables and return a single row or multiple rows as the result set of the query.

For example, the following host strings qualify:

```
EXECUTE IMMEDIATE SELECT DEPARTMENT_ID, SALARY
FROM EMPLOYEES
WHERE DEPARTMENT_ID = :dep_num
INTO DEP_ID, SAL;
```

You may also have queries that accept a known number of input variables and return multiple rows. In such a case, you use `OPEN FOR`, `FETCH`, and `CLOSE` statements to retrieve the data into a cursor and perform operations on it.

Method 4

Method 4 is required for dynamic SQL statements that contain an unknown number of select-list items or input host variables. With this method, you use the `DBMS_SQL` package, which is covered in the following slides.

Note

For additional information about the four dynamic SQL methods, see the following lessons in the *Pro*C/C++ Programmer's Guide*.

- *Lesson 13, Oracle Dynamic SQL*
- *Lesson 15, Oracle Dynamic SQL: Method 4*

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Dynamic SQL and its features
- NDS
- Using the `DBMS_SQL` package





Hey Alice, can you create a simple interface where I can just enter the table name, column names, and output I want to see in the user interface, and I get the job done on submission to the server?

Yes...I'll be able to do that.

For the audience that is wondering how we write the query, the DBMS_SQL package is the answer



0

For Instructor Use Only.
This document should not be distributed.



In a situation of uncertainty, where you aren't sure about the table on which the query would execute or about the number of columns you would retrieve from the table or about the output, you can use the procedures provided by the DBMS_SQL package.

You first construct the query based on user input with respect to the tables, columns, and so on. Then, you execute the query. The DBMS_SQL package provides various procedures for the purpose.

0

For Instructor Use Only.
This document should not be distributed.

Using the DBMS_SQL Package

- The `DBMS_SQL` package is used to write dynamic SQL in stored procedures and to parse DDL statements.
- You must use the `DBMS_SQL` package to execute a dynamic SQL statement that has an unknown number of input or output variables, also known as Method 4.
- In most cases, NDS is easier to use and performs better than `DBMS_SQL`, except when dealing with Method 4.
- For example, you must use the `DBMS_SQL` package in the following situations:
 - You do not know the `SELECT` list at compile time.
 - You do not know how many columns a `SELECT` statement will return or what their data types will be.

25

By using `DBMS_SQL`, you can write stored procedures and anonymous PL/SQL blocks that use dynamic SQL, such as executing DDL statements in PL/SQL. An example is executing a `DROP TABLE` statement. The operations provided by this package are performed under the current user, and not under the package owner `SYS`.

Method 4: Method 4 refers to situations where, in a dynamic SQL statement, the number of columns selected for a query or the number of bind variables set is not known until run time. In this case, you should use the `DBMS_SQL` package.

Using the DBMS_SQL Package Subprograms

The procedures used through the execution flow of dynamic SQL statements using DBMS_SQL package:

- OPEN_CURSOR
- PARSE
- BIND_VARIABLE or BIND_ARRAY
- DEFINE_COLUMN, DEFINE_COLUMN_LONG or DEFINE_ARRAY
- EXECUTE
- FETCH_ROWS or EXECUTE_AND_FETCH
- VARIABLE_VALUE, COLUMN_VALUE or COLUMN_VALUE_LONG
- CLOSE_CURSOR

26

The DBMS_SQL package provides the following subprograms to execute dynamic SQL:

- **OPEN_CURSOR:** To process a SQL statement, you must have an open cursor. The DBMS_SQL package provides OPEN_CURSOR functions, which when called return a cursor ID number for a valid cursor. These cursors are used only by the DBMS_SQL package.
- **PARSE** procedures: Every SQL statement must be parsed by calling the PARSE procedures. Parsing the statement checks the statement's syntax and associates it with the cursor in your program. DDL statements are immediately executed when parsed.
- **BIND_VARIABLE OR BIND_ARRAY:** You have to define bind variables when the SQL statement you execute need input data to be supplied at runtime. The SQL statement should have placeholders to mark where the data has to be supplied. You must invoke a BIND_ARRAY or BIND_VARIABLE procedure for each placeholder.
- **DEFINE_COLUMN OR DEFINE_ARRAY:** If the result of the query has to be received in the PL/SQL block, then you must call one of the define procedures of the package. You use the DEFINE_COLUMN procedure when you have to receive a column of values and you use the DEFINE_ARRAY procedure when you have to retrieve a collection of values.
- **EXECUTE:** The EXECUTE function is invoked to execute the SQL statement and return the number of rows processed.
- **FETCH:** The FETCH_ROWS function is invoked to retrieve the rows from the result set of a query. You execute FETCH_ROWS multiple times to retrieve consequent set of rows.
- **COLUMN_VALUE:** The COLUMN_VALUE procedure is invoked to retrieve the value of the column fetched through FETCH_ROWS.
- **CLOSE_CURSOR:** The CLOSE_CURSOR procedure is used to close the specified cursor.

Using DBMS_SQL with a DML Statement: Deleting Rows

```
CREATE OR REPLACE FUNCTION delete_all_rows
(p_table_name VARCHAR2) RETURN NUMBER IS
  v_cur_id      INTEGER;
  v_rows_del    NUMBER;
BEGIN
  v_cur_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(v_cur_id,
    'DELETE FROM ' || p_table_name, DBMS_SQL.NATIVE);
  v_rows_del := DBMS_SQL.EXECUTE (v_cur_id);
  DBMS_SQL.CLOSE_CURSOR(v_cur_id);
  RETURN v_rows_del;
END;
/
```

```
CREATE TABLE temp_emp AS SELECT * FROM employees;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Rows Deleted: ' || delete_all_rows('temp_emp'));
END;
/
```

27

In the slide, the table name is passed into the `delete_all_rows` function. The function uses dynamic SQL to delete rows from the specified table and returns a count representing the number of rows that are deleted after successful execution of the statement.

To process a DML statement dynamically, perform the following steps:

1. Use `OPEN_CURSOR` to establish an area in memory to process a SQL statement.
2. Use `PARSE` to establish the validity of the SQL statement. `DBMS_SQL.NATIVE` is a constant that is the value for `language_flag` parameter of `PARSE` procedure. It implies normal behavior for the connected database.
3. Use the `EXECUTE` function to run the SQL statement. This function returns the number of rows processed.
4. Use `CLOSE_CURSOR` to close the cursor.

The steps to execute a DDL statement are similar, but step 3 is optional because a DDL statement is immediately executed when the `PARSE` is successfully done—that is, the statement syntax and semantics are correct. If you use the `EXECUTE` function with a DDL statement, then it does not do anything and returns a value of 0 for the number of rows processed because DDL statements do not process rows.

```
Function DELETE_ALL_ROWS compiled
Table TEMP_EMP created.
PL/SQL procedure successfully completed.
Rows Deleted: 107
Table TEMP_EMP dropped.
```

For Instructor Use Only.
This document should not be distributed.

Using DBMS_SQL with a Parameterized DML Statement

```
CREATE PROCEDURE insert_row (p_table_name VARCHAR2,
p_id VARCHAR2, p_name VARCHAR2, p_region NUMBER) IS
  v_cur_id      INTEGER;
  v_stmt        VARCHAR2(200);
  v_rows_added  NUMBER;
BEGIN
  v_stmt := 'INSERT INTO ' || p_table_name ||
    ' VALUES (:cid, :cname, :rid)';
  v_cur_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(v_cur_id, v_stmt, DBMS_SQL.NATIVE);
  DBMS_SQL.BIND_VARIABLE(v_cur_id, ':cid', p_id);
  DBMS_SQL.BIND_VARIABLE(v_cur_id, ':cname', p_name);
  DBMS_SQL.BIND_VARIABLE(v_cur_id, ':rid', p_region);
  v_rows_added := DBMS_SQL.EXECUTE(v_cur_id);
  DBMS_SQL.CLOSE_CURSOR(v_cur_id);
  DBMS_OUTPUT.PUT_LINE(v_rows_added || ' row added');
END;
/
```

29

The example in the slide performs the DML operation to insert a row into a specified table. The example demonstrates the extra step required to associate values to bind variables that exist in the SQL statement. For example, a call to the procedure shown in the slide is:

```
EXECUTE insert_row('countries', 'LB', 'Lebanon', 4)
```

After the statement is parsed, you must call the `DBMS_SQL.BIND_VARIABLE` procedure to assign values for each bind variable that exists in the statement. The binding of values must be done before executing the code. To process a `SELECT` statement dynamically, perform the following steps after opening and before closing the cursor:

1. Execute `DBMS_SQL.DEFINE_COLUMN` for each column selected.
2. Execute `DBMS_SQL.BIND_VARIABLE` for each bind variable in the query.
3. For each row, perform the following steps:
 - a. Execute `DBMS_SQL.FETCH_ROWS` to retrieve a row and return the number of rows fetched. Stop additional processing when a zero value is returned.
 - b. Execute `DBMS_SQL.COLUMN_VALUE` to retrieve each selected column value into each PL/SQL variable for processing.

Although this coding process is not complex, it is more time consuming to write and is prone to error compared to using the Native Dynamic SQL approach.

Quiz



The full text of the dynamic SQL statement might be unknown until run time; therefore, its syntax is checked at *run time* rather than at *compile time*.

- a. True
- b. False

30

Answer: a

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Describe the execution flow of SQL statements
- Build and execute SQL statements dynamically by using NDS
- Identify situations when you must use the `DBMS_SQL` package instead of NDS to build and execute SQL statements dynamically



31

In this lesson, you learned how to dynamically create a SQL statement and execute it by using NDS statements. Dynamically executing SQL and PL/SQL code extends the capabilities of PL/SQL beyond query and transactional operations. For earlier releases of the database, you could achieve similar results with the `DBMS_SQL` package.

For Instructor Use Only.
This document should not be distributed.

Practice 17 Overview: Using Dynamic SQL

This practice covers the following topics:

- Creating a package that uses Native Dynamic SQL to create or drop a table and to populate, modify, and delete rows from a table
- Creating a package that compiles the PL/SQL code in your schema



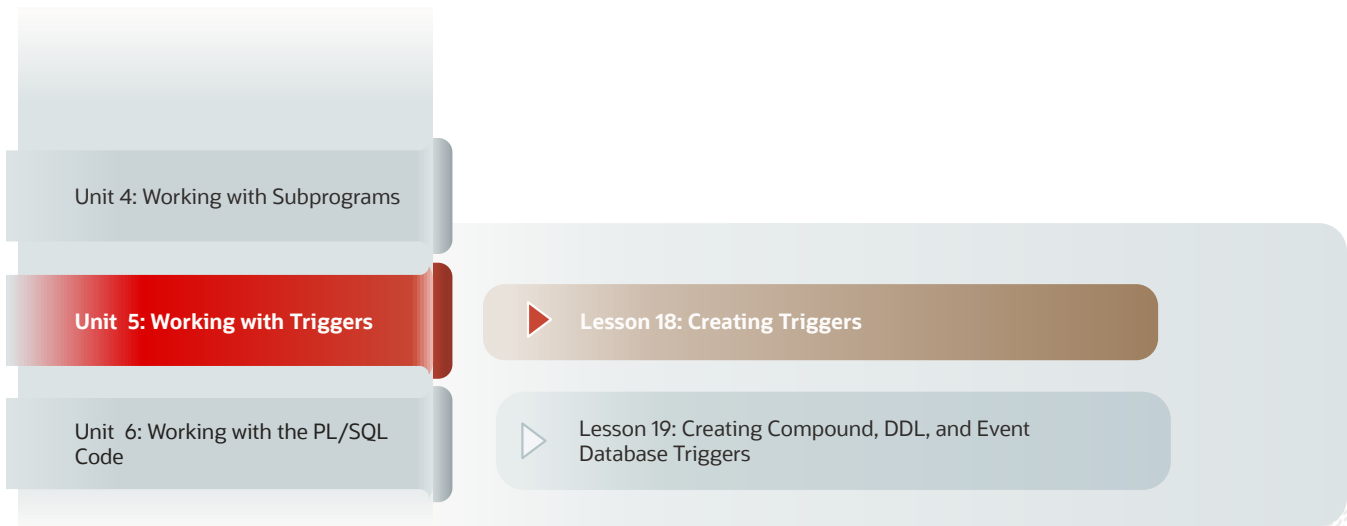
32

In this practice, you write code to perform the following tasks:

- Create a package that uses Native Dynamic SQL to create or drop a table, and to populate, modify, and delete rows from the table.
- Create a package that compiles the PL/SQL code in your schema, either all the PL/SQL code or only code that has an `INVALID` status in the `USER_OBJECTS` table.

Creating Triggers

Course Road Map



2

In Unit 5, you learn to create and use triggers in database applications.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Describe database triggers and their uses
- Describe the different types of triggers
- Create database triggers
- Describe database trigger-firing rules
- Remove database triggers
- Display trigger information



For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Understanding the usage of triggers
- Creating triggers
- Creating row-level triggers
- Creating INSTEAD OF triggers
- Managing Triggers





Hey Alice, I have a situation where I have to keep track of the amount of salary increase given to an employee whenever it is given. I can request the managers to update a certain table whenever they give a hike, but usually they forget about this update. Can you automate this?

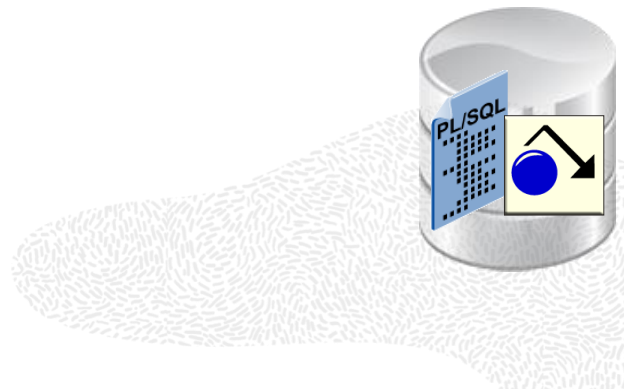
I can definitely create something for this requirement. So, friends, let's get to work!

We have triggers in PL/SQL that can help us in creating something that the HR manager has asked for. Let's look at the concept first.



What are Triggers?

- A trigger is a PL/SQL block that is stored in the database and fired (executed) in response to a specified event.
- The Oracle database automatically executes a trigger when specified conditions occur.

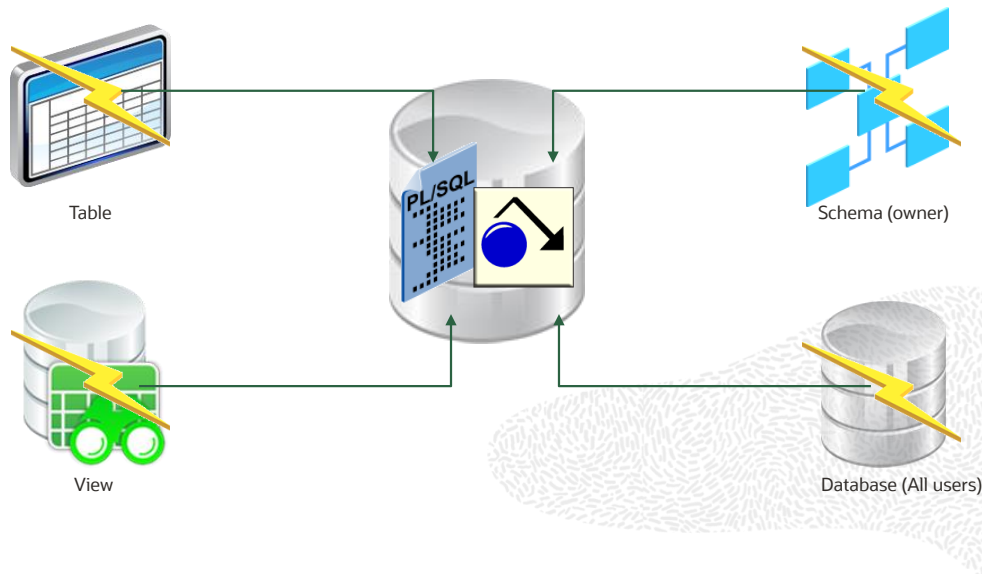


6

Triggers are similar to stored procedures. A trigger stored in the database contains PL/SQL in the form of an anonymous block, a call statement, or a compound trigger block. However, procedures and triggers differ in the way that they are invoked. A procedure is explicitly run by a user, application, or trigger. Triggers are implicitly fired by the Oracle database when a triggering event occurs, regardless of which user is connected and which application is being used.

Defining Triggers

A trigger can be defined on the table, view, schema (schema owner), or database (all users).



7

You can create a trigger on a table, view, schema, or database. You can also specify the time when the trigger is expected to be invoked. You generally define a trigger for an event and define it to be invoked before or after the event.

Why do you use Triggers?

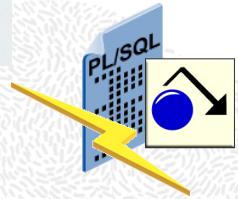
You might use a trigger for one of the following purposes:

- To log events
- To gather statistics on table access
- To modify table data when DML statements are issued against views
- To enforce referential integrity constraints when child and parent tables are on different nodes
- To prevent DML operations on a table based on application requirements
- To prevent invalid transactions
- To implement complex business rules

Trigger Event Types

You can write triggers that fire whenever one of the following operations occurs in the database:

DML Statements	DDL Statements	Database Events
• DELETE • INSERT • UPDATE	• CREATE • ALTER • DROP	• SERVERERROR • LOGON • LOGOFF • STARTUP • SHUTDOWN



9

A triggering event or statement is the SQL statement, database event, or user event that causes a trigger to fire. A triggering event can be one or more of the following:

- An INSERT, UPDATE, or DELETE statement on a specific table (or view, in some cases)
- A CREATE, ALTER, or DROP statement on any schema object
- A database startup or instance shutdown
- A specific error message or any error message
- A user login or logoff

For Instructor Use Only.
This document should not be distributed.

Available Trigger Types

- Simple DML triggers
 - BEFORE
 - AFTER
 - INSTEAD OF
- Compound triggers
- System triggers
 - DDL event triggers
 - Database event triggers



10

Triggers are classified based on the time when they are defined to be executed, how they are organized, and for what type of statements they execute.

Simple DML triggers are defined for DML statements. These triggers can execute BEFORE, AFTER, or INSTEAD OF the triggering statement.

Compound triggers are those that have a single trigger defined to execute over multiple events.

System triggers are also known as Schema triggers, which are defined to execute on events pertaining to the database or schema. The triggering event might be either a DDL statement or a database operation.

Note: In this lesson, you will learn about the BEFORE, AFTER, and INSTEAD OF triggers. The other trigger types are discussed in the lesson titled “Creating Compound, DDL, and Event Database Triggers.”

Trigger Event Types and Body

- A trigger event type determines which DML statement causes the trigger to execute. The possible events are:

- INSERT
- UPDATE [OF column]
- DELETE

- A trigger body determines what action is performed and is a PL/SQL block or a CALL to a procedure.

11

The triggering event or statement can be an INSERT, UPDATE, or DELETE statement on a table.

- When the triggering event is an UPDATE statement, you can include a column list to identify which columns must be changed to fire the trigger. You cannot specify a column list for an INSERT or for a DELETE statement because it always affects entire rows.

```
. . . UPDATE OF salary . . .
```

- The triggering event can contain one, two, or all three of these DML operations.

```
. . . INSERT or UPDATE or DELETE
```

```
. . . INSERT or UPDATE OF job_id . . .
```

The trigger body defines the action—that is, what needs to be done when the triggering event is issued. The PL/SQL block can contain SQL and PL/SQL statements, and can define PL/SQL constructs such as variables, cursors, exceptions, and so on. You can also call a PL/SQL procedure or a Java procedure.

Note: The size of a trigger cannot be greater than 32 KB.

Lesson Agenda

- Understanding the usage of triggers
- Creating triggers
- Creating row-level triggers
- Creating INSTEAD OF triggers
- Managing Triggers



Creating DML Triggers by Using the CREATE TRIGGER Statement

```
CREATE [OR REPLACE] TRIGGER trigger_name
timing -- when to fire the trigger
event1 [OR event2 OR event3]
ON object_name
[REFERENCING OLD AS old | NEW AS new]
FOR EACH ROW -- default is statement level trigger
WHEN (condition)]
DECLARE]
BEGIN
... trigger_body -- executable statements
[EXCEPTION . . .]
END [trigger_name];
```

```
timing = BEFORE | AFTER | INSTEAD OF
```

```
event = INSERT | DELETE | UPDATE | UPDATE OF column_list
```

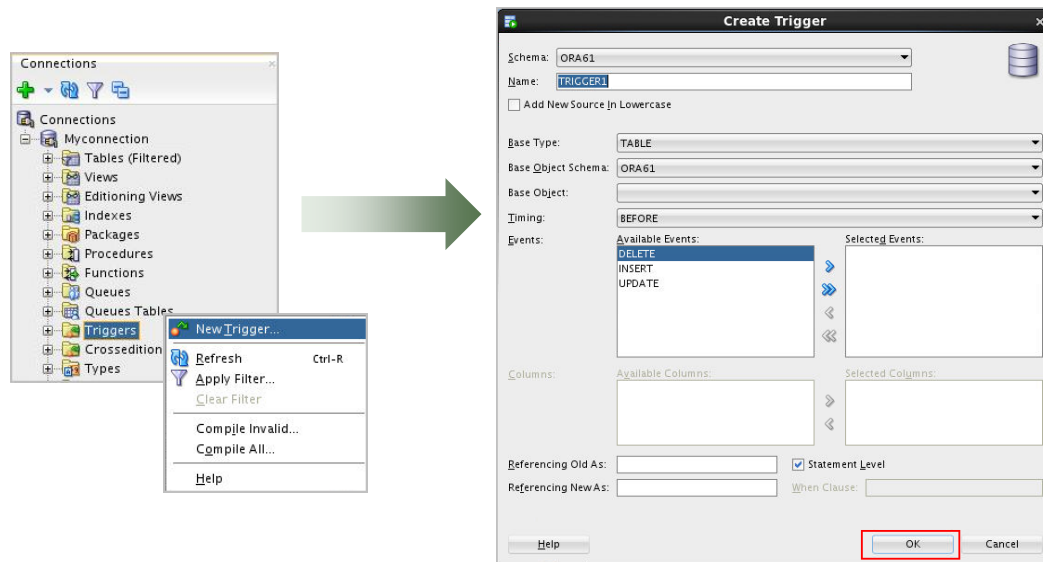
13

The components of the trigger syntax are:

- **trigger_name**: uniquely identifies the trigger
- **timing**: indicates when the trigger fires in relation to the triggering event. Values are BEFORE, AFTER, and INSTEAD OF.
- **event**: identifies the DML operation causing the trigger to fire
- **Values** are INSERT, UPDATE [OF column], and DELETE
- **object_name**: indicates the table or view associated with the trigger
- For row triggers, you can specify:
 - A **REFERENCING** clause to choose correlation names for referencing the old and new values of the current row (default values are OLD and NEW)
 - **FOR EACH ROW** to designate that the trigger is a row trigger
 - A **WHEN** clause to apply a conditional predicate, in parentheses, which is evaluated for each row to determine whether or not to execute the trigger body
- **trigger_body** is the action performed by the trigger, implemented as either of the following:
 - An anonymous block with a **DECLARE** or **BEGIN**, and an **END**
 - A **CALL** clause to invoke a standalone or packaged stored procedure, such as:

```
CALL my_procedure;
```

Creating DML Triggers by Using SQL Developer



14

The slide demonstrates how to create a trigger in SQL Developer.

To create DML triggers in SQL Developer, perform the following steps:

1. Right-click **Triggers** under your connection name.
2. Click **New Trigger**. This opens a trigger-creating wizard as shown in the slide.
3. In the wizard, provide the trigger name, type, the event on which the trigger has to fire, and other details.
4. Click **OK**.
5. A code editor with the skeletal trigger code opens; define the body of the trigger.

Specifying the Trigger Execution Time

You can specify the trigger timing as to whether to run the trigger's action before or after the triggering statement:

- **BEFORE:** Execute the trigger body before the triggering DML event on a table.
- **AFTER:** Execute the trigger body after the triggering DML event on a table.
- **INSTEAD OF:** Execute the trigger body instead of the triggering statement. This is used for views that are not otherwise modifiable.

15

The **BEFORE** trigger timing is used in the following situations:

- To determine whether the triggering statement should execute or not. The triggering statement would not execute if an exception raises in the trigger execution.
- To derive column values before completing an **INSERT** or **UPDATE** statement
- To initialize global variables or flags, and to validate complex business rules

The **AFTER** triggers are used in the following situations:

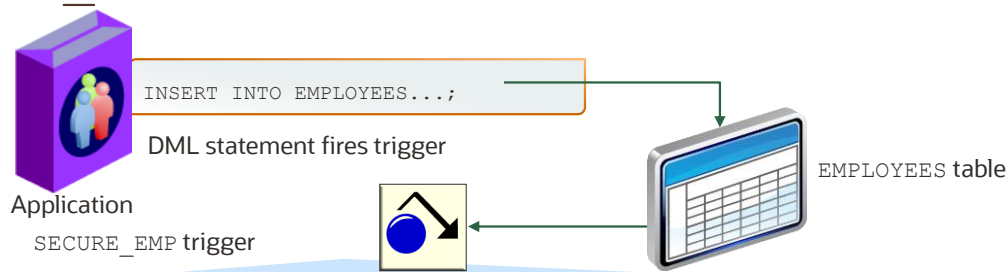
- To complete the triggering statement before executing the triggering action
- To perform different actions on the same triggering statement if a **BEFORE** trigger is already present

The **INSTEAD OF** triggers provide a transparent way of modifying views that cannot be modified directly through SQL DML statements because a view is not always modifiable. You can write appropriate DML statements inside the body of an **INSTEAD OF** trigger to perform actions directly on the underlying tables of views.

If it is practical, replace the set of individual triggers with different timing points with a single compound trigger that explicitly codes the actions in the order that you intend. If two or more triggers are defined with the same timing point, and the order in which they fire is important, you can control the firing order by using the **FOLLOWS** and **PRECEDES** clauses.

Creating a DML Statement Trigger Example:

SECURE_EMP



```
CREATE OR REPLACE TRIGGER secure_emp
BEFORE INSERT ON employees
BEGIN
  IF (TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN')) OR
     (TO_CHAR(SYSDATE, 'HH24:MI')
      NOT BETWEEN
        '08:00' AND '18:00') THEN
    RAISE_APPLICATION_ERROR(-20500, 'You may insert'
      || ' into EMPLOYEES table only during '
      || ' normal business hours.');
```

16

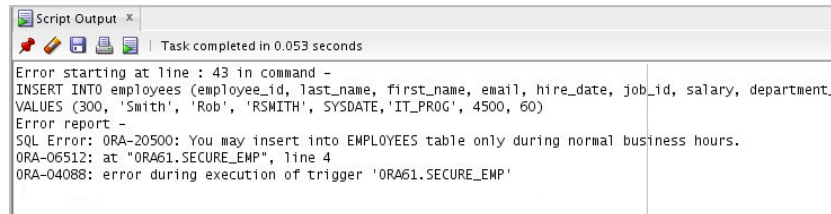
In the example in the slide, the SECURE_EMP database trigger is a BEFORE statement trigger that prevents the INSERT operation from succeeding if the business condition is violated. In this case, the trigger restricts inserts into the EMPLOYEES table during certain business hours, Monday through Friday.

If you attempt to insert a row into the EMPLOYEES table on Saturday, then you'll an error message, the trigger fails, and the triggering statement is rolled back. Remember that the RAISE_APPLICATION_ERROR is a server-side built-in procedure that returns an error to the user and causes the PL/SQL block to fail.

When a database trigger fails, the triggering statement is automatically rolled back by the Oracle server.

Testing Trigger SECURE_EMP

```
INSERT INTO employees (employee_id, last_name, first_name, email, hire_date, job_id, salary, department_id)
VALUES (300, 'Smith', 'Rob', 'RSMITH', SYSDATE, 'IT_PROG', 4500, 60);
```



17

To test the trigger, insert a row into the `EMPLOYEES` table during nonbusiness hours. When the date and time are out of the business hours specified in the trigger, you receive the error message shown in the slide.

Using Conditional Predicates

```
CREATE OR REPLACE TRIGGER secure_emp BEFORE
INSERT OR UPDATE OR DELETE ON employees
BEGIN
    IF (TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN')) OR
       (TO_CHAR(SYSDATE, 'HH24')
        NOT BETWEEN '08' AND '18') THEN
        IF DELETING THEN RAISE_APPLICATION_ERROR(
            -20502, 'You may delete from EMPLOYEES table'||
            'only during normal business hours.');
```

```
        ELSEIF INSERTING THEN RAISE_APPLICATION_ERROR(
            -20500, 'You may insert into EMPLOYEES table'||
            'only during normal business hours.');
```

```
        ELSIF UPDATING ('SALARY') THEN
            RAISE_APPLICATION_ERROR(-20503, 'You may '||
            'update SALARY only normal during business hours.');
```

```
        ELSE RAISE_APPLICATION_ERROR(-20504, 'You may'||
            ' update EMPLOYEES table only during'||
            ' normal business hours.');
```

```
    END IF;
END IF;
END;
```

18

Detecting the DML Operation that Fired a Trigger

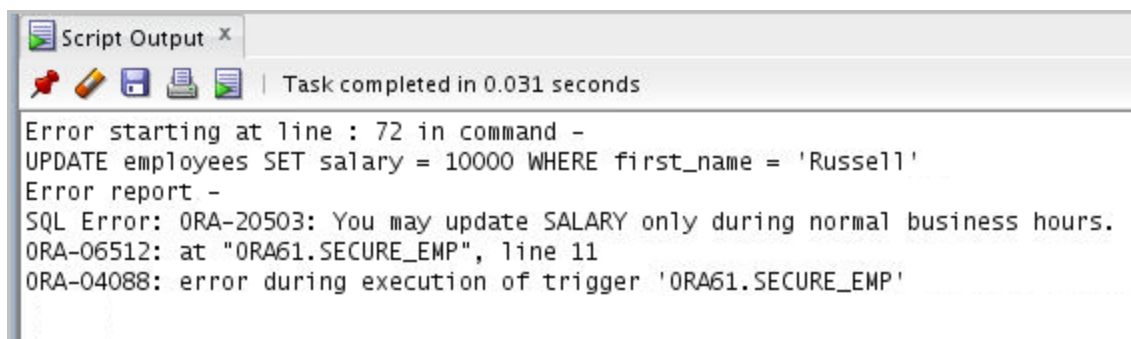
If more than one type of DML operation can fire a trigger (for example, ON INSERT OR DELETE OR UPDATE OF Emp_tab), the trigger body can use the conditional predicates INSERTING, DELETING, and UPDATING to check which type of statement fired the trigger.

You can combine several triggering events into one by taking advantage of the special conditional predicates INSERTING, UPDATING, and DELETING within the trigger body.

Example

Create one trigger to restrict all data manipulation events on the EMPLOYEES table to certain business hours, 8 AM to 6 PM, Monday through Friday.

The following is the response from the trigger when you perform an insertion on a Saturday evening.



The screenshot shows a 'Script Output' window with a toolbar at the top. Below the toolbar, it says 'Task completed in 0.031 seconds'. The main text area displays the following error message:

```
Error starting at line : 72 in command -
UPDATE employees SET salary = 10000 WHERE first_name = 'Russell'
Error report -
SQL Error: ORA-20503: You may update SALARY only during normal business hours.
ORA-06512: at "ORA61.SECURE_EMP", line 11
ORA-04088: error during execution of trigger 'ORA61.SECURE_EMP'
```

Multiple Triggers of the Same Type

- You can have multiple triggers defined for a timing instance.
- To resolve the sequence of execution, you can use:
 - PRECEDES
 - FOLLOWS
- To execute `trigger2` after `trigger1` for a table `employees` BEFORE UPDATE, you may write it syntactically as:

```
CREATE OR REPLACE TRIGGER2 ON EMPLOYEES BEFORE UPDATE  
FOLLOWS TRIGGER1...
```

CALL Statements in Triggers

```
CREATE [OR REPLACE] TRIGGER trigger_name
timing
event1 [OR event2 OR event3]
ON table_name
[REFERENCING OLD AS old | NEW AS new]
[FOR EACH ROW]
[WHEN condition]
CALL procedure name
/
```

```
CREATE OR REPLACE PROCEDURE log_execution IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('log_execution: Employee Inserted');
END;
/
CREATE OR REPLACE TRIGGER log_employee
BEFORE INSERT ON EMPLOYEES
CALL log_execution -- no semicolon needed
/
```

20

A `CALL` statement enables you to call a stored procedure, rather than code the PL/SQL body in the trigger itself. The procedure can be implemented in PL/SQL, C, or Java.

The call can reference the trigger attributes `:NEW` and `:OLD` as parameters, as in the following example:

```
CREATE OR REPLACE TRIGGER salary_check
    BEFORE UPDATE OF salary, job_id ON employees
    FOR EACH ROW
    WHEN (NEW.job_id <> 'AD_PRES')
    CALL sal_status(:NEW.job_id, :NEW.salary)
```

Note: There is no semicolon at the end of the `CALL` statement.

In the preceding example, the trigger calls the `sal_status` procedure. The procedure compares the new salary with the salary range for the new job ID from the `JOBS` table.

Lesson Agenda

- Understanding the usage of triggers
- Creating triggers
- **Creating row-level triggers**
- Creating INSTEAD OF triggers
- Managing Triggers



For Instructor Use Only.
This document should not be distributed.

Statement-Level Triggers Versus Row-Level Triggers

Statement-Level Triggers	Row-Level Triggers
Is the default when creating a trigger	Use the FOR EACH ROW clause when creating a trigger
Fires once for the triggering event	Fires once for each row affected by the triggering event
Fires once even if no rows are affected	Does not fire if the triggering event does not affect any rows

22

Types of DML Triggers

You can specify that the trigger will be executed once for every row affected by the triggering statement (such as a multiple-row `UPDATE`) or once for the triggering statement, no matter how many rows it affects.

Statement Trigger

A statement trigger is fired once on behalf of the triggering event, even if no rows are affected at all. Statement triggers are useful if the trigger action does not depend on the data from rows that are affected or on data provided by the triggering event itself (for example, a trigger that performs a complex security check on the current user).

Row Trigger

A row trigger fires each time the table is affected by the triggering event. If the triggering event affects no rows, a row trigger is not executed. Row triggers are useful if the trigger action depends on data of the rows that are affected or on data provided by the triggering event itself.

Note: Row triggers use correlation names to access the old and new column values of the row being processed by the trigger.



In a situation where I have to log insertion of every row while inserting a bunch of 100 rows, I will use a row-level trigger.
While creating the trigger, I add FOR EACH ROW in the syntax.



In a situation where I have to check the date and time of insertion and then allow it if it is in a valid range, I will use a statement-level trigger.

23

Consider a scenario where you have to log every row for backup while inserting a bunch of 100 rows. You may write a trigger with options `ON INSERT FOR EACH ROW` and define the trigger to perform insertions into the backup table.

In a situation where you have to check the day and time of insertion before performing an insertion and allow the insertion only if it is on a valid day and in a valid time range, you may write a trigger with the option `BEFORE INSERT` and define the trigger body to perform the range checks.

Creating a DML Row Trigger

```
CREATE OR REPLACE TRIGGER restrict_salary
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    IF NOT (:NEW.job_id IN ('AD_PRES', 'AD_VP'))
        AND :NEW.salary > 15000 THEN
        RAISE_APPLICATION_ERROR (-20202,
            'Employee cannot earn more than $15,000.');
```

```
END IF;
END;
```

```
UPDATE employees
SET salary = 15500
WHERE last_name = 'Russell';
```

```
Error starting at line : 105 in command -
UPDATE employees
SET salary = 15500
WHERE last_name = 'Russell'
Error report -
SQL Error: ORA-20202: Employee cannot earn more than $15,000.
ORA-06512: at "ORA61.RESTRICT_SALARY", line 4
ORA-04088: error during execution of trigger 'ORA61.RESTRICT_SALARY'
```

24

You can create a **BEFORE** row trigger in order to prevent the triggering operation from succeeding if a certain condition is violated.

In the first example in the slide, a trigger is created to allow only employees whose job IDs are either **AD_PRES** or **AD_VP** to earn a salary of more than 15,000. If you try to update the salary of employee **Russell** whose employee ID is **SA_MAN**, the trigger raises the exception displayed in the slide.

Note: Before executing the first code example in the slide, make sure you disable the `secure_emp` and `secure_employees` triggers.

Correlation names and Pseudorecords

- Correlation names are used to refer to values of a column before and after the trigger firing event.
- There are three different correlation names – OLD, NEW, and PARENT
- Pseudorecords refer to the values of the rows before and after the trigger-firing event when they are manipulated in the trigger body as a record.
- Pseudorecords are also referred to with OLD, NEW, and PARENT qualifiers.

Correlation Names and Pseudorecords

Let us consider the logging of update operation by a logging utility in the database.



- While logging the update operation, the old value has to be written into the log table
- When you perform the log operation through a trigger, you have to refer to the value before **UPDATE**.
- You can refer to the value before the update with the **OLD** qualifier.
- The value after the update is referred with the **NEW** qualifier.

Using OLD and NEW Qualifiers

- When a row-level trigger fires, the PL/SQL runtime engine creates and populates two data structures:
 - OLD: Stores the original values of the record processed by the trigger
 - NEW: Contains the new values
- NEW and OLD have the same structure as a record declared by using the %ROWTYPE on the table to which the trigger is attached.

Data Operationsl	Old Value	New Value
INSERT	NULL	Inserted value
UPDATE	Value before update	Value after update
DELETE	Value before delete	NULL

27

Within a ROW trigger, you can reference the value of a column before and after the data change by prefixing it with the OLD and NEW qualifiers.

Note

- The OLD and NEW qualifiers are available only in ROW triggers.
- Prefix these qualifiers with a colon (:) in every SQL and PL/SQL statement.
- There is no colon (:) prefix if the qualifiers are referenced in the WHEN restricting condition.
- Row triggers can decrease the performance if you perform many updates on larger tables.

Using OLD and NEW Qualifiers: Example

```
CREATE TABLE audit_emp (  
  user_name      VARCHAR2(30),  
  time_stamp     date,  
  id             NUMBER(6),  
  old_last_name  VARCHAR2(25),  
  new_last_name  VARCHAR2(25),  
  old_title      VARCHAR2(10),  
  new_title      VARCHAR2(10),  
  old_salary     NUMBER(8,2),  
  new_salary     NUMBER(8,2) )  
/  
CREATE OR REPLACE TRIGGER audit_emp_values  
AFTER DELETE OR INSERT OR UPDATE ON employees  
FOR EACH ROW  
BEGIN  
  INSERT INTO audit_emp(user_name, time_stamp, id,  
    old_last_name, new_last_name, old_title,  
    new_title, old_salary, new_salary)  
VALUES (USER, SYSDATE, :OLD.employee_id,  
  :OLD.last_name, :NEW.last_name, :OLD.job_id,  
  :NEW.job_id, :OLD.salary, :NEW.salary);  
END;
```

28

In the example in the slide, the `AUDIT_EMP_VALUES` trigger is created on the `EMPLOYEES` table. The trigger adds rows to a user table, `AUDIT_EMP`, logging a user's activity against the `EMPLOYEES` table. The trigger records the values of several columns both before and after the data changes by using the `OLD` and `NEW` qualifiers with the respective column name.

Using OLD and NEW Qualifiers: Example

```
INSERT INTO employees (employee_id, last_name, job_id, salary, email, hire_date)
VALUES (999, 'Temp emp', 'SA_REP', 6000, 'TEMPEMP', TRUNC(SYSDATE))
/
UPDATE employees
SET salary = 7000, last_name = 'Smith'
WHERE employee_id = 999
/
SELECT *
FROM audit_emp;
```



Query Result x

All Rows Fetched: 2 in 0.005 seconds

	USER_NAME	TIME_STAMP	ID	OLD_LAST_NAME	NEW_LAST_NAME	OLD_TITLE	NEW_TITLE	OLD_SALARY	NEW_SALARY
1	ORA61	03-SEP-16	(null)	(null)	Temp emp	(null)	SA_REP	(null)	6000
2	ORA61	03-SEP-16	999	Temp emp	Smith	SA_REP	SA_REP	6000	7000

29

The slide demonstrates two instances where the trigger `AUDIT_EMP_VALUES` is fired.

1. When you insert a row into the `EMPLOYEES` table, the trigger is fired and the first row is inserted into the `AUDIT_EMP` table.
2. When you update the salary of employee Smith, then the second row is inserted into the `AUDIT_EMP` table.

Using the WHEN Clause to Fire a Row Trigger Based on a Condition

```
CREATE OR REPLACE TRIGGER derive_commission_pct
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
WHEN (NEW.job_id = 'SA_REP')
BEGIN
    IF INSERTING THEN
        :NEW.commission_pct := 0;
    ELSIF :OLD.commission_pct IS NULL THEN
        :NEW.commission_pct := 0;
    ELSE
        :NEW.commission_pct := :OLD.commission_pct+0.05;
    END IF;
END;
/
```

30

Optionally, you can include a trigger restriction in the definition of a row trigger by specifying a Boolean SQL expression in a **WHEN** clause. If you include a **WHEN** clause in the trigger, then the expression in the **WHEN** clause is evaluated for each row that the trigger affects.

If the expression evaluates to **TRUE** for a row, then the trigger body executes on behalf of that row. However, if the expression evaluates to **FALSE** or **NOT TRUE** for a row (unknown, as with nulls), then the trigger body does not execute for that row. The evaluation of the **WHEN** clause does not have an effect on the execution of the triggering SQL statement (in other words, the triggering statement is not rolled back if the expression in a **WHEN** clause evaluates to **FALSE**).

Note: A **WHEN** clause cannot be included in the definition of a statement trigger.

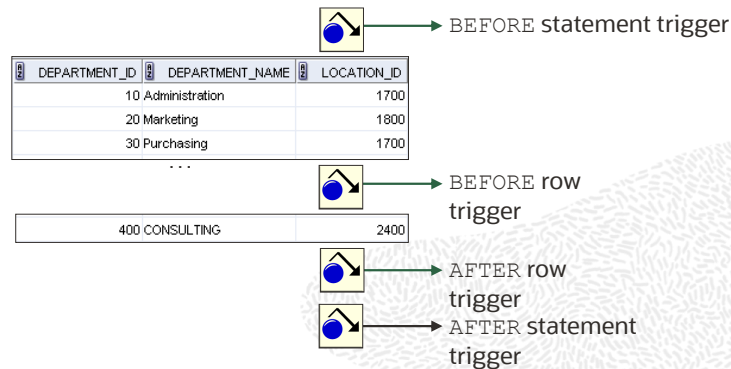
In the example in the slide, a trigger is created on the **EMPLOYEES** table to calculate an employee's commission when a row is added to the **EMPLOYEES** table, or when an employee's salary is modified.

The **NEW** qualifier cannot be prefixed with a colon in the **WHEN** clause because the **WHEN** clause is outside the PL/SQL blocks.

Trigger-Firing Sequence: Single-Row Manipulation

Use the following firing sequence for a trigger on a table when a single row is manipulated:

```
INSERT INTO departments
(department_id, department_name, location_id)
VALUES (400, 'CONSULTING', 2400);
```



31

You create a statement trigger when the statements in the trigger body have to execute only once for the event. You create a row-level trigger when the statements in the trigger body have to execute once for every row updated due to the trigger event.

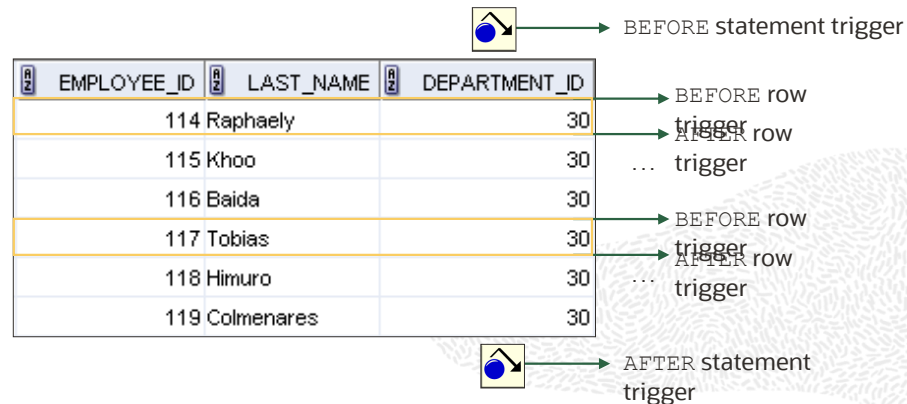
The slide shows the sequence of execution of multiple triggers when the triggering event effects only one row in a table. The INSERT statement in the slide inserts a new row into the table departments. Assuming that there are triggers defined on the INSERT event on the departments table for all the four instances (BEFORE, BEFORE with FOR EACH ROW, AFTER with FOR EACH ROW and AFTER). The sequence of execution of these triggers is:

1. Trigger defined to execute before the event
2. Trigger defined to execute before the modification of the effecting row
3. Trigger defined to execute after the modification of the effecting row
4. Trigger defined to execute after the event

Trigger-Firing Sequence: Multirow Manipulation

Use the following firing sequence for a trigger on a table when many rows are manipulated:

```
UPDATE employees
SET salary = salary * 1.1
WHERE department_id = 30;
```



32

When the triggering DML statement affects many rows, the statement trigger fires exactly once, and the row trigger fires once for every row affected by the statement.

Consider the SQL statement in the slide causes a row-level trigger to fire a number of times equal to the number of rows that satisfy the `WHERE` clause (that is, the number of employees reporting to department 30).

Assuming that there are triggers defined to execute on employees table for all the four instances (BEFORE, BEFORE with FOR EACH ROW, AFTER with FOR EACH ROW, AFTER). Then following is the order in which the triggers will execute:

1. The BEFORE trigger
2. The BEFORE...FOR EACH ROW trigger before the update of each row
3. The AFTER...FOR EACH ROW trigger after the update of each row
4. Steps 2 and 3 will repeat for the number of times equal to the number of rows being updated (In case of the query shown in the slide, equal to the number of employees in department number 30.)
5. The AFTER trigger

Summary of the Trigger Execution Model

1. Execute all `BEFORE STATEMENT` triggers.
2. Loop for each row affected by the SQL statement:
 - a. Execute all `BEFORE ROW` triggers *for that row*.
 - b. Execute the DML statement and perform integrity constraint checking *for that row*.
 - c. Execute all `AFTER ROW` triggers *for that row*.
3. Execute all `AFTER STATEMENT` triggers.

33

A single DML statement can potentially fire up to four types of triggers:

- `BEFORE` and `AFTER` statement triggers
- `BEFORE` and `AFTER` row triggers

A triggering event or a statement within the trigger can cause one or more integrity constraints to be checked. However, you can defer constraint checking until a `COMMIT` operation is performed.

Triggers can also cause other triggers, known as cascading triggers, to fire.

All actions and checks performed as a result of a SQL statement must succeed. If an exception is raised within a trigger and the exception is not explicitly handled, then all actions performed because of the original SQL statement are rolled back (including actions performed by firing triggers). This guarantees that integrity constraints can never be compromised by triggers.

When a trigger fires, the tables referenced in the trigger action may undergo changes by other users' transactions. In all cases, a read-consistent image is guaranteed for the modified values that the trigger needs to read (query) or write (update).

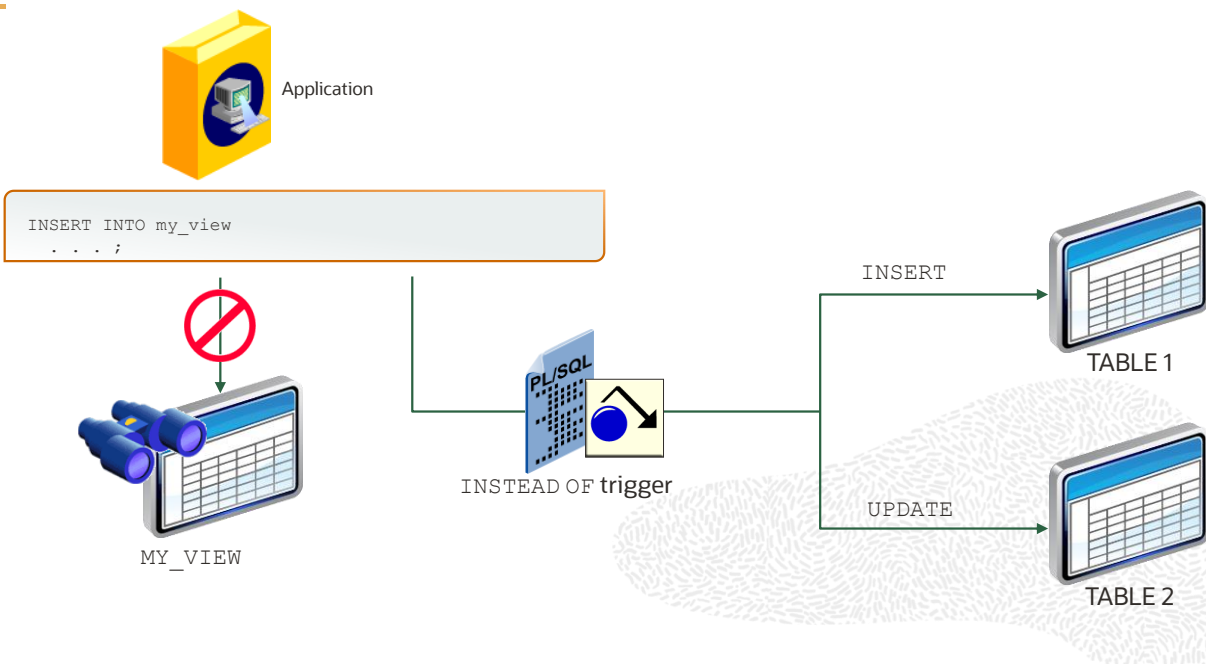
Note: Integrity checking can be deferred until the `COMMIT` operation is performed.

Lesson Agenda

- Understanding the usage of triggers
- Creating triggers
- Creating row-level triggers
- Creating INSTEAD OF triggers
- Managing Triggers



INSTEAD OF Triggers



35

Use `INSTEAD OF` triggers to modify data in which the DML statement has been issued against an inherently unupdatable view. These triggers are called `INSTEAD OF` triggers because, unlike other triggers, the Oracle server fires the trigger instead of executing the triggering statement. These triggers are used to perform `INSERT`, `UPDATE`, and `DELETE` operations directly on the underlying tables.

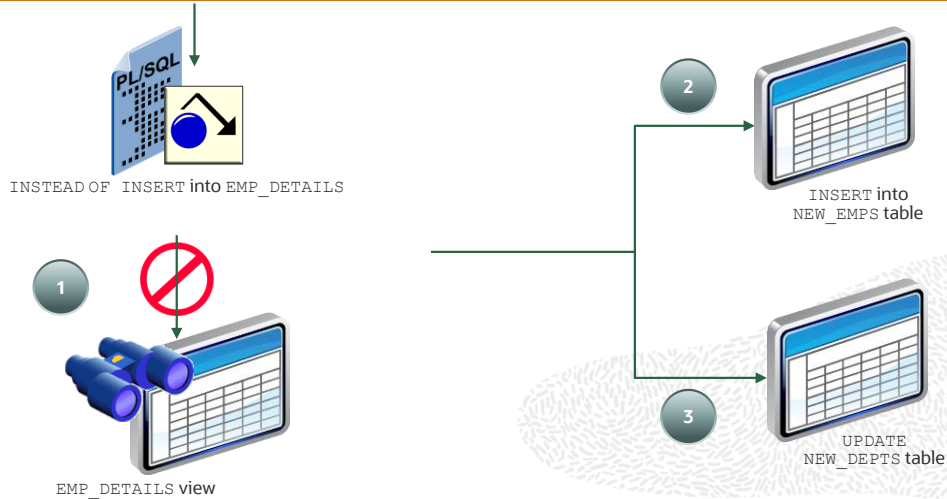
You can write `INSERT`, `UPDATE`, and `DELETE` statements against a view, and the `INSTEAD OF` trigger works invisibly in the background to make the right actions take place. A view cannot be modified by normal DML statements if the view query contains set operators, group functions, clauses such as `GROUP BY`, `CONNECT BY`, `START`, the `DISTINCT` operator, or joins.

For example, if a view consists of more than one table, an insert to the view may entail an insertion into one table and an update to another. So you write an `INSTEAD OF` trigger that fires when you write an insert against the view. Instead of the original insertion, the trigger body executes, which results in an insertion of data into one table and an update to another table.

Note: If a view is inherently updatable and has `INSTEAD OF` triggers, then the triggers take precedence. `INSTEAD OF` triggers are row triggers. The `CHECK` option for views is not enforced when insertions or updates to the view are performed by using `INSTEAD OF` triggers. The `INSTEAD OF` trigger body must enforce the check.

Creating an INSTEAD OF Trigger: Example

```
INSERT INTO emp_details  
VALUES (9001,'ABBOTT',3000, 10, 'Administration');
```



36

You can create an INSTEAD OF trigger in order to maintain the base tables on which a view is based.

The example in the slide illustrates an employee's details being inserted into the view EMP_DETAILS, whose query is based on the EMPLOYEES and DEPARTMENTS tables. The NEW_EMP_DEPT (INSTEAD OF) trigger executes in place of the INSERT operation that causes the trigger to fire. The INSTEAD OF trigger then issues the appropriate INSERT and UPDATE to the base tables used by the EMP_DETAILS view. Therefore, instead of inserting the new employee record into the EMPLOYEES table, the following actions take place:

1. The NEW_EMP_DEPT INSTEAD OF trigger fires.
2. A row is inserted into the NEW_EMPS table.
3. The DEPT_SAL column of the NEW_DEPTS table is updated. The salary value supplied for the new employee is added to the existing total salary of the department to which the new employee has been assigned.

Note: Before you run the example in the slide, you must create the required structures shown in the next two slides.

Creating an INSTEAD OF Trigger to Perform DML on Complex Views

```
CREATE TABLE new_emps AS
  SELECT employee_id, last_name, salary, department_id
     FROM employees;

CREATE TABLE new_depts AS
  SELECT d.department_id, d.department_name,
         sum(e.salary) dept_sal
     FROM employees e, departments d
    WHERE e.department_id = d.department_id;

CREATE VIEW emp_details AS
  SELECT e.employee_id, e.last_name, e.salary,
         e.department_id, d.department_name
     FROM employees e, departments d
    WHERE e.department_id = d.department_id
    GROUP BY d.department_id, d.department_name;
```

37

The example in the slide creates two new tables, `NEW_EMPS` and `NEW_DEPTS`, that are based on the `EMPLOYEES` and `DEPARTMENTS` tables, respectively. It also creates an `EMP_DETAILS` view from the `EMPLOYEES` and `DEPARTMENTS` tables.

If a view has a complex query structure, it is not always possible to perform DML directly on the view to affect the underlying tables. The example requires creation of an `INSTEAD OF` trigger, called `NEW_EMP_DEPT`, shown on the next page. The `NEW_DEPT_EMP` trigger handles DML in the following way:

- When a row is inserted into the `EMP_DETAILS` view, instead of inserting the row directly into the view, rows are added into the `NEW_EMPS` and `NEW_DEPTS` tables, by using the data values supplied with the `INSERT` statement.
- When a row is modified or deleted through the `EMP_DETAILS` view, corresponding rows in the `NEW_EMPS` and `NEW_DEPTS` tables are affected.

Note: `INSTEAD OF` triggers can be written only for views, and the `BEFORE` and `AFTER` timing options are not valid.

```

CREATE OR REPLACE TRIGGER new_emp_dept
INSTEAD OF INSERT OR UPDATE OR DELETE ON emp_details
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO new_emps
        VALUES (:NEW.employee_id, :NEW.last_name,
                :NEW.salary, :NEW.department_id);
        UPDATE new_depts
        SET dept_sal = dept_sal + :NEW.salary
        WHERE department_id = :NEW.department_id;
    ELSIF DELETING THEN
        DELETE FROM new_emps
        WHERE employee_id = :OLD.employee_id;
        UPDATE new_depts
        SET dept_sal = dept_sal - :OLD.salary
        WHERE department_id = :OLD.department_id;
    ELSIF UPDATING ('salary') THEN
        UPDATE new_emps
        SET salary = :NEW.salary
        WHERE employee_id = :OLD.employee_id;
        UPDATE new_depts
        SET dept_sal = dept_sal +
            (:NEW.salary - :OLD.salary)
        WHERE department_id = :OLD.department_id;
    ELSIF UPDATING ('department_id') THEN
        UPDATE new_emps
        SET department_id = :NEW.department_id
        WHERE employee_id = :OLD.employee_id;
        UPDATE new_depts
        SET dept_sal = dept_sal - :OLD.salary
        WHERE department_id = :OLD.department_id;
        UPDATE new_depts
        SET dept_sal = dept_sal + :NEW.salary
        WHERE department_id = :NEW.department_id;
    END IF;
END;
/

```

You can check the execution of the trigger by executing the following statements:

```

insert into emp_details values(999,'Himuro',3000,10,'Administration');
Select * from new_emps;
Select * from new_depts;

```

Lesson Agenda

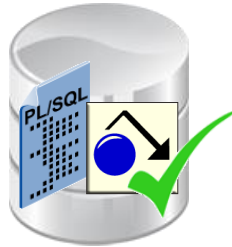
- Understanding the usage of triggers
- Creating triggers
- Creating row-level triggers
- Creating INSTEAD OF triggers
- Managing Triggers



The Status of a Trigger

A trigger can be in either of the following distinct modes:

- **Enabled:** The trigger runs its trigger action if a triggering statement is issued and the trigger restriction (if any) evaluates to true (default).
- **Disabled:** The trigger does not run its trigger action, even if a triggering statement is issued and the trigger restriction (if any) would evaluate to true.



40

A trigger is in either of two distinct modes:

- **Enabled:** The trigger runs its trigger action if a triggering statement is issued and the trigger restriction (if any) evaluates to true (default).
- **Disabled:** The trigger does not run its trigger action, even if a triggering statement is issued and the trigger restriction (if any) would evaluate to true.

System Privileges Required to Manage Triggers

The following system privileges are required to manage triggers:

- The privileges that enable you to create, alter, and drop triggers in any schema:
 - `GRANT CREATE TRIGGER TO ora61`
 - `GRANT ALTER ANY TRIGGER TO ora61`
 - `GRANT DROP ANY TRIGGER TO ora61`
- The privilege that enables you to create a trigger on the database:
 - `GRANT ADMINISTER DATABASE TRIGGER TO ora61`
- The `EXECUTE` privilege (if your trigger refers to any objects that are not in your schema)

41

To create a trigger in your schema, you need the `CREATE TRIGGER` system privilege, and you must own the table specified in the triggering statement, have the `ALTER` privilege for the table in the triggering statement, or have the `ALTER ANY TABLE` system privilege. You can alter or drop your triggers without any further privileges being required.

If the `ANY` keyword is used, you can create, alter, or drop your own triggers and those in another schema and can be associated with any user's table.

You do not need any privileges to invoke a trigger in your schema. A trigger is invoked by DML statements that you issue. But if your trigger refers to any objects that are not in your schema, the user creating the trigger must have the `EXECUTE` privilege on the referenced procedures, functions, or packages, and not through roles.

To create a trigger on `DATABASE`, you must have the `ADMINISTER DATABASE TRIGGER` privilege. If this privilege is later revoked, you can drop the trigger but you cannot alter it.

Note: Similar to stored procedures, statements in the trigger body use the privileges of the trigger owner, and not the privileges of the user executing the operation that fires the trigger.

Managing Triggers by Using the ALTER and DROP SQL Statements

```
-- Disable or reenable a database trigger:  
ALTER TRIGGER trigger_name DISABLE | ENABLE;
```

```
-- Disable or reenable all triggers for a table:  
ALTER TABLE table_name DISABLE | ENABLE ALL TRIGGERS;
```

```
-- Recompile a trigger for a table:  
ALTER TRIGGER trigger_name COMPILE;
```

```
-- Remove a trigger from the database:  
DROP TRIGGER trigger_name;
```

42

Managing Triggers

A trigger has two modes or states: `ENABLED` and `DISABLED`. When a trigger is first created, it is enabled by default. You can create a disabled trigger on creation by adding `DISABLE`, as shown below:

```
CREATE OR REPLACE TRIGGER trigger_name  
BEFORE INSERT ON table_name FOR EACH ROW DISABLE
```

A disabled trigger wouldn't fire even if the triggering event occurs.

The Oracle server checks integrity constraints for enabled triggers and guarantees that triggers cannot compromise them.

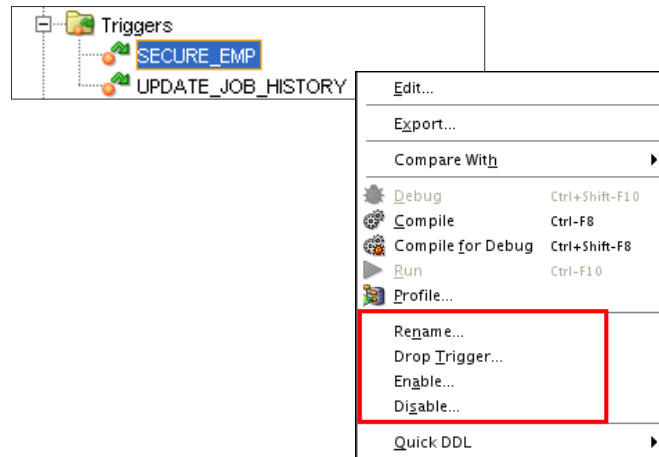
Disabling a Trigger

Use the `ALTER TRIGGER` command to disable a trigger. You can also disable all triggers on a table by using the `ALTER TABLE` command. You can disable triggers to improve performance or to avoid data integrity checks when loading massive amounts of data with utilities such as `SQL*Loader`.

Removing Triggers

When a trigger is no longer required, use `DROP TRIGGER` SQL statement in SQL Developer or `SQL*Plus` to remove it. When you remove a table, all triggers on that table are also removed.

Managing Triggers by Using SQL Developer



43

You can use the Triggers node in the Connections navigation tree to manage triggers. Right-click a trigger name, and then select one of the following options:

- Edit
- Compile
- Compile for Debug
- Rename
- Drop Trigger
- Enable
- Disable

Viewing Trigger Information

You can view the following trigger information:

Data Dictionary View	Description
USER_OBJECTS	Displays object information
USER/ALL/DBA_TRIGGERS	Displays trigger information
USER_ERRORS	Displays PL/SQL syntax errors for a trigger

44

The slide shows the data dictionary views that you can access to get information regarding the triggers.

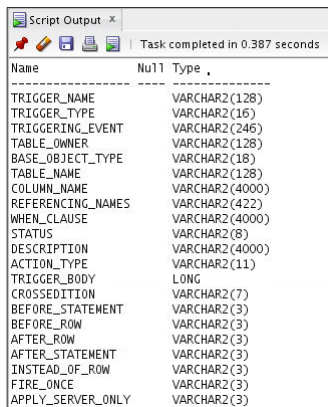
The `USER_OBJECTS` view contains the name and status of the trigger and the date and time when the trigger was created.

The `USER_ERRORS` view contains the details about the compilation errors that occurred while a trigger was compiling. The contents of these views are similar to those for subprograms.

The `USER_TRIGGERS` view contains details such as name, type, triggering event, the table on which the trigger is created, and the body of the trigger.

Using USER_TRIGGERS

```
DESCRIBE user_triggers
```



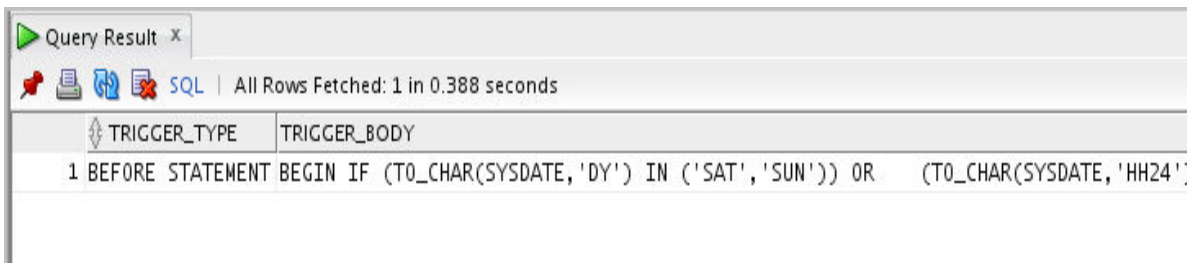
Script Output x
Task completed in 0.387 seconds

Name	Null	Type
TRIGGER_NAME		VARCHAR2(128)
TRIGGER_TYPE		VARCHAR2(16)
TRIGGERING_EVENT		VARCHAR2(246)
TABLE_OWNER		VARCHAR2(128)
BASE_OBJECT_TYPE		VARCHAR2(18)
TABLE_NAME		VARCHAR2(128)
COLUMN_NAME		VARCHAR2(4000)
REFERENCING_NAMES		VARCHAR2(422)
WHEN_CLAUSE		VARCHAR2(4000)
STATUS		VARCHAR2(8)
DESCRIPTION		VARCHAR2(4000)
ACTION_TYPE		VARCHAR2(11)
TRIGGER_BODY		LONG
CROSSITION		VARCHAR2(7)
BEFORE_STATEMENT		VARCHAR2(3)
BEFORE_ROW		VARCHAR2(3)
AFTER_ROW		VARCHAR2(3)
AFTER_STATEMENT		VARCHAR2(3)
INSTEAD_OF_ROW		VARCHAR2(3)
FIRE_ONCE		VARCHAR2(3)
APPLY_SERVER_ONLY		VARCHAR2(3)

```
SELECT trigger_type, trigger_body  
FROM user_triggers  
WHERE trigger_name = 'SECURE_EMP';
```

45

If the source file is unavailable, you can use the SQL Worksheet in SQL Developer or SQL*Plus to regenerate it from USER_TRIGGERS. You can also examine the ALL_TRIGGERS and DBA_TRIGGERS views, each of which contains the additional column OWNER, for the owner of the object. The result for the second example in the slide is as follows:



Query Result x
SQL | All Rows Fetched: 1 in 0.388 seconds

TRIGGER_TYPE	TRIGGER_BODY
1 BEFORE STATEMENT	BEGIN IF (TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN')) OR (TO_CHAR(SYSDATE, 'HH24')

The TRIGGER_BODY column has the code of the trigger.

For Instructor Use Only.
This document should not be distributed.

Testing Triggers

- Test each triggering and non-triggering data operation.
- Test each case of the `WHEN` clause.
- Make the trigger fire directly from a basic data operation as well as indirectly from a procedure.
- Test the effect of the trigger on other triggers.
- Test the effect of other triggers on the trigger.

46

Testing code can be a time-consuming process. Perform the following when you are testing triggers:

- Ensure that the trigger works properly by testing a number of cases separately:
 - Test the most common success scenarios first.
 - Test the most common failure conditions to see that they are properly managed.
- The more complex the trigger, the more detailed your testing is likely to be. For example, if you have a row trigger with a `WHEN` clause specified, then you should ensure that the trigger fires when the conditions are satisfied. Or, if you have cascading triggers, you need to test the effect of one trigger on the other and ensure that you end up with the desired results.
- Use the `DBMS_OUTPUT` package to debug triggers.

Quiz



A triggering event can be one or more of the following:

- a. An `INSERT`, `UPDATE`, or `DELETE` statement on a specific table (or view, in some cases)
- b. A `CREATE`, `ALTER`, or `DROP` statement on any schema object
- c. A database startup or instance shutdown
- d. A specific error message or any error message
- e. A user logon or logoff

47

Answer: a, b, c, d, e

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Create database triggers that are invoked by DML operations
- Create statement and row trigger types
- Use database trigger-firing rules
- Enable, disable, and manage database triggers
- Develop a strategy for testing triggers
- Remove database triggers



48

This lesson covered creating database triggers that execute before, after, or instead of a specified DML operation. Triggers are associated with database tables or views. The `BEFORE` and `AFTER` timings apply to DML operations on tables. The `INSTEAD OF` trigger is used as a way to replace DML operations on a view with appropriate DML statements against other tables in the database.

Triggers are enabled by default but can be disabled to suppress their operation until enabled again. If business rules change, triggers can be removed or altered as required.

For Instructor Use Only.
This document should not be distributed.

Practice 18 Overview: Creating Statement and Row Triggers

This practice covers the following topics:

- Creating row triggers
- Creating a statement trigger
- Calling procedures from a trigger



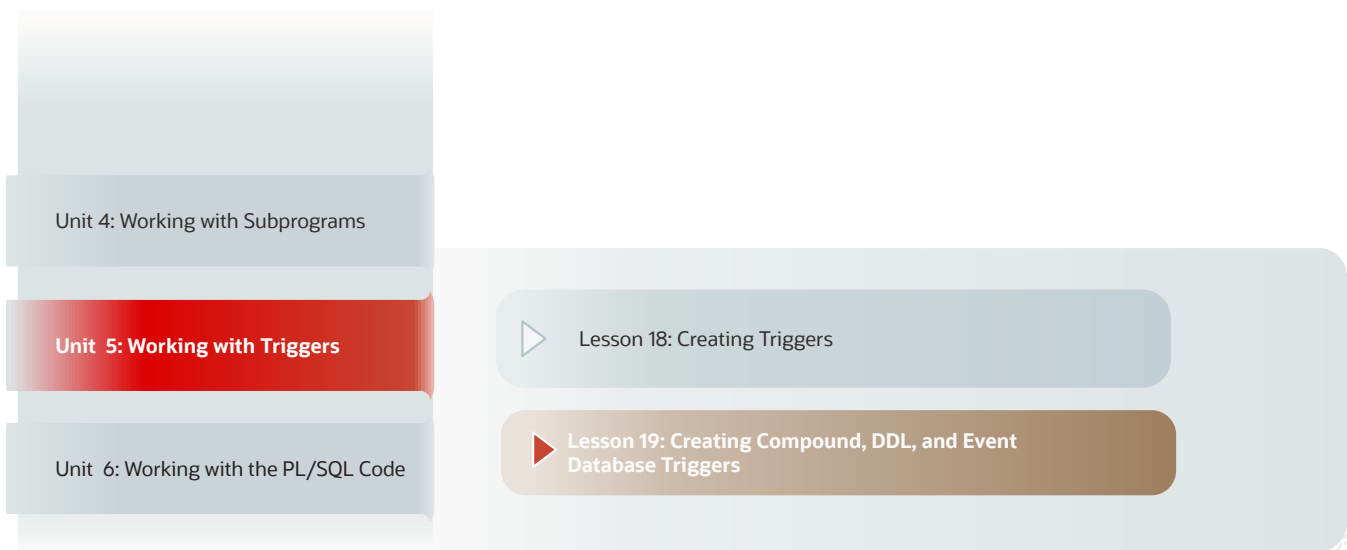
0

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Creating Compound, DDL, and Event Database Triggers

Course Road Map



2

In Unit 5, you learn to create and use triggers in database applications.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Describe compound triggers
- Describe mutating tables
- Create triggers on DDL statements
- Create triggers on system events
- Display information about triggers



0

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Create compound DML triggers
- Resolve mutating table errors
- Schema triggers
- Database triggers
- Guidelines for designing triggers



What is a Compound Trigger?

- A single trigger on a table that allows you to specify actions for each of the following four timing points:
 - **BEFORE**
 - **BEFORE EACH ROW**
 - **AFTER EACH ROW**
 - **AFTER**
- Each timing instance has an executable part and an optional exception handling section.
- All the timing instances access a common PL/SQL state.

5

A compound trigger is a single trigger on a table that allows you to specify actions for each of the four triggering timing points:

- Before the triggering statement
- Before each row that the triggering statement affects
- After each row that the triggering statement affects
- After the triggering statement

You use a compound trigger when you want all the timing instances to access a common PL/SQL state. The common PL/SQL state is established when the triggering statement starts and is destroyed when the triggering statement completes.

However, in some situations where you may have to write DML statements in the trigger body, you can use compound triggers.

Working with Compound Triggers

- The compound trigger body supports a common PL/SQL state that the code for each timing point can access.
- The compound trigger common state is:
 - Established when the triggering statement starts
 - Destroyed when the triggering statement completes
- A compound trigger has a declaration section and a section for each of its timing points.
- The section of each timing point may have an optional exception handling section

6

The compound trigger body supports a common PL/SQL state that the code for each timing point can access. The common state is automatically destroyed when the firing statement completes, even when the firing statement causes an error.

Your applications can avoid the mutating table error by allowing rows destined for a second table (such as a history table or an audit table) to accumulate and then bulk-insert them.

Compound triggers make PL/SQL easier for you to use and improve runtime performance and scalability.

Why Compound Triggers?

You can use compound triggers to:

- Program an approach where you want the actions you implement for the various timing points to share common data
- Accumulate rows destined for a second table so that you can periodically bulk-insert them
- Avoid the mutating-table error (ORA-04091)

7

The current PL/SQL state is stored before starting the execution of the trigger and destroyed after completing the execution of the trigger. All the trigger timing points can access the same PL/SQL state in such scenario.

If you are inserting values into a second table while executing the trigger body, compound triggers allow you to accumulate all the newly inserted rows and bulk insert them into the table at periodic intervals. Thus improving the performance of the PL/SQL block.

What is ORA-4091?

Consider a trigger, you have defined for event `BEFORE INSERT` on a table `T`. In the trigger body definition, you execute a DML statement on the same table `T`, such action would result in `ORA-4091: table T mutating, trigger/function may not see it`. The trigger execution here is initiated to modify the table. When you try to modify it again in the trigger body, it might lead to an inconsistent state, thereby giving the `ORA-4091` error.

Compound Trigger Structure

```
CREATE OR REPLACE TRIGGER schema.trigger
FOR dml_event_clause ON schema.table
COMPOUND TRIGGER
[ declare_section ]
timing_point_section [ timing_point_section ]... END [ trigger ] ;
```

8

The slide illustrates the structure of a compound trigger. The difference between a simple trigger and a compound trigger syntax is the `COMPOUND TRIGGER` clause.

The optional declarative part of a compound trigger declares variables and subprograms that all of its timing-point sections can use. When the trigger fires, the declarative part runs before any timing-point sections run, thus creating the PL/SQL state of the trigger. The variables and subprograms exist for the duration of the triggering statement.

The compound triggers further have the timing point sections for the timing points – `BEFORE`, `BEFORE FOR EACH ROW`, `AFTER`, `AFTER FOR EACH ROW`.

Compound Trigger Structure for Views

```
CREATE trigger FOR dml_event_clause ON view
COMPOUND TRIGGER
INSTEAD OF EACH ROW IS
BEGIN
    statement;
END INSTEAD OF EACH ROW;
```

9

A compound trigger defined on a view has only an `INSTEAD OF` timing point section defined in the trigger body.

Compound Trigger Restrictions

- A compound trigger must be a DML trigger defined on either a table or a view.
- An exception that occurs in one section must be handled in that section. It cannot transfer control to another section.
- A timing-point section cannot handle exceptions raised in another timing-point section.
- `:OLD` and `:NEW` cannot appear in the declaration, `BEFORE STATEMENT`, or the `AFTER STATEMENT` sections.
- Only the `BEFORE EACH ROW` section can change the value of `:NEW`.

10

Following are some of the restrictions when working with compound triggers:

- A compound trigger must be a DML trigger.
- A compound trigger must be defined on either a table or a view.
- An exception that occurs in one section must be handled in that section. It cannot transfer control to another section.
- `:OLD`, `:NEW`, and `:PARENT` cannot appear in the declaration section, the `BEFORE STATEMENT` section, or the `AFTER STATEMENT` section.

Lesson Agenda

- Create compound DML triggers
- Resolve mutating table errors
- Schema triggers
- Database triggers
- Guidelines for designing triggers



For Instructor Use Only.
This document should not be distributed.

Mutating Tables

- A mutating table is:
 - A table that is being modified by an `UPDATE`, `DELETE`, or `INSERT` statement
 - or
 - A table that might be updated by the effects of a `DELETE CASCADE` constraint
- The session that issued the triggering statement cannot query or modify a mutating table.
- This restriction prevents a trigger from seeing inconsistent data.
- This restriction applies to all triggers that use the `FOR EACH ROW` clause.
- Views being modified in the `INSTEAD OF` triggers are not considered mutating.

12

Rules Governing Triggers

Reading and writing data by using triggers is subject to certain rules. The restrictions apply only to row triggers, unless a statement trigger is fired as a result of `ON DELETE CASCADE`.

Mutating Tables

A mutating table is a table that is currently being modified by an `UPDATE`, `DELETE`, or `INSERT` statement, or a table that might need to be updated by the effects of a declarative `DELETE CASCADE` referential integrity action. For `STATEMENT` triggers, a table is not considered a mutating table.

A mutating table error (`ORA-4091`) occurs when a row-level trigger attempts to change or examine a table that is already undergoing change via a DML statement.

The triggered table itself is a mutating table, as well as any table referencing it with the `FOREIGN KEY` constraint. This restriction prevents a row trigger from seeing an inconsistent set of data.

Mutating Table: Example

```
CREATE OR REPLACE TRIGGER check_salary
  BEFORE INSERT OR UPDATE OF salary, job_id
  ON employees
  FOR EACH ROW
  WHEN (NEW.job_id <> 'AD_PRES')
DECLARE
  v_minsalary employees.salary%TYPE;
  v_maxsalary employees.salary%TYPE;
BEGIN
  SELECT MIN(salary), MAX(salary)
    INTO   v_minsalary, v_maxsalary
  FROM     employees
  WHERE    job_id = :NEW.job_id;
  IF :NEW.salary < v_minsalary OR :NEW.salary > v_maxsalary THEN
    RAISE_APPLICATION_ERROR(-20505, 'Out of range');
  END IF;
END;
/
```

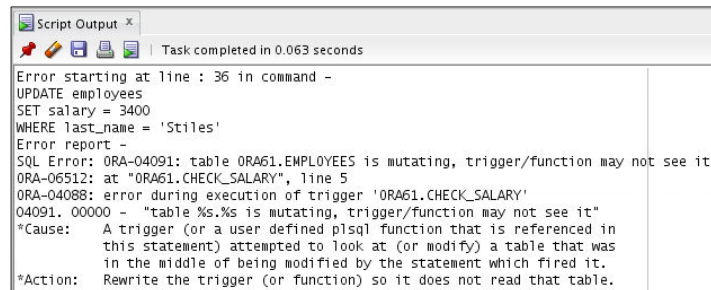
13

The `CHECK_SALARY` trigger in the slide example attempts to guarantee that whenever a new employee is added to the `EMPLOYEES` table or whenever an existing employee's salary or job ID is changed, the employee's salary falls within the established salary range for the employee's job.

When an employee record is updated, the `CHECK_SALARY` trigger is fired for each row that is updated. The trigger code queries the same table that is being updated. Therefore, it is said that the `EMPLOYEES` table is a mutating table.

Mutating Table: Example

```
UPDATE employees
SET salary = 3400
WHERE last_name = 'Stiles';
```

A screenshot of a 'Script Output' window from a database client. The window title is 'Script Output' with a close button. Below the title bar, there's a status bar that says 'Task completed in 0.063 seconds'. The main area displays an error report. It starts with 'Error starting at line : 36 in command -' followed by the SQL command: 'UPDATE employees', 'SET salary = 3400', and 'WHERE last_name = 'Stiles''. Below this, it says 'Error report -'. The error details include: 'SQL Error: ORA-04091: table ORA61.EMPLOYEES is mutating, trigger/function may not see it', 'ORA-06512: at "ORA61.CHECK_SALARY", line 5', and 'ORA-04088: error during execution of trigger 'ORA61.CHECK_SALARY''. A detailed message follows: 'ORA-04091. 00000 - "table %s.%s is mutating, trigger/function may not see it"', followed by the cause: '*Cause: A trigger (or a user defined plsql function that is referenced in this statement) attempted to look at (or modify) a table that was in the middle of being modified by the statement which fired it.' and the action: '*Action: Rewrite the trigger (or function) so it does not read that table.'

```
Script Output x
Task completed in 0.063 seconds

Error starting at line : 36 in command -
UPDATE employees
SET salary = 3400
WHERE last_name = 'Stiles'
Error report -
SQL Error: ORA-04091: table ORA61.EMPLOYEES is mutating, trigger/function may not see it
ORA-06512: at "ORA61.CHECK_SALARY", line 5
ORA-04088: error during execution of trigger 'ORA61.CHECK_SALARY'
ORA-04091. 00000 - "table %s.%s is mutating, trigger/function may not see it"
*Cause: A trigger (or a user defined plsql function that is referenced in
this statement) attempted to look at (or modify) a table that was
in the middle of being modified by the statement which fired it.
*Action: Rewrite the trigger (or function) so it does not read that table.
```

14

In the example in the slide, the trigger code tries to read or select data from a mutating table.

If you restrict the salary within a range between the minimum existing value and the maximum existing value, then you get a runtime error. The `EMPLOYEES` table is mutating or is in a state of change; therefore, the trigger cannot read from it.

Remember that functions can also cause a mutating table error when they are invoked in a DML statement.

Possible Solutions

Possible solutions to this mutating table problem include the following:

- Use a compound trigger as described earlier in this lesson.
- Store the summary data (the minimum salaries and the maximum salaries) in another summary table, which is kept up-to-date with other DML triggers.
- Store the summary data in a PL/SQL package, and access the data from the package. This can be done in a `BEFORE` statement trigger.

Using a Compound Trigger to Resolve the Mutating Table Error

```
CREATE OR REPLACE TRIGGER check_salary
FOR INSERT OR UPDATE OF salary, job_id
ON employees
WHEN (NEW.job_id <> 'AD_PRES')
COMPOUND TRIGGER

  TYPE salaries_t          IS TABLE OF employees.salary%TYPE;
  min_salaries             salaries_t;
  max_salaries             salaries_t;

  TYPE department_ids_t    IS TABLE OF employees.department_id%TYPE;
  department_ids           department_ids_t;

  TYPE department_salaries_t IS TABLE OF employees.salary%TYPE
                           INDEX BY VARCHAR2(80);
  department_min_salaries  department_salaries_t;
  department_max_salaries  department_salaries_t;

  -- example continues on next slide
```

15

The `CHECK_SALARY` compound trigger resolves the mutating table error in the earlier example. This is achieved by storing the values in PL/SQL collections, and then performing a bulk insert/update in the “before statement” section of the compound trigger.

In the example in the slide, PL/SQL collections are used. The element types used are based on the `SALARY` and `DEPARTMENT_ID` columns from the `EMPLOYEES` table.

To create collections, you define a collection type and then declare variables of that type. Collections are instantiated when you enter a block or subprogram, and cease to exist when you exit. `min_salaries` is used to hold the minimum salary for each department and `max_salaries` is used to hold the maximum salary for each department.

Using a Compound Trigger to Resolve the Mutating Table Error

```
...  
BEFORE STATEMENT IS  
BEGIN  
    SELECT MIN(salary), MAX(salary), NVL(department_id, -1)  
    BULK COLLECT INTO min_salaries, max_salaries, department_ids  
    FROM employees  
    GROUP BY department_id;  
    FOR j IN 1..department_ids.COUNT() LOOP  
        department_min_salaries(department_ids(j)) := min_salaries(j);  
        department_max_salaries(department_ids(j)) := max_salaries(j);  
    END LOOP;  
END BEFORE STATEMENT;  
AFTER EACH ROW IS  
BEGIN  
    IF :NEW.salary < department_min_salaries(:NEW.department_id)  
    OR :NEW.salary > department_max_salaries(:NEW.department_id) THEN  
        RAISE_APPLICATION_ERROR(-20505, 'New Salary is out of acceptable  
        range');  
    END IF;  
END AFTER EACH ROW;  
END check_salary;
```

16

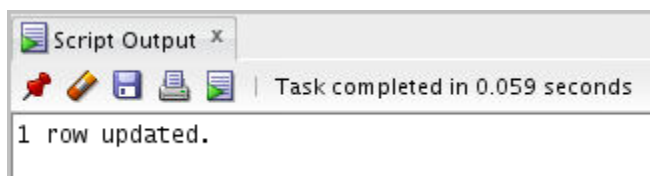
department_ids is used to hold the department IDs. If the employee who earns the minimum or maximum salary does not have an assigned department, you use the NVL function to store -1 for the department id, instead of NULL.

Next, you collect the minimum salary, maximum salary, and the department ID using a bulk insert into the min_salaries, max_salaries, and department_ids respectively grouped by department ID. The select statement returns 13 rows. The values of the department_ids are used as an index for the department_min_salaries and department_max_salaries tables. Therefore, the index for those two tables (VARCHAR2) represents the actual department_ids.

After each row is added, if the new salary is less than the minimum salary for that department or greater than the department's maximum salary, then an error message is displayed.

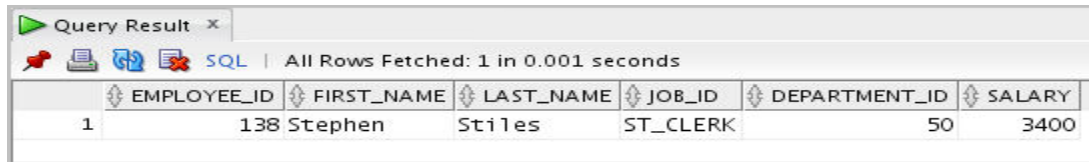
To test the newly created compound trigger, issue the following statement:

```
UPDATE employees  
SET salary = 3400  
WHERE last_name = 'Stiles';
```



To ensure that the salary for employee Stiles was updated, issue the following query by using the F9 key in SQL Developer:

```
SELECT employee_id, first_name,  
last_name, job_id, department_id,salary  
FROM employees  
WHERE last_name = 'Stiles';
```



The screenshot shows the 'Query Result' window in SQL Developer. It displays the results of the query for employee Stiles. The window title is 'Query Result x'. Below the title bar, there are icons for saving, refreshing, and other actions, followed by the text 'SQL | All Rows Fetched: 1 in 0.001 seconds'. The results are shown in a table with 7 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, JOB_ID, DEPARTMENT_ID, and SALARY. The first row shows the data for employee Stiles.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	JOB_ID	DEPARTMENT_ID	SALARY
1	138	Stephen	Stiles	ST_CLERK	50	3400

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Create compound DML triggers
- Resolve mutating table errors
- **Schema triggers**
- Database triggers
- Guidelines for designing triggers



Creating Triggers on DDL Statements

```
CREATE [OR REPLACE] TRIGGER trigger_name
BEFORE | AFTER -- Timing
[ddl_event1 [OR ddl_event2 OR ...]]
ON {DATABASE | SCHEMA}
trigger_body
```

Sample DDL Events	Fires When
CREATE	Any database object is created using the CREATE command.
ALTER	Any database object is altered using the ALTER command.
DROP	Any database object is dropped using the DROP command.

19

You can specify one or more types of DDL statements that can cause the trigger to fire. You can create triggers for these events on `DATABASE` or `SCHEMA` unless otherwise noted. You can also specify `BEFORE` and `AFTER` for the timing of the trigger. The Oracle database fires the trigger in the existing user transaction.

You cannot specify as a triggering event any DDL operation performed through a PL/SQL procedure.

The trigger body in the syntax in the slide represents a complete PL/SQL block.

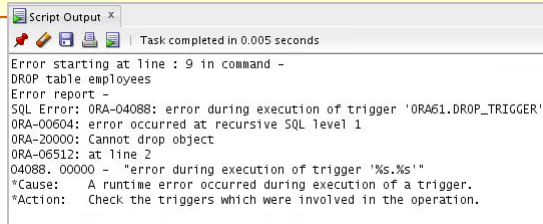
DDL triggers fire only if the object being created is a cluster, function, index, package, procedure, role, sequence, synonym, table, tablespace, trigger, type, view, or user.

Creating Triggers on DDL Statements -Example

```
CREATE OR REPLACE TRIGGER drop_trigger
BEFORE DROP ON ora61.SCHEMA
BEGIN
RAISE_APPLICATION_ERROR ( num => -20000, msg => 'Cannot drop object');
END;
/
```

To check the execution of this trigger

```
DROP TABLE employees;
```



Script Output x | Task completed in 0.005 seconds

Error starting at line : 9 in command -
DROP table employees

Error report -
SQL Error: ORA-04088: error during execution of trigger 'ORA61.DROP_TRIGGER'
ORA-00604: error occurred at recursive SQL level 1
ORA-20000: Cannot drop object
ORA-06512: at line 2
04088. 00000 - "error during execution of trigger '%s.%s'"
*Cause: A runtime error occurred during execution of a trigger.
*Action: Check the triggers which were involved in the operation.

Lesson Agenda

- Create compound DML triggers
- Resolve mutating table errors
- Schema triggers
- **Database triggers**
- Guidelines for designing triggers



For Instructor Use Only.
This document should not be distributed.

Creating Database Triggers

- Triggering user event:
 - Logging on or off
- Triggering a database event:
 - Shutting down or starting up the database
 - A specific error (or any error) being raised

22

Before coding the trigger body, decide on the components of the trigger.

Triggers on system events can be defined at the database or schema level. For example, a database shutdown trigger is defined at the database level. Triggers on DDL statements, or a user logging on or off, can also be defined at either the database level or the schema level. Triggers on DML statements are defined on a specific table or a view.

A trigger defined at the database level fires for all users, whereas a trigger defined at the schema or table level fires only when the triggering event involves that schema or table.

Creating Triggers on System Events

```
CREATE [OR REPLACE] TRIGGER trigger_name
BEFORE | AFTER -- timing
[database_event1 [OR database_event2 OR ...]]
ON {DATABASE | SCHEMA}
trigger_body
```

Database Event	Triggers Fires When
AFTER SERVERERROR	An Oracle error is raised
AFTER LOGON	A user logs on to the database
BEFORE LOGOFF	A user logs off the database
AFTER STARTUP	The database is opened
BEFORE SHUTDOWN	The database is shut down normally

23

You can create triggers for the events listed in the table in the slide on DATABASE or SCHEMA, except SHUTDOWN and STARTUP, which apply only to DATABASE.

LOGON and LOGOFF Triggers: Example

```
-- Create the log_trig_table shown in the notes page
-- first
CREATE OR REPLACE TRIGGER logon_trig
AFTER LOGON ON SCHEMA
BEGIN
    INSERT INTO log_trig_table(user_id,log_date,action)
    VALUES (USER, SYSDATE, 'Logging on');
END;
/
```

```
CREATE OR REPLACE TRIGGER logoff_trig
BEFORE LOGOFF ON SCHEMA
BEGIN
    INSERT INTO log_trig_table(user_id,log_date,action)
    VALUES (USER, SYSDATE, 'Logging off');
END;
/
```

24

You can create these triggers to monitor how often you log on and off, or you may want to write a report that monitors the length of time for which you are logged on. When you specify ON SCHEMA, the trigger fires for the specific user. If you specify ON DATABASE, the trigger fires for all users.

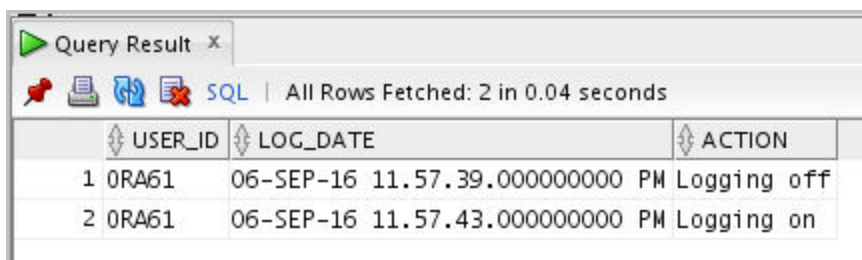
The definition of the log_trig_table used in the slide examples is as follows:

```
CREATE TABLE log_trig_table(
    user_id VARCHAR2(30),
    log_date TIMESTAMP,
    action VARCHAR2(40))
/
```

To see the execution of these triggers you can disconnect from the database and connect with the database. Then execute

```
select * from log_trig_table;
```

You can see the output as:



The screenshot shows a 'Query Result' window with the following data:

	USER_ID	LOG_DATE	ACTION
1	ORA61	06-SEP-16 11.57.39.000000000 PM	Logging off
2	ORA61	06-SEP-16 11.57.43.000000000 PM	Logging on

Lesson Agenda

- Create compound DML triggers
- Resolve mutating table errors
- Schema triggers
- Database triggers
- Guidelines for designing triggers



Guidelines for Designing Triggers

- You can design triggers to:
 - ensure that whenever a specific event occurs, any necessary actions are done.
- Don't create triggers:
 - which duplicate the function of the database
 - which are recursive
- You can create stored procedures and invoke them in a trigger, if the PL/SQL code is very lengthy.

26

- Use triggers to ensure that whenever a specific event occurs, any necessary actions are done.
For example, use a trigger to ensure that whenever anyone updates a table, its log file is updated.
- Don't create triggers that duplicate database features.
For example, don't create a trigger to reject invalid data if you can do the same with constraints.
- Don't create recursive triggers.
For example, don't create an AFTER UPDATE trigger that issues an UPDATE statement on the table on which the trigger is defined. The trigger fires recursively until it runs out of memory.

Quiz



Which of the following statements are true for a trigger?

- a. A trigger is defined with a `CREATE TRIGGER` statement.
- b. A trigger's source code is contained in the `USER_TRIGGERS` data dictionary.
- c. A trigger is explicitly invoked.
- d. A trigger is implicitly invoked by DML.
- e. `COMMIT`, `SAVEPOINT`, and `ROLLBACK` are not allowed when working with a trigger.

27

Answer: a, b, d, e

For Instructor Use Only.

This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Describe compound triggers
- Describe mutating tables
- Create triggers on DDL statements
- Create triggers on system events
- Display information about triggers



For Instructor Use Only.
This document should not be distributed.

Practice 19 Overview: Creating Compound, DDL, and Event Database Triggers

This practice covers the following topics:

- Creating advanced triggers to manage data integrity rules
- Creating triggers that cause a mutating table exception
- Creating triggers that use package state to solve the mutating table problem



29

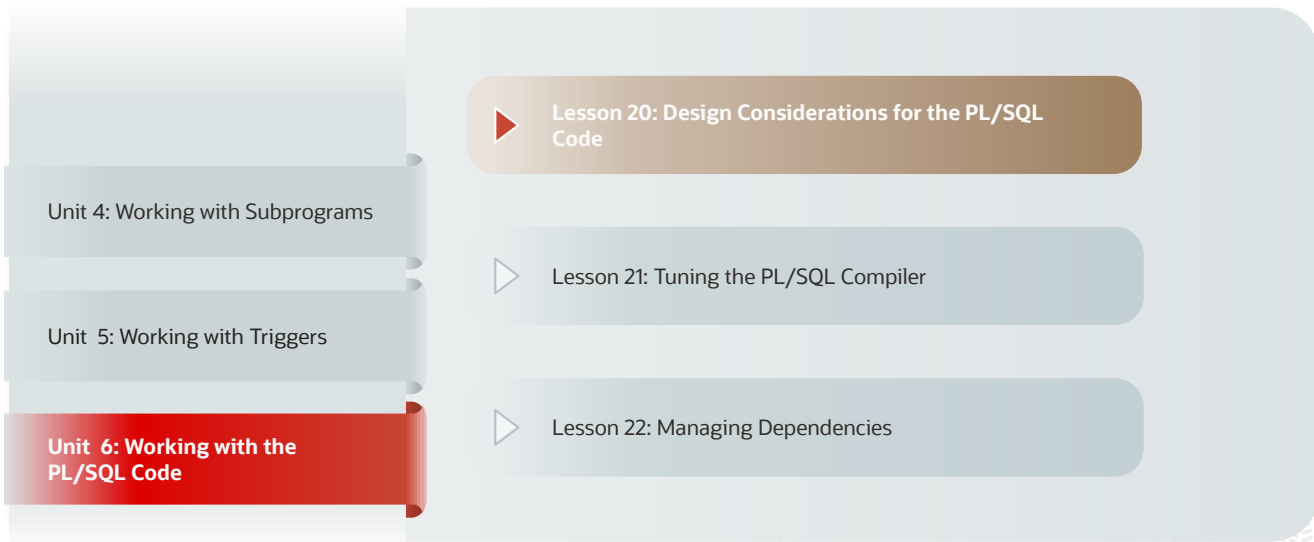
In this practice, you implement a simple business rule for ensuring data integrity of employees' salaries with respect to the valid salary range for their job. You create a trigger for this rule.

During this process, your new triggers cause a cascading effect with triggers created in the practice section of the previous lesson. The cascading effect results in a mutating table exception on the `JOBS` table. You then create a PL/SQL package and additional triggers to solve the mutating table issue.

For Instructor Use Only.
This document should not be distributed.

Design Considerations for the PL/SQL Code

Course Road Map



2

In Unit 6, you will learn the design aspects of the PL/SQL code. You will also learn how to use the PL/SQL compiler and how to manage dependencies among different objects in the database application.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Write standard PL/SQL code
- Grant and control runtime privileges of subprograms
- Create and use autonomous transactions
- Use `NOCOPY`, `PARALLEL_ENABLE`, and `DETERMINISTIC` clauses for optimization
- Use result caching for optimization
- Use bulk binding for optimization



0

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Standardization of the code
- Managing security for PL/SQL packages and subprograms
- Design considerations for autonomous transactions
- Performance optimization in PL/SQL blocks
 - NO COPY clause
 - PARALLEL_ENABLE clause
 - RESULT_CACHE clause
 - DETERMINISTIC clause
 - RETURNING clause
- Bulk Binding

4



For Instructor Use Only.

0

This document should not be distributed.

Standardizing Constants and Exceptions

- Standardizing helps to:
 - Develop programs that are consistent
 - Promote a higher degree of code reuse
 - Ease code maintenance
 - Implement company standards across entire applications
- Constants and exceptions are typically implemented by using a bodiless package (that is, a package specification).
- Start with standardization of:
 - Exception names
 - Constant definitions

5

When several developers are writing their own exception handlers in an application, there could be inconsistencies in the handling of error situations. Unless certain standards are adhered to, the situation can become confusing because of the different approaches followed in handling the same error or because of the display of conflicting error messages that confuse users. To overcome these, you can:

- Implement company standards that use a consistent approach to error handling across the entire application
- Create predefined, generic exception handlers that produce consistency in the application
- Write and call programs that produce consistent error messages

Good programming environments promote naming and coding standards. In PL/SQL, a good place to start implementing naming and coding standards is with commonly used constants and exceptions that occur in the application domain.

The PL/SQL package specification construct is an excellent component to support standardization because all identifiers declared in the package specification are public. They are visible to the subprograms that are developed by the owner of the package and all code with `EXECUTE` rights to the package specification.

Standardizing Exceptions

Create a standardized error-handling package that includes all named and programmer-defined exceptions to be used in the application.

```
CREATE OR REPLACE PACKAGE error_pkg IS
  e_fk_err          EXCEPTION;
  e_seq_nbr_err     EXCEPTION;
  PRAGMA EXCEPTION_INIT (e_fk_err, -2292);
  PRAGMA EXCEPTION_INIT (e_seq_nbr_err, -2277);
  ...
END error_pkg;
/
```

6

In the slide, the `error_pkg` package is a standardized exception package. It declares a set of programmer-defined exception identifiers. An internally defined exception does not have a name unless either PL/SQL or you give it one. The example shown in the slide uses the `PRAGMA EXCEPTION_INIT` directive to associate selected exception names with an Oracle database error number. This enables you to refer to any of the exceptions in a standard way in your applications, as in the following example:

```
BEGIN
  DELETE FROM departments
WHERE department_id = deptno;
  ...
EXCEPTION
  WHEN error_pkg.e_fk_err THEN
  ...
  WHEN OTHERS THEN
  ...
END;
```

The PL/SQL block here refers to `e_fk_err` mapped onto error number `ORA-2292`, which is a constraint violation error.

Standardizing Exception Handling

Consider writing a subprogram with common exception handling. The exception-handling code should:

- Display errors based on `SQLCODE` and `SQLERRM` values for exceptions
- Track runtime errors by using parameters in your code to identify:
 - The procedure in which the error occurred
 - The location (line number) of the error
 - `RAISE_APPLICATION_ERROR` with appropriate error code and message

```
RAISE_APPLICATION_ERROR(-20001, 'My first error', TRUE);
```

7

Standardized exception handling can be implemented either as a standalone subprogram or a subprogram added to the package that defines the standard exceptions.

Consider creating a package for exception handling in your application with the following:

- Every named exception that is to be used in the application
- All unnamed, programmer-defined exceptions that are used in the application. These are error numbers –20000 through –20999.
- A program to call `RAISE_APPLICATION_ERROR` based on package exceptions
- A program to display an error based on the values of `SQLCODE` and `SQLERRM`
- Additional objects, such as error log tables, and programs to access the tables

A common practice is to use parameters that identify the name of the procedure and the location in which the error has occurred. This enables you to keep track of runtime errors more easily. An alternative is to use the `RAISE_APPLICATION_ERROR` built-in procedure to keep a stack trace of exceptions that can be used to track the call sequence leading to the error. To do this, set the third optional argument to `TRUE`. If you specify `TRUE` for the third parameter, PL/SQL puts `error_code` on top of the error stack. Otherwise, PL/SQL replaces the error stack with `error_code`.

For example:

```
RAISE_APPLICATION_ERROR(-20001, 'My first error', TRUE);
```

Standardizing Constants

For programs that use local variables whose values should not change:

- Convert the variables to constants to reduce maintenance and debugging
- Create one central package specification and place all constants in it

```
CREATE OR REPLACE PACKAGE constant_pkg IS
  c_order_received CONSTANT VARCHAR(2) := 'OR';
  c_order_shipped   CONSTANT VARCHAR(2) := 'OS';
  c_min_sal         CONSTANT NUMBER(3)  := 900;
END constant_pkg;
```

8

By definition, a variable's value changes, whereas a constant's value cannot be changed. If you have programs that use local variables whose values should not and do not change, then convert those variables to constants. This can help with the maintenance and debugging of your code.

Consider creating a single shared package with all your constants in it. This makes maintenance and change of the constants much easier. This procedure or package can be loaded on system startup for better performance.

The example in the slide shows the `constant_pkg` package containing a few constants. Refer to any of the package constants in your application as required. Here is an example:

```
BEGIN

  UPDATE employees

    SET salary = salary + 200

  WHERE salary <= constant_pkg.c_min_sal;

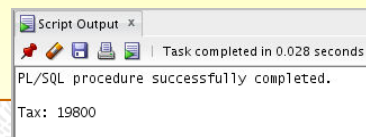
END;

/
```

Local Subprograms

A local subprogram is a `PROCEDURE` or `FUNCTION` defined at the end of the declarative section in a subprogram.

```
CREATE PROCEDURE employee_sal(p_id NUMBER) IS
  v_emp employees%ROWTYPE;
  FUNCTION tax(p_salary VARCHAR2) RETURN NUMBER IS
  BEGIN
    RETURN p_salary * 0.825;
  END tax;
BEGIN
  SELECT * INTO v_emp
  FROM EMPLOYEES WHERE employee_id = p_id;
  DBMS_OUTPUT.PUT_LINE('Tax: ' || tax(v_emp.salary));
END;
/
EXECUTE employee_sal(100)
```



9

Local subprograms reduce the size of a module by removing redundant code. This is one of the main reasons for creating a local subprogram. If a module needs the same routine several times and only this module needs the routine, then define it as a local subprogram.

You can define a named PL/SQL block in the declarative section of any PL/SQL program, procedure or function. Local subprograms have the following characteristics:

- They are accessible to only the block in which they are defined.
- They are compiled as part of their enclosing blocks.

The benefits of local subprograms are:

- Reduction of repetitive code
- Improved code readability and ease of maintenance
- Less administration because there is one program to maintain instead of two

The concept is simple. The example shown in the slide illustrates this with a basic example of an income tax calculation of an employee's salary.

Lesson Agenda

- Standardization of the code
- Managing security for PL/SQL packages and subprograms
- Design considerations for autonomous transactions
- Performance optimization in PL/SQL blocks
 - NO COPY clause
 - PARALLEL_ENABLE clause
 - RESULT_CACHE clause
 - DETERMINISTIC clause
 - RETURNING clause
- Bulk Binding

10



For Instructor Use Only.
This document should not be distributed.

Definer's and Invoker's Rights

- You can control access to privileges required to execute a PL/SQL subprogram by using Definer's or Invoker's rights
- With Definer's rights, the subprogram executes with the access privileges of the owner of the procedure.
- With Invoker's rights, the subprogram executes with the privileges of the user who invokes the procedures

11

Suppose user **bixby** creates a procedure that is designed to modify table `cust_records` and then he/she grants the `EXECUTE` privilege on this procedure to user **rlayton**. If **bixby** had created the procedure with definer's rights, then the procedure looks for table `cust_records` in **bixby's** schema. Had the procedure been created with invoker's rights, then when **rlayton** runs it, the procedure looks for table `cust_records` in **rlayton's** schema.

By default, all procedures are considered definer's rights procedures. You can designate a procedure to be an invoker's rights procedure by using the `AUTHID CURRENT_USER` clause when you create or modify it, or you can use the `AUTHID DEFINER` clause to make it a definer's rights procedure.

Specifying Invoker's Rights: Setting AUTHID to CURRENT_USER

```
CREATE OR REPLACE PROCEDURE add_dept(  
  p_id NUMBER, p_name VARCHAR2) AUTHID CURRENT_USER IS  
BEGIN  
  INSERT INTO departments  
    VALUES (p_id, p_name, NULL, NULL);  
END;
```

When used with standalone functions, procedures, or packages:

- Names used in queries, DML, Native Dynamic SQL, and DBMS_SQL package are resolved in the invoker's schema
- Calls to other packages, functions, and procedures are resolved in the definer's schema

12

Specifying Invoker's Rights

You can set the invoker's rights for different PL/SQL subprogram constructs as follows:

```
CREATE FUNCTION name RETURN type AUTHID CURRENT_USER IS...  
CREATE PROCEDURE name AUTHID CURRENT_USER IS...  
CREATE PACKAGE name AUTHID CURRENT_USER IS...  
CREATE TYPE name AUTHID CURRENT_USER IS OBJECT...
```

The default is AUTHID DEFINER, which specifies that the subprogram executes with the privileges of its owner. Most supplied PL/SQL packages such as DBMS_LOB, DBMS_ROWID, and so on, are invoker-rights packages.

Name Resolution

For a definer's rights procedure, all external references are resolved in the definer's schema. For an invoker's rights procedure, the resolution of external references depends on the kind of statement in which they appear:

- Names used in queries, DML statements, dynamic SQL, and DBMS_SQL are resolved in the invoker's schema.
- All other statements, such as calls to packages, functions, and procedures, are resolved in the definer's schema.

Granting Privileges to Invoker's Rights Unit

- When the invoker is a lower-privilege user, then the definer can grant privileges to execute a subprogram or a PL/SQL package to the invoking user.
- The SQL `GRANT` command grants roles to PL/SQL packages and standalone subprograms:
 - Create an IR unit.
 - Grant roles using the IR unit.
- The following command grants the read and execute roles to

```
GRANT read,execute TO FUNCTION scott . func, PACKAGE sys . pkg
```



13

Before Oracle Database 12c, a definer's rights unit always ran with the privileges of the definer and an invoker's rights unit always ran with the privileges of the invoker. If you wanted to create a PL/SQL unit that all users could invoke, even if their privileges were lower than yours, it had to be a definer's rights unit. The unit always ran with all the definer privileges, regardless of which user invoked it.

As of Oracle Database 12c, you can grant roles to individual PL/SQL packages and standalone subprograms. Instead of a designer's rights unit, you can create an invoker's rights unit and then grant it roles. The invoker's rights unit runs with the privileges of both the invoker and the roles, but without any additional privileges that you have.

You grant roles to an invoker's rights unit so that users with lower privileges than yours can run the unit with only the privileges needed to do so. There is no reason to grant roles to a designer's rights unit, because its invokers run it with all your privileges.

Using the SQL `GRANT` command, you can grant roles to PL/SQL packages and standalone stored subprograms. Roles granted to a PL/SQL unit do not affect compilation. They affect the privilege checking of SQL statements that the unit issues at run time; the unit runs with the privileges of both its own roles and any other currently enabled roles.

Lesson Agenda

- Standardization of the code
- Managing security for PL/SQL packages and subprograms
- Design considerations for autonomous transactions
- Performance optimization in PL/SQL blocks
 - NO COPY clause
 - PARALLEL_ENABLE clause
 - RESULT_CACHE clause
 - DETERMINISTIC clause
 - RETURNING clause
- Bulk Binding

14

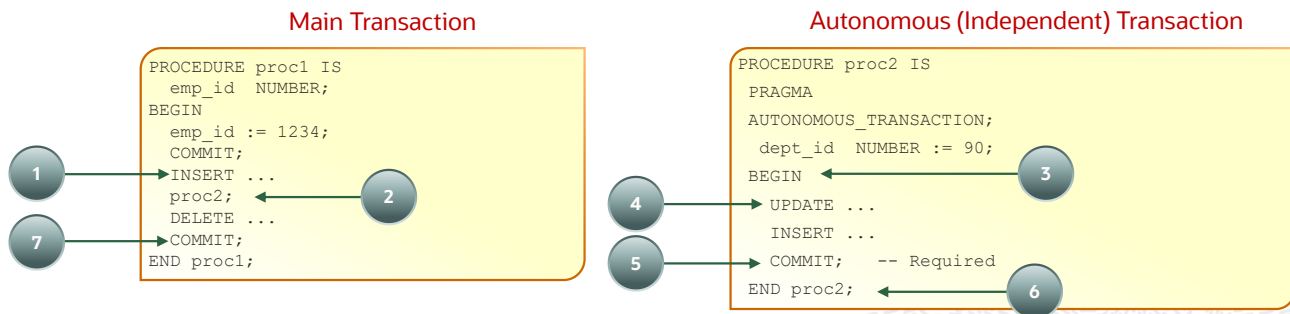


0

For Instructor Use Only.
This document should not be distributed.

Autonomous Transactions

- Are independent transactions started by another main transaction
- Are defined with `PRAGMA AUTONOMOUS_TRANSACTION`



15

A transaction is a series of statements doing a logical unit of work that completes or fails as an integrated unit. Often, one transaction starts another that may need to operate outside the scope of the transaction that started it. That is, in an existing transaction, a required independent transaction may need to commit or roll back changes without affecting the outcome of the starting transaction. For example, in a stock-purchase transaction, the customer's information must be committed regardless of whether the overall stock purchase completes. Or, while running that same transaction, you want to log messages to a table even if the overall transaction rolls back.

The slide depicts the behavior of an autonomous transaction:

1. The main transaction begins.
2. A `proc2` procedure is called to start the autonomous transaction.
3. The main transaction is suspended.
4. The autonomous transactional operation begins.
5. The autonomous transaction ends with a commit or rollback operation.
6. The main transaction is resumed.
7. The main transaction ends.

Features of Autonomous Transactions

- Are independent of the main transaction
- Suspend the calling transaction until the autonomous transactions are completed
- Are not nested transactions
- Don't roll back if the main transaction rolls back
- Enable the changes to become visible to other transactions upon a commit
- Are started and ended by individual subprograms and not by nested or anonymous PL/SQL blocks

16

Autonomous transactions exhibit the following features:

- Although called within a transaction, autonomous transactions are independent of that transaction; that is, they aren't nested transactions.
- If the main transaction rolls back, autonomous transactions don't.
- Changes made by an autonomous transaction become visible to other transactions when the autonomous transaction commits.
- With their stack-like functionality, only the "top" transaction is accessible at any given time. After completion, the autonomous transaction is popped and the calling transaction is resumed.
- There are no limits other than resource limits on how many autonomous transactions can be recursively called.
- Autonomous transactions must be explicitly committed or rolled back; otherwise, an error is returned when attempting to return from the autonomous block.
- You can't use `PRAGMA` to mark all subprograms in a package as autonomous. Only individual routines can be marked autonomous.
- You can't mark a nested or anonymous PL/SQL block as autonomous.

Using Autonomous Transactions: Example

```
CREATE TABLE usage (card_id NUMBER, loc NUMBER)
/
CREATE TABLE txn (acc_id NUMBER, amount NUMBER)
/
CREATE OR REPLACE PROCEDURE log_usage (p_card_id NUMBER, p_loc NUMBER) IS
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO usage
        VALUES (p_card_id, p_loc);
    COMMIT;
END log_usage;
/
CREATE OR REPLACE PROCEDURE bank_trans(p_cardnbr NUMBER,p_loc NUMBER) IS
BEGIN
    INSERT INTO txn VALUES (9001, 1000);
    log_usage (p_cardnbr, p_loc);
END bank_trans;
/
EXECUTE bank_trans(50, 2000)
```

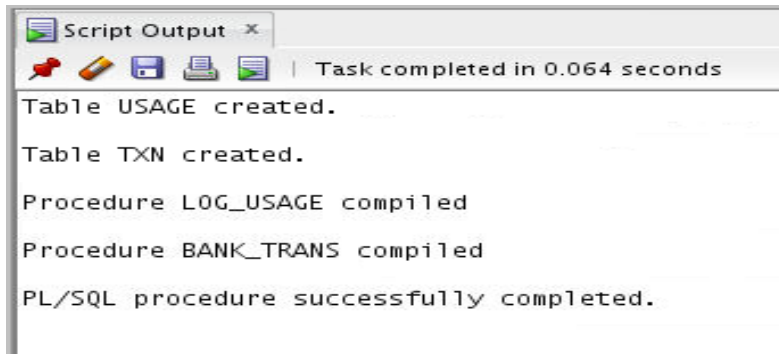
17

To define autonomous transactions, you use `PRAGMA AUTONOMOUS_TRANSACTION`. `PRAGMA` instructs the PL/SQL compiler to mark a routine as autonomous (independent). In this context, the term “routine” includes top-level (not nested) anonymous PL/SQL blocks; local, standalone, and packaged functions and procedures; methods of a SQL object type; and database triggers. You can code `PRAGMA` anywhere in the declarative section of a routine. However, for readability, it is best placed at the top of the Declaration section.

In the example in the slide, you track where the bankcard is used, regardless of whether the transaction is successful. The following are the benefits of autonomous transactions:

- After starting, an autonomous transaction is fully independent. It shares no locks, resources, or commit dependencies with the main transaction, so you can log events, increment retry counters, and so on even if the main transaction rolls back.
- Autonomous transactions help you build modular, reusable software components. For example, stored procedures can start and finish autonomous transactions on their own. A calling application need not know about a procedure’s autonomous operations and the procedure need not know about the application’s transaction context. That makes autonomous transactions less error-prone than regular transactions and easier to use.

The output of the previous slide examples, the `TXN` and `USAGE` tables, are as follows:



To run the code on the previous page, enter the following code:

```
EXECUTE bank_trans(50, 2000)
```

Use the Data tab in the Tables node of the Object Navigator tree to display the values in the `TXN` and `USAGE` tables as follows:

TXN		
Columns Data Constraints Grants Statistics Triggers Flashback Dependencies Details Partitions Indexes SQL		
	ACC_ID	AMOUNT
1	9001	1000

USAGE		
Columns Data Constraints Grants Statistics Triggers Flashback Dependencies Details Partitions Indexes SQL		
	CARD_ID	LOC
1	50	2000

Lesson Agenda

- Standardization of the code
- Managing security for PL/SQL packages and subprograms
- Design considerations for autonomous transactions
- Performance optimization in PL/SQL blocks
 - NO COPY clause
 - PARALLEL_ENABLE clause
 - RESULT_CACHE clause
 - DETERMINISTIC clause
 - RETURNING clause
- Bulk Binding

19



0

For Instructor Use Only.
This document should not be distributed.

Using the NOCOPY Hint

- Allows the PL/SQL compiler to pass OUT and IN OUT parameters by reference rather than by value
- Enhances performance by reducing overhead when passing parameters

```
DECLARE
  TYPE      rec_emp_type IS TABLE OF employees%ROWTYPE;
  rec_emp   rec_emp_type;
  PROCEDURE populate(p_tab IN OUT NOCOPY empTABtype) IS
  BEGIN
    . . .
  END;
BEGIN
  populate(rec_emp);
END;
/
```

20

Note that PL/SQL subprograms support three parameter-passing modes: IN, OUT, and IN OUT. By default:

- The IN parameter is passed by reference. A pointer to the IN actual parameter is passed to the corresponding formal parameter. So, both the parameters reference the same memory location, which holds the value of the actual parameter.
- The OUT and IN OUT parameters are passed by value. The value of the OUT or IN OUT actual parameter is copied into the corresponding formal parameter. Then, if the subprogram exits normally, the values assigned to the OUT and IN OUT formal parameters are copied into the corresponding actual parameters.

Copying parameters that represent large data structures (such as collections, records, and instances of object types) with OUT and IN OUT parameters slows down execution and uses up memory. To prevent this overhead, you can specify the NOCOPY hint, which enables the PL/SQL compiler to pass the OUT and IN OUT parameters by reference.

The slide shows an example of declaring an IN OUT parameter with the NOCOPY hint.

Effects of the NOCOPY Hint

- If the subprogram exits with an exception that is not handled:
 - You cannot rely on the values of the actual parameters passed to a `NOCOPY` parameter
 - Any incomplete modifications are not “rolled back”
- The remote procedure call (RPC) protocol enables you to pass parameters only by value.

21

As a trade-off for better performance, the `NOCOPY` hint enables you to trade well-defined exception semantics for better performance. Its use affects exception handling in the following ways:

- Because `NOCOPY` is a hint and not a directive, the compiler can pass `NOCOPY` parameters to a subprogram by value or by reference. So, if the subprogram exits with an unhandled exception, you cannot rely on the values of the `NOCOPY` actual parameters.
- By default, if a subprogram exits with an unhandled exception, the values assigned to its `OUT` and `IN OUT` formal parameters are not copied to the corresponding actual parameters, and the changes appear to roll back. However, when you specify `NOCOPY`, assignments to the formal parameters immediately affect the actual parameters as well. So, if the subprogram exits with an unhandled exception, the (possibly unfinished) changes are not “rolled back.”
- Currently, the RPC protocol enables you to pass parameters only by value. So, exception semantics can change without notification when you partition applications. For example, if you move a local procedure with `NOCOPY` parameters to a remote site, those parameters are no longer passed by reference.

When Does the PL/SQL Compiler Ignore the NOCOPY Hint?

The `NOCOPY` hint has no effect if:

- The actual parameter:
 - Is an element of associative arrays (index-by tables)
 - Is constrained (for example, by scale or `NOT NULL`)
 - And formal parameter are records, where one or both records were declared by using `%ROWTYPE` or `%TYPE`, and constraints on corresponding fields in the records differ
 - Requires an implicit data type conversion
- The subprogram is involved in an external or RPC

22

In the following cases, the PL/SQL compiler ignores the `NOCOPY` hint and uses the by-value parameter-passing method (with no error generated):

- The actual parameter is an element of associative arrays (index-by tables). This restriction does not apply to entire associative arrays.
- The actual parameter is constrained (by scale or `NOT NULL`). This restriction does not extend to constrained elements or attributes. Also, it does not apply to size-constrained character strings.
- The actual and formal parameters are records; one or both records were declared by using `%ROWTYPE` or `%TYPE`, and constraints on corresponding fields in the records differ.
- The actual and formal parameters are records; the actual parameter was declared (implicitly) as the index of a cursor `FOR` loop, and constraints on corresponding fields in the records differ.
- Passing the actual parameter requires an implicit data-type conversion.
- The subprogram is involved in an external or RPC.

Using the PARALLEL_ENABLE Hint

- Can be used in functions as an optimization hint
- Indicates that a function can be used in a parallelized query or parallelized DML statement

```
CREATE OR REPLACE FUNCTION f2 (p_p1 NUMBER)
  RETURN NUMBER PARALLEL_ENABLE IS
BEGIN
  RETURN p_p1 * 2;
END f2;
```

23

The `PARALLEL_ENABLE` keyword can be used in the syntax for declaring a function. It is an optimization hint that indicates that the function can be used in a parallelized query or parallelized DML statement. Oracle's parallel execution feature divides the work of executing a SQL statement across multiple processes. Functions called from a SQL statement that is run in parallel can have a separate copy run in each of these processes, with each copy called for only the subset of rows that are handled by that process.

For DML statements, before Oracle 8i, the parallelization optimization looked to see whether a function was noted as having all four of `RNDS`, `WNDS`, `RNPS`, and `WNPS` specified in a `PRAGMA RESTRICT_REFERENCES` declaration; those functions that were marked as neither reading nor writing to either the database or package variables could run in parallel. Again, those functions defined with a `CREATE FUNCTION` statement had their code implicitly examined to determine whether they were actually pure enough; parallelized execution might occur even though a `PRAGMA` cannot be specified on these functions.

The `PARALLEL_ENABLE` keyword is placed after the return value type in the declaration of the function, as shown in the example in the slide.

Note: The function should not use session state, such as package variables, because those variables may not be shared among the parallel execution servers.

Using the Cross-Session PL/SQL Function Result Cache

- Each time a result-cached PL/SQL function is called with different parameter values, those parameters and their results are stored in cache.
- The function result cache is stored in a shared global area (SGA), making it available to any session that runs your application.
- Subsequent calls to the same function with the same parameters uses the result from cache.
- Performance and scalability are improved.
- This feature is used with functions that are called frequently and dependent on information that changes infrequently.

24

This caching mechanism provides you with a language-supported and system-managed means for storing the results of PL/SQL functions in a SGA, which is available to every session that runs your application. The caching mechanism is both efficient and easy to use, and it relieves you of the burden of designing and developing your own caches and cache-management policies.

Each time a result-cached PL/SQL function is called with different parameter values, those parameters and their results are stored in the cache. Subsequently, when the same function is called with the same parameter values, the result is retrieved from the cache, instead of being recomputed. If a database object that was used to compute a cached result is updated, the cached result becomes invalid and must be recomputed.

Use the result-caching feature with functions that are called frequently and are dependent on information that never changes or changes infrequently.

Note: For additional information about Cross-Session PL/SQL Function Result Cache, refer to the *Oracle Database Advanced PL/SQL* course, the *Oracle Database SQL and PL/SQL New Features* course, or *Oracle Database PL/SQL Language Reference*.

Declaring and Defining a Result-Cached Function: Example

- Use the `RESULT_CACHE` clause to enable result-caching for a function.
- Example:

```
CREATE OR REPLACE FUNCTION get_hire_date (emp_id NUMBER)
RETURN VARCHAR
  RESULT_CACHE
  AUTHID CURRENT_USER
IS
  date_hired DATE;
BEGIN
  SELECT hire_date INTO date_hired
  from employees
  WHERE employee_id = emp_id;
  RETURN TO_CHAR(date_hired);
END;
/
SELECT get_hire_date(206) from DUAL;
```

25

Before Oracle Database 12c, an invoker's rights function could not be result-cached. As of 12c, the invoker's rights function can be result-cached.

To enable result-caching for a function, use the `RESULT_CACHE` clause. When a result-cached function is invoked, the system checks the cache. If the cache contains the result from a previous invocation of the function with the same parameter values, the system returns the cached result to the invoker and doesn't execute the function body again. If the cache does not contain the result, the system runs the function body and adds the result (for these parameter values) to the cache before returning control to the invoker.

Note: If function execution results in an unhandled exception, the exception result is not stored in the cache.

The code example in the slide depicts the creation of a function `get_hire_date` that takes the `employee_id` as a parameter and returns the `hire_date`.

To enable result caching in a package function, you have to include the `RESULT_CACHE` clause in both function declaration and function definition.

Using the DETERMINISTIC Clause with Functions

- Specify `DETERMINISTIC` to indicate that the function returns the same result value whenever it is called with the same values for its arguments.
- Add the `DETERMINISTIC` clause in function declaration and definition.
- This helps the optimizer avoid redundant function calls.
- If a function was called previously with the same arguments, the optimizer can elect to use the previous result.
- Do not specify `DETERMINISTIC` for a function whose result depends on the state of session variables or schema objects.

26

You can use the `DETERMINISTIC` function clause to indicate that the function returns the same result value whenever it is called with the same values for its arguments.

When Oracle Database encounters a deterministic function, it attempts to use previously calculated results when possible rather than execute the function again. If you subsequently change the semantics of the function, you must manually rebuild all dependent function-based indexes and materialized views.

Don't specify this clause to define a function that uses package variables or that accesses the database in any way that might affect the return result of the function. The results of doing so will not be captured if Oracle Database chooses not to re-execute the function.

Note

- Don't specify `DETERMINISTIC` for a function whose result depends on the state of session variables or schema objects because results might vary across calls. Instead, consider making the function result-cached.
- For more information about the `DETERMINISTIC` clause, refer to *Oracle Database SQL Language Reference*.

Using the RETURNING Clause

- Improves performance by returning column values with INSERT, UPDATE, and DELETE statements
- Eliminates the need for a SELECT statement

```
CREATE OR REPLACE PROCEDURE update_salary(p_emp_id NUMBER) IS
  v_name      employees.last_name%TYPE;
  v_new_sal    employees.salary%TYPE;
BEGIN
  UPDATE employees
    SET salary = salary * 1.1
  WHERE employee_id = p_emp_id
  RETURNING last_name, salary INTO v_name, v_new_sal;
  DBMS_OUTPUT.PUT_LINE(v_name || ' new salary is ' ||
    v_new_sal);
END update_salary;
/
```

27

Often, applications need information about the row affected by a SQL operation; for example, to generate a report or to take a subsequent action. The INSERT, UPDATE, and DELETE statements can include a RETURNING clause, which returns column values from the affected row into PL/SQL variables or host variables. This eliminates the need to SELECT the row after an INSERT or UPDATE, or before a DELETE. As a result, fewer network round trips, less server CPU time, fewer cursors, and less server memory are required.

The example in the slide shows how to update the salary of an employee and, at the same time, retrieve the employee's last name and new salary into a local PL/SQL variable. To test the code example, issue the following commands that check the salary of employee_id 108 before updating it. Next, the procedure is invoked with the employee_id 108 as the parameter.

```
select last_name, salary from employees where employee_id=108;
/
EXECUTE update_salary(108)
```

Query Result x		Script Output x	
All Rows Fetched: 1 in 0.048 seconds		Task completed in 0.016 seconds	
LAST_NAME	SALARY	PL/SQL procedure successfully completed.	
1 Greenberg	12008	Greenberg new salary is 13208.8	

Lesson Agenda

- Standardization of code
- Managing security for PL/SQL packages and subprograms
- Design considerations for autonomous transactions
- Performance optimization in PL/SQL blocks
 - NO COPY clause
 - PARALLEL_ENABLE clause
 - RESULT_CACHE clause
 - DETERMINISTIC clause
 - RETURNING clause
- Bulk Binding

28

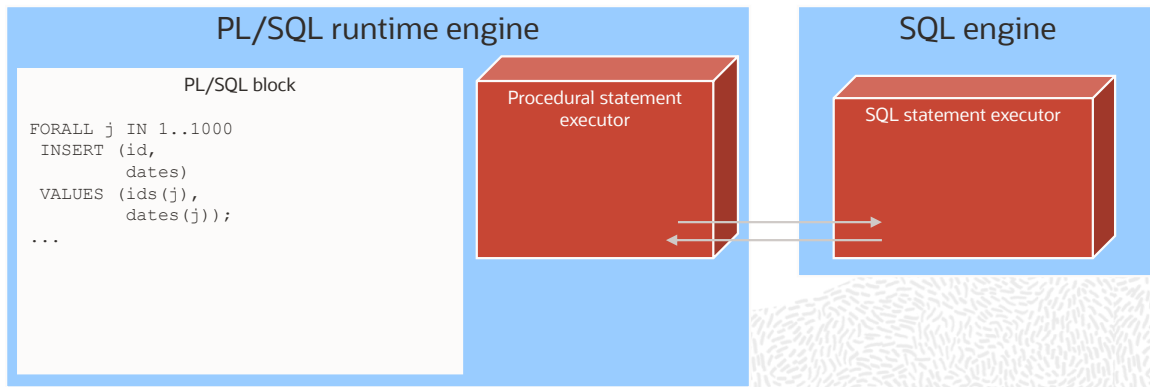


0

For Instructor Use Only.
This document should not be distributed.

Using Bulk Binding

Binds whole arrays of values in a single operation, rather than using a loop to perform a `FETCH`, `INSERT`, `UPDATE`, and `DELETE` operation multiple times



29

The Oracle server uses two engines to run PL/SQL blocks and subprograms:

- The PL/SQL runtime engine, which runs procedural statements, but passes the SQL statements to the SQL engine
- The SQL engine, which parses and executes the SQL statement and, in some cases, returns data to the PL/SQL engine

During execution, every SQL statement causes a context switch between the two engines, which results in a performance penalty for excessive amounts of SQL processing. This is typical of applications that have a SQL statement in a loop that uses values from indexed collections. Collections include nested tables, varying arrays, index-by tables, and host arrays.

Performance can be substantially improved by minimizing the number of context switches through the use of bulk binding. Bulk binding causes an entire collection to be bound in one call, a context switch, to the SQL engine. That is, a bulk bind process passes the entire collection of values back and forth between the two engines in a single context switch, compared with incurring a context switch for each collection element in an iteration of a loop. The more rows affected by a SQL statement, the greater the performance gain with bulk binding.

Bulk Binding: Syntax and Keywords

- The `FORALL` keyword instructs the PL/SQL engine to bulk bind input collections before sending them to the SQL engine.

```
FORALL index IN lower_bound .. upper_bound
[SAVE EXCEPTIONS]
sql_statement;
```

- The `BULK COLLECT` keyword instructs the SQL engine to bulk bind output collections before returning them to the PL/SQL engine.

```
... BULK COLLECT INTO
collection_name[,collection_name] ...
```

30

Use bulk binds to improve the performance of:

- DML statements that reference collection elements
- `SELECT` statements that reference collection elements
- Cursor `FOR` loops that reference collections and the `RETURNING INTO` clause

The `FORALL` keyword instructs the PL/SQL engine to bulk bind input collections before sending them to the SQL engine. Although the `FORALL` statement contains an iteration scheme, it is not a `FOR` loop.

The `BULK COLLECT` keyword instructs the SQL engine to bulk bind output collections, before returning them to the PL/SQL engine. This enables you to bind locations into which SQL can return the retrieved values in bulk. Thus, you can use these keywords in the `SELECT INTO`, `FETCH INTO`, and `RETURNING INTO` clauses.

Bulk Binding FORALL : Example

```
CREATE TABLE parts1 ( pnum INTEGER, pname VARCHAR2(15) );
/
CREATE TABLE parts2 ( pnum INTEGER, pname VARCHAR2(15) );
/
DECLARE
TYPE NumTab IS TABLE OF parts1.pnum%TYPE INDEX BY PLS_INTEGER;
TYPE NameTab IS TABLE OF parts1.pname%TYPE INDEX BY PLS_INTEGER;
pnums NumTab;
pnames NameTab;
iterations CONSTANT PLS_INTEGER := 50000;
t1 INTEGER;
t2 INTEGER;
t3 INTEGER;
```

Bulk Binding FORALL : Example

```
BEGIN
  FOR j IN 1..iterations LOOP -- populate collections
    pnums(j) := j;
    pnames(j) := 'Part No. ' || TO_CHAR(j);
  END LOOP;
  t1 := DBMS_UTILITY.get_time;
  FOR i IN 1..iterations LOOP
    INSERT INTO parts1 (pnum, pname) VALUES (pnums(i), pnames(i));
  END LOOP;
  t2 := DBMS_UTILITY.get_time;
  FORALL i IN 1..iterations
    INSERT INTO parts2 (pnum, pname) VALUES (pnums(i), pnames(i));
  t3 := DBMS_UTILITY.get_time;
```

Bulk Binding FORALL : Example

```
DBMS_OUTPUT.PUT_LINE('Execution Time (secs)');
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('FOR LOOP: ' || TO_CHAR((t2 - t1)/100));
DBMS_OUTPUT.PUT_LINE('FORALL: ' || TO_CHAR((t3 - t2)/100));
COMMIT;
END;
/
```

```
PL/SQL procedure successfully completed.
Execution Time (secs)
-----
FOR LOOP: .62
FORALL: .02
```


Using BULK COLLECT INTO with Queries

The SELECT statement supports the BULK COLLECT INTO syntax.

```
CREATE PROCEDURE get_departments(p_loc NUMBER) IS
  TYPE dept_tab_type IS
    TABLE OF departments%ROWTYPE;
  v_depts dept_tab_type;
BEGIN
  SELECT * BULK COLLECT INTO v_depts
  FROM departments
  WHERE location_id = p_loc;
  FOR i IN 1 .. v_depts.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(v_depts(i).department_id
      || ' ' || v_depts(i).department_name);
  END LOOP;
END;
```

34

By using BULK COLLECT INTO, you can quickly obtain a set of rows without using a cursor mechanism.

The example reads all the department rows for a specified region into a PL/SQL table, whose contents are displayed with the FOR loop that follows the SELECT statement.

Using BULK COLLECT INTO with Cursors

The FETCH statement has been enhanced to support the BULK COLLECT INTO syntax.

```
CREATE OR REPLACE PROCEDURE get_departments(p_loc NUMBER) IS
  CURSOR cur_dept IS
    SELECT * FROM departments
    WHERE location_id = p_loc;
  TYPE dept_tab_type IS TABLE OF cur_dept%ROWTYPE;
  v_depts dept_tab_type;
BEGIN
  OPEN cur_dept;
  FETCH cur_dept BULK COLLECT INTO v_depts;
  CLOSE cur_dept;
  FOR i IN 1 .. v_depts.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(v_depts(i).department_id
      || ' ' || v_depts(i).department_name);
  END LOOP;
END;
```

35

0

The example in the slide shows how BULK COLLECT INTO can be used with cursors.

You can also add a LIMIT clause to control the number of rows fetched in each operation. The code example in the slide could be modified as follows:

```
CREATE OR REPLACE PROCEDURE get_departments(p_loc NUMBER,
  p_nrows NUMBER) IS
  CURSOR dept_csr IS SELECT *
  FROM departments
  WHERE location_id = p_loc;
  TYPE dept_tabtype IS TABLE OF dept_csr%ROWTYPE;
  depts dept_tabtype;
BEGIN
  OPEN dept_csr;
  FETCH dept_csr BULK COLLECT INTO depts LIMIT p_nrows;
  CLOSE dept_csr;
  DBMS_OUTPUT.PUT_LINE(depts.COUNT || ' rows read');
END;
```

For Instructor Use Only.
This document should not be distributed.

Using BULK COLLECT INTO with a RETURNING Clause

```
CREATE OR REPLACE PROCEDURE raise_salary(p_rate NUMBER) IS
  TYPE emplist_type IS TABLE OF NUMBER;
  TYPE numlist_type IS TABLE OF employees.salary%TYPE
    INDEX BY BINARY_INTEGER;
  v_emp_ids emplist_type := emplist_type(100,101,102,104);
  v_new_sals numlist_type;
BEGIN
  FORALL i IN v_emp_ids.FIRST .. v_emp_ids.LAST
    UPDATE employees
      SET commission_pct = p_rate * salary
      WHERE employee_id = v_emp_ids(i)
      RETURNING salary BULK COLLECT INTO v_new_sals;
  FOR i IN 1 .. v_new_sals.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(v_new_sals(i));
  END LOOP;
END;
```

36

Bulk binds can be used to improve the performance of `FOR` loops that reference collections and return DML. If you have, or plan to have, PL/SQL code that does this, then you can use the `FORALL` keyword, along with the `RETURNING` and `BULK COLLECT INTO` keywords, to improve performance.

In the slide, the salary information is retrieved from the `EMPLOYEES` table and collected into the `new_sals` array. The `new_sals` collection is returned in bulk to the PL/SQL engine.

The example in the slide shows a `FOR` loop that is used to iterate through the new salary data received from the `UPDATE` operation and then process the results.

Quiz



The `NOCOPY` hint allows the PL/SQL compiler to pass `OUT` and `IN OUT` parameters by reference rather than by value. This enhances performance by reducing overhead when passing parameters.

- a. True
- b. False

37

Answer: a

PL/SQL subprograms support three parameter-passing modes: `IN`, `OUT`, and `IN OUT`.

By default:

- The `IN` parameter is passed by reference. A pointer to the `IN` actual parameter is passed to the corresponding formal parameter. So, both the parameters reference the same memory location, which holds the value of the actual parameter.
- The `OUT` and `IN OUT` parameters are passed by value. The value of the `OUT` or `IN OUT` actual parameter is copied into the corresponding formal parameter. Then, if the subprogram exits normally, the values assigned to the `OUT` and `IN OUT` formal parameters are copied into the corresponding actual parameters.

Copying parameters that represent large data structures (such as collections, records, and instances of object types) with `OUT` and `IN OUT` parameters slows down execution and uses up memory. To prevent this overhead, you can specify the `NOCOPY` hint, which enables the PL/SQL compiler to pass `OUT` and `IN OUT` parameters by reference.

Summary

In this lesson, you should have learned how to:

- Create standard constants and exceptions
- Write and call local subprograms
- Control the runtime privileges of a subprogram
- Perform autonomous transactions
- Grant roles to PL/SQL packages and stored subprograms
- Pass parameters by reference using a `NOCOPY` hint
- Use the `PARALLEL ENABLE` hint for optimization
- Use the cross-session PL/SQL function result cache
- Use the `DETERMINISTIC` clause with functions
- 38 • Use the `RETURNING` clause and bulk binding with DML



The lesson provides insights into managing your PL/SQL code by defining constants and exceptions in a package specification. This enables a high degree of reuse and standardization of code.

Local subprograms can be used to simplify and modularize a block of code where the subprogram functionality is repeatedly used in the local block.

The runtime security privileges of a subprogram can be altered by using definer's or invoker's rights.

Autonomous transactions can be executed without affecting an existing main transaction.

You should understand how to obtain performance gains by using the `NOCOPY` hint, bulk binding and the `RETURNING` clauses in SQL statements, and the `PARALLEL_ENABLE` hint for optimization of functions.

Practice 20 Overview: Design Considerations for PL/SQL Code

This practice covers the following topics:

- Creating a package that uses bulk fetch operations
- Creating a local subprogram to perform an autonomous transaction to audit a business operation



39

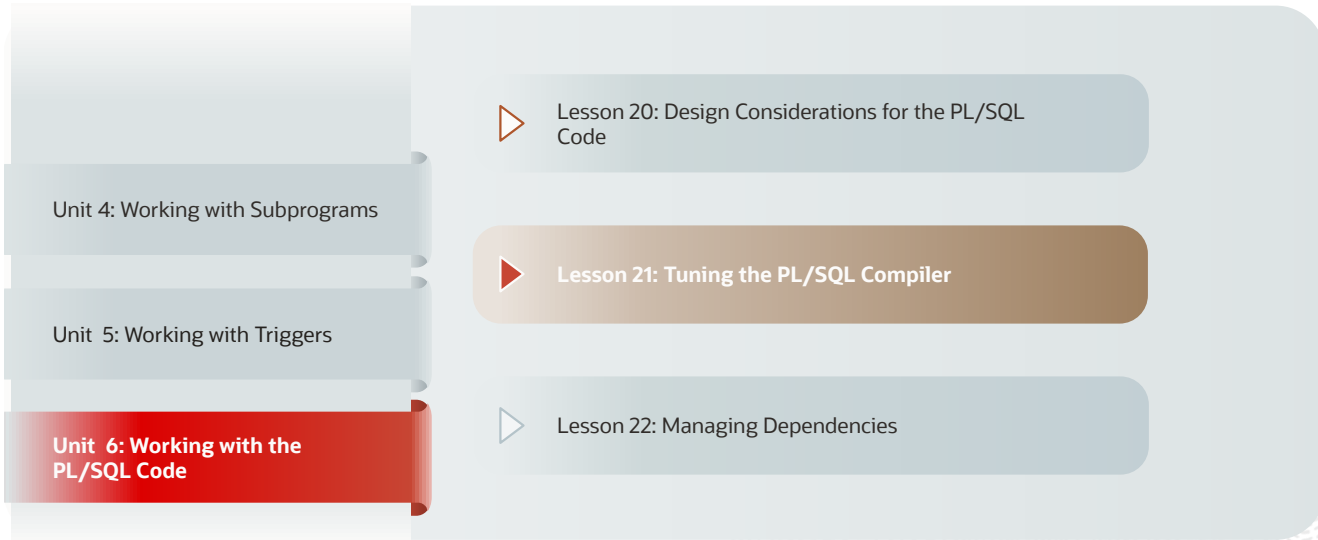
In this practice, you create a package that performs a bulk fetch of employees in a specified department. The data is stored in a PL/SQL table in the package. You also provide a procedure to display the contents of the table.

You add an `add_employee` procedure to the package that inserts new employees. The procedure uses a local autonomous subprogram to write a log record each time the `add_employee` procedure is called, whether it successfully adds a record or not.

For Instructor Use Only.
This document should not be distributed.

Tuning the PL/SQL Compiler

Course Road Map



2

In Unit 6, you will learn the design aspects of the PL/SQL code. You will also learn how to use PL/SQL compiler and how to manage dependencies among different objects in the database application.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Use the PL/SQL compiler initialization parameters
- Use the PL/SQL compile-time warnings



For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Using PL/SQL initialization parameters
- Using the PL/SQL compile-time warnings:
 - Using the `PLSQL_WARNING` initialization parameter
 - Using the `DBMS_WARNING` package subprograms



Optimizing PL/SQL Compiler Performance

- The PL/SQL compiler implicitly rearranges the code for better performance.
- The compiler behavior is defined by its initialization parameters.
- You can use initialization parameters to tune the performance of the compiler according to the application requirement.



0

For Instructor Use Only.
This document should not be distributed.

Initialization Parameters for PL/SQL Compilation

Compilation Parameter	Description
<code>PLSCOPE_SETTINGS</code>	Controls the compile-time collection, cross-reference, and storage of the PL/SQL source text identifier data
<code>PLSQL_CCFLAGS</code>	Lets you control conditional compilation of each PL/SQL unit independently
<code>PLSQL_CODE_TYPE</code>	Specifies the compilation mode for PL/SQL units - <code>INTERPRETED</code> (the default) or <code>NATIVE</code>
<code>PLSQL_OPTIMIZE_LEVEL</code>	Specifies the optimization level at which to compile PL/SQL units
<code>PLSQL_WARNINGS</code>	Enables or disables the reporting of warning messages by the PL/SQL compiler, and specifies which warning messages to show as errors
<code>NLS_LENGTH_SEMANTICS</code>	Lets you create <code>CHAR</code> and <code>VARCHAR2</code> columns by using either byte-length or character-length semantics

- `PLSCOPE_SETTINGS` determines whether the compiler should collect data about the identifiers in the PL/SQL program unit and provide it to the static data dictionary views. The collected data includes information about identifier types, usages, and location of each usage in the source code.

The `PLSCOPE_SETTINGS` parameter can assume either `'IDENTIFIERS:ALL'` or `'IDENTIFIERS:NONE'`.

- `PLSQL_CCFLAGS` lets you control the conditional compilation of each program unit. Conditional compilation lets you customize the functionality of a PL/SQL application without removing the source text. For example:
 - Using new features with the latest database release and disabling them when running the application in an older database release.
 - Activate debugging or tracing statements in the development environment and hide them when running the application at a production site.
- The `PLSQL_CODE_TYPE` parameter can have either `INTERPRETED` or `NATIVE` as its value. It determines the mode of compilation.

The PL/SQL statements in a PL/SQL unit are compiled into an intermediate form, system code, which is stored in the catalog and interpreted at run time. This is the default behavior when the `PLSQL_CODE_TYPE` parameter is set to `INTERPRETED` value.

When the `PLSQL_CODE_TYPE` value is set to `NATIVE`, the PL/SQL statements in a PL/SQL unit are compiled into native code and stored in the catalog. The native code need not be interpreted at run time, so it runs faster.

- The `PLSQL_OPTIMIZE_LEVEL` parameter can have a value in the 0-3 range. A higher value of the parameter implies a greater level of optimization.
- The `PLSQL_WARNINGS` parameter configures the response of the application to various PL/SQL warnings in the application.
- The `NLS_LENGTH_SEMANTICS` parameter determines whether the compiler should use byte-length or character-length semantics for the `VARCHAR2` and `CHAR` variables in the PL/SQL program unit.

The values at the time of compilation of `PLSQL_CCFLAGS`, `PLSQL_CODE_TYPE`, `PLSQL_OPTIMIZE_LEVEL`, `PLSQL_WARNINGS`, and `NLS_LENGTH_SEMANTICS` initialization parameters are stored with the unit's metadata. You can view information about the settings of these parameters with the `ALL_PLSQL_OBJECT_SETTINGS` view.

Using the Initialization Parameters for PL/SQL Compilation

- `PLSQL_CODE_TYPE`: Specifies the compilation mode for PL/SQL library units
`PLSQL_CODE_TYPE = { INTERPRETED | NATIVE }`
- `PLSQL_OPTIMIZE_LEVEL`: Specifies the optimization level to be used to compile PL/SQL library units
`PLSQL_OPTIMIZE_LEVEL = { 0 | 1 | 2 | 3 }`
- 19c: to compile PL/SQL units for debugging, specify `PLSQL_OPTIMIZE_LEVEL=1`

8

`PLSQL_CODE_TYPE` determines the compilation mode for the PL/SQL program unit. When the value of this parameter is changed, it has no effect on PL/SQL library units that have already been compiled. The value of this parameter is stored persistently with each library unit. If a PL/SQL library unit is compiled natively, all subsequent automatic recompilations of that library unit will use native compilation.

`PLSQL_OPTIMIZE_LEVEL` defines the extent of optimization applicable to the compilation of the program unit.

`PLSQL_OPTIMIZE_LEVEL`, when set to 0: the compiler would not reorder the code for better performance.

`PLSQL_OPTIMIZE_LEVEL`, when set to 1: applies a wide range of optimizations to PL/SQL programs, including the elimination of unnecessary computations and exceptions, but generally does not move the source code out of its original source order

`PLSQL_OPTIMIZE_LEVEL`, when set to 2: applies a wide range of modern optimization techniques beyond those of level 1, including changes that may move the source code relatively far from its original location.

`PLSQL_OPTIMIZE_LEVEL`, when set to 3: applies a wide range of optimization techniques beyond those of level 2, automatically including techniques not specifically requested

New for 19c: to compile PL/SQL units for debugging, specify `PLSQL_OPTIMIZE_LEVEL=1`

Displaying the PL/SQL Initialization Parameters

Use the `USER | ALL | DBA_PLSQL_OBJECT_SETTINGS` data dictionary views to display the settings for a PL/SQL object:

```
DESCRIBE USER_PLSQL_OBJECT_SETTINGS;
```

Name	Null	Type
NAME	NOT NULL	VARCHAR2(128)
TYPE		VARCHAR2(12)
PLSQL_OPTIMIZE_LEVEL		NUMBER
PLSQL_CODE_TYPE		VARCHAR2(4000)
PLSQL_DEBUG		VARCHAR2(4000)
PLSQL_WARNINGS		VARCHAR2(4000)
NLS_LENGTH_SEMANTICS		VARCHAR2(4000)
PLSQL_CCFLAGS		VARCHAR2(4000)
PLSCOPE_SETTINGS		VARCHAR2(4000)
ORIGIN_CON_ID		NUMBER

9

The columns of the `USER_PLSQL_OBJECTS_SETTINGS` data dictionary view are:

- **Owner:** The owner of the object. This column is not displayed in the `USER_PLSQL_OBJECTS_SETTINGS` view.
- **Name:** The name of the object
- **Type:** The available choices are `PROCEDURE`, `FUNCTION`, `PACKAGE`, `PACKAGE BODY`, `TRIGGER`, `TYPE`, `TYPE BODY`, or any other user-defined objects.
- **PLSQL_OPTIMIZE_LEVEL:** The optimization level that was used to compile the object
- **PLSQL_CODE_TYPE:** The compilation mode for the object
- **PLSQL_DEBUG:** Specifies whether or not the object was compiled for debugging (deprecated in version 12.1)
- **PLSQL_WARNINGS:** The compiler warning settings used to compile the object
- **NLS_LENGTH_SEMANTICS:** The NLS length semantics used to compile the object
- **PLSQL_CCFLAGS:** The conditional compilation flag used to compile the object
- **PLSCOPE_SETTINGS:** Controls the compile-time collection, cross reference, and storage of PL/SQL source code identifier data
- **ORIGIN_CON_ID:** The identifier of the container where the data originates. This value is 0 for non-container databases.

Displaying and Setting PL/SQL Initialization Parameters

```
SELECT name, type, plsql_code_type,  
       plsql_optimize_level  
FROM   user_plsql_object_settings;
```

NAME	TYPE	PLSQL_OPTIMIZE_LEVEL
1 ADD_COL	PROCEDURE	INTERPRETED
2 ADD_DEPARTMENT	PROCEDURE	INTERPRETED
3 ADD_DEPARTMENT_NOEX	PROCEDURE	INTERPRETED
4 ADD_DEPT	PROCEDURE	INTERPRETED
5 ADD_JOB_HISTORY	PROCEDURE	INTERPRETED
6 ADD_ROW	PROCEDURE	INTERPRETED
7 ANNUAL_SAL	FUNCTION	INTERPRETED
8 AUDIT_EMP_VALUES	TRIGGER	INTERPRETED
9 BANK_TRANS	PROCEDURE	INTERPRETED
10 CHECK_SALARY	TRIGGER	INTERPRETED

- Set the compiler initialization parameter's value by using the `ALTER SYSTEM` or `ALTER SESSION` statements.
- The parameters' values are accessed when the `CREATE OR REPLACE` statement is executed.

10

You can set the value of the compilation parameter through `ALTER SESSION` as follows:

```
ALTER SESSION SET PLSQL_OPTIMIZE_LEVEL=1;
```

Changing PL/SQL Initialization Parameters: Example

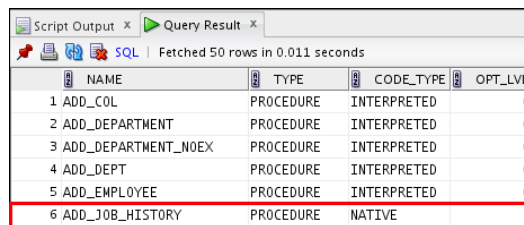
```
ALTER SESSION SET PLSQL_OPTIMIZE_LEVEL = 1;  
ALTER SESSION SET PLSQL_CODE_TYPE = 'NATIVE';
```

Session altered.

Session altered.

```
-- code displayed in the notes page  
CREATE OR REPLACE PROCEDURE add_job_history  
...
```

@code_11_09_sa.sql



	NAME	TYPE	CODE_TYPE	OPT_LVL
1	ADD_COL	PROCEDURE	INTERPRETED	0
2	ADD_DEPARTMENT	PROCEDURE	INTERPRETED	0
3	ADD_DEPARTMENT_NOEX	PROCEDURE	INTERPRETED	0
4	ADD_DEPT	PROCEDURE	INTERPRETED	0
5	ADD_EMPLOYEE	PROCEDURE	INTERPRETED	0
6	ADD_JOB_HISTORY	PROCEDURE	NATIVE	1

11

To change a compiled PL/SQL object from the interpreted code type to the native code type, you must first set the `PLSQL_CODE_TYPE` parameter to `NATIVE` (optionally set the other parameters) and then recompile the program.

To enforce native compilation to all PL/SQL code, you must recompile each one. Scripts (in the `rdmbs/admin` directory) are provided for you to achieve conversion to full native compilation (`dbmsupgnv.sql`) or full interpreted compilation (`dbmsupgin.sql`). The `add_job_history` procedure is created as follows:

```
CREATE OR REPLACE PROCEDURE add_job_history  
( p_emp_id          job_history.employee_id%type  
  , p_start_date     job_history.start_date%type  
  , p_end_date       job_history.end_date%type  
  , p_job_id         job_history.job_id%type  
  , p_department_id  job_history.department_id%type )  
IS  
BEGIN  
    INSERT INTO job_history (employee_id, start_date,  
                           end_date, job_id, department_id)  
    VALUES (p_emp_id, p_start_date, p_end_date,  
            p_job_id, p_department_id);  
END add_job_history;  
/
```

Lesson Agenda

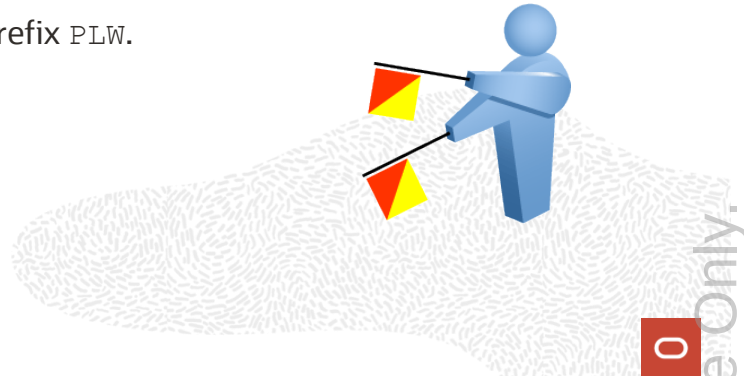
- Using PL/SQL initialization parameters
- Using the PL/SQL compile-time warnings:
 - Using the `PLSQL_WARNING` initialization parameter
 - Using the `DBMS_WARNING` package subprograms



For Instructor Use Only.
This document should not be distributed.

PL/SQL Compile-Time Warnings

- Messages from compiler that help in improving code maintainability
- Developers can use them to reduce the chances of bugs creeping in to the application.
- A PL/SQL unit flagged with warnings may execute successfully, but might exhibit unexpected behavior or inefficient performance.
- All PL/SQL warning messages use the prefix `PLW`.



13

To make your programs more robust and avoid problems at run time, you can turn on checking for certain warning conditions. These conditions are not serious enough to produce an error and keep you from compiling a subprogram. They may point out something in the subprogram that produces an undefined result or might create a performance problem.

With the PL/SQL compiler-warning feature, compiling a PL/SQL program could have the following two possible outcomes:

- Success with compilation warnings
- Failure with compilation errors and compilation warnings

Note that the compiler may issue warning messages even on a successful compile. A compilation error must be corrected to be able to use the stored procedure, whereas a warning is for informational purposes.

Examples of warning messages

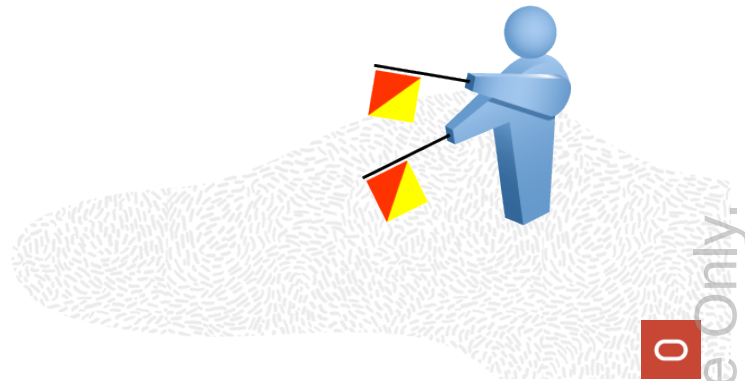
SP2-0804: Procedure created with compilation warnings

PLW-07203: Parameter "`IO_TBL`" may benefit from use of the `NOCOPY` compiler hint.

For Instructor Use Only.
This document should not be distributed.

Benefits of Compiler Warnings

- Make programs more robust and avoid problems at run time
- Identify potential performance problems
- Identify factors that produce undefined results



14

Using compiler warnings can help you to:

- Make your programs more robust and avoid problems at run time
- Identify potential performance problems
- Identify factors that produce undefined results

All PL/SQL warning messages use the prefix `PLW`.

0
For Instructor Use Only.
This document should not be distributed.

Categories of PL/SQL Compile-Time Warning Messages

SEVERE	PERFORMANCE	INFORMATIONAL
Condition might cause unexpected action or wrong results.	Condition might cause performance problems.	Condition does not affect performance or correctness, but you might want to change it for better maintainability.

15

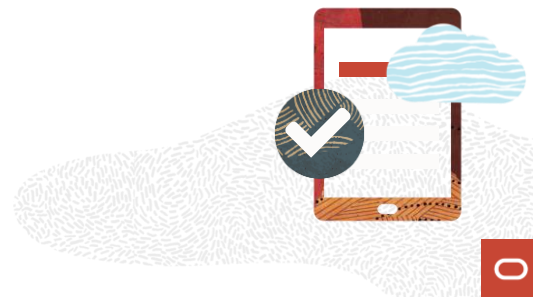
PL/SQL warning messages are divided into categories, so that you can suppress or display groups of similar warnings during compilation. The categories are:

- **SEVERE:** Messages for conditions that may cause unexpected behavior or wrong results, such as aliasing problems with parameters
- **PERFORMANCE:** Messages for conditions that may cause performance problems, such as passing a `VARCHAR2` value to a `NUMBER` column in an `INSERT` statement
- **INFORMATIONAL:** Messages for conditions that do not have an effect on performance or correctness, but that you may want to change to make the code more maintainable, such as unreachable code that can never be executed

Enabling Warning Messages

You can enable displaying warning messages by using one of the following methods:

- The `PLSQL_WARNINGS` initialization parameter
- The `DBMS_WARNING` package



16

You can enable the display of compiler warning messages by using one of the following methods:

The `PLSQL_WARNINGS` Initialization Parameter

The `PLSQL_WARNINGS` parameter enables or disables warning messages being reported by the PL/SQL compiler and specifies which warning messages to show as errors. The settings for the `PLSQL_WARNINGS` parameter are stored along with each compiled subprogram. You can use the `PLSQL_WARNINGS` initialization parameter to do the following:

- Enable or disable the reporting of all warnings, warnings of a selected category, or specific warning messages.
- Treat all warnings, a selected category of warning, or specific warning messages as errors.
- Following is the syntax to do so:

```
PLSQL_WARNINGS = '<DISABLE | ENABLE | ERROR>:<ALL | INFORMATIONAL |  
PERFORMANCE | SEVERE>'
```

```
PLSQL_WARNINGS = 'DISABLE:<warning#>'ENABLE:<warning#>',  
'ERROR:<warning#>'
```

You can disable or enable certain category of warnings. You can also configure the compiler to report certain category of warnings as errors.

Through the `PLSQL_WARNINGS` parameter you can enable, disable, or report specific warnings as errors.

The `DBMS_WARNING` Package

The `DBMS_WARNING` package provides a way to manipulate the behavior of PL/SQL warning messages, in particular by reading and changing the setting of the `PLSQL_WARNINGS` initialization parameter, to control what kinds of warnings are suppressed, displayed, or treated as errors. This package provides the interface to query, modify, and delete current system or session settings.

Setting Compiler Warning Levels: Using PLSQL_WARNINGS, Examples

```
ALTER SESSION
SET plsql_warnings = 'enable:severe',
    'enable:performance',
    'disable:informational';
```

Session altered.

```
ALTER SESSION
SET plsql_warnings = 'enable:severe';
```

Session altered.

```
ALTER SESSION SET PLSQL_WARNINGS='ENABLE:SEVERE',
    'DISABLE:PERFORMANCE' , 'ERROR:05003';
```

Session altered.

17

You can use the `ALTER SESSION` or `ALTER SYSTEM` command to change the `PLSQL_WARNINGS` initialization parameter. The graphic in the slide shows the various examples of enabling and disabling compiler warnings.

Example 1

In this example, you are enabling `SEVERE` and `PERFORMANCE` warnings and disabling `INFORMATIONAL` warnings.

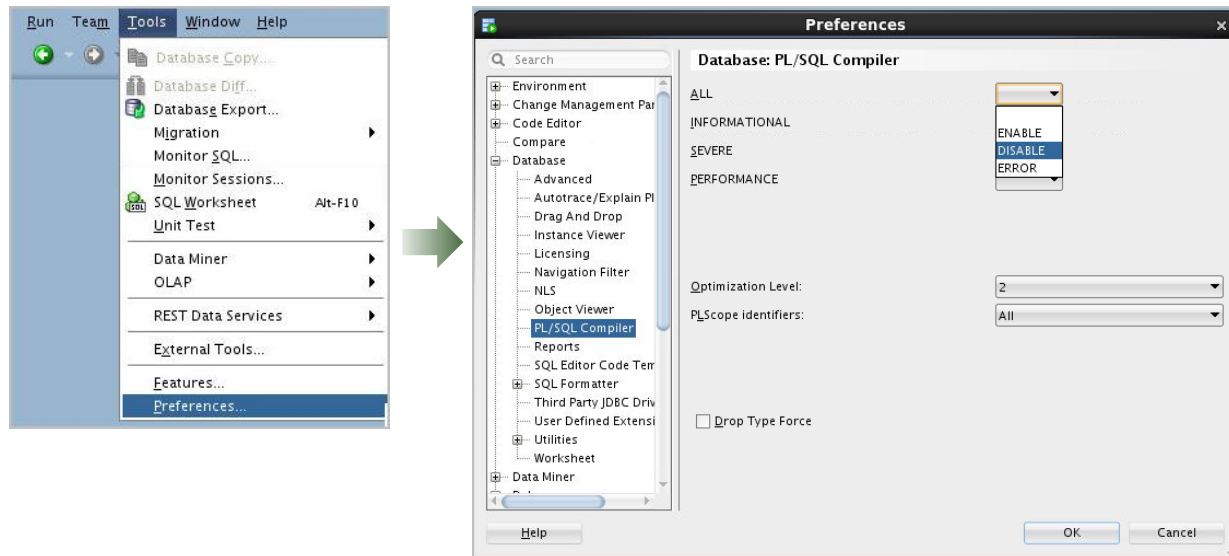
Example 2

In the second example, you are enabling only `SEVERE` warnings.

Example 3

You can treat particular messages as errors, instead of warnings. In this example, if you know that the warning message `PLW-05003` represents a serious problem in your code; `ERROR:05003` in the `PLSQL_WARNINGS` setting makes that condition trigger an error message (`PLS_05003`) instead of a warning message. An error message causes the compilation to fail. In this example, you are also disabling `PERFORMANCE` warnings.

Enabling Compiler Warnings: Using PLSQL_WARNINGS in SQL Developer



18

The PL/SQL Compiler pane specifies options for compilation of PL/SQL subprograms.

Setting and Viewing the PL/SQL Compile-Time Warning Messages Categories in SQL Developer

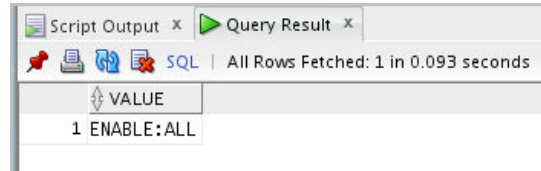
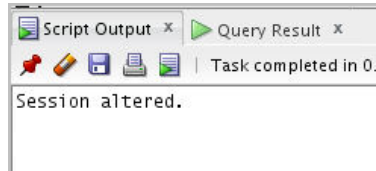
You can control the display of informational, severe, and performance-related messages. The **ALL** type overrides any individual specifications for the other types of messages. For each type of message, you can specify any of the following:

- **No entry (blank):** Use any value specified for **ALL**; and if no value is specified, use the Oracle default.
- **Enable:** Enable the display of all messages of this category.
- **Disable:** Disable the display of all messages of this category.
- **Error:** Enable the display of only error messages of this category.

Viewing the Current Setting of PLSQL_WARNINGS

You can examine the current setting for the PLSQL_WARNINGS parameter by issuing a SELECT statement on the V\$PARAMETER view.

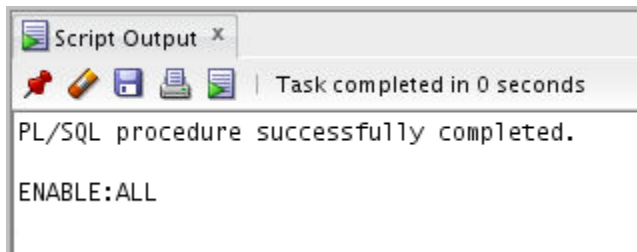
```
ALTER SESSION SET plsql_warnings = 'enable:severe', 'enable:performance','enable:informational';  
/  
SELECT value FROM v$parameter WHERE name='plsql_warnings';  
/
```



19

Alternatively, you can use the DBMS_WARNING.GET_WARNING_SETTING_STRING package and procedure to retrieve the current settings for the PLSQL_WARNINGS parameter:

```
DECLARE s VARCHAR2(1000);  
BEGIN  
    s := dbms_warning.get_warning_setting_string();  
    dbms_output.put_line (s);  
END;  
/
```



Viewing Compiler Warnings

- You can view compiler warnings through:
 - SQL*Plus with SHOW ERRORS command
 - Data Dictionary views

20

You can use SQL*Plus to see any warnings raised as a result of the compilation of a PL/SQL block. SQL*Plus indicates that a compilation warning has occurred. The **“SP2-08xx: <object> created with compilation warnings.”** message is displayed for objects compiled. You must enable the compiler warnings before compiling the program. You can display the compiler warning messages by using any of the following methods:

Using the SQL*Plus SHOW ERRORS Command

This command displays any compiler errors including the new compiler warnings and informational messages. This command is invoked immediately after a `CREATE [PROCEDURE | FUNCTION | PACKAGE]` command is used. The `SHOW ERRORS` command displays warnings and compiler errors. New compiler warnings and informational messages are “interleaved” with compiler errors when `SHOW ERRORS` is invoked.

Using Data Dictionary Views

You can select from the `USER_ | ALL_ | DBA_ERRORS` data dictionary views to display PL/SQL compiler warnings. The `ATTRIBUTES` column of these views has a new attribute called `WARNING` and the warning message displays in the `TEXT` column.

SQL*Plus Warning Messages: Example

```
CREATE OR REPLACE PROCEDURE bad_proc(p_out ...) IS
BEGIN
    ...;
END;
/
```

SP2-0804: Procedure created with compilation warnings.

```
Procedure BAD_PROC compiled
Errors: check compiler log
Errors for PROCEDURE ORA61.BAD_PROC:

LINE/COL ERROR
-----
1/26      PLS-00103: Encountered the symbol "?" when expecting one of the following:

          in out <an identifier> <a double-quoted delimited-identifier>
          table ... columns long double ref char time timestamp
          interval date binary national character nchar
```

21

Use the `SHOW ERRORS` command in SQL*Plus to display the compilation errors of a stored procedure. When you specify this option with no arguments, SQL*Plus displays the compilation errors for the most recently created or altered stored procedure. If SQL*Plus displays a compilation warnings message after you create or alter a stored procedure, you can use `SHOW ERRORS` commands to obtain more information.

With the introduction of support for PL/SQL warnings, the range of feedback messages is expanded to include a third message as follows:

```
SP2-08xx: <object> created with compilation warnings.
```

This enables you to differentiate between the occurrence of a compilation warning and a compilation error. You must correct an error if you want to use the stored procedure, whereas a warning is for informational purposes only.

The `SP2` prefix is included with the warning message because this provides you with the ability to look up the corresponding message number in the *SQL*Plus User's Guide and Reference* to determine the cause and action for the particular message.

Note: You can also view the compiler errors and warnings by using the `USER_ | ALL_ | DBA_ERRORS` data dictionary views.

Defining PLSQL_WARNINGS for Program Units

- The settings for the `PLSQL_WARNINGS` parameter are stored along with each compiled subprogram.
- If you recompile the subprogram using one of the following statements, the current settings for that session are used:
 - `CREATE OR REPLACE`
 - `ALTER ... COMPILE`
- If you recompile the subprogram using the `ALTER ... COMPILE` statement with the `REUSE SETTINGS` clause, the original setting stored with the program is used.

22

As already stated, the `PLSQL_WARNINGS` parameter can be set at the session level or at the system level.

The settings for the `PLSQL_WARNINGS` parameter are stored along with each compiled subprogram. If you recompile the subprogram with a `CREATE OR REPLACE` statement, the current settings for that session are used. If you recompile the subprogram with an `ALTER ... COMPILE` statement, then the current session setting is used unless you specify the `REUSE SETTINGS` clause in the statement, which uses the original setting that is stored with the subprogram.

Lesson Agenda

- Using the PL/SQL initialization parameters
- Using the PL/SQL compile-time warnings:
 - Using the `PLSQL_WARNING` initialization parameter
 - Using the `DBMS_WARNING` package subprograms



For Instructor Use Only.
This document should not be distributed.

Using the DBMS_WARNING Package

You can use the `DBMS_WARNING` package to:

- Manipulate the behavior of PL/SQL warning messages
- Change the definition of `PLSQL_WARNINGS` parameter
- Query, modify, and delete current system or session settings



24

Use the `DBMS_WARNING` package to programmatically manipulate the behavior of PL/SQL warning settings for the current system or session. The `DBMS_WARNING` package provides a way to manipulate the behavior of PL/SQL warning messages, in particular by reading and changing the setting of the `PLSQL_WARNINGS` initialization parameter, to control the types of warnings suppressed, displayed, or treated as errors. This package provides the interface to query, modify, and delete current system or session settings.

The `DBMS_WARNING` package is valuable if you are writing a development environment that compiles PL/SQL subprograms. By using the package interface routines, you can control PL/SQL warning messages programmatically to suit your requirements.

Using the DBMS_WARNING Package Subprograms

Scenario	Subprograms to Use
Set warnings	ADD_WARNING_SETTING_CAT (procedure) ADD_WARNING_SETTING_NUM (procedure)
Query warnings	GET_WARNING_SETTING_CAT (function) GET_WARNING_SETTING_NUM (function) GET_WARNING_SETTING_STRING (function)
Replace warnings	SET_WARNING_SETTING_STRING (procedure)
Get the warnings' categories names	GET_CATEGORY (function)

25

The following is a list of the DBMS_WARNING subprograms:

- **ADD_WARNING_SETTING_CAT:** Modifies the current session or system warning settings of the `warning_category` previously supplied
- **ADD_WARNING_SETTING_NUM:** Modifies the current session or system warning settings of the `warning_number` previously supplied
- **GET_CATEGORY:** Returns the category name, given the message number
- **GET_WARNING_SETTING_CAT:** Returns the specific warning category in the session
- **GET_WARNING_SETTING_NUM:** Returns the specific warning number in the session
- **GET_WARNING_SETTING_STRING:** Returns the entire warning string for the current session
- **SET_WARNING_SETTING_STRING:** Replaces previous settings with the new value

Note: For additional information about the preceding subprograms, refer to *Oracle Database PL/SQL Packages and Types Reference* or *Oracle Database PL/SQL Packages and Types Reference* (as applicable to the version you are using).

The DBMS_WARNING Procedures: Syntax, Parameters, and Allowed Values

```
EXECUTE DBMS_WARNING.ADD_WARNING_SETTING_CAT (-  
  warning_category      IN      VARCHAR2,  
  warning_value         IN      VARCHAR2,  
  scope                 IN      VARCHAR2);
```

```
EXECUTE DBMS_WARNING.ADD_WARNING_SETTING_NUM (-  
  warning_number        IN      NUMBER,  
  warning_value         IN      VARCHAR2,  
  scope                 IN      VARCHAR2);
```

```
EXECUTE DBMS_WARNING.SET_WARNING_SETTING_STRING (-  
  warning_value         IN      VARCHAR2,  
  scope                 IN      VARCHAR2);
```

26

The `warning_category` is the name of the category. The allowed values are:

- ALL
- INFORMATIONAL
- SEVERE
- PERFORMANCE.

The `warning_value` is the value for the category. The allowed values are:

- ENABLE
- DISABLE
- ERROR

The `warning_number` is the warning message number. The allowed values are all valid warning numbers.

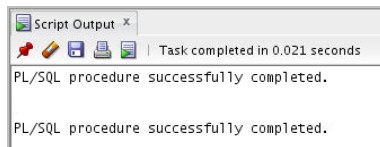
The `scope` specifies whether the changes are being performed in the session context or the system context. The allowed values are `SESSION` or `SYSTEM`. Using `SYSTEM` requires the `ALTER SYSTEM` privilege.

The DBMS_WARNING Procedures: Example

```
-- Establish the following warning setting string in the
-- current session:
-- ENABLE:INFORMATIONAL,
-- DISABLE:PERFORMANCE,
-- ENABLE:SEVERE

EXECUTE DBMS_WARNING.SET_WARNING_SETTING_STRING(-
  'ENABLE:ALL', 'SESSION');

EXECUTE DBMS_WARNING.ADD_WARNING_SETTING_CAT(-
  'PERFORMANCE','DISABLE','SESSION');
```



27

By using the SET_WARNING_SETTING_STRING procedure, you can set one warning setting. If you have multiple warning settings, you should perform the following steps:

1. Call SET_WARNING_SETTING_STRING to set the initial warning setting string.
2. Call ADD_WARNING_SETTING_CAT (or ADD_WARNING_SETTING_NUM) repeatedly to add more settings to the initial string.

The example in the slide establishes the following warning setting string in the current session:

```
ENABLE:INFORMATIONAL,DISABLE:PERFORMANCE,ENABLE:SEVERE
```

The DBMS_WARNING Functions: Syntax, Parameters, and Allowed Values

```
DBMS_WARNING.GET_WARNING_SETTING_CAT (-  
warning_category IN VARCHAR2) RETURN warning_value;
```

```
DBMS_WARNING.GET_WARNING_SETTING_NUM (-  
warning_number IN NUMBER) RETURN warning_value;
```

```
DBMS_WARNING.GET_WARNING_SETTING_STRING  
RETURN pls_integer;
```

```
DBMS_WARNING.GET_CATEGORY (-  
warning_number IN pls_integer) RETURN VARCHAR2;
```

28

The `warning_category` is the name of the category. The allowed values are:

- ALL
- INFORMATIONAL
- SEVERE
- PERFORMANCE

The `warning_number` is the warning message number. The allowed values are all valid warning numbers.

The `scope` specifies whether the changes are being performed in the session context or the system context. The allowed values are `SESSION` or `SYSTEM`. Using `SYSTEM` requires the `ALTER SYSTEM` privilege.

Note: Use the `GET_WARNING_SETTING_STRING` function when you do not have the `SELECT` privilege on the `v$parameter` or `v$parameter2` fixed tables, or if you want to parse the warning string yourself, and then modify and set the new value by using `SET_WARNING_SETTING_STRING`.

The DBMS_WARNING Functions: Example

```
-- Determine the current session warning settings
SET SERVEROUTPUT ON
EXECUTE DBMS_OUTPUT.PUT_LINE( -
DBMS_WARNING.GET_WARNING_SETTING_STRING);
```

```
PL/SQL procedure successfully completed.
ENABLE:INFORMATIONAL,DISABLE:PERFORMANCE,ENABLE:SEVERE
```

```
-- Determine the category for warning message number
-- PLW-07203
SET SERVEROUTPUT ON
EXECUTE DBMS_OUTPUT.PUT_LINE( -
DBMS_WARNING.GET_CATEGORY(7203));
```

```
PL/SQL procedure successfully completed.
PERFORMANCE
```

29

Note: The message numbers must be specified as positive integers because the data type for the GET_CATEGORY parameter is PLS_INTEGER (allowing positive integer values).

Using DBMS_WARNING: Example

```
CREATE OR REPLACE PROCEDURE compile_code(p_pkg_name VARCHAR2) IS
  v_warn_value  VARCHAR2(200);
  v_compile_stmt VARCHAR2(200) :=
    'ALTER PACKAGE '|| p_pkg_name ||' COMPILE';

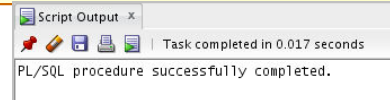
BEGIN
  v_warn_value := DBMS_WARNING.GET_WARNING_SETTING_STRING;
  DBMS_OUTPUT.PUT_LINE('Current warning settings: '||
    v_warn_value);
  DBMS_WARNING.ADD_WARNING_SETTING_CAT(
    'PERFORMANCE', 'DISABLE', 'SESSION');
  DBMS_OUTPUT.PUT_LINE('Modified warning settings: '||
    DBMS_WARNING.GET_WARNING_SETTING_STRING);
  EXECUTE IMMEDIATE v_compile_stmt;
  DBMS_WARNING.SET_WARNING_SETTING_STRING(v_warn_value,
    'SESSION');
  DBMS_OUTPUT.PUT_LINE('Restored warning settings: '||
    DBMS_WARNING.GET_WARNING_SETTING_STRING);
END;
/
```

30

In the example in the slide, the `compile_code` procedure is designed to compile a named PL/SQL package. The code suppresses the `PERFORMANCE` category warnings. The calling environment's warning settings must be restored after the compilation is performed. The code does not know what the calling environment warning settings are; it uses the `GET_WARNING_SETTING_STRING` function to save the current setting. This value is used to restore the calling environment setting using the `DBMS_WARNING.SET_WARNING_SETTING_STRING` procedure in the last line of the example code. Before compiling the package using Native Dynamic SQL, the `compile_code` procedure alters the current session warning level by disabling warnings for the `PERFORMANCE` category. The code also prints the original, modified, and the restored warning settings.

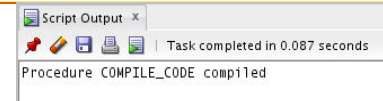
Using DBMS_WARNING: Example

```
EXECUTE DBMS_WARNING.SET_WARNING_SETTING_STRING(-  
  'ENABLE:ALL', 'SESSION');
```



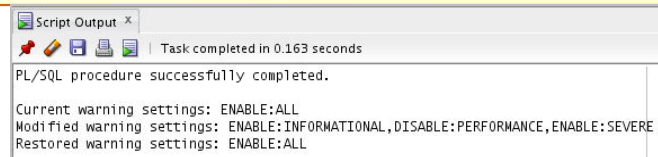
Script Output x
Task completed in 0.017 seconds
PL/SQL procedure successfully completed.

```
@/home/oracle/labs/plpu/code_ex/code_ex_scripts/code_12_33_s.sql
```



Script Output x
Task completed in 0.087 seconds
Procedure COMPILE_CODE compiled

```
EXECUTE compile_code('EMP_PKG');
```



Script Output x
Task completed in 0.163 seconds
PL/SQL procedure successfully completed.
Current warning settings: ENABLE:ALL
Modified warning settings: ENABLE:INFORMATIONAL, DISABLE:PERFORMANCE, ENABLE:SEVERE
Restored warning settings: ENABLE:ALL

31

The example in the slide tests the example provided in the previous slide. First, enable all compiler warnings. Next, run the script on the previous page. Finally, call the `compile_code` procedure and pass it an existing package name, `MY_PKG`, as a parameter.

Quiz



The categories of PL/SQL compile-time warning messages are:

- a. SEVERE
- b. PERFORMANCE
- c. INFORMATIONAL
- d. All
- e. CRITICAL

32

Answer: a, b, c, d

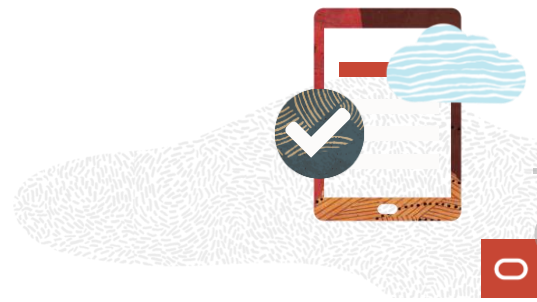
PL/SQL warning messages are divided into categories, so that you can suppress or display groups of similar warnings during compilation. The categories are:

- **SEVERE:** Messages for conditions that may cause unexpected behavior or wrong results, such as aliasing problems with parameters
- **PERFORMANCE:** Messages for conditions that may cause performance problems, such as passing a `VARCHAR2` value to a `NUMBER` column in an `INSERT` statement
- **INFORMATIONAL:** Messages for conditions that do not have an effect on performance or correctness, but that you may want to change to make the code more maintainable, such as unreachable code that can never be executed
- **ALL:** Displays all categories

Summary

In this lesson, you should have learned how to:

- Use the PL/SQL compiler initialization parameters
- Use the PL/SQL compile-time warnings

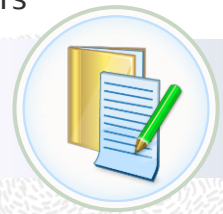


For Instructor Use Only.
This document should not be distributed.

Practice 21 Overview: Tuning PL/SQL Compiler

This practice covers the following topics:

- Displaying the compiler initialization parameters
- Enabling native compilation for your session and compiling a procedure
- Disabling the compiler warnings, and then restoring the original session-warning settings
- Identifying the categories for some compiler-warning message numbers



0

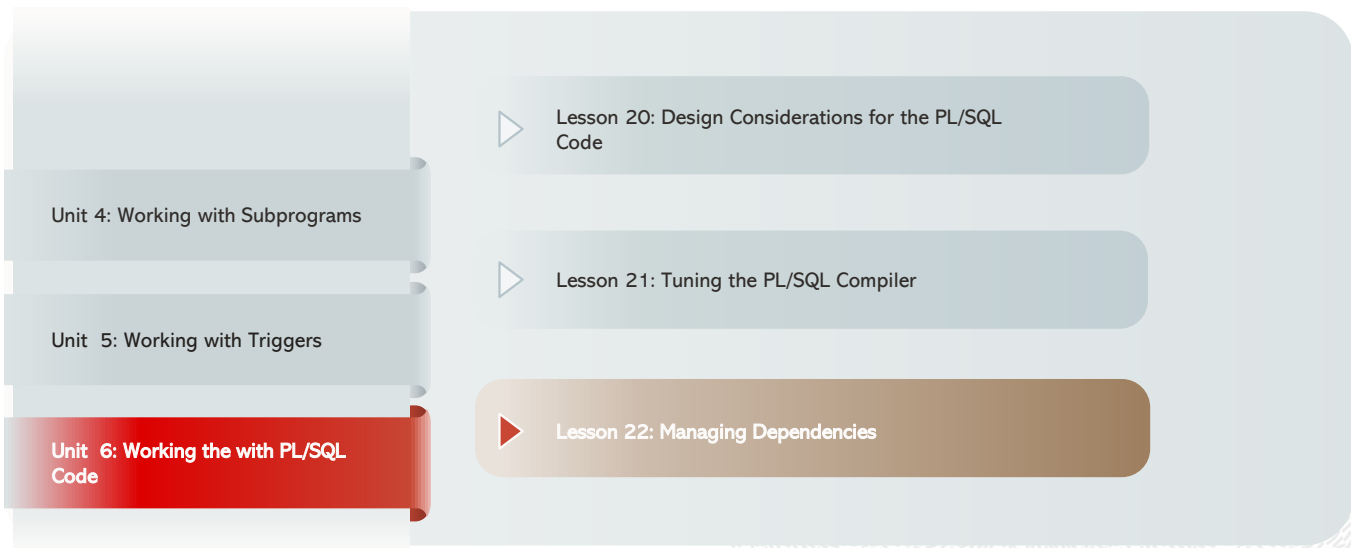
34

In this practice, you display the compiler initialization parameters. You then enable native compilation for your session and compile a procedure. You then suppress all compiler warnings categories and then restore the original session-warning settings. Finally, you identify the categories for some compiler-warning message numbers.

For Instructor Use Only.
This document should not be distributed.

Managing Dependencies

Course Road Map



2

In Unit 6, you will learn the design aspects of the PL/SQL code. You will also learn how to use PL/SQL compiler and how to manage dependencies among different objects in the database application.

Objectives

After completing this lesson, you should be able to:

- Track procedural dependencies
- Predict the effect of changing a database object on procedures and functions
- Manage procedural dependencies
- Manage local and remote dependencies



0

This lesson introduces you to object dependencies, and implicit and explicit recompilation of invalid objects.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Schema object dependencies
- Direct and indirect dependencies
- Fine-grained dependency management
- Managing remote dependencies
- Revalidating PL/SQL program units
- Package dependency management

What are Dependencies in a Schema?

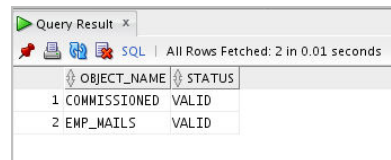
- Dependency refers to a situation where one schema object depends on another object for its definition.
- Examples
 - There is a dependency between a view and the set of tables on which it is defined.
 - There is a dependency between the procedure and the tables on which the procedure is defined.

How Dependencies Work?

Let us create the following views in the HR schema:

```
CREATE OR REPLACE VIEW commissioned AS SELECT first_name, last_name, commission_pct FROM employees WHERE  
commission_pct > 0.00;  
  
CREATE OR REPLACE VIEW emp_mails AS SELECT first_name, last_name, email FROM employees;
```

```
SELECT object_name, status  
FROM user_objects  
WHERE object_type = 'VIEW'  
ORDER BY object_name;
```



	OBJECT_NAME	STATUS
1	COMMISSIONED	VALID
2	EMP_MAILS	VALID

6

In the slide, we create two views – `commissioned` and `emp_mails` – on the table `employees`. Both the views are dependent on the schema object `employees` table.

There is a dependency between the table `employees`, and views `commissioned` and `emp_mails`. The views are dependent on the table `employees`.

The view `commissioned` uses columns `first_name`, `last_name`, and `commission_pct` columns of the `employees` table.

The view `emp_mails` uses columns `first_name`, `last_name`, and `email` columns of the `employees` table.

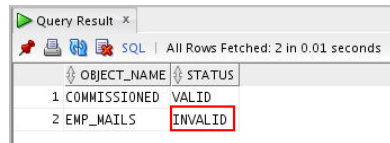
How Dependencies Work?

Let us modify the email column of the employees table:

```
ALTER TABLE employees MODIFY email VARCHAR2(100);
```

And check the state of the views:

```
SELECT object_name, status
FROM user_objects
WHERE object_type = 'VIEW'
ORDER BY object_name;
```



	OBJECT_NAME	STATUS
1	COMMISSIONED	VALID
2	EMP_MAILS	INVALID

You can see that after the email column in the employees table is modified, the status emp_mails view has become invalid.

7

When we modify the definition of the email column in the employees table, the view emp_mails, which uses the email column, has been invalidated because of the dependency between the view and the table.

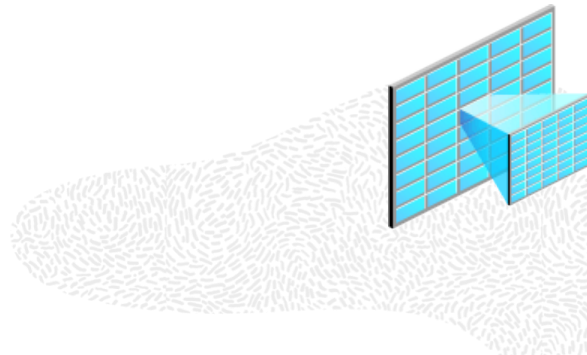
The view was defined on a different version of the email column, which has now changed. Therefore, the emp_mails view is invalidated.

Dependent and Referenced Objects

Some types of schema objects can reference other objects in their definitions.

For example:

- A view is defined by a query that references tables.
 - View is a dependent object.
 - Tables are referenced objects.



0

8

Some types of schema objects can reference other objects in their definitions. For example, a view is defined by a query that references tables or other views, and the body of a subprogram can include SQL statements that reference other objects. If the definition of object A references object B, then A is a dependent object (with respect to B) and B is a referenced object (with respect to A).

Dependency Issues

- If you alter the definition of a referenced object, dependent objects may or may not continue to work properly. For example, if the table definition is changed, the procedure may or may not continue to work without error.
- The Oracle server automatically records dependencies among objects. To manage dependencies, all schema objects have a status (valid or invalid) that is recorded in the data dictionary and you can view the status in the `USER_OBJECTS` data dictionary view.
- If the status of a schema object is `VALID`, then the object has been compiled and can be immediately used when referenced.
- If the status of a schema object is `INVALID`, then the schema object must be compiled before it can be used.

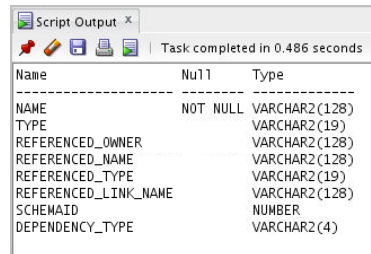
The following table shows schema objects can be dependent, referenced, or both

Object Type	Can Be Dependent or Referenced
Package body	Dependent only
Package specification	Both
Sequence	Referenced only
Subprogram	Both
Synonym	Both
Table	Both
Trigger	Both
User-defined object	Both
User-defined collection	Both
View	Both

For Instructor Use Only.
This document should not be distributed.

Querying Object Dependencies: Using the USER_DEPENDENCIES View

```
desc user_dependencies;
```



Script Output x
Task completed in 0.486 seconds

Name	Null	Type
NAME	NOT NULL	VARCHAR2(128)
TYPE		VARCHAR2(19)
REFERENCED_OWNER		VARCHAR2(128)
REFERENCED_NAME		VARCHAR2(128)
REFERENCED_TYPE		VARCHAR2(19)
REFERENCED_LINK_NAME		VARCHAR2(128)
SCHEMAID		NUMBER
DEPENDENCY_TYPE		VARCHAR2(4)

```
SELECT name, type, referenced_name, referenced_type  
FROM user_dependencies  
WHERE referenced_name IN ('EMPLOYEES', 'EMP_VW' )  
  
...
```

	NAME	TYPE	REFERENCED_NAME	REFERENCED_TYPE
1	SECURE_EMPLOYEES	TRIGGER	EMPLOYEES	TABLE
2	DML_CALL_SQL	FUNCTION	EMPLOYEES	TABLE
3	GET_EMP	FUNCTION	EMPLOYEES	TABLE
4	SECURE_EMP	TRIGGER	EMPLOYEES	TABLE
5	EMPLOYEE_SAL	PROCEDURE	EMPLOYEES	TABLE
6	EMP_MAILS	VIEW	EMPLOYEES	TABLE
7	UPDATE_JOB_HISTORY	TRIGGER	EMPLOYEES	TABLE
8	RAISE_SAL	PROCEDURE	EMPLOYEES	TABLE

10

You can determine dependencies in the schema from the `USER_DEPENDENCIES` data dictionary view.

The `ALL_DEPENDENCIES` and `DBA_DEPENDENCIES` views contain the additional `OWNER` column, which references the owner of the object.

The USER_DEPENDENCIES Data Dictionary View Columns

The columns of the `USER_DEPENDENCIES` data dictionary view are as follows:

- **NAME:** The name of the dependent object
- **TYPE:** The type of the dependent object (PROCEDURE, FUNCTION, PACKAGE, PACKAGE BODY, TRIGGER, or VIEW)
- **REFERENCED_OWNER:** The schema of the referenced object
- **REFERENCED_NAME:** The name of the referenced object
- **REFERENCED_TYPE:** The type of the referenced object
- **REFERENCED_LINK_NAME:** The database link used to access the referenced object
- **SCHEMA_ID:** Opaque schema identifier (16 bytes)
- **DEPENDENCY_TYPE:** Indicates whether the dependency is a REF dependency

Querying an Object's Status

Every database object has one of the following status values:

Status	Description
VALID	The object was successfully compiled by using the current definition in the data dictionary.
COMPILED WITH ERRORS	The most recent attempt to compile the object produced errors.
INVALID	The object is marked invalid because an object that it references has changed. (Only a dependent object can be invalid.)
UNAUTHORIZED	An access privilege on a referenced object was revoked. (Only a dependent object can be unauthorized.)

11

Every database object has one of the status values shown in the table in the slide.

Note: The `USER_OBJECTS`, `ALL_OBJECTS`, and `DBA_OBJECTS` static data dictionary views do not distinguish between `Compiled with errors`, `Invalid`, and `Unauthorized`; instead, they describe all these as `INVALID`.

Categorizing Dependencies

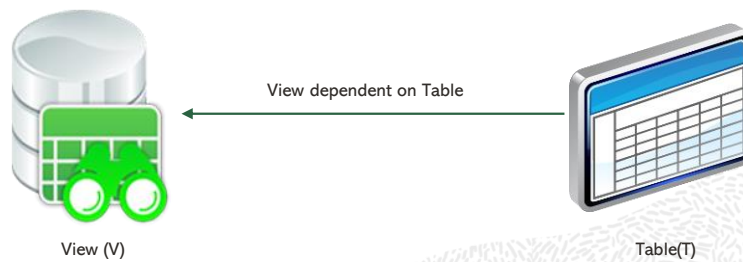
- Local dependencies – Both the referenced and dependent objects are on the same database. Oracle Database manages them implicitly. There are two types of dependencies in local dependencies using the internal tables.
 - Direct dependencies
 - Indirect dependencies
- Remote dependencies – The referenced and dependent objects are on different nodes across the network. Oracle Database uses different mechanisms to manage remote dependencies, depending on the objects involved.

Lesson Agenda

- Schema object dependencies
- **Direct and indirect dependencies**
- Fine-grained dependency management
- Managing remote dependencies
- Revalidating PL/SQL program units
- Package dependency management

Direct Dependencies

- When a view *V* is defined on a table *T*, there is a direct dependency of the view *V* on the table *T*.
- Direct dependents are invalidated only by changes to the referenced objects that affect them.

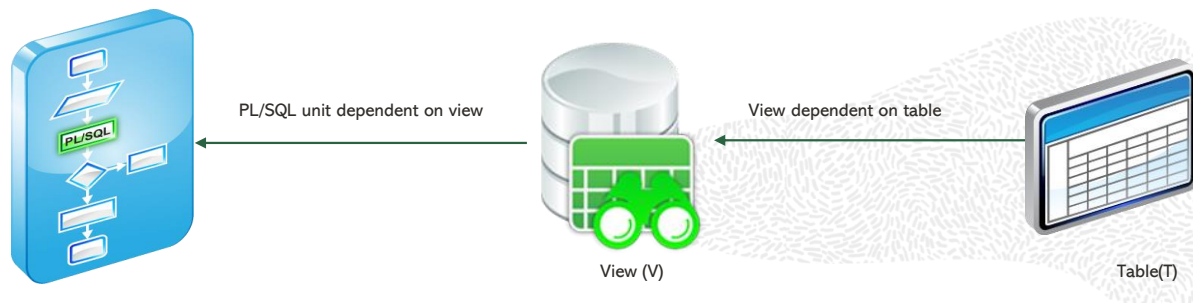


14

The slide shows a direct dependency between the table and the view defined on that table. A modification to the table structure might invalidate the view, as we have seen in the case of view earlier.

Indirect Dependencies

- Consider a PL/SQL program unit dependent on the view V, which in turn is dependent on the table T. The PL/SQL program unit is indirectly dependent on the table T.
- Indirect dependents can be invalidated by changes to the referenced object that do not affect them.
- The PL/SQL unit may be invalidated if there is a modification to the table.



15

The slide shows an indirect dependency between the PL/SQL unit and the table. So, the PL/SQL unit is an indirect dependent on the table.

Indirect dependents can be invalidated by changes to the reference object that do not affect them. A modification to the table may invalidate the PL/SQL unit. This is called cascading invalidation.

Displaying Direct and Indirect Dependencies

- Run the `utldtree.sql` script that creates the objects that enable you to display the direct and indirect dependencies.

```
@/home/oracle/labs/plpu/labs/utldtree.sql
```

- This script creates four objects, as given below:
 - A table `deptree temptab` to hold dependency data
 - A procedure `deptree_fill` to populate the table
 - Two views – `deptree` and `ideptree` – to select and format dependency data from the populated table

16

Displaying Direct and Indirect Dependencies by Using Views Provided by Oracle

You can display direct and indirect dependencies from additional user views, called `DEPTREE` and `IDEPTREE`; these views are provided by Oracle.

Example

1. Make sure that the `utldtree.sql` script has been executed. This script is located in the `$ORACLE_HOME/labs/plpu/labs` folder. You can run the script as follows:

```
@?/labs/plpu/labs/utldtree.sql
```

Note: In this class, this script is supplied in the `labs` folder of your class files. The code example above uses the student account `ORA61`. (This applies to a Linux environment. If the file is not found, locate the file in your labs subdirectory.)

2. Populate the `DEPTREE_TEMPLTAB` table with information for a particular referenced object by invoking the `DEPTREE_FILL` procedure. There are three parameters for this procedure:

<i>object_type</i>	Type of the referenced object
<i>object_owner</i>	Schema of the referenced object
<i>object_name</i>	Name of the referenced object

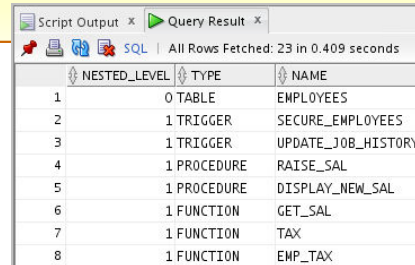
Displaying Direct and Indirect Dependencies

- Execute the DEPTREE_FILL procedure.

```
EXECUTE deptree_fill('TABLE', 'ORA61', 'EMPLOYEES')
```

- Display the dependencies

```
SELECT  nested_level, type, name
FROM    deptree
ORDER BY seq#;
```

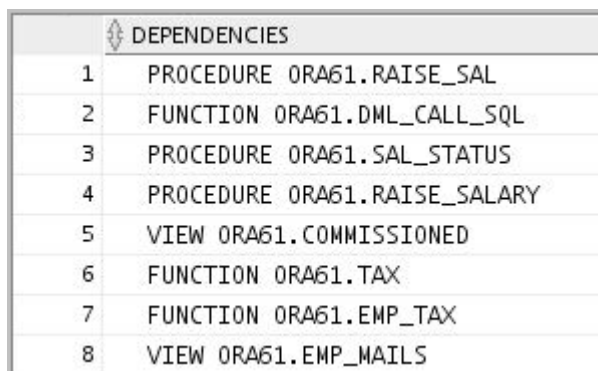


NESTED_LEVEL	TYPE	NAME
1	TABLE	EMPLOYEES
2	TRIGGER	SECURE_EMPLOYEES
3	TRIGGER	UPDATE_JOB_HISTORY
4	PROCEDURE	RAISE_SAL
5	PROCEDURE	DISPLAY_NEW_SAL
6	FUNCTION	GET_SAL
7	FUNCTION	TAX
8	FUNCTION	EMP_TAX

17

You can display a tabular representation of all dependent objects by querying the DEPTREE view. You can display an indented representation of the same information by querying the IDEPTREE view, which consists of a single column named DEPENDENCIES as follows:

```
SELECT *
FROM    ideptree;
```



DEPENDENCIES
1 PROCEDURE ORA61.RAISE_SAL
2 FUNCTION ORA61.DML_CALL_SQL
3 PROCEDURE ORA61.SAL_STATUS
4 PROCEDURE ORA61.RAISE_SALARY
5 VIEW ORA61.COMMISSIONED
6 FUNCTION ORA61.TAX
7 FUNCTION ORA61.EMP_TAX
8 VIEW ORA61.EMP_MAILS

...

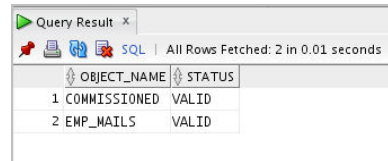
Lesson Agenda

- Schema object dependencies
- Direct and indirect dependencies
- **Fine-grained dependency management**
- Managing remote dependencies
- Revalidating PL/SQL program units
- Package dependency management

Fine-Grained Dependency Management

```
CREATE OR REPLACE VIEW commissioned AS SELECT first_name, last_name, commission_pct FROM employees WHERE  
commission_pct > 0.00;  
CREATE OR REPLACE VIEW emp_mails AS SELECT first_name, last_name, email FROM employees;
```

```
SELECT object_name, status  
FROM user_objects  
WHERE object_type = 'VIEW'  
ORDER BY object_name;
```



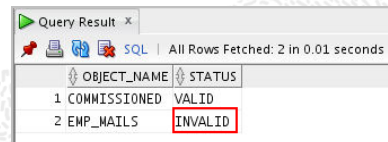
Query Result x

All Rows Fetched: 2 in 0.01 seconds

	OBJECT_NAME	STATUS
1	COMMISSIONED	VALID
2	EMP_MAILS	VALID

```
ALTER TABLE employees MODIFY email VARCHAR2(100);
```

```
SELECT object_name, status  
FROM user_objects  
WHERE object_type = 'VIEW'  
ORDER BY object_name;
```



Query Result x

All Rows Fetched: 2 in 0.01 seconds

	OBJECT_NAME	STATUS
1	COMMISSIONED	VALID
2	EMP_MAILS	INVALID

19

Let us reconsider the scenario that we discussed earlier. We created two views, `commissioned` and `emp_mails`, on the `employees` table.

The `commissioned` view has the `first_name`, `last_name`, and `commission_pct` columns of the `employees` table.

The `emp_mails` view has the `first_name`, `last_name`, and `email` columns of the `employees` table.

When we altered the definition of the `email` column of the `employees` table, only the `emp_mails` view is invalidated. The `commissioned` view is still valid.

This process of invalidating specific objects based on their definition is fine-grained dependency management. Whenever a referenced object is modified, only those dependent objects are invalidated that are effected by the modification, and not all the dependent objects.

Fine-Grained Dependency Management

- Starting with Oracle Database 11g, dependencies are now tracked at the level of element within unit.
- Element-based dependency tracking covers the following:
 - Dependency of a single-table view on its base table
 - Dependency of a PL/SQL program unit (package specification, package body, or subprogram) on the following:
 - Other PL/SQL program units
 - Tables
 - Views

Fine-Grained Dependency Management: Example 1

```
CREATE TABLE t2 (col_a NUMBER, col_b NUMBER, col_c NUMBER);
CREATE VIEW v AS SELECT col_a, col_b FROM t2;
```

```
SELECT ud.name, ud.type, ud.referenced_name,
       ud.referenced_type, uo.status
FROM user_dependencies ud, user_objects uo
WHERE ud.name = uo.object_name AND ud.name = 'V';
```

Results:					
	NAME	TYPE	REFERENCED_NAME	REFERENCED_TYPE	STATUS
1	V	VIEW	T2	TABLE	VALID

```
ALTER TABLE t2 ADD (col_d VARCHAR2(20));
```

```
SELECT ud.name, ud.type, ud.referenced_name,
       ud.referenced_type, uo.status
FROM user_dependencies ud, user_objects uo
WHERE ud.name = uo.object_name AND ud.name = 'V';
```

Results:					
	NAME	TYPE	REFERENCED_NAME	REFERENCED_TYPE	STATUS
1	V	VIEW	T2	TABLE	VALID

21

Example of Dependency of a Single-Table View on Its Base Table

In the first example in the slide, table T2 is created with three columns: COL_A, COL_B, and COL_C. A view named V is created based on columns COL_A and COL_B of table T2. The dictionary views are queried and view V is dependent on table T and its status is valid.

In the third example, table T2 is altered. A new column named COL_D is added. The dictionary views still report that view V is dependent because element-based dependency tracking grasps that columns COL_A and COL_B are not modified and, therefore, the view does not need to be invalidated.

Fine-Grained Dependency Management: Example 1

```
ALTER TABLE t2 MODIFY (col_a VARCHAR2(20));  
SELECT ud.name, ud.referenced_name, ud.referenced_type, uo.status  
FROM user_dependencies ud, user_objects uo  
WHERE ud.name = uo.object_name AND ud.name = 'V';
```

Results:				
	NAME	REFERENCED_NAME	REFERENCED_TYPE	STATUS
1	V	T2	TABLE	INVALID

22

In the example in the slide, the view is invalidated because its element (COL_A) is modified in the table on which the view is dependent.

Fine-Grained Dependency Management: Example 2

```
CREATE OR REPLACE PACKAGE pkg IS
  PROCEDURE proc_1;
END pkg;
/
CREATE OR REPLACE PROCEDURE p IS
BEGIN
  pkg.proc_1();
END p;
/
CREATE OR REPLACE PACKAGE pkg
IS
  PROCEDURE proc_1;
  PROCEDURE unheard_of;
END pkg;
/
```

```
PACKAGE PKG compiled
PROCEDURE P compiled
PACKAGE PKG compiled
```

23

In the example in the slide, you create a package named `PKG` that has procedure `PROC_1` declared.

A procedure named `P` invokes `PKG.PROC_1`.

The definition of the `PKG` package is modified and another subroutine is added to the package declaration.

When you query the `USER_OBJECTS` dictionary view for the status of the `P` procedure, it is still valid as shown because the element you added to the definition of `PKG` is not referenced through procedure `P`.

```
SELECT status FROM user_objects
WHERE object_name = 'P';
```

Results:	
	STATUS
1	VALID

Guidelines for Reducing Invalidation

To reduce invalidation of dependent objects:

- Add new items to the end of the package
- Reference each table through a view

24

Add New Items to the End of the Package

When adding new items to a package, add them to the end of the package. This preserves the slot numbers and entry-point numbers of existing top-level package items, thereby preventing their invalidation. For example, consider the following package:

```
CREATE OR REPLACE PACKAGE pkg1 IS
  FUNCTION get_var RETURN VARCHAR2;
  PROCEDURE set_var (v VARCHAR2);
END;
```

Adding an item to the end of `pkg1` does not invalidate dependents that reference `get_var`. Inserting an item between the `get_var` function and the `set_var` procedure invalidates dependents that reference the `set_var` function.

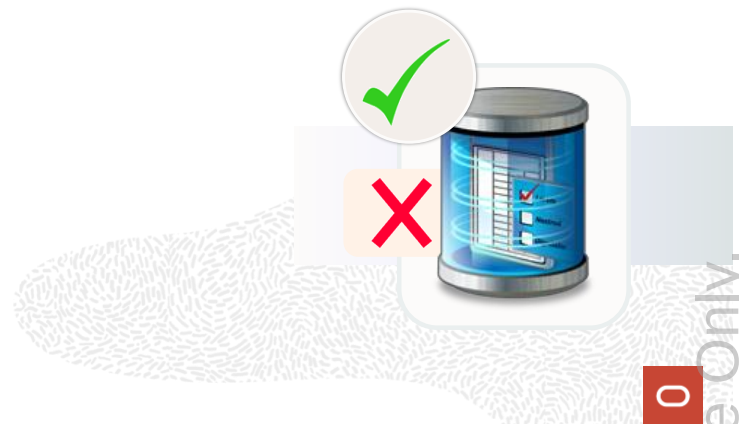
Reference Each Table Through a View

Reference tables indirectly by using views. This allows you to do the following:

- Add columns to the table without invalidating dependent views or dependent PL/SQL objects
- Modify or delete columns not referenced by the view without invalidating dependent objects

Object Revalidation

- An object that is not valid when it is referenced must be validated before it can be used.
- Validation occurs automatically when an object is referenced; it does not require explicit user action.
- Revalidation may not happen automatically if:
 - The object is compiled with errors
 - The object is an unauthorized object
 - The object is an invalid object



25

The compiler cannot automatically revalidate an object that compiled with errors. The compiler recompiles the object, and if it recompiles without errors, it is revalidated; otherwise, it remains invalid.

The compiler checks whether the unauthorized object has access privileges to all of its referenced objects. If so, the compiler revalidates the unauthorized object without recompiling it. If not, the compiler issues appropriate error messages.

The SQL compiler recompiles the invalid object. If the object recompiles without errors, it is revalidated; otherwise, it remains invalid.

For an invalid PL/SQL program unit (procedure, function, or package), the PL/SQL compiler checks whether any referenced object is an invalid object.

- If so, the compiler recompiles the invalid object. If the object recompiles without errors, it is revalidated; otherwise, it remains invalid.
- If not, the compiler revalidates the invalid object without recompiling it. Fast revalidation is usually performed on objects that were invalidated due to cascading invalidation.

Lesson Agenda

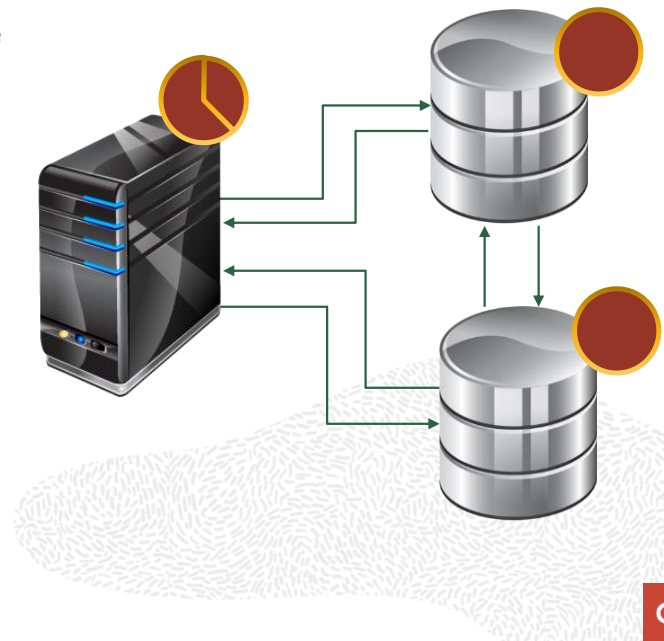
- Schema object dependencies
- Direct and indirect dependencies
- Fine-grained dependency management
- **Managing remote dependencies**
- Revalidating PL/SQL program units
- Package dependency management



For Instructor Use Only.
This document should not be distributed.

Remote Dependencies

- The referenced and dependent objects are on different nodes across the network.
- Oracle Database does not manage dependencies among remote schema objects other than local-procedure-to-remote-procedure dependencies.



27

In the case of remote dependencies, the objects are on separate nodes. The local stored procedure and all its dependent objects are invalidated, but are automatically recompile when called for the first time. Oracle Database does not manage dependencies among remote schema objects other than local-procedure-to-remote-procedure dependencies.

For example, assume that a local view is created and defined by a query that references a remote table. Also assume that a local procedure includes a SQL statement that references the same remote table. Later, the definition of the table is altered.

As a result, the local view and procedure are never invalidated, even if the view or procedure is used after the table is altered and even if the view or procedure now returns errors when used. In this case, the view or procedure must be altered manually so that errors are not returned. In such cases, lack of dependency management is preferable to unnecessary recompilations of dependent objects.

Recompilation of Dependent Objects: Local and Remote

- Verify successful explicit recompilation of the dependent remote procedures and implicit recompilation of the dependent local procedures by checking the status of these procedures within the `USER_OBJECTS` view.
- If an automatic implicit recompilation of the dependent local procedures fails, the status remains invalid and the Oracle server issues a runtime error. Therefore, to avoid disrupting production, it is strongly recommended that you recompile local dependent objects manually, rather than rely on an automatic mechanism.

Managing Remote Procedure Dependencies

Mechanisms used for managing remote procedure dependencies are:

- Timestamp checking



- Signature checking



28

TIMESTAMP Checking

Each PL/SQL program unit carries a time stamp that is set when it is created or recompiled. Whenever you alter a PL/SQL program unit or a relevant schema object, all its dependent program units are marked as invalid and must be recompiled before they can execute. The actual time stamp comparison occurs when a statement in the body of a local procedure calls a remote procedure.

SIGNATURE Checking

For each PL/SQL program unit, both the time stamp and the signature are recorded. The signature of a PL/SQL construct contains information about the following:

- The name of the construct (procedure, function, or package)
- The base types of the parameters of the construct
- The modes of the parameters (IN, OUT, or IN OUT)
- The number of the parameters

The recorded time stamp in the calling program unit is compared with the current time stamp in the called remote program unit. If the time stamps match, the call proceeds. If they do not match, the remote procedure call layer performs a simple comparison of the signature to determine whether the call is safe or not. If the signature has not been changed in an incompatible manner, execution continues; otherwise, an error is returned.

For Instructor Use Only.
This document should not be distributed.

Setting the REMOTE_DEPENDENCIES_MODE Parameter

- You can define whether a remote procedure call should undergo Timestamp checking or Signature checking by setting the REMOTE_DEPENDENCIES_MODE parameter.
 - At the system level:
`ALTER SYSTEM SET REMOTE_DEPENDENCIES_MODE = TIMESTAMP | SIGNATURE`
 - At the session level:
`ALTER SESSION SET REMOTE_DEPENDENCIES_MODE = TIMESTAMP | SIGNATURE`
 - As an `init.ora` parameter:
`REMOTE_DEPENDENCIES_MODE = TIMESTAMP | SIGNATURE`
- If the REMOTE_DEPENDENCIES_MODE parameter is not specified, either in the `init.ora` parameter file or by using the ALTER SESSION or ALTER SYSTEM DDL statements, TIMESTAMP is the default value.

Timestamp Checking



Remote procedure B:
compiles and is **VALID**
at 8:00 AM

30

Local Procedures Referencing Remote Procedures

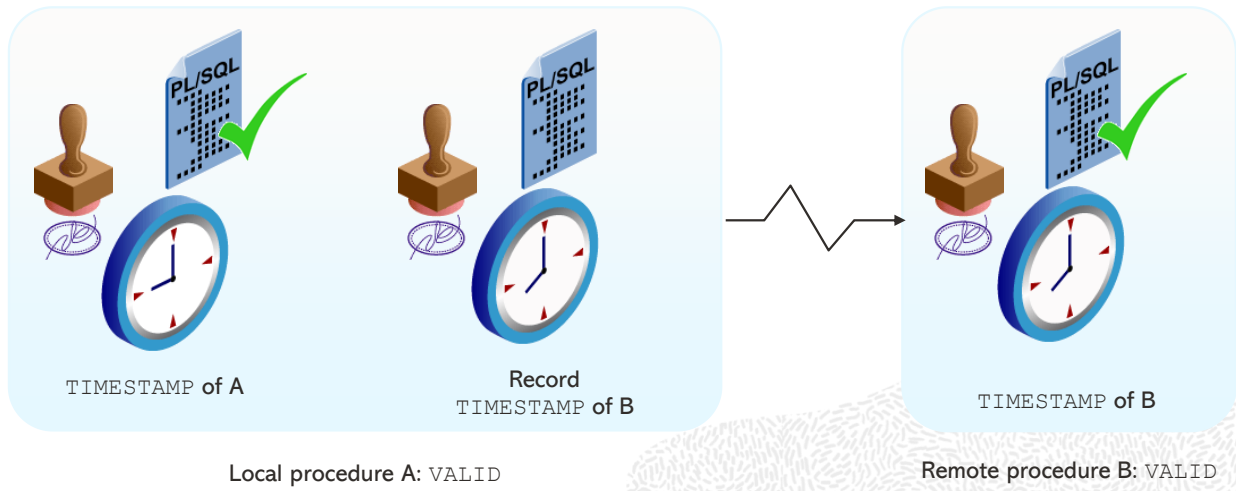
A local procedure that references a remote procedure is invalidated by the Oracle server if the remote procedure is recompiled after the local procedure is compiled.

Automatic Remote Dependency Mechanism

When a procedure compiles, the Oracle server records the time stamp of that compilation within the **P** code of the procedure.

In the slide, when the remote procedure B is successfully compiled at 8:00 AM, this time is recorded as its time stamp.

Timestamp Checking



31

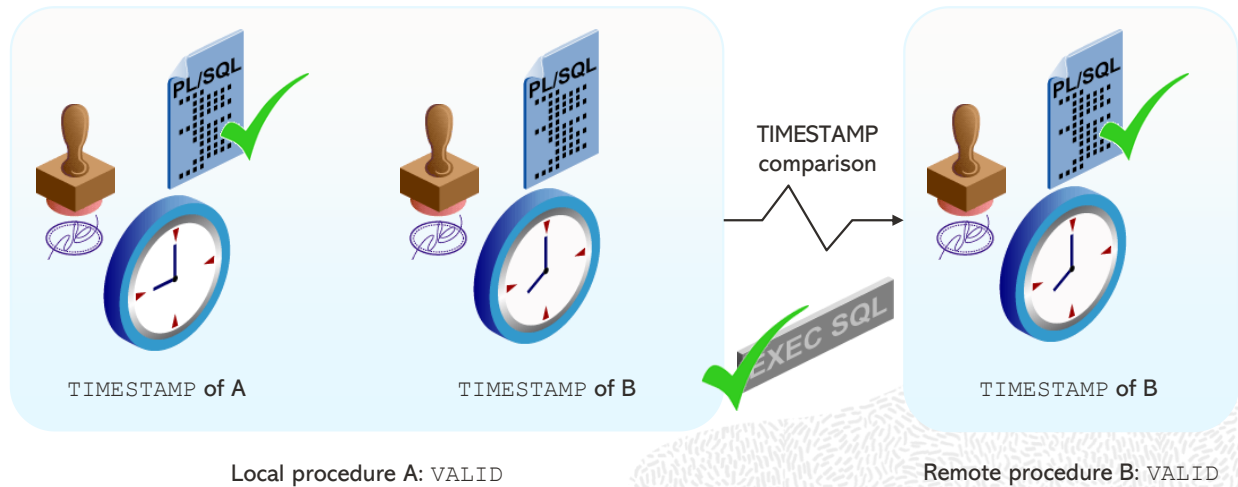
Local Procedures Referencing Remote Procedures

Automatic Remote Dependency Mechanism

When a local procedure referencing a remote procedure compiles, the Oracle server also records the time stamp of the remote procedure in the P code of the local procedure.

In the slide, local procedure A (which is dependent on remote procedure B) is compiled at 9:00 AM. The time stamps of both procedure A and remote procedure B are recorded in the P code of procedure A.

Timestamp Checking



32

Automatic Remote Dependency

When the local procedure is invoked at run time, the Oracle server compares the two time stamps of the referenced remote procedure.

If the time stamps are equal (indicating that the remote procedure has not recompiled), then the Oracle server executes the local procedure.

In the example in the slide, the time stamp recorded with the `P` code of remote procedure B is the same as that recorded with local procedure A. Therefore, local procedure A is valid.

Timestamp Checking



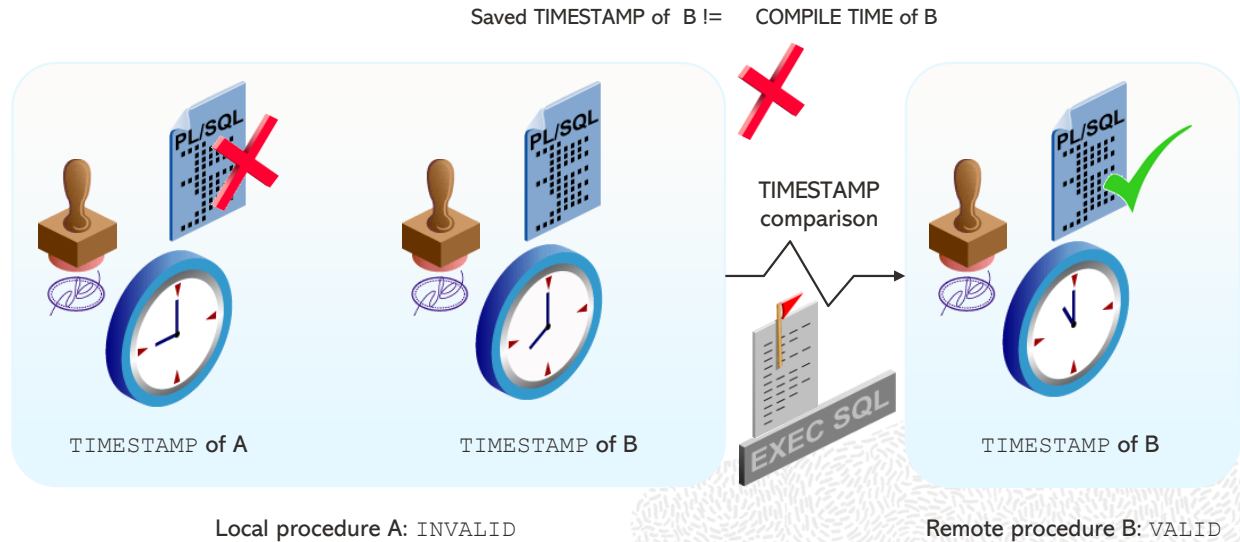
Remote procedure B:
Recompiles and is `VALID`
at 11:00 AM

33

Local Procedures Referencing Remote Procedures

Assume that remote procedure B is successfully recompiled at 11:00 AM. The new time stamp is recorded along with its `P` code.

Timestamp Checking



34

Automatic Remote Dependency

If the time stamps are not equal (indicating that the remote procedure has recompiled), then the Oracle server invalidates the local procedure and returns a runtime error. If the local procedure (which is now tagged as invalid) is invoked a second time, then the Oracle server recompiles it before executing, in accordance with the automatic local dependency mechanism.

Note: If a local procedure returns a runtime error the first time it is invoked (indicating that the remote procedure's time stamp has changed), then you should develop a strategy to reinvoke the local procedure.

In the example in the slide, the remote procedure is recompiled at 11:00 AM and this time is recorded as its time stamp in the P code. The P code of local procedure A still has 8:00 AM as the time stamp for remote procedure B. Because the time stamp recorded with the P code of local procedure A is different from that recorded with the remote procedure B, the local procedure is marked invalid. When the local procedure is invoked for the second time, it can be successfully compiled and marked valid.

A disadvantage of the time stamp mode is that it is unnecessarily restrictive. Recompilation of dependent objects across the network is often performed when not strictly necessary, leading to performance degradation.

Signature Checking

- The signature of a procedure is:
 - The name of the procedure
 - The data types of the parameters
 - The modes of the parameters
- The signature of the remote procedure is saved in the local procedure.
- When executing a dependent procedure, the signature of the referenced remote procedure is compared.

35

To alleviate some of the problems with the time stamp-only dependency model, you can use the signature model. This allows the remote procedure to be recompiled without affecting the local procedures. This is important if the database is distributed.

The signature of a subprogram contains the following information:

- The name of the subprogram
- The data types of the parameters
- The modes of the parameters
- The number of parameters
- The data type of the return value for a function

If a remote program is changed and recompiled but the signature does not change, then the local procedure can execute the remote procedure. With the time stamp method, an error would have been raised because the time stamps would not have matched.

Lesson Agenda

- Schema object dependencies
- Direct and indirect dependencies
- Fine-grained dependency management
- Managing remote dependencies
- **Revalidating PL/SQL program units**
- Package dependency management

Revalidating PL/SQL Program Units

You revalidate a PL/SQL program unit by recompiling it. There are two ways of recompiling:

- Implicit runtime recompilation
 - Local dependencies are resolved implicitly
- Explicit recompilation with the `ALTER` statement
 - Here is the syntax for using the `ALTER` statement.

```
ALTER PROCEDURE|FUNCTION | TRIGGER|PACKAGE [SCHEMA.]object_name COMPILE;
```

37

Recompiling PL/SQL Objects

If the recompilation is successful, the object becomes valid. If not, the Oracle server returns an error and the object remains invalid. When you recompile a PL/SQL object, the Oracle server first recompiles any invalid object on which it depends. This is implicit recompilation or revalidation of the objects. Local dependencies are generally resolved implicitly.

You explicitly resolve dependencies by using the `ALTER` statement. You may have to explicitly recompile the PL/SQL program units if the implicit recompilation is unsuccessful. You have to resolve such dependencies by using the `ALTER` statement.

Unsuccessful Recompilation

Recompiling-dependent procedures and functions are unsuccessful when:

- The referenced objects are compiled with errors
- The referenced objects are dropped or renamed
- The data type of the referenced column is changed
- The referenced column is dropped
- A referenced view is replaced by a view with different columns
- The parameter list of a referenced procedure is modified

38

Sometimes, a recompilation of dependent procedures is unsuccessful (for example, when a referenced table is dropped or renamed).

The success of any recompilation is based on the exact dependency. If a referenced view is recreated, any object that is dependent on the view needs to be recompiled. The success of the recompilation depends on the columns that the view now contains, as well as the columns that the dependent objects require for their execution. If the required columns are not part of the new view, then the object remains invalid.

Successful Recompilation

Recompiling-dependent procedures and functions are successful if:

- The referenced table has new columns
- The data type of referenced columns has not changed
- A private table is dropped, but a public table that has the same name and structure exists
- The PL/SQL body of a referenced procedure has been modified and recompiled successfully

39

The recompilation of dependent objects is successful if:

- New columns are added to a referenced table
- All `INSERT` statements include a column list
- No new column is defined as `NOT NULL`

When a private table is referenced by a dependent procedure and the private table is dropped, the status of the dependent procedure would become invalid. When the procedure is recompiled (either explicitly or implicitly) and a public table exists, the procedure can recompile successfully but is now dependent on the public table. The recompilation is successful only if the public table contains the columns that the procedure requires; otherwise, the status of the procedure remains invalid.

Recompiling Procedures

Minimize dependency failures by:

- Declaring records with the `%ROWTYPE` attribute
- Declaring variables with the `%TYPE` attribute
- Querying with the `SELECT *` notation
- Including a column list with `INSERT` statements

40

You can minimize recompilation failure by following the guidelines that are shown in the slide.

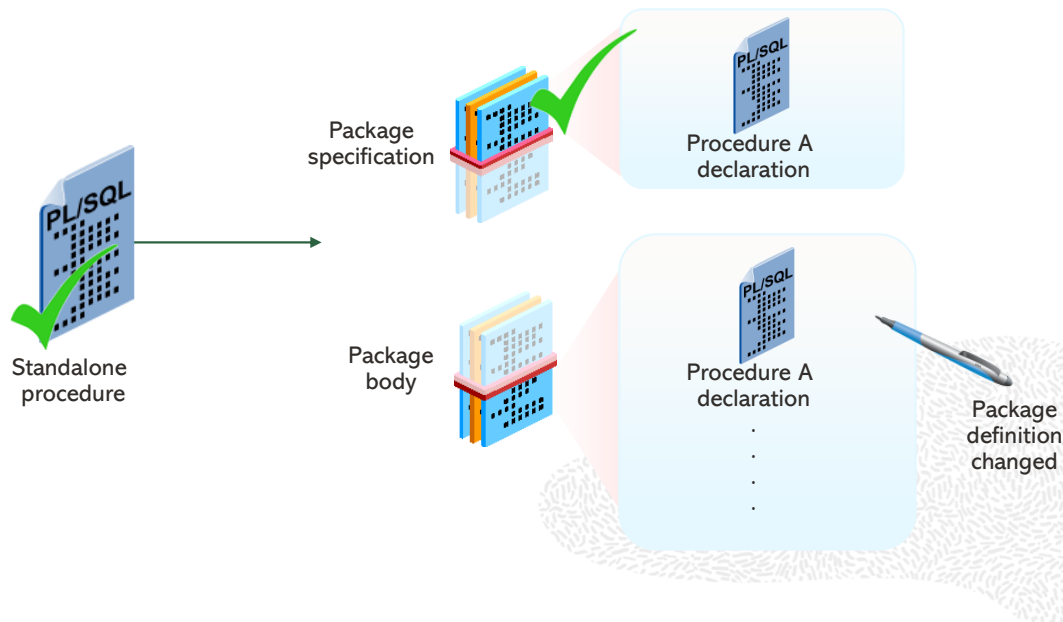
Lesson Agenda

- Schema object dependencies
- Direct and indirect dependencies
- Fine-grained dependency management
- Managing remote dependencies
- Revalidating PL/SQL program units
- Package dependency management



For Instructor Use Only.
This document should not be distributed.

Packages and Dependencies: Subprogram References the Package

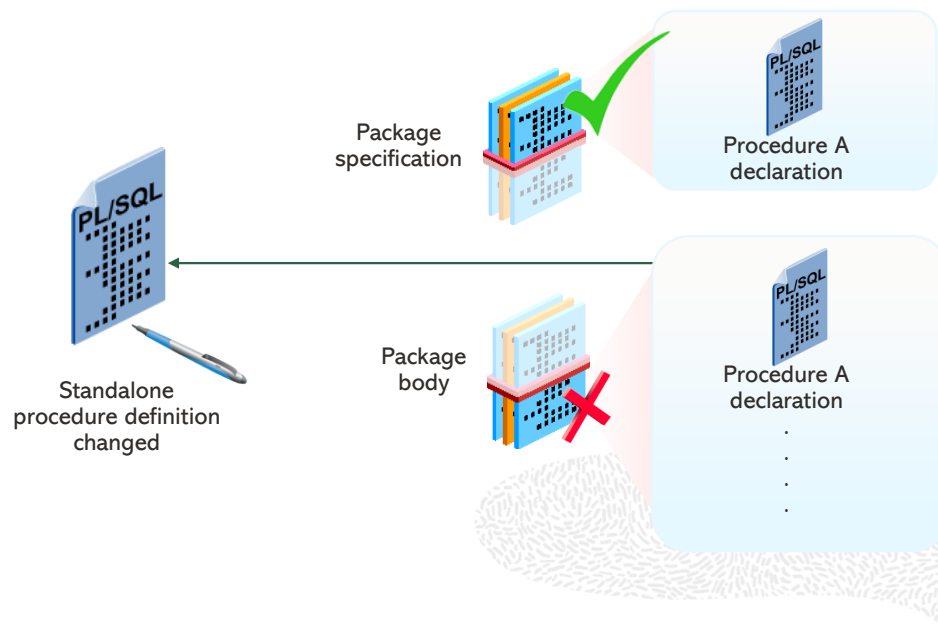


42

You can simplify dependency management with packages when referencing a package procedure or function from a standalone procedure or function.

- If the package body changes and the package specification does not change, then the standalone procedure that references a package construct remains valid.
- If the package specification changes, then the outside procedure referencing a package construct is invalidated.

Packages and Dependencies: Package Subprogram References Procedure



43

If a standalone procedure that is referenced within the package changes, then the entire package body is invalidated, but the package specification remains valid. Therefore, it is recommended that all the procedures referenced by the procedures in a package body should be in the package body.

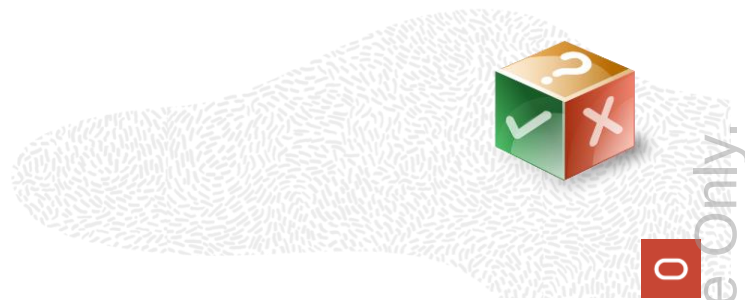
Quiz



You can display direct and indirect dependencies by running the `utldtree.sql` script, populating the `DEPTREE_TEMPTAB` table with information for a particular referenced object, and querying the `DEPTREE` or `IDEPTREE` views.

- a. True
- b. False

44



0

Answer: a

Displaying Direct and Indirect Dependencies

You can display direct and indirect dependencies as follows:

1. Run the `utldtree.sql` script, which creates the objects that enable you to display the direct and indirect dependencies.
2. Populate the `DEPTREE_TEMPTAB` table with information for a particular referenced object by executing the `DEPTREE_FILL` procedure.
3. Query the `DEPTREE` or `IDEPTREE` views.

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Track procedural dependencies
- Predict the effect of changing a database object on procedures and functions
- Manage procedural dependencies
- Manage local and remote dependencies



45

Avoid disrupting production by keeping a track of dependent procedures and recompiling them manually as soon as possible after the definition of a database object changes.

For Instructor Use Only.
This document should not be distributed.

Practice 22 Overview: Managing Dependencies in Your Schema

This practice covers the following topics:

- Using `DEPTREE_FILL` and `IDEPTREE` to view dependencies
- Recompiling procedures, functions, and packages



0

46

In this practice, you use the `DEPTREE_FILL` procedure and the `IDEPTREE` view to investigate dependencies in your schema. In addition, you recompile invalid procedures, functions, packages, and views.

For Instructor Use Only.
This document should not be distributed.

ORACLE

A

Using SQL Developer

0

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this appendix, you should be able to do the following:

- List the key features of Oracle SQL Developer
- Identify the menu items of Oracle SQL Developer
- Create a database connection
- Manage database objects
- Use SQL Worksheet
- Save and run SQL scripts
- Create and save reports
- Browse the Data Modeling options in SQL Developer



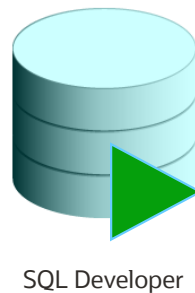
2

In this appendix, you are introduced to the graphical tool called SQL Developer. You learn how to use SQL Developer for your database development tasks. You learn how to use SQL Worksheet to execute SQL statements and SQL scripts.

For Instructor Use Only.
This document should not be distributed.

What Is Oracle SQL Developer?

- Oracle SQL Developer is a graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema by using standard Oracle database authentication.



3

Oracle SQL Developer is a free graphical tool designed to improve your productivity and simplify the development of everyday database tasks. With just a few clicks, you can easily create and debug stored procedures, test SQL statements, and view optimizer plans.

SQL Developer, which is the visual tool for database development, simplifies the following tasks:

- Browsing and managing database objects
- Executing SQL statements and scripts
- Editing and debugging PL/SQL statements
- Creating reports

You can connect to any target Oracle database schema by using standard Oracle database authentication. When connected, you can perform operations on objects in the database.

SQL Developer is the interface to administer the Oracle Application Express Listener. The new interface enables you to specify global settings and multiple database settings with different database connections for the Application Express Listener. SQL Developer provides the option to drag and drop objects by table or column name onto the worksheet. It provides improved DB Diff comparison options, GRANT statements support in the SQL editor, and DB Doc reporting. Additionally, SQL Developer includes support for 19c Database features.

Specifications of SQL Developer

- Is shipped along with Oracle Database 19c
- Is developed in Java
- Supports Windows, Linux, and Mac OS X platforms
- Enables default connectivity using the JDBC Thin driver
- Connects to Oracle Database version 9.2.0.1 and later

4

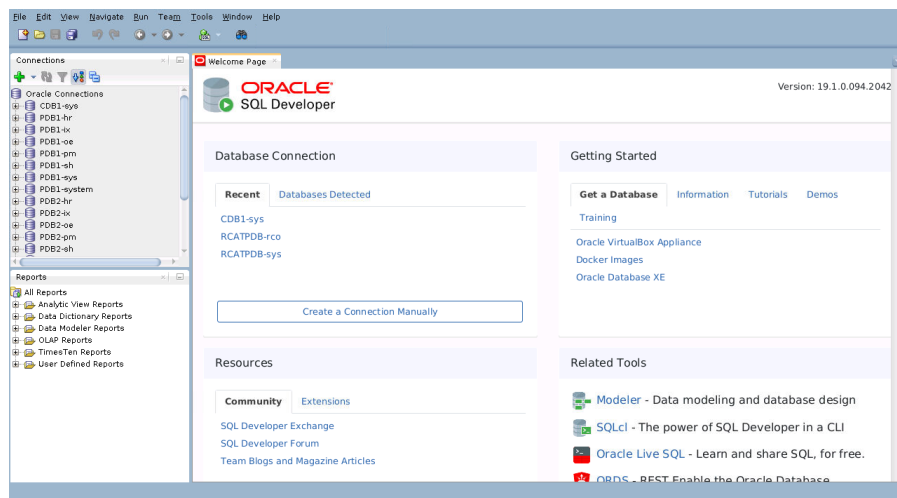
Oracle SQL Developer is shipped along with Oracle Database 19c by default. SQL Developer is developed in Java, leveraging the Oracle JDeveloper integrated development environment (IDE). Therefore, it is a cross-platform tool. The tool runs on Windows, Linux, and Mac operating system (OS) X platforms.

The default connectivity to the database is through the Java Database Connectivity (JDBC) Thin driver, and therefore, no Oracle Home is required. SQL Developer does not require an installer and you need to simply unzip the downloaded file. With SQL Developer, users can connect to Oracle Databases 9.2.0.1 and later, and all Oracle database editions, including Express Edition.

Note

- For Oracle Database 19c, you will have to download and install SQL Developer. SQL Developer can be downloaded for free from the following link:
<http://www.oracle.com/technetwork/developer-tools/sql-developer/downloads/index.html>
- For instructions on how to install SQL Developer, see the website at:
<http://www.oracle.com/technetwork/developer-tools/sql-developer/overview/index.html>

SQL Developer Interface



5

The SQL Developer interface contains three main navigation tabs, from left to right:

- **Connections tab:** By using this tab, you can browse database objects and users to which you have access.
- **Reports tab:** Identified by the Reports icon, this tab enables you to run predefined reports or create and add your own reports.
- **Files tab:** Identified by the Files folder icon, this tab enables you to access files from your local machine without having to use the File > Open menu.

General Navigation and Use

SQL Developer uses the left side for navigation to find and select objects, and the right side to display information about selected objects. You can customize many aspects of the appearance and behavior of SQL Developer by setting preferences.

Note: You need to define at least one connection to be able to connect to a database schema and issue SQL queries or run procedures and functions.

Menus

The following menus contain standard entries, plus entries for features that are specific to SQL Developer:

- **View:** Contains options that affect what is displayed in the SQL Developer interface
- **Navigate:** Contains options for navigating to panes and for executing subprograms
- **Run:** Contains the Run File and Execution Profile options that are relevant when a function or procedure is selected, and also debugging options
- **Versioning:** Provides integrated support for the following versioning and source control systems
 - Concurrent Versions System (CVS) and Subversion
- **Tools:** Invokes SQL Developer tools such as SQL*Plus, Preferences, and SQL Worksheet. It also contains options related to migrating third-party databases to Oracle.

Note: The Run menu also contains options that are relevant when a function or procedure is selected for debugging.

For Instructor Use Only.
This document should not be distributed.

Creating a Database Connection

- You must have at least one database connection to use SQL Developer.
- You can create and test connections for:
 - Multiple databases
 - Multiple schemas
- SQL Developer automatically imports any connections defined in the `tnsnames.ora` file on your system.
- You can export connections to an Extensible Markup Language (XML) file.
- Each additional database connection created is listed in the Connections Navigator hierarchy.

7

A connection is a SQL Developer object that specifies the necessary information for connecting to a specific database as a specific user of that database. To use SQL Developer, you must have at least one database connection, which may be existing, created, or imported.

You can create and test connections for multiple databases and for multiple schemas.

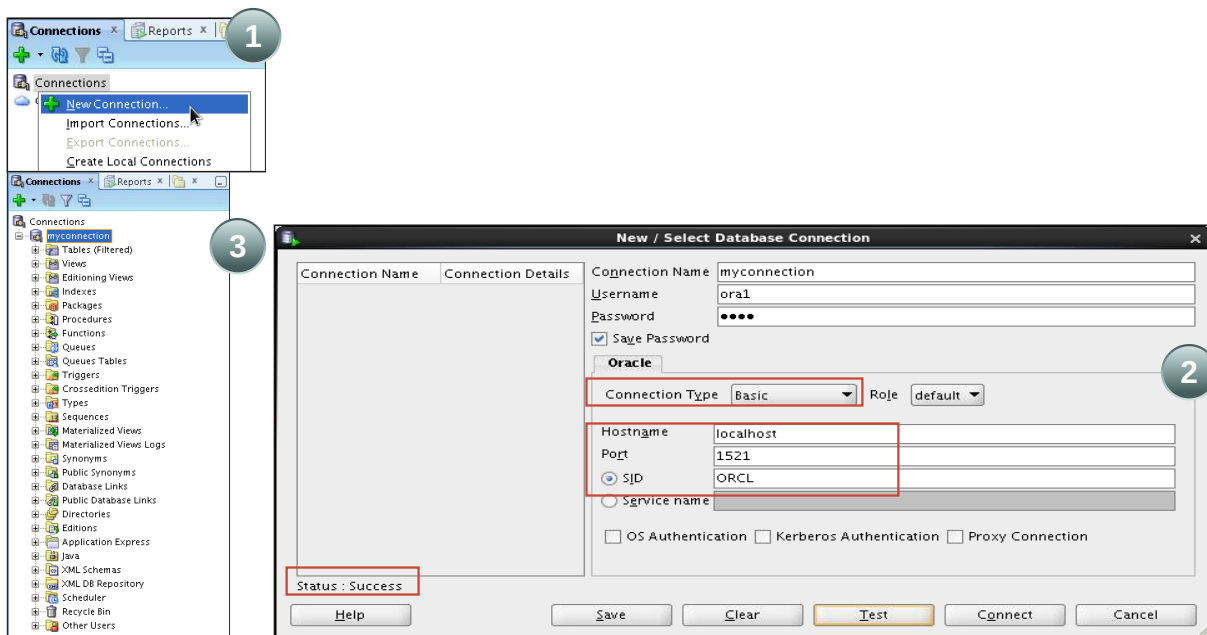
By default, the `tnsnames.ora` file is located in the `$ORACLE_HOME/network/admin` directory, but it can also be in the directory specified by the `TNS_ADMIN` environment variable or registry value. When you start SQL Developer and open the Database Connections dialog box, SQL Developer automatically imports any connections defined in the `tnsnames.ora` file on your system.

Note: On Windows, if the `tnsnames.ora` file exists, but its connections are not being used by SQL Developer, define `TNS_ADMIN` as a system environment variable.

You can export connections to an XML file so that you can reuse it.

You can create additional connections as different users to the same database or to connect to the different databases.

Creating a Database Connection



8

To create a database connection, perform the following steps:

1. On the Connections tabbed page, right-click **Connections** and select **New Connection**.
2. In the New/Select Database Connection window, enter the connection name. Enter the username and password of the schema that you want to connect to.
 - a. From the Role drop-down list, you can select either *default* or *SYSDBA*. (You choose *SYSDBA* for the *sys* user or any user with database administrator privileges.)
 - b. You can select the connection type as:

Basic: In this type, enter host name and SID for the database that you want to connect to. Port is already set to 1521. You can also choose to enter the Service name directly if you use a remote database connection.

TNS: You can select any one of the database aliases imported from the `tnsnames.ora` file.

LDAP: You can look up database services in Oracle Internet Directory, which is a component of Oracle Identity Management.

Advanced: You can define a custom Java Database Connectivity (JDBC) URL to connect to the database.

Local/Bequeath: If the client and database exist on the same computer, a client connection can be passed directly to a dedicated server process without going through the listener.

- c. Click Test to ensure that the connection has been set correctly.
- d. Click Connect.

If you select the Save Password check box, the password is saved to an XML file. So, after you close the SQL Developer connection and open it again, you are not prompted for the password.

- 3. The connection gets added in the Connections Navigator. You can expand the connection to view the database objects and view object definitions—for example, dependencies, details, statistics, and so on.

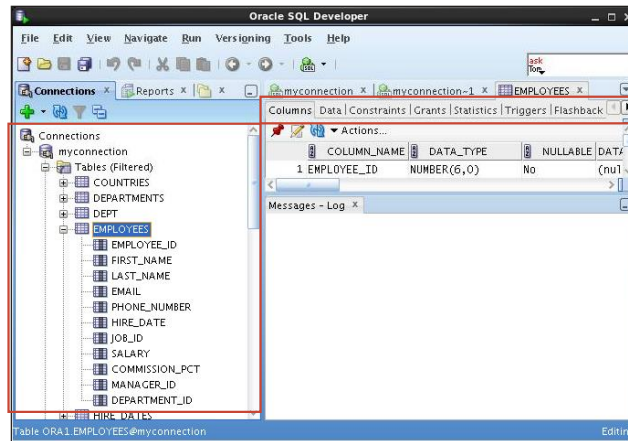
Note: From the same New/Select Database Connection window, you can define connections to non-Oracle data sources using the Access, MySQL, and SQL Server tabs. However, these connections are read-only connections that enable you to browse objects and data in that data source.

For Instructor Use Only.
This document should not be distributed.

Browsing Database Objects

Use the Connections Navigator to:

- Browse through many objects in a database schema
- Review the definitions of objects at a glance



10

After you create a database connection, you can use the Connections Navigator to browse through many objects in a database schema, including Tables, Views, Indexes, Packages, Procedures, Triggers, and Types.

SQL Developer uses the left side for navigation to find and select objects, and the right side to display information about the selected objects. You can customize many aspects of the appearance of SQL Developer by setting preferences.

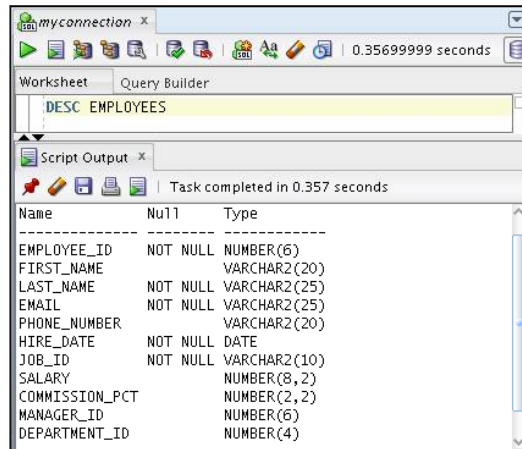
You can see the definition of the objects broken into tabs of information that is pulled out of the data dictionary. For example, if you select a table in the Navigator, details about columns, constraints, grants, statistics, triggers, and so on are displayed on an easy-to-read tabbed page.

If you want to see the definition of the `EMPLOYEES` table as shown in the slide, perform the following steps:

1. Expand the Connections node in the Connections Navigator.
2. Expand Tables.
3. Click `EMPLOYEES`. By default, the Columns tab is selected. It shows the column description of the table. Using the Data tab, you can view the table data and also enter new rows, update data, and commit these changes to the database.

Displaying the Table Structure

Use the `DESCRIBE` command to display the structure of a table:

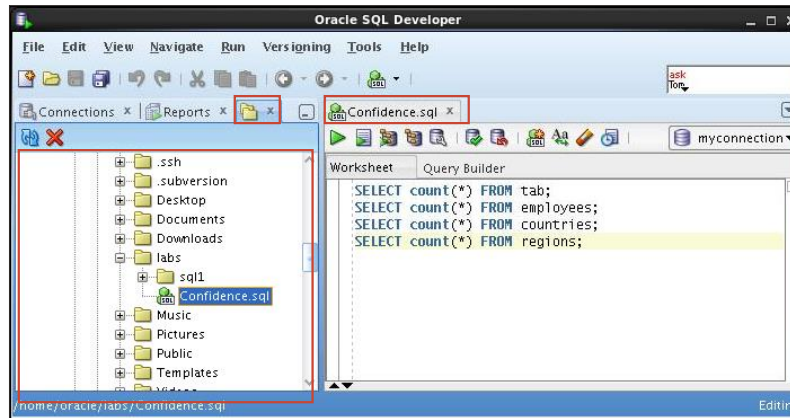


11

In SQL Developer, you can also display the structure of a table using the `DESCRIBE` command. The result of the command is a display of column names and data types, as well as an indication if a column must contain data.

Browsing Files

Use the File Navigator to explore the file system and open system files.



12

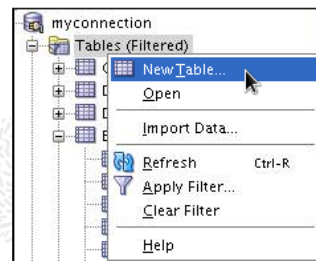
Browsing Database Objects

You can use the File Navigator to browse and open system files.

- To view the File Navigator, click the View tab and select Files, or select View > Files.
- To view the contents of a file, double-click a file name to display its contents in the SQL Worksheet area.

Creating a Schema Object

- SQL Developer supports the creation of any schema object by:
 - Executing a SQL statement in SQL Worksheet
 - Using the context menu
- Edit the objects by using an edit dialog box or one of the many context-sensitive menus.
- View the data definition language (DDL) for adjustments such as creating a new object or editing an existing schema object.



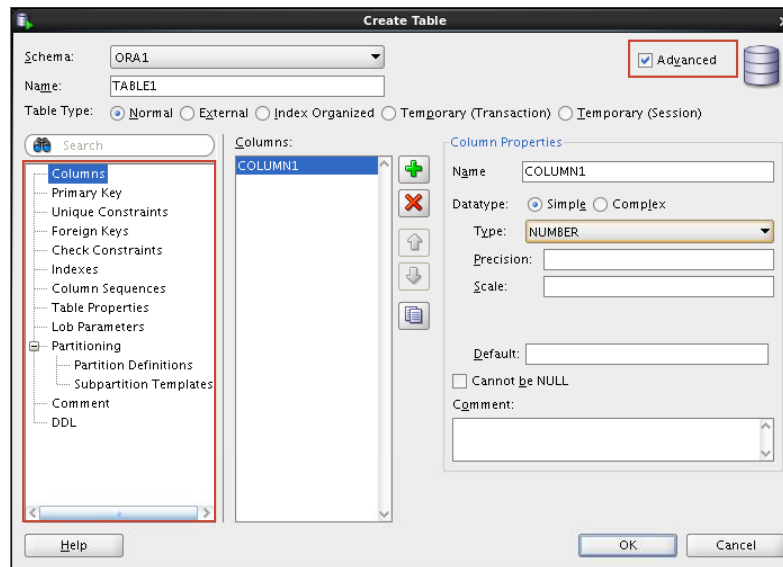
13

SQL Developer supports the creation of any schema object by executing a SQL statement in SQL Worksheet. Alternatively, you can create objects by using the context menus. When created, you can edit objects using an edit dialog box or one of the many context-sensitive menus.

As new objects are created or existing objects are edited, the DDL for those adjustments is available for review. An Export DDL option is available if you want to create the full DDL for one or more objects in the schema.

The slide shows how to create a table using the context menu. To open a dialog box for creating a new table, right-click Tables and select New Table. The dialog boxes to create and edit database objects have multiple tabs, each reflecting a logical grouping of properties for that type of object.

Creating a New Table: Example



14

In the Create Table dialog box, if you do not select the Advanced check box, you can create a table quickly by specifying columns and some frequently-used features.

If you select the Advanced check box, the Create Table dialog box changes to one with multiple options, in which you can specify an extended set of features while you create the table.

The example in the slide shows how to create the `DEPENDENTS` table by selecting the Advanced check box.

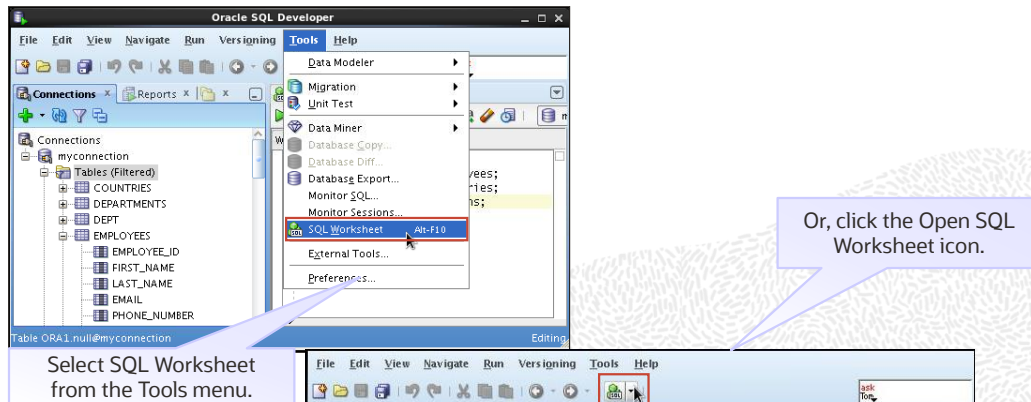
To create a new table, perform the following steps:

1. In the Connections Navigator, right-click Tables and select Create TABLE.
2. In the Create Table dialog box, select Advanced.
3. Specify the column information.
4. Click OK.

Although it is not required, you should also specify a primary key by using the Primary Key tab in the dialog box. Sometimes, you may want to edit the table that you have created; to do so, right-click the table in the Connections Navigator and select Edit.

Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL *Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



15

When you connect to a database, a SQL Worksheet window for that connection automatically opens. You can use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL *Plus statements. The SQL Worksheet supports SQL*Plus statements to a certain extent. SQL*Plus statements that are not supported by the SQL Worksheet are ignored and not passed to the database.

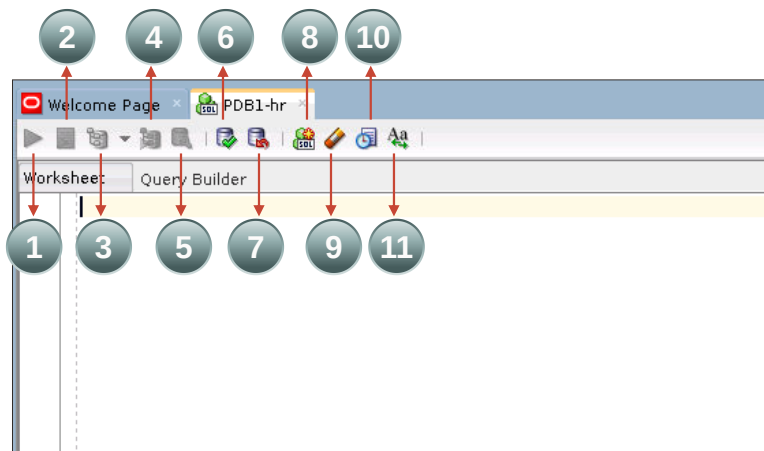
You can specify the actions that can be processed by the database connection associated with the worksheet, such as:

- Creating a table
- Inserting data
- Creating and editing a trigger
- Selecting data from a table
- Saving the selected data to a file

You can display a SQL Worksheet by using one of the following:

- Select Tools > SQL Worksheet.
- Click the Open SQL Worksheet icon.

Using the SQL Worksheet



16

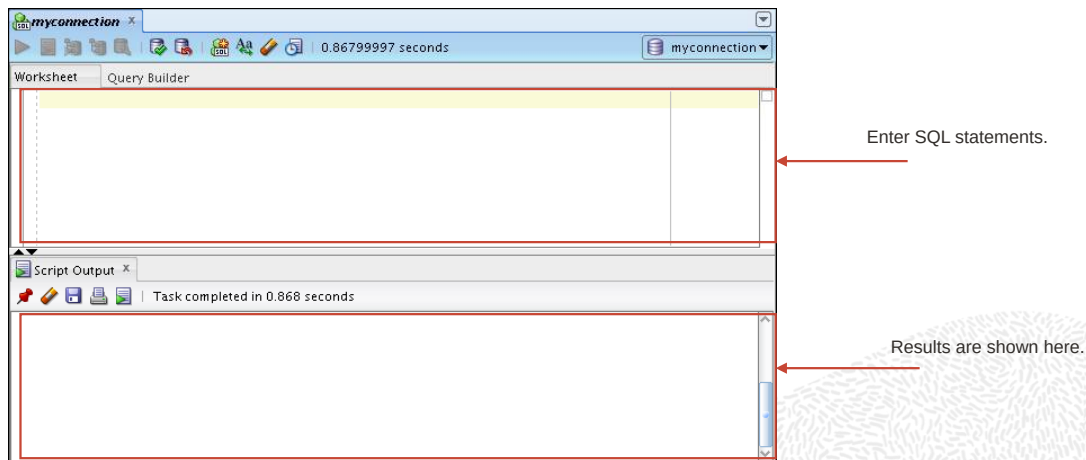
You may want to use the shortcut keys or icons to perform certain tasks such as executing a SQL statement, running a script, and viewing the history of the SQL statements that you have executed. You can use the SQL Worksheet toolbar that contains icons to perform the following tasks:

1. **Run Statement:** Executes the statement where the cursor is located in the Enter SQL Statement box. You can use bind variables in the SQL statements, but not substitution variables.
2. **Run Script:** Executes all the statements in the Enter SQL Statement box by using the Script Runner. You can use substitution variables in the SQL statements, but not bind variables.
3. **Explain Plan:** Generates the execution plan, which you can see by clicking the Explain tab
4. **Autotrace:** Generates trace information for the statement
5. **SQL Tuning Advisory:** Analyzes high-volume SQL statements and offers tuning recommendations
6. **Commit:** Writes any changes to the database and ends the transaction
7. **Rollback:** Discards any changes to the database, without writing them to the database, and ends the transaction

8. **Unshared SQL Worksheet:** Creates a separate unshared SQL Worksheet for a connection
9. **Clear:** Erases the statement or statements in the Enter SQL Statement box
10. **SQL History:** Displays a dialog box with information about the SQL statements that you have executed
11. **To Upper/Lower/InitCap:** Changes the selected text to uppercase, lowercase, or initcap, respectively

For Instructor Use Only.
This document should not be distributed.

Using the SQL Worksheet



18

When you connect to a database, a SQL Worksheet window for that connection automatically opens. You can use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements. All SQL and PL/SQL commands are supported as they are passed directly from the SQL Worksheet to the Oracle database. The SQL*Plus commands that are used in SQL Developer must be interpreted by the SQL Worksheet before being passed to the database.

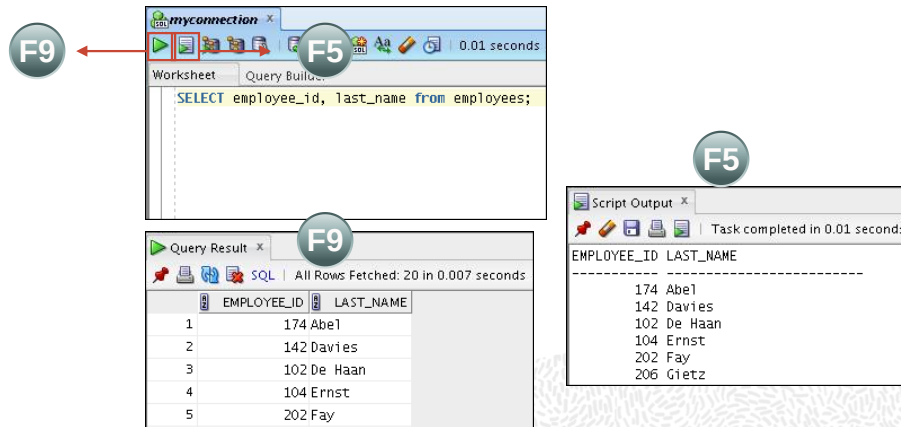
The SQL Worksheet currently supports a number of SQL*Plus commands. Commands that are not supported by the SQL Worksheet are ignored and not sent to the Oracle database. Through the SQL Worksheet, you can execute the SQL statements and some of the SQL*Plus commands.

You can display a SQL Worksheet by using any of the following options:

- Select Tools > SQL Worksheet.
- Click the Open SQL Worksheet icon.

Executing SQL Statements

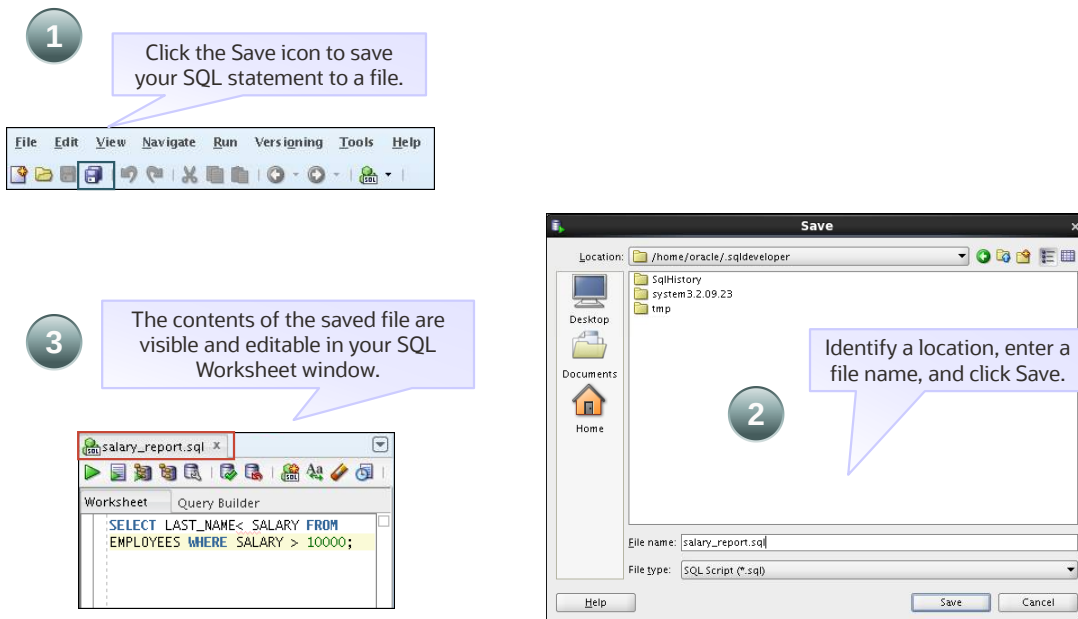
Use the Enter SQL Statement box to enter single or multiple SQL statements.



19

The example in the slide shows the difference in output for the same query when the F9 key or Execute Statement is used versus the output when F5 or Run Script is used.

Saving SQL Scripts



20

You can save your SQL statements from the SQL Worksheet to a text file. To save the contents of the Enter SQL Statement box, perform the following steps:

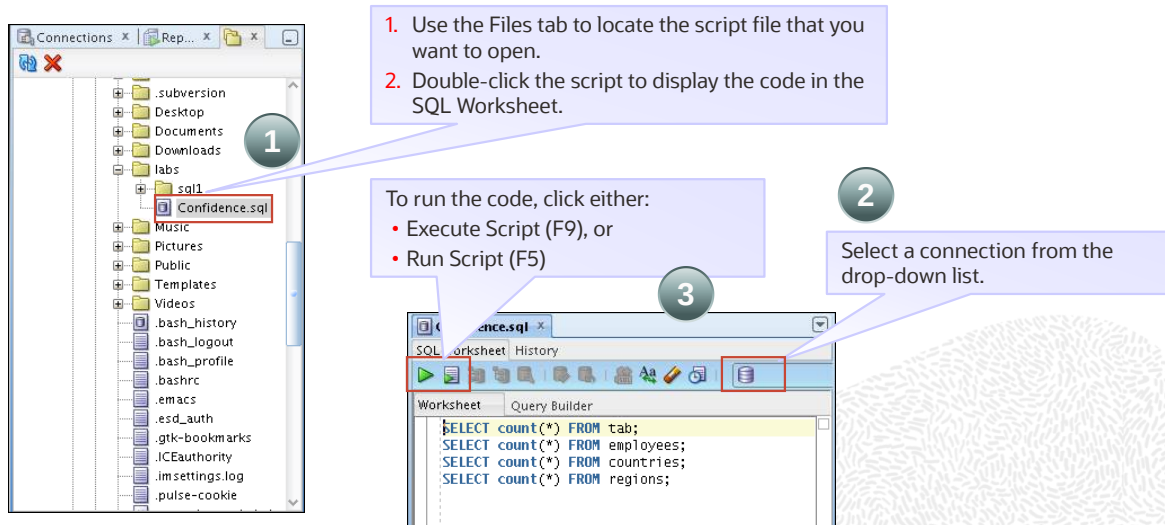
1. Click the Save icon or use the File > Save menu item.
2. In the Save dialog box, enter a file name and the location where you want the file saved.
3. Click Save.

After you save the contents to a file, the Enter SQL Statement window displays a tabbed page of your file contents. You can have multiple files open at the same time. Each file is displayed as a tabbed page.

Script Pathing

You can select a default path to look for scripts and to save scripts. Under Tools > Preferences > Database > Worksheet Parameters, enter a value in the “Select default path to look for scripts” field.

Executing Saved Script Files: Method 1



21

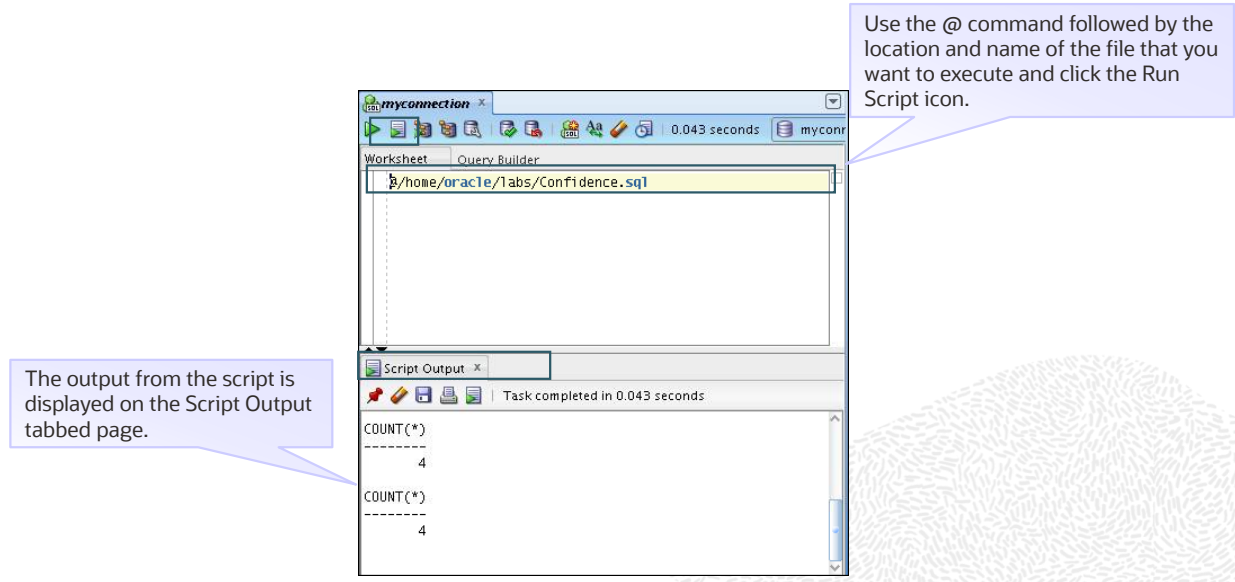
To open a script file and display the code in the SQL Worksheet area, perform the following steps:

1. In the files navigator, select (or navigate to) the script file that you want to open.
2. Double-click to open. The code of the script file is displayed in the SQL Worksheet area.
3. Select a connection from the connection drop-down list.
4. To run the code, click the Run Script (F5) icon on the SQL Worksheet toolbar. If you have not selected a connection from the connection drop-down list, a connection dialog box will appear. Select the connection that you want to use for the script execution.

Alternatively, you can also do the following:

1. Select File > Open. The Open dialog box is displayed.
2. In the Open dialog box, select (or navigate to) the script file that you want to open.
3. Click Open. The code of the script file is displayed in the SQL Worksheet area.
4. Select a connection from the connection drop-down list.
5. To run the code, click the Run Script (F5) icon on the SQL Worksheet toolbar. If you have not selected a connection from the connection drop-down list, a connection dialog box will appear. Select the connection that you want to use for the script execution.

Executing Saved Script Files: Method 2



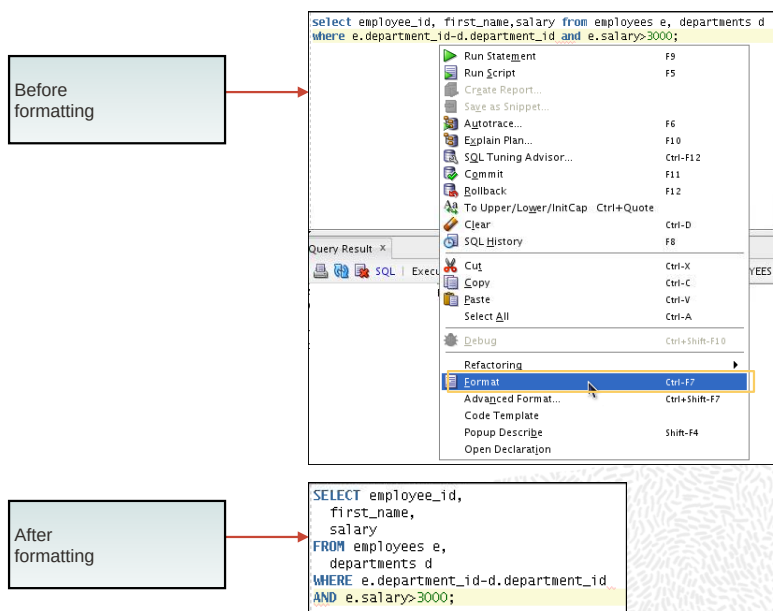
22

To run a saved SQL script, perform the following steps:

1. Use the @ command followed by the location and the name of the file that you want to run in the Enter SQL Statement window.
2. Click the Run Script icon.

The results from running the file are displayed on the Script Output tabbed page. You can also save the script output by clicking the Save icon on the Script Output tabbed page. The File Save dialog box appears and you can identify a name and location for your file.

Formatting the SQL Code



23

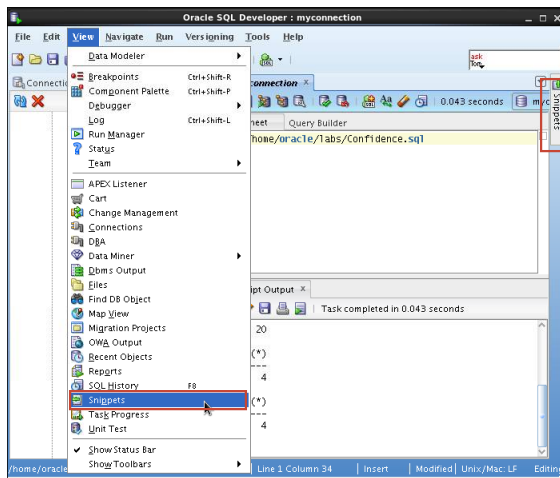
You may want to format the indentation, spacing, capitalization, and line separation of the SQL code. SQL Developer has a feature for formatting SQL code.

To format the SQL code, right-click in the statement area and select Format.

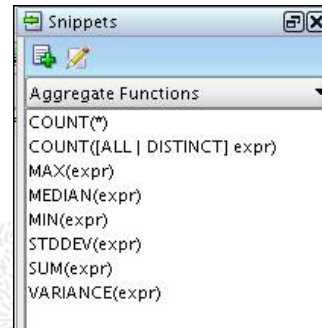
In the example in the slide, before formatting, the SQL code has the keywords not capitalized and the statement not properly indented. After formatting, the SQL code is beautified with the keywords capitalized and the statement properly indented.

Using Snippets

Snippets are code fragments that may be just syntax or examples.



When you place your cursor here, it shows the Snippets window. From the drop-down list, you can select the functions category that you want.



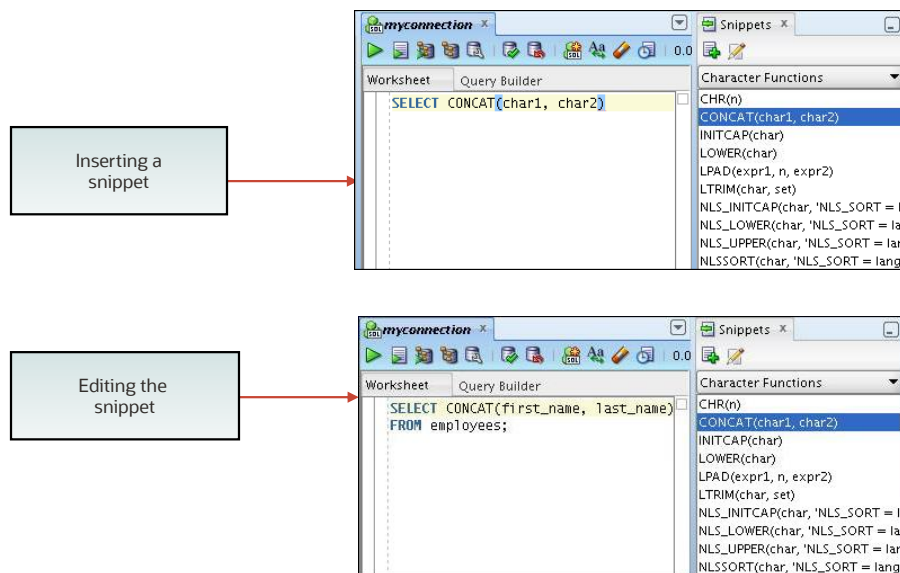
24

You may want to use certain code fragments when you use the SQL Worksheet or create or edit a PL/SQL function or procedure. SQL Developer has a feature called Snippets. Snippets are code fragments such as SQL functions, optimizer hints, and miscellaneous PL/SQL programming techniques. You can drag snippets to the Editor window.

To display Snippets, select View > Snippets.

The Snippets window is displayed on the right. You can use the drop-down list to select a group. A Snippets button is placed in the right window margin, so that you can display the Snippets window if it becomes hidden.

Using Snippets: Example



25

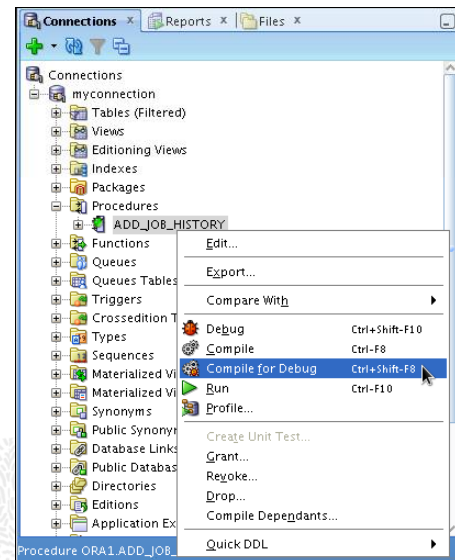
To insert a Snippet into your code in a SQL Worksheet or in a PL/SQL function or procedure, drag the snippet from the Snippets window to the desired place in your code. Then you can edit the syntax so that the SQL function is valid in the current context. To see a brief description of a SQL function in a tool tip, place the cursor over the function name.

The example in the slide shows that `CONCAT(char1, char2)` is dragged from the Character Functions group in the Snippets window. Then the `CONCAT` function syntax is edited and the rest of the statement is added as in the following:

```
SELECT CONCAT(first_name, last_name)
FROM employees;
```


Debugging Procedures and Functions

- Use SQL Developer to debug PL/SQL functions and procedures.
- Use the Compile for Debug option to perform a PL/SQL compilation so that the procedure can be debugged.
- Use the Debug menu options to set breakpoints, and to perform step into, step over tasks.



26

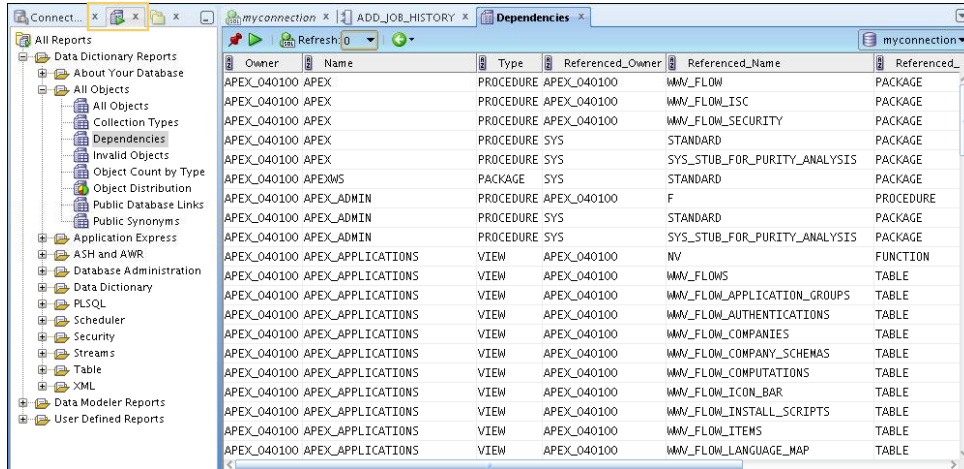
In SQL Developer, you can debug PL/SQL procedures and functions. Using the Debug menu options, you can perform the following debugging tasks:

- **Find Execution Point** goes to the next execution point.
- **Resume** continues execution.
- **Step Over** bypasses the next method and goes to the next statement after the method.
- **Step Into** goes to the first statement in the next method.
- **Step Out** leaves the current method and goes to the next statement.
- **Step to End of Method** goes to the last statement of the current method.
- **Pause** halts execution, but does not exit, thus allowing you to resume execution.
- **Terminate** halts and exits the execution. You cannot resume execution from this point; instead, to start running or debugging from the beginning of the function or procedure, click the Run or Debug icon on the Source tab toolbar.
- **Garbage Collection** removes invalid objects from the cache in favor of more frequently accessed and more valid objects.

These options are also available as icons on the Debugging tab of the output window.

Database Reporting

SQL Developer provides a number of predefined reports about the database and its objects.



Owner	Name	Type	Referenced_Owner	Referenced_Name	Referenced_Type
APEX_040100	APEX	PROCEDURE	APEX_040100	MMV_FLOW	PACKAGE
APEX_040100	APEX	PROCEDURE	APEX_040100	MMV_FLOW_ISC	PACKAGE
APEX_040100	APEX	PROCEDURE	APEX_040100	MMV_FLOW_SECURITY	PACKAGE
APEX_040100	APEX	PROCEDURE	SYS	STANDARD	PACKAGE
APEX_040100	APEX	PROCEDURE	SYS	SYS_STUB_FOR_PURITY_ANALYSIS	PACKAGE
APEX_040100	APEXMS	PACKAGE	SYS	STANDARD	PACKAGE
APEX_040100	APEX_ADMIN	PROCEDURE	APEX_040100	F	PROCEDURE
APEX_040100	APEX_ADMIN	PROCEDURE	SYS	STANDARD	PACKAGE
APEX_040100	APEX_ADMIN	PROCEDURE	SYS	SYS_STUB_FOR_PURITY_ANALYSIS	PACKAGE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	NV	FUNCTION
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOWS	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_APPLICATION_GROUPS	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_AUTHENTICATIONS	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_COMPANIES	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_COMPANY_SCHEMAS	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_COMPUTATIONS	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_ICON_BAR	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_INSTALL_SCRIPTS	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_ITEMS	TABLE
APEX_040100	APEX_APPLICATIONS	VIEW	APEX_040100	MMV_FLOW_LANGUAGE_MAP	TABLE

27

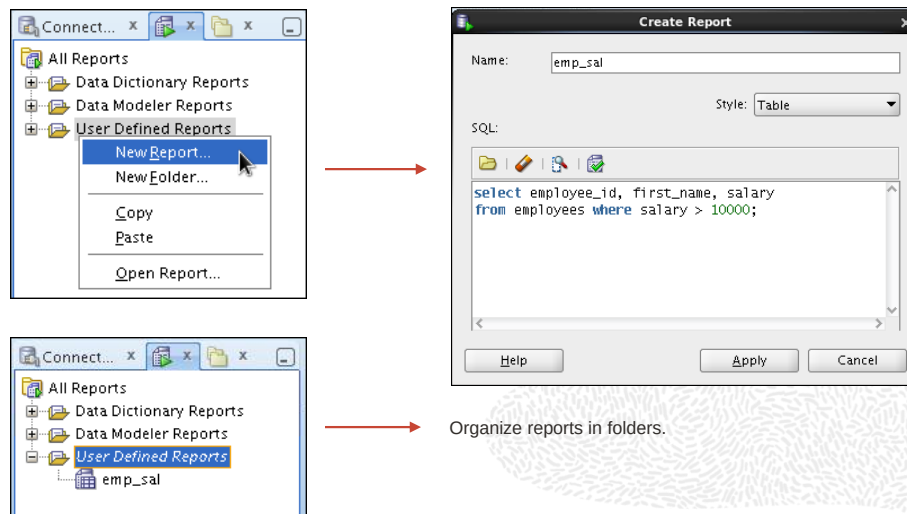
SQL Developer provides many reports about the database and its objects. These reports can be grouped into the following categories:

- About Your Database reports
- Database Administration reports
- Table reports
- PL/SQL reports
- Security reports
- XML reports
- Jobs reports
- Streams reports
- All Objects reports
- Data Dictionary reports
- User-Defined reports

To display reports, click the Reports tab on the left of the window. Individual reports are displayed in tabbed panes on the right of the window; and for each report, you can select (using a drop-down list) the database connection for which to display the report. For reports about objects, the objects shown are only those visible to the database user associated with the selected database connection, and the rows are usually ordered by the Owner. You can also create your own user-defined reports.

Creating a User-Defined Report

Create and save user-defined reports for repeated use.



28

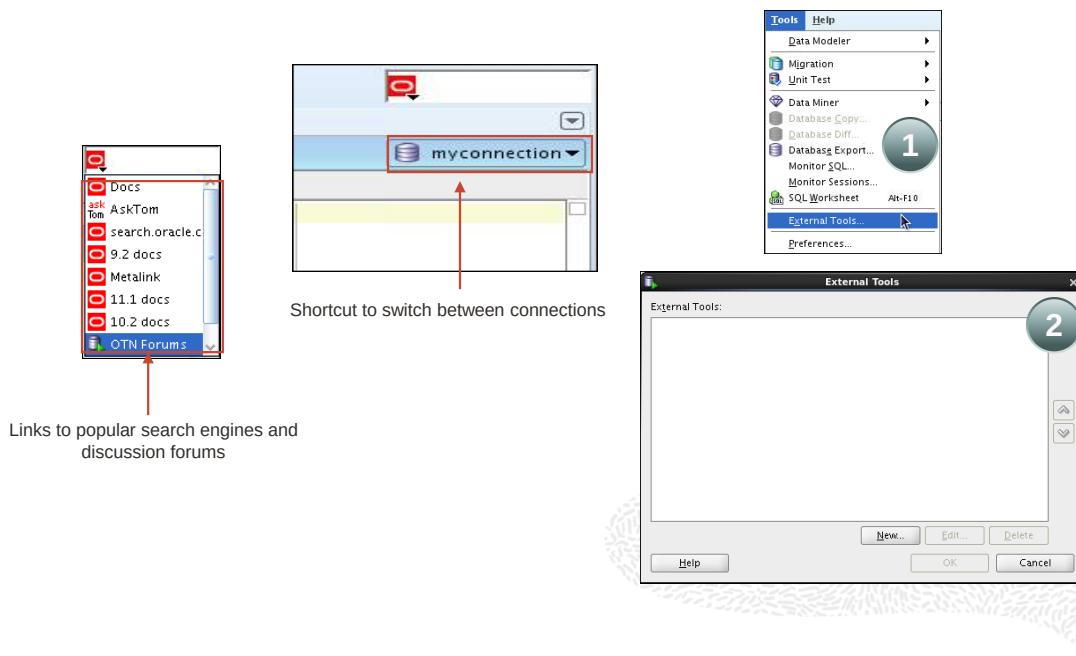
User-defined reports are reports created by SQL Developer users. To create a user-defined report, perform the following steps:

1. Right-click the User Defined Reports node under Reports and select Add Report.
2. In the Create Report dialog box, specify the report name and the SQL query to retrieve information for the report. Then click Apply.

In the example in the slide, the report name is specified as `emp_sal`. An optional description is provided indicating that the report contains details of employees with `salary >= 10000`. The complete SQL statement for retrieving the information to be displayed in the user-defined report is specified in the SQL box. You can also include an optional tool tip to be displayed when the cursor stays briefly over the report name in the Reports navigator display.

You can organize user-defined reports in folders and you can create a hierarchy of folders and subfolders. To create a folder for user-defined reports, right-click the User Defined Reports node or any folder name under that node and select Add Folder. Information about user-defined reports, including any folders for these reports, is stored in a file named `UserReports.xml` in the directory for user-specific information.

Search Engines and External Tools



29

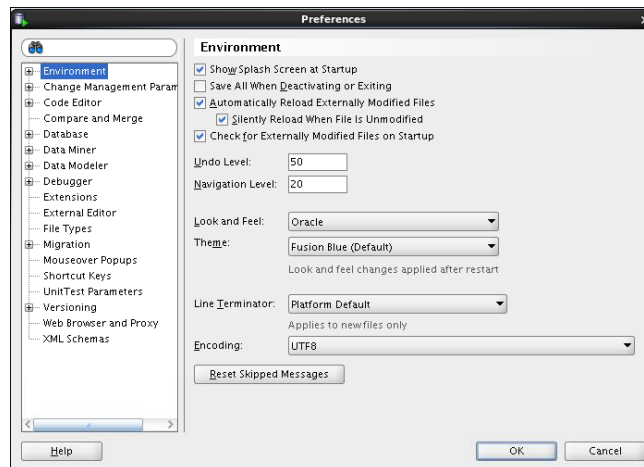
To enhance the productivity of developers, SQL Developer has added quick links to popular search engines and discussion forums such as AskTom, Google, and so on. Also, you have shortcut icons to some of the frequently-used tools such as Notepad, Microsoft Word, and Dreamweaver, available to you.

You can add external tools to the existing list or even delete shortcuts to the tools that you do not use frequently. To do so, perform the following steps:

1. From the Tools menu, select External Tools.
2. In the External Tools dialog box, select New to add new tools. Select Delete to remove any tool from the list.

Setting Preferences

- Customize the SQL Developer interface and environment.
- In the Tools menu, select Preferences.



30

You can customize many aspects of the SQL Developer interface and environment by modifying SQL Developer preferences according to your needs. To modify SQL Developer preferences, select Tools, and then Preferences.

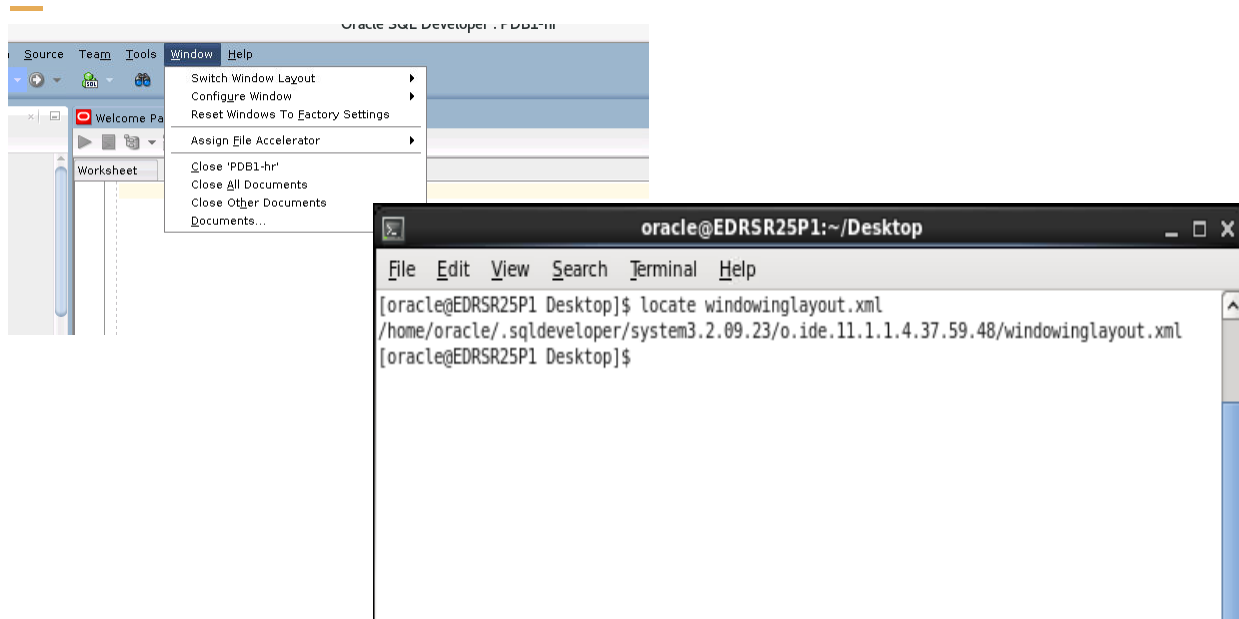
The preferences are grouped into the following categories:

- Environment
- Change Management parameter
- Code Editors
- Compare and Merge
- Database
- Data Miner
- Data Modeler
- Debugger
- Extensions
- External Editor
- File Types
- Migration

- Mouseover Popups
- Shortcut Keys
- Unit Test Parameters
- Versioning
- Web Browser and Proxy
- XML Schemas

For Instructor Use Only.
This document should not be distributed.

Resetting the SQL Developer Layout



32

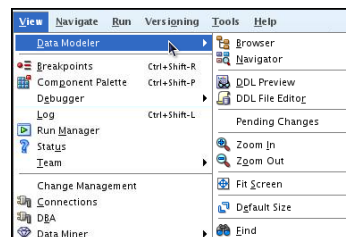
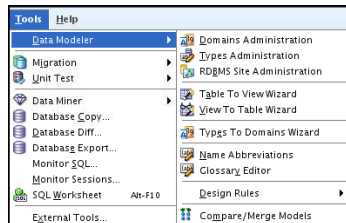
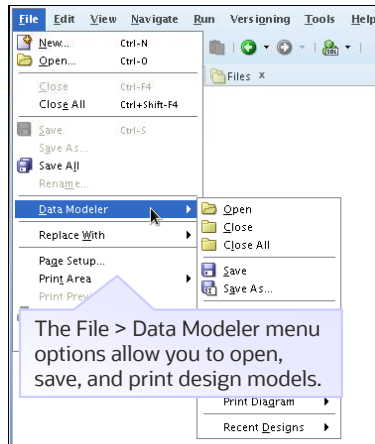
While working with SQL Developer, if the Connections Navigator disappears or if you cannot dock the Log window in its original place, perform the following steps to fix the problem.

If using a version of SQL Developer that has a 'Reset Windows' parameter, go to Window>Reset Windows to Factory Settings. Otherwise, follow these 4 steps:

1. Exit from SQL Developer.
2. Open a terminal window and use the locate command to find the location of windowinglayout.xml.
3. Go to the directory that has windowinglayout.xml and delete it.
4. Restart SQL Developer.

Data Modeler in SQL Developer

SQL Developer includes an integrated version of SQL Developer Data Modeler.



33

Using the integrated version of the SQL Developer Data Modeler, you can:

- Create, open, import, and save a database design
- Create, modify, and delete Data Modeler objects

To display Data Modeler in a pane, click Tools, and then Data Modeler. The Data Modeler menu under Tools includes additional commands, for example, that enable you to specify design rules and preferences.

Summary

In this appendix, you should have learned how to use SQL Developer to do the following:

- Browse, create, and edit database objects
- Execute SQL statements and scripts in SQL Worksheet
- Create and save custom reports
- Browse the Data Modeling options in SQL Developer



34

SQL Developer is a free graphical tool to simplify database development tasks. Using SQL Developer, you can browse, create, and edit database objects. You can use SQL Worksheet to run SQL statements and scripts. SQL Developer enables you to create and save your own special set of reports for repeated use.

For Instructor Use Only.
This document should not be distributed.

ORACLE

B

Using SQL*Plus

0

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this appendix, you should be able to do the following:

- Log in to SQL*Plus
- Edit SQL commands
- Format the output by using SQL*Plus commands
- Interact with script files



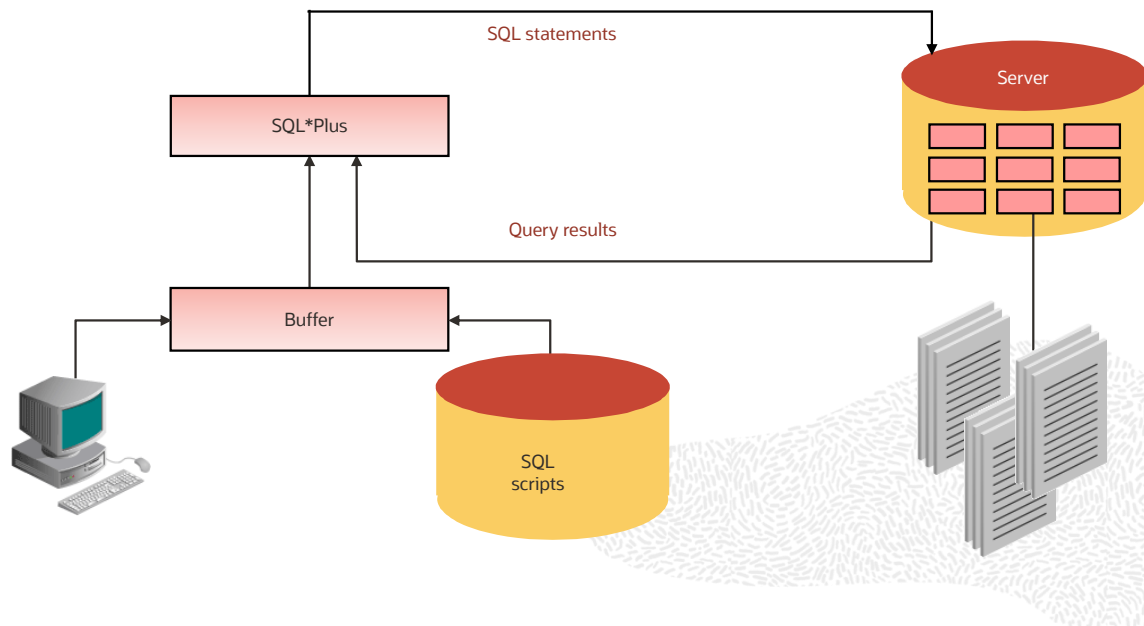
0

2

You might want to create `SELECT` statements that can be used repeatedly. This appendix covers the use of SQL*Plus commands to execute SQL statements. You learn how to format the output by using SQL*Plus commands, edit SQL commands, and save scripts in SQL*Plus.

For Instructor Use Only.
This document should not be distributed.

SQL and SQL*Plus Interaction



3

SQL and SQL*Plus

SQL is a command language that is used for communication with the Oracle server from any tool or application. Oracle SQL contains many extensions. When you enter a SQL statement, it is stored in a part of memory called the *SQL buffer* and remains there until you enter a new SQL statement. SQL*Plus is an Oracle tool that recognizes and submits SQL statements to the Oracle19c Server for execution. It contains its own command language.

Features of SQL

- Can be used by a range of users, including those with little or no programming experience
- Is a nonprocedural language
- Reduces the amount of time required for creating and maintaining systems
- Is an English-like language

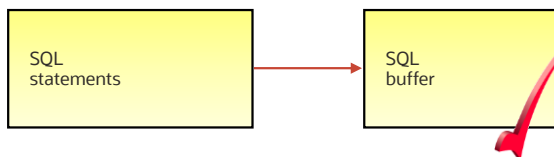
Features of SQL*Plus

- Accepts ad hoc entry of statements
- Accepts SQL input from files
- Provides a line editor for modifying SQL statements
- Controls environmental settings
- Formats query results into basic reports
- Accesses local and remote databases

SQL Statements Versus SQL*Plus Commands

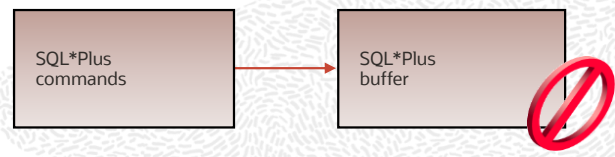
SQL

- A language
- ANSI-standard
- Keywords cannot be abbreviated.
- Statements manipulate data and table definitions in the database.



SQL*Plus

- An environment
- Oracle-proprietary
- Keywords can be abbreviated.
- Commands do not allow manipulation of values in the database.



4

The following table compares SQL and SQL*Plus:

SQL	SQL*Plus
Is a language for communicating with the Oracle server to access data	Recognizes SQL statements and sends them to the server
Is based on American National Standards Institute (ANSI)–standard SQL	Is the Oracle-proprietary interface for executing SQL statements
Manipulates data and table definitions in the database	Does not allow manipulation of values in the database
Is entered into the SQL buffer on one or more lines	Is entered one line at a time, not stored in the SQL buffer
Does not have a continuation character	Uses a dash (–) as a continuation character if the command is longer than one line
Cannot be abbreviated	Can be abbreviated
Uses a termination character to execute commands immediately	Does not require termination characters; executes commands immediately
Uses functions to perform some formatting	Uses commands to format data

Overview of SQL*Plus

- Log in to SQL*Plus.
- Describe the table structure.
- Edit your SQL statement.
- Execute SQL from SQL*Plus.
- Save SQL statements to files and append SQL statements to files.
- Execute saved files.
- Load commands from the file to buffer to edit.

5

SQL*Plus

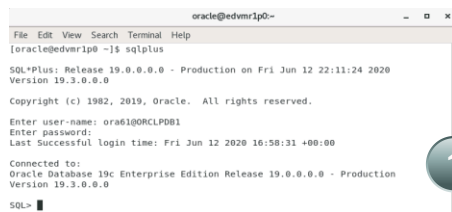
SQL*Plus is an environment in which you can:

- Execute SQL statements to retrieve, modify, add, and remove data from the database
- Format, perform calculations on, store, and print query results in the form of reports
- Create script files to store SQL statements for repeated use in the future

SQL*Plus commands can be divided into the following main categories:

Category	Purpose
Environment	Affect the general behavior of SQL statements for the session
Format	Format query results
File manipulation	Save, load, and run script files
Execution	Send SQL statements from the SQL buffer to the Oracle server
Edit	Modify SQL statements in the buffer
Interaction	Create and pass variables to SQL statements, print variable values, and print messages to the screen
Miscellaneous	Connect to the database, manipulate the SQL*Plus environment, and display column definitions

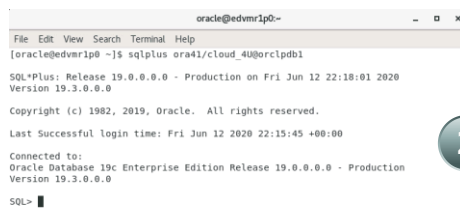
Logging In to SQL*Plus



A terminal window titled 'oracle@edvmr1p0~' showing the execution of the 'sqlplus' command. The output displays the SQL*Plus version (19.0.0.0.0) and prompts for a username and password. The user enters 'ora61@ORCLPDB1' and a password, resulting in a successful login to the Oracle Database 19c Enterprise Edition.

1

```
sqlplus [username[/password[@database]]]
```



A terminal window titled 'oracle@edvmr1p0~' showing the execution of the 'sqlplus ora41/cloud_4u@orclpdb1' command. The output displays the SQL*Plus version (19.0.0.0.0) and prompts for a username and password. The user enters 'ora41/cloud_4u@orclpdb1' and a password, resulting in a successful login to the Oracle Database 19c Enterprise Edition.

2

0

How you invoke SQL*Plus depends on the type of operating system that you are running Oracle Database on.

To log in from a Linux environment, perform the following steps:

1. Right-click your Linux desktop and select terminal.
2. Enter the `sqlplus` command shown in the slide.
3. Enter the username, password, and database name.

In the syntax:

<i>username</i>	Your database username
<i>password</i>	Your database password (Your password is visible if you enter it here.)
<i>@database</i>	The database connect string

Note: To ensure the integrity of your password, do not enter it at the operating system prompt. Instead, enter only your username. Enter your password at the password prompt.

Displaying the Table Structure

Use the SQL*Plus `DESCRIBE` command to display the structure of a table:

```
DESC[RIBE] tablename
```

7

In SQL*Plus, you can display the structure of a table using the `DESCRIBE` command. The result of the command is a display of column names and data types, as well as an indication if a column must contain data.

In the syntax:

tablename The name of any existing table, view, or synonym that is accessible to the user

To describe the `DEPARTMENTS` table, use the following command:

```
SQL> DESCRIBE DEPARTMENTS
```

Name	Null	Type
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

Displaying the Table Structure

```
DESCRIBE departments
```

Name	Null	Type
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

8

The example in the slide displays information about the structure of the `DEPARTMENTS` table. In the result:

Null: Specifies whether a column must contain data (`NOT NULL` indicates that a column must contain data.)

Type: Displays the data type for a column

SQL*Plus Editing Commands

- `A[PPEND] text`
- `C[HANGE] / old / new`
- `C[HANGE] / text /`
- `CL[EAR] BUFF[ER]`
- `DEL`
- `DEL n`
- `DEL m n`

9

SQL*Plus commands are entered one line at a time and are not stored in the SQL buffer.

Command	Description
<code>A[PPEND] text</code>	Adds text to the end of the current line
<code>C[HANGE] / old / new</code>	Changes <i>old</i> text to <i>new</i> in the current line
<code>C[HANGE] / text /</code>	Deletes <i>text</i> from the current line
<code>CL[EAR] BUFF[ER]</code>	Deletes all lines from the SQL buffer
<code>DEL</code>	Deletes current line
<code>DEL n</code>	Deletes line <i>n</i>
<code>DEL m n</code>	Deletes lines <i>m</i> to <i>n</i> inclusive

Guidelines

- If you press Enter before completing a command, SQL*Plus prompts you with a line number.
- You terminate the SQL buffer either by entering one of the terminator characters (semicolon or slash) or by pressing Enter twice. The SQL prompt appears.

SQL*Plus Editing Commands

- `I [NPUT]`
- `I [NPUT] text`
- `L [IST]`
- `L [IST] n`
- `L [IST] m n`
- `R [UN]`
- `n`
- `n text`
- `0 text`

10

Command	Description
<code>I [NPUT]</code>	Inserts an indefinite number of lines
<code>I [NPUT] text</code>	Inserts a line consisting of <i>text</i>
<code>L [IST]</code>	Lists all lines in the SQL buffer
<code>L [IST] n</code>	Lists one line (specified by <i>n</i>)
<code>L [IST] m n</code>	Lists a range of lines (<i>m</i> to <i>n</i>) inclusive
<code>R [UN]</code>	Displays and runs the current SQL statement in the buffer
<code>n</code>	Specifies the line to make the current line
<code>n text</code>	Replaces line <i>n</i> with <i>text</i>
<code>0 text</code>	Inserts a line before line 1

Note: You can enter only one SQL*Plus command for each SQL prompt. SQL*Plus commands are not stored in the buffer. To continue a SQL*Plus command on the next line, end the first line with a hyphen (-).

Using LIST, n, and APPEND

```
LIST
1  SELECT last_name
2* FROM   employees
```

```
1
1* SELECT last_name
```

```
A , job_id
1* SELECT last_name, job_id
```

```
LIST
1  SELECT last_name, job_id
2* FROM   employees
```

11

- Use the `L[IST]` command to display the contents of the SQL buffer. The asterisk (*) beside line 2 in the buffer indicates that line 2 is the current line. Any edits that you made apply to the current line.
- Change the number of the current line by entering the number (n) of the line that you want to edit. The new current line is displayed.
- Use the `A[PPEND]` command to add text to the current line. The newly edited line is displayed. Verify the new contents of the buffer by using the `LIST` command.

Note: Many SQL*Plus commands, including `LIST` and `APPEND`, can be abbreviated to just their first letter. `LIST` can be abbreviated to `L`; `APPEND` can be abbreviated to `A`.

Using the CHANGE Command

```
LIST
1* SELECT * from employees
```

```
c/employees/departments
1* SELECT * from departments
```

```
LIST
1* SELECT * from departments
```

12

- Use `L[IST]` to display the contents of the buffer.
- Use the `C[HANGE]` command to alter the contents of the current line in the SQL buffer. In this case, replace the `employees` table with the `departments` table. The new current line is displayed.
- Use the `L[IST]` command to verify the new contents of the buffer.

SQL*Plus File Commands

- `SAVE filename`
- `GET filename`
- `START filename`
- `@ filename`
- `EDIT filename`
- `SPOOL filename`
- `EXIT`

13

SQL statements communicate with the Oracle server. SQL*Plus commands control the environment, format query results, and manage files. You can use the commands described in the following table:

Command	Description
<code>SAV[E] filename [.ext]</code> <code>[REPL[ACE]APP[END]]</code>	Saves the current contents of the SQL buffer to a file. Use <code>APPEND</code> to add to an existing file; use <code>REPLACE</code> to overwrite an existing file. The default extension is <code>.sql</code> .
<code>GET filename [.ext]</code>	Writes the contents of a previously saved file to the SQL buffer. The default extension for the file name is <code>.sql</code> .
<code>STA[RT] filename [.ext]</code>	Runs a previously saved command file
<code>@ filename</code>	Runs a previously saved command file (same as <code>START</code>)
<code>ED[IT]</code>	Invokes the editor and saves the buffer contents to a file named <code>afiedt.buf</code>
<code>ED[IT] [filename [.ext]]</code>	Invokes the editor to edit the contents of a saved file
<code>SPO[OL] [filename [.ext]]</code> <code>OFF OUT</code>	Stores query results in a file. <code>OFF</code> closes the spool file. <code>OUT</code> closes the spool file and sends the file results to the printer.
<code>EXIT</code>	Quits SQL*Plus

Using the SAVE, START Commands

```
LIST
1  SELECT last_name, manager_id, department_id
2* FROM employees
```

```
SAVE my_query
Created file my_query
```

```
START my_query

LAST_NAME          MANAGER_ID DEPARTMENT_ID
-----
King                90
Kochhar             100          90
...
107 rows selected.
```

14

SAVE

Use the `SAVE` command to store the current contents of the buffer in a file. Thus, you can store frequently-used scripts for use in the future.

START

Use the `START` command to run a script in SQL*Plus. You can also, alternatively, use the symbol `@` to run a script.

```
@my_query
```

SERVEROUTPUT Command

- Use the `SET SERVEROUT[PUT]` command to control whether to display the output of stored procedures or PL/SQL blocks in SQL*Plus.
- The `DBMS_OUTPUT` line length limit is increased from 255 bytes to 32767 bytes.
- The default size is now unlimited.
- Resources are not preallocated when `SERVEROUTPUT` is set.
- Because there is no performance penalty, use `UNLIMITED` unless you want to conserve physical memory.

```
SET SERVEROUT[PUT] {ON | OFF} [SIZE {n | UNLIMITED}] [FOR[MAT] {WRA[PPED] |  
WOR[D_WRAPPED] | TRU[NCATED]}]
```

15

Most of the PL/SQL programs perform input and output through SQL statements, to store data in database tables or query those tables. All other PL/SQL input/output is done through APIs that interact with other programs. For example, the `DBMS_OUTPUT` package has procedures, such as `PUT_LINE`. To see the result outside of PL/SQL requires another program, such as SQL*Plus, to read and display the data passed to `DBMS_OUTPUT`.

SQL*Plus does not display `DBMS_OUTPUT` data unless you first issue the SQL*Plus command `SET SERVEROUTPUT ON` as follows:

```
SET SERVEROUTPUT ON
```

Note

- `SIZE` sets the number of bytes of the output that can be buffered within the Oracle Database server. The default is `UNLIMITED`. `n` cannot be less than 2000 or greater than 1,000,000.
- For additional information about `SERVEROUTPUT`, see *Oracle Database PL/SQL User's Guide and Reference 19c*.

Using the SQL*Plus SPOOL Command

```
SPO[OL] [file_name[.ext] [CRE[ATE] | REP[LACE] | APP[END]] | OFF | OUT]
```

Option	Description
file_name[.ext]	Spools output to the specified file name
CRE[ATE]	Creates a new file with the name specified
REP[LACE]	Replaces the contents of an existing file. If the file does not exist, REPLACE creates the file.
APP[END]	Adds the contents of the buffer to the end of the file that you specify
OFF	Stops spooling
OUT	Stops spooling and sends the file to your computer's standard (default) printer

16

The `SPOOL` command stores query results in a file or optionally sends the file to a printer. The `SPOOL` command has been enhanced. You can now append to, or replace an existing file, where previously you could use `SPOOL` only to create (and replace) a file. `REPLACE` is the default.

To spool the output generated by commands in a script without displaying the output on screen, use `SET TERMOUT OFF`. `SET TERMOUT OFF` does not affect the output from commands that run interactively.

You must use quotation marks around file names that contain white space. To create a valid HTML file using `SPOOL APPEND` commands, you must use `PROMPT` or a similar command to create the HTML page header and footer. The `SPOOL APPEND` command does not parse HTML tags. `SET SQLPLUSCOMPAT[IBILITY]` to 9.2 or earlier to disable the `CREATE`, `APPEND`, and `SAVE` parameters.

Using the AUTOTRACE Command

- It displays a report after the successful execution of SQL DML statements such as `SELECT`, `INSERT`, `UPDATE`, or `DELETE`.
- The report can now include execution statistics and the query execution path.

```
SET AUTOT[RACE] {ON | OFF | TRACE[ONLY]} [EXP[LAIN]] [STAT[ISTICS]]
```

```
SET AUTOTRACE ON
-- The AUTOTRACE report includes both the optimizer
-- execution path and the SQL statement execution
-- statistics
```

17

`EXPLAIN` shows the query execution path by performing an `EXPLAIN PLAN`. `STATISTICS` displays SQL statement statistics. The formatting of your `AUTOTRACE` report may vary depending on the version of the server to which you are connected and the configuration of the server. The `DBMS_XPLAN` package provides an easy way to display the output of the `EXPLAIN PLAN` command in several predefined formats.

Note

- For additional information about the package and subprograms, refer to Oracle Database *PL/SQL Packages and Types Reference*: <https://docs.oracle.com/en/database/oracle/oracle-database/19/arpls/index.html>
- For additional information about the `EXPLAIN PLAN`, refer to *Oracle Database SQL Language Reference*: <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/index.html>
- For additional information about Execution Plans and the statistics, refer to *Oracle Database Database Performance Tuning Guide*: <https://docs.oracle.com/en/database/oracle/oracle-database/19/tgdba/index.html>

Summary

In this appendix, you should have learned how to use SQL*Plus as an environment to do the following:

- Execute SQL statements
- Edit SQL statements
- Format the output
- Interact with script files



18

SQL*Plus is an execution environment that you can use to send SQL commands to the database server and to edit and save SQL commands. You can execute commands from the SQL prompt or from a script file.

For Instructor Use Only.
This document should not be distributed.

Commonly Used SQL Commands

Objectives

After completing this appendix, you should be able to:

- Execute a basic `SELECT` statement
- Create, alter, and drop a table using the data definition language (DDL) statements
- Insert, update, and delete rows from one or more tables using data manipulation language (DML) statements
- Commit, roll back, and create save points using the transaction control statements
- Perform join operations on one or more tables



2

This lesson explains how to obtain data from one or more tables using the `SELECT` statement, how to use DDL statements to alter the structure of data objects, how to manipulate data in the existing schema objects by using the DML statements, how to manage the changes made by DML statements, and how to use joins to display data from multiple tables using SQL:1999 join syntax.

For Instructor Use Only.
This document should not be distributed.

Basic SELECT Statement

- Use the `SELECT` statement to:
 - Identify the columns to be displayed
 - Retrieve data from one or more tables, object tables, views, object views, or materialized views
- A `SELECT` statement is also known as a query because it queries a database.
- Syntax:

```
SELECT {*|[DISTINCT] column|expression [alias],...}  
FROM   table;
```

3

In its simplest form, a `SELECT` statement must include the following:

- A `SELECT` clause, which specifies the columns to be displayed
- A `FROM` clause, which identifies the table containing the columns that are listed in the `SELECT` clause

In the syntax:

<code>SELECT</code>	Is a list of one or more columns
<code>*</code>	Selects all columns
<code>DISTINCT</code>	Suppresses duplicates
<code>column expression</code>	Selects the named column or the expression
<code>alias</code>	Gives different headings to the selected columns
<code>FROM table</code>	Specifies the table containing the columns

Note: Throughout this course, the words *keyword*, *clause*, and *statement* are used as follows:

- A *keyword* refers to an individual SQL element—for example, `SELECT` and `FROM` are keywords.
- A *clause* is a part of a SQL statement (for example, `SELECT employee_id, last_name`).
- A *statement* is a combination of two or more clauses (for example, `SELECT * FROM employees`).

SELECT Statement

- Select all columns:

```
SELECT *  
FROM job_history;
```

	EMPLOYEE_ID	START_DATE	END_DATE	JOB_ID	DEPARTMENT_ID
1	102	13-JAN-01	24-JUL-06	IT_PROG	60
2	101	21-SEP-97	27-OCT-01	AC_ACCOUNT	110
3	101	28-OCT-01	15-MAR-05	AC_MGR	110
4	201	17-FEB-04	19-DEC-07	MK_REP	20
5	114	24-MAR-06	31-DEC-07	ST_CLERK	50
6	122	01-JAN-07	31-DEC-07	ST_CLERK	50
7	200	17-SEP-95	17-JUN-01	AD_ASST	90
8	176	24-MAR-06	31-DEC-06	SA_REP	80
9	176	01-JAN-07	31-DEC-07	SA_MAN	80
10	200	01-JUL-02	31-DEC-06	AC_ACCOUNT	90

- Select specific columns:

```
SELECT manager_id, job_id  
FROM employees;
```

	MANAGER_ID	JOB_ID
1	(null)	AD_PRES
2	100	AD_VP
3	100	AD_VP
4	102	IT_PROG
5	103	IT_PROG
6	103	IT_PROG
7	100	ST_MAN
8	124	ST_CLERK
9	124	ST_CLERK
10	124	ST_CLERK

4

You can display all columns of data in a table by following the `SELECT` keyword with an asterisk (*) or by listing all the column names after the `SELECT` keyword. The first example in the slide displays all the rows from the `job_history` table. Specific columns of the table can be displayed by specifying the column names, separated by commas. The second example in the slide displays the `manager_id` and `job_id` columns from the `employees` table.

In the `SELECT` clause, specify the columns in the order in which you want them to appear in the output. For example, the following SQL statement displays the `location_id` column before displaying the `department_id` column:

```
SELECT location_id, department_id FROM departments;
```

Note: You can enter your SQL statement in a SQL Worksheet and click the Run Statement icon or press F9 to execute a statement in SQL Developer. The output displayed on the Results tabbed page appears as shown in the slide.

WHERE Clause

- Use the optional `WHERE` clause to:
 - Filter rows in a query
 - Produce a subset of rows
- Syntax:

```
SELECT * FROM      table
[WHERE condition];
```

- Example:

```
SELECT location_id from departments
WHERE department_name = 'Marketing';
```

5

The `WHERE` clause specifies a condition to filter rows, producing a subset of the rows in the table. A condition specifies a combination of one or more expressions and logical (Boolean) operators. It returns a value of `TRUE`, `FALSE`, or `NULL`. The example in the slide retrieves the `location_id` of the marketing department.

The `WHERE` clause can also be used to update or delete data from the database.

For example:

```
UPDATE departments
SET  department_name = 'Administration'
WHERE department_id = 20;

and

DELETE from departments
WHERE department_id =20;
```


ORDER BY Clause

- Use the optional ORDER BY clause to specify the row order.
- Syntax:

```
SELECT * FROM      table
[WHERE condition]
[ORDER BY {<column>|<position> } [ASC|DESC] [, ...] ];
```

- Example:

```
SELECT last_name, department_id, salary
FROM employees
ORDER BY department_id ASC, salary DESC;
```

6

The ORDER BY clause specifies the order in which the rows should be displayed. The rows can be sorted in ascending or descending fashion. By default, the rows are displayed in ascending order.

The example in the slide retrieves rows from the EMPLOYEES table ordered first by ascending order of department_id, and then by descending order of salary.

GROUP BY Clause

- Use the optional `GROUP BY` clause to group columns that have matching values into subsets.
- Each group has no two rows having the same value for the grouping column or columns.
- Syntax:

```
SELECT <column1, column2, ... column_n>
FROM table
[WHERE condition]
[GROUP BY <column> [, ...] ]
[ORDER BY <column> [, ...] ] ;
```

- Example:

```
SELECT department_id, MIN(salary), MAX (salary)
FROM employees
GROUP BY department_id ;
```

7

The `GROUP BY` clause is used to group selected rows based on the value of `expr(s)` for each row. The clause groups rows but does not guarantee order of the result set. To order the groupings, use the `ORDER BY` clause.

Any `SELECT` list elements that are not included in aggregation functions must be included in the `GROUP BY` list of elements. This includes both columns and expressions. The database returns a single row of summary information for each group.

The example in the slide returns the minimum and maximum salaries for each department in the `EMPLOYEES` table.

Data Definition Language

- DDL statements are used to define, structurally change, and drop schema objects.
- The commonly used DDL statements are:
 - CREATE TABLE, ALTER TABLE, and DROP TABLE
 - GRANT, REVOKE
 - TRUNCATE

8

DDL statements enable you to alter the attributes of an object without altering the applications that access the object. You can also use DDL statements to alter the structure of objects while database users are performing work in the database. These statements are most frequently used to:

- Create, alter, and drop schema objects and other database structures, including the database itself and database users
- Delete all the data in schema objects without removing the structure of these objects
- Grant and revoke privileges and roles

Oracle Database implicitly commits the current transaction before and after every DDL statement.

CREATE TABLE Statement

- Use the `CREATE TABLE` statement to create a table in the database.
- Syntax:

```
CREATE TABLE tablename (  
  {column-definition | Table-level constraint}  
  [ , {column-definition | Table-level constraint} ] * )
```

- Example:

```
CREATE TABLE teach_dept (  
  department_id NUMBER(3) PRIMARY KEY,  
  department_name VARCHAR2(10));
```

9

Use the `CREATE TABLE` statement to create a table in the database. To create a table, you must have the `CREATE TABLE` privilege and a storage area in which to create objects.

The table owner and the database owner automatically gain the following privileges on the table after it is created:

- `INSERT`
- `SELECT`
- `REFERENCES`
- `ALTER`
- `UPDATE`

The table owner and the database owner can grant the preceding privileges to other users.

ALTER TABLE Statement

- Use the ALTER TABLE statement to modify the definition of an existing table in the database.

- Example 1:

```
ALTER TABLE teach_dept  
ADD location_id NUMBER NOT NULL;
```

- Example 2:

```
ALTER TABLE teach_dept  
MODIFY department_name VARCHAR2(30) NOT NULL;
```

10

The ALTER TABLE statement allows you to make changes to an existing table.

You can:

- Add a column to a table
- Add a constraint to a table
- Modify an existing column definition
- Drop a column from a table
- Drop an existing constraint from a table
- Increase the width of the VARCHAR and CHAR columns
- Change a table to have read-only status

Example 1 in the slide adds a new column called LOCATION_ID to the TEACH_DEPT table.

Example 2 updates the existing DEPARTMENT_NAME column from VARCHAR2 (10) to VARCHAR2 (30), and adds a NOT NULL constraint to it.

DROP TABLE Statement

- The `DROP TABLE` statement removes the table and all its data from the database.

- Example:

```
DROP TABLE teach_dept;
```

- `DROP TABLE` with the `PURGE` clause drops the table and releases the space that is associated with it.

```
DROP TABLE teach_dept PURGE;
```

- The `CASCADE CONSTRAINTS` clause drops all referential integrity constraints from the table.

```
DROP TABLE teach_dept CASCADE CONSTRAINTS;
```

11

The `DROP TABLE` statement allows you to remove a table and its contents from the database, and pushes it to the recycle bin. Dropping a table invalidates dependent objects and removes object privileges on the table.

Use the `PURGE` clause along with the `DROP TABLE` statement to release back to the tablespace the space allocated for the table. You cannot roll back a `DROP TABLE` statement with the `PURGE` clause, nor can you recover the table if you have dropped it with the `PURGE` clause.

The `CASCADE CONSTRAINTS` clause allows you to drop the reference to the primary key and unique keys in the dropped table.

GRANT Statement

- The `GRANT` statement assigns privilege to perform the following operations:
 - Insert or delete data.
 - Create a foreign key reference to the named table or to a subset of columns from a table.
 - Select data, a view, or a subset of columns from a table.
 - Create a trigger on a table.
 - Execute a specified function or procedure.
- Example:

```
GRANT SELECT any table to PUBLIC;
```

12

You can use the `GRANT` statement to:

- Assign privileges to a specific user or role, or to all users, to perform actions on database objects
- Grant a role to a user, to `PUBLIC`, or to another role

Before you issue a `GRANT` statement, check that the `derby.database.sql Authorization` property is set to `True`. This property enables the SQL Authorization mode. You can grant privileges on an object if you are the owner of the database.

You can grant privileges to all users by using the `PUBLIC` keyword. When `PUBLIC` is specified, the privileges or roles affect all current and future users.

Privilege Types

Assign the following privileges using the `GRANT` statement:

- `ALL PRIVILEGES`
- `DELETE`
- `INSERT`
- `REFERENCES`
- `SELECT`
- `UPDATE`

13

Oracle Database provides a variety of privilege types to grant privileges to a user or role:

- Use the `ALL PRIVILEGES` privilege type to grant all privileges to the user or role for the specified table.
- Use the `DELETE` privilege type to grant permission to delete rows from the specified table.
- Use the `INSERT` privilege type to grant permission to insert rows into the specified table.
- Use the `REFERENCES` privilege type to grant permission to create a foreign key reference to the specified table.
- Use the `SELECT` privilege type to grant permission to perform `SELECT` statements on a table or view.
- Use the `UPDATE` privilege type to grant permission to use the `UPDATE` statement on the specified table.

REVOKE Statement

- Use the `REVOKE` statement to remove privileges from a user to perform actions on database objects.
- Revoke a *system privilege* from a user:

```
REVOKE DROP ANY TABLE  
FROM hr;
```

- Revoke a *role* from a user:

```
REVOKE dw_manager  
FROM sh;
```

14

The `REVOKE` statement removes privileges from a specific user (or users) or role to perform actions on database objects. It performs the following operations:

- Revokes a role from a user, from `PUBLIC`, or from another role
- Revokes privileges for an object if you are the owner of the object or the database owner

Note: To revoke a role or system privilege, you must have been granted the privilege with the `ADMIN OPTION`.

TRUNCATE TABLE Statement

- Use the `TRUNCATE TABLE` statement to remove all the rows from a table.
- Example:

```
TRUNCATE TABLE employees_demo;
```

- By default, Oracle Database performs the following tasks:
 - Deallocates space used by the removed rows
 - Sets the `NEXT` storage parameter to the size of the last extent removed from the segment by the truncation process

15

The `TRUNCATE TABLE` statement deletes all the rows from a specific table. Removing rows with the `TRUNCATE TABLE` statement can be more efficient than dropping and re-creating a table. Dropping and re-creating a table:

- Invalidates the dependent objects of the table
- Requires you to regrant object privileges
- Requires you to re-create indexes, integrity constraints, and triggers
- Requires you to respecify its storage parameters

The `TRUNCATE TABLE` statement spares you from these efforts.

Note: You cannot roll back a `TRUNCATE TABLE` statement.

Data Manipulation Language

- DML statements query or manipulate data in the existing schema objects.
- A DML statement is executed when:
 - New rows are added to a table by using the `INSERT` statement
 - Existing rows in a table are modified using the `UPDATE` statement
 - Existing rows are deleted from a table by using the `DELETE` statement
- A *transaction* consists of a collection of DML statements that form a logical unit of work.

16

Data Manipulation Language (DML) statements enable you to query or change the contents of an existing schema object. These statements are most frequently used to:

- Add new rows of data to a table or view by specifying a list of column values or using a subquery to select and manipulate existing data
- Change column values in the existing rows of a table or view
- Remove rows from tables or views

A collection of DML statements that forms a logical unit of work is called a transaction. Unlike DDL statements, DML statements do not implicitly commit the current transaction.

INSERT Statement

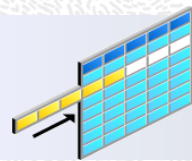
- Use the `INSERT` statement to add new rows to a table.
- Syntax:

```
INSERT INTO table [(column [, column...])]  
VALUES (value [, value...]);
```

- Example:

```
INSERT INTO departments  
VALUES (200, 'Development', 104, 1400);
```

```
1 rows inserted.
```



17

The `INSERT` statement adds rows to a table. Make sure to insert a new row containing values for each column and to list the values in the default order of the columns in the table. Optionally, you can also list the columns in the `INSERT` statement.

For example:

```
INSERT INTO job_history (employee_id, start_date, end_date, job_id)  
VALUES (120, '25-JUL-06', '12-FEB_08', 'AC_ACCOUNT');
```

The syntax discussed in the slide allows you to insert a single row at a time. The `VALUES` keyword assigns the values of expressions to the corresponding columns in the column list.

UPDATE Statement Syntax

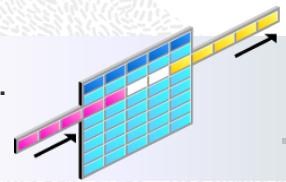
- Use the `UPDATE` statement to modify the existing rows in a table.
- Update more than one row at a time (if required).

```
UPDATE    table
SET       column = value [, column = value, ...]
[WHERE    condition];
```

- Example:

```
UPDATE    copy_emp
SET
22 rows updated
```

- Specify `SET column_name= NULL` to update a column value to `NULL`.



18

The `UPDATE` statement modifies the existing values in a table. Confirm the update operation by querying the table to display the updated rows. You can modify a specific row or rows by specifying the `WHERE` clause.

For example:

```
UPDATE employees
SET    salary = 17500
WHERE  employee_id = 102;
```

In general, use the primary key column in the `WHERE` clause to identify the row to update. For example, to update a specific row in the `EMPLOYEES` table, use `employee_id` to identify the row instead of `employee_name`, because more than one employee may have the same name.

Note: Typically, the `condition` keyword is composed of column names, expressions, constants, subqueries, and comparison operators.

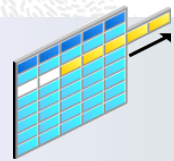
DELETE Statement

- Use the `DELETE` statement to delete the existing rows from a table.
- Syntax:

```
DELETE [FROM] table
[WHERE condition];
```

- Write the `DELETE` statement using the `WHERE` clause to delete specific rows from a table.

```
DELETE FROM departments
WHERE department_name = 'Finance';
1 rows deleted
```



19

The `DELETE` statement removes existing rows from a table. You must use the `WHERE` clause to delete a specific row or rows from a table based on the condition. The `condition` identifies the rows to be deleted. It may contain column names, expressions, constants, subqueries, and comparison operators.

The first example in the slide deletes the finance department from the `DEPARTMENTS` table. You can confirm the delete operation by using the `SELECT` statement to query the table.

```
SELECT *
FROM departments
WHERE department_name = 'Finance';
```

If you omit the `WHERE` clause, all rows in the table are deleted. For example:

```
DELETE FROM copy_emp;
```

The preceding example deletes all the rows from the `COPY_EMP` table.

Transaction Control Statements

- Transaction control statements are used to manage the changes made by DML statements.
- The DML statements are grouped into transactions.
- Transaction control statements include:
 - COMMIT
 - ROLLBACK
 - SAVEPOINT

20

A transaction is a sequence of SQL statements that Oracle Database treats as a single unit. Transaction control statements are used in a database to manage the changes made by DML statements and to group these statements into transactions.

Each transaction is assigned a unique `transaction_id` and it groups SQL statements so that they are either all committed, which means they are applied to the database, or all rolled back, which means they are undone from the database.

COMMIT Statement

- Use the `COMMIT` statement to:
 - Permanently save the changes made to the database during the current transaction
 - Erase all savepoints in the transaction
 - Release transaction locks
- Example:

```
INSERT INTO departments
VALUES      (201, 'Engineering', 106, 1400);
COMMIT;
```

```
1 rows inserted.
Committed.
```

21

The `COMMIT` statement ends the current transaction by making all the pending data changes permanent. It releases all row and table locks, and erases any savepoints that you may have marked since the last commit or rollback. The changes made using the `COMMIT` statement are visible to all users.

Oracle recommends that you explicitly end every transaction in your application programs with a `COMMIT` or `ROLLBACK` statement, including the last transaction, before disconnecting from Oracle Database. If you do not explicitly commit the transaction and the program terminates abnormally, the last uncommitted transaction is automatically rolled back.

Note: Oracle Database issues an implicit `COMMIT` before and after any data definition language (DDL) statement.

ROLLBACK Statement

- Use the `ROLLBACK` statement to undo changes made to the database during the current transaction.
- Use the `TO SAVEPOINT` clause to undo a part of the transaction after the savepoint.
- Example:

```
UPDATE      employees
SET         salary = 7000
WHERE       last_name = 'Ernst';
SAVEPOINT   Ernst_sal;

UPDATE      employees
SET         salary = 12000
WHERE       last_name = 'Mourgos';

ROLLBACK TO SAVEPOINT Ernst_sal;
```

22

The `ROLLBACK` statement undoes work done in the current transaction. To roll back the current transaction, no privileges are necessary.

Using `ROLLBACK` with the `TO SAVEPOINT` clause performs the following operations:

- Rolls back only the portion of the transaction after the savepoint
- Erases all savepoints created after that savepoint. The named savepoint is retained, so you can roll back to the same savepoint multiple times.

Using `ROLLBACK` without the `TO SAVEPOINT` clause performs the following operations:

- Ends the transaction
- Undoes all the changes in the current transaction
- Erases all savepoints in the transaction

SAVEPOINT Statement

- Use the `SAVEPOINT` statement to name and mark the current point in the processing of a transaction.
- Specify a name to each savepoint.
- Use distinct savepoint names within a transaction to avoid overriding.
- Syntax:

```
SAVEPOINT savepoint;
```

23

The `SAVEPOINT` statement identifies a point in a transaction to which you can later roll back. You must specify a distinct name for each savepoint. If you create a second savepoint with the same identifier as an earlier savepoint, the earlier savepoint is erased.

After a savepoint has been created, you can continue processing, commit your work, roll back the entire transaction, or roll back to the savepoint.

A simple rollback or commit erases all savepoints. When you roll back to a savepoint, any savepoints marked after that savepoint are erased. The savepoint to which you have rolled back is retained.

When savepoint names are reused within a transaction, the Oracle Database moves (overrides) the save point from its old position to the current point in the transaction.

Joins

Use a join to query data from more than one table:

```
SELECT table1.column, table2.column
FROM   table1, table2
WHERE  table1.column1 = table2.column2;
```

- Write the join condition in the `WHERE` clause.
- Prefix the column name with the table name when the same column name appears in more than one table.

24

When data from more than one table in the database is required, a *join* condition is used. Rows in one table can be joined to rows in another table according to common values that exist in the corresponding columns (usually primary and foreign key columns).

To display data from two or more related tables, write a simple join condition in the `WHERE` clause.

In the syntax:

```
table1.column
table1.column1 =
table2.column2
```

Denotes the table and column from which data is retrieved
Is the condition that joins (or relates) the tables together

Guidelines

- When writing a `SELECT` statement that joins tables, precede the column name with the table name for clarity and to enhance database access.
- If the same column name appears in more than one table, the column name must be prefixed with the table name.
- To join n tables together, you need a minimum of $n-1$ join conditions. For example, to join four tables, a minimum of three joins is required. This rule may not apply if your table has a concatenated primary key, in which case more than one column is required to uniquely identify each row.

Types of Joins

- Natural join
- Equijoin
- Nonequijoin
- Outer join
- Self-join
- Cross join

25

To join tables, you can use Oracle's join syntax.

Note: Before the Oracle9i release, the join syntax was proprietary. The SQL:1999-compliant join syntax does not offer any performance benefits over the Oracle-proprietary join syntax.

Qualifying Ambiguous Column Names

- Use table prefixes to qualify column names that are in multiple tables.
- Use table prefixes to improve performance.
- Use table aliases, instead of full table name prefixes.
- Table aliases give a table a shorter name.
 - This keeps SQL code smaller and uses less memory.
- Use column aliases to distinguish columns that have identical names, but reside in different tables.

26

When joining two or more tables, you need to qualify the names of the columns with the table name to avoid ambiguity. Without the table prefixes, the `DEPARTMENT_ID` column in the `SELECT` list could be from either the `DEPARTMENTS` table or the `EMPLOYEES` table. Therefore, it is necessary to add the table prefix to execute your query. If there are no common column names between the two tables, there is no need to qualify the columns. However, using a table prefix improves performance, because you tell the Oracle server exactly where to find the columns.

Qualifying column names with table names can be very time consuming, particularly if table names are lengthy. Therefore, you can use *table aliases*, instead of table names. Just as a column alias gives a column another name, a table alias gives a table another name. Table aliases help to keep SQL code smaller, thereby using less memory.

The table name is specified in full, followed by a space, and then the table alias. For example, the `EMPLOYEES` table can be given an alias of `e`, and the `DEPARTMENTS` table an alias of `d`.

Guidelines

- Table aliases can be up to 30 characters in length, but shorter aliases are better than longer ones.
- If a table alias is used for a particular table name in the `FROM` clause, that table alias must be substituted for the table name throughout the `SELECT` statement.
- Table aliases should be meaningful.
- A table alias is valid only for the current `SELECT` statement.

Natural Join

- The `NATURAL JOIN` clause is based on all the columns in the two tables that have the same name.
- It selects rows from tables that have the same names and data values of columns.
- Example:

```
SELECT country_id, location_id, country_name, city
FROM countries NATURAL JOIN locations;
```

COUNTRY_ID	LOCATION_ID	COUNTRY_NAME	CITY
1 US	1400	United States of America	Southlake
2 US	1500	United States of America	South San Francisco
3 US	1700	United States of America	Seattle
4 CA	1800	Canada	Toronto
5 UK	2500	United Kingdom	Oxford

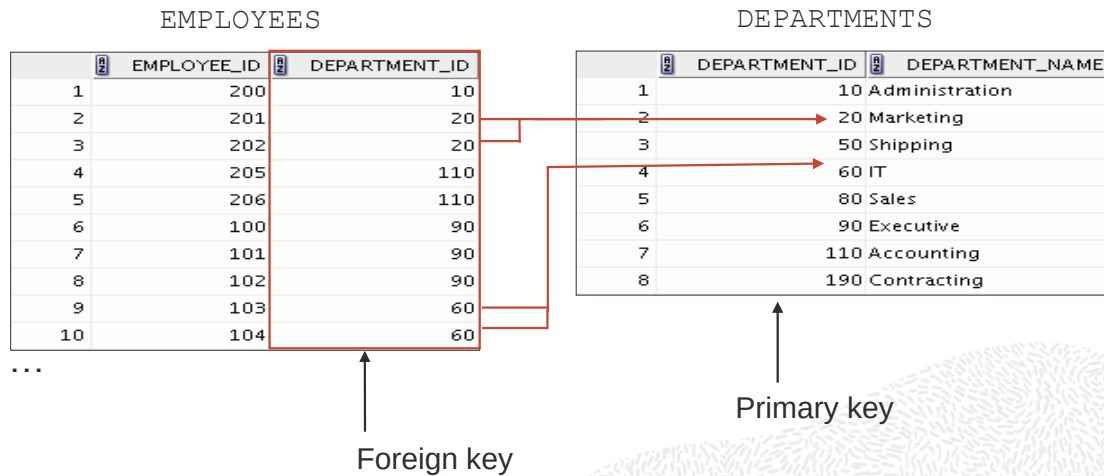
27

You can join tables automatically based on the columns in the two tables that have matching data types and names. You do this by using the `NATURAL JOIN` keywords.

Note: The join can happen only on those columns that have the same names and data types in both tables. If the columns have the same name but different data types, the `NATURAL JOIN` syntax causes an error.

In the example in the slide, the `COUNTRIES` table is joined to the `LOCATIONS` table by the `COUNTRY_ID` column, which is the only column of the same name in both tables. If other common columns were present, the join would have used them all.

Equijoins



28

An **equijoin** is a join with a join condition containing an equality operator. An equijoin combines rows that have equivalent values for the specified columns. To determine an employee's department name, you compare the values in the `DEPARTMENT_ID` column in the `EMPLOYEES` table with the `DEPARTMENT_ID` values in the `DEPARTMENTS` table. The relationship between the `EMPLOYEES` and `DEPARTMENTS` tables is an *equijoin*; that is, values in the `DEPARTMENT_ID` column in both tables must be equal. Often, this type of join involves primary and foreign key complements.

Note: Equijoins are also called *simple joins*.

Retrieving Records with Equijoins

```
SELECT e.employee_id, e.last_name, e.department_id,  
       d.department_id, d.location_id  
FROM   employees e JOIN departments d  
ON e.department_id = d.department_id;
```

	EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_ID_1	LOCATION_ID
1	200	Whalen	10	10	1700
2	201	Hartstein	20	20	1800
3	202	Fay	20	20	1800
4	144	Vargas	50	50	1500
5	143	Matos	50	50	1500
6	142	Davies	50	50	1500
7	141	Rajs	50	50	1500
8	124	Mourgos	50	50	1500
9	103	Hunold	60	60	1400
10	104	Ernst	60	60	1400
11	107	Lorentz	60	60	1400

...

29

In the example in the slide:

- **The `SELECT` clause specifies the column names to retrieve:**
 - Employee last name, employee ID, and department ID, which are columns in the `EMPLOYEES` table
 - Department ID and location ID, which are columns in the `DEPARTMENTS` table
- **The `FROM` clause specifies the two tables that the database must access:**
 - `EMPLOYEES` table
 - `DEPARTMENTS` table
- **The `WHERE` clause specifies how the tables are to be joined:**

```
e.department_id = d.department_id
```

Because the `DEPARTMENT_ID` column is common to both tables, it must be prefixed with the table alias to avoid ambiguity. Other columns that are not present in both the tables need not be qualified by a table alias, but it is recommended for better performance.

Note: When you use the Execute Statement icon to run the query, SQL Developer suffixes a “_1” to differentiate between the two `DEPARTMENT_IDS`.

Additional Search Conditions Using the AND and WHERE Operators

```
SELECT d.department_id, d.department_name, l.city
FROM   departments d JOIN locations l
ON     d.location_id = l.location_id
AND d.department_id IN (20, 50);
```

	DEPARTMENT_ID	DEPARTMENT_NAME	CITY
1	20	Marketing	Toronto
2	50	Shipping	South San Francisco

```
SELECT d.department_id, d.department_name, l.city
FROM   departments d JOIN locations l
ON     d.location_id = l.location_id
WHERE  d.department_id IN (20, 50);
```

30

In addition to the join, you may have criteria for your `WHERE` clause to restrict the rows in consideration for one or more tables in the join. The example in the slide performs a join on the `DEPARTMENTS` and `LOCATIONS` tables and, in addition, displays only those departments with ID equal to 20 or 50. To add additional conditions to the `ON` clause, you can add `AND` clauses. Alternatively, you can use a `WHERE` clause to apply additional conditions.

Both queries produce the same output.

	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Matos	50	Shipping

Retrieving Records with Nonequijoins

```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e JOIN job_grades j
ON     e.salary
      BETWEEN j.lowest_sal AND j.highest_sal;
```

	LAST_NAME	SALARY	GRADE_LEVEL
1	Vargas	2500	A
2	Matos	2600	A
3	Davies	3100	B
4	Rajs	3500	B
5	Lorentz	4200	B
6	Whalen	4400	B
7	Fay	6000	C

...

31

The example in the slide creates a nonequijoin to evaluate an employee's salary grade. The salary must be *between* any pair of the low and high salary ranges.

It is important to note that all employees appear exactly once when this query is executed. No employee is repeated in the list. There are two reasons for this:

- None of the rows in the job grade table contain grades that overlap. That is, the salary value for an employee can lie only between the low salary and high salary values of one of the rows in the salary grade table.
- All of the employees' salaries lie within the limits that are provided by the job grade table. That is, no employee earns less than the lowest value contained in the `LOWEST_SAL` column or more than the highest value contained in the `HIGHEST_SAL` column.

Note: Other conditions (such as `<=` and `>=`) can be used, but `BETWEEN` is the simplest. Remember to specify the low value first and the high value last when using the `BETWEEN` condition. The Oracle server translates the `BETWEEN` condition to a pair of `AND` conditions. Therefore, using `BETWEEN` has no performance benefits, but should be used only for logical simplicity.

Table aliases have been specified in the example in the slide for performance reasons, not because of possible ambiguity.

Retrieving Records by Using the USING Clause

- You can use the `USING` clause to match only one column when more than one column matches.
- You cannot specify this clause with a `NATURAL` join.
- Do not qualify the column name with a table name or table alias.
- Example:

```
SELECT country_id, country_name, location_id, city
FROM   countries JOIN locations
      USING (country_id) ;
```

	COUNTRY_ID	COUNTRY_NAME	LOCATION_ID	CITY
1	US	United States of America	1400	Southlake
2	US	United States of America	1500	South San Francisco
3	US	United States of America	1700	Seattle
4	CA	Canada	1800	Toronto
5	UK	United Kingdom	2500	Oxford

32

In the example in the slide, the `COUNTRY_ID` columns in the `COUNTRIES` and `LOCATIONS` tables are joined and thus the `LOCATION_ID` of the location where an employee works is shown.

Retrieving Records by Using the ON Clause

- The join condition for the natural join is basically an equijoin of all columns with the same name.
- Use the ON clause to specify arbitrary conditions or specify columns to join.
- The ON clause makes code easy to understand.

```
SELECT e.employee_id, e.last_name, j.department_id,  
FROM   employees e JOIN job_history j  
ON     (e.employee_id = j.employee_id);
```

	EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
1	101	Kochhar	110
2	101	Kochhar	110
3	102	De Haan	60
4	176	Taylor	80
5	176	Taylor	80
6	200	Whalen	90
7	200	Whalen	90
8	201	Hartstein	20

33

Use the ON clause to specify a join condition. With this, you can specify join conditions separate from any search or filter conditions in the WHERE clause.

In this example, the EMPLOYEE_ID columns in the EMPLOYEES and JOB_HISTORY tables are joined using the ON clause. Wherever an employee ID in the EMPLOYEES table equals an employee ID in the JOB_HISTORY table, the row is returned. The table alias is necessary to qualify the matching column names.

You can also use the ON clause to join columns that have different names. The parentheses around the joined columns, as in the example in the slide, (e.employee_id = j.employee_id), is optional. So, even ON e.employee_id = j.employee_id will work.

Note: When you use the Execute Statement icon to run the query, SQL Developer suffixes a '_1' to differentiate between the two employee_ids.

Left Outer Join

- A join between two tables that returns all matched rows, as well as the unmatched rows from the left table is called a `LEFT OUTER JOIN`.
- Example:

```
SELECT c.country_id, c.country_name, l.location_id, l.city
FROM   countries c LEFT OUTER JOIN locations l
ON     (c.country_id = l.country_id) ;
```

	COUNTRY_ID	COUNTRY_NAME	LOCATION_ID	CITY
1	CA	Canada	1800	Toronto
2	DE	Germany	(null)	(null)
3	UK	United Kingdom	2500	Oxford
4	US	United States of America	1400	Southlake
5	US	United States of America	1500	South San Francisco
6	US	United States of America	1700	Seattle

34

This query retrieves all the rows in the `COUNTRIES` table, which is the left table, even if there is no match in the `LOCATIONS` table.

Right Outer Join

- A join between two tables that returns all matched rows, as well as the unmatched rows from the right table is called a `RIGHT OUTER JOIN`.
- Example:

```
SELECT d.department_id, d.department_name, l.location_id, l.city
FROM   departments d RIGHT OUTER JOIN locations l
ON     (d.location_id = l.location_id) ;
```

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID	CITY
1	10 Administration	1700	Seattle
2	20 Marketing	1800	Toronto
3	30 Purchasing	1700	Seattle
4	40 Human Resources	2400	London
...			
26	260 Recruiting	1700	Seattle
27	270 Payroll	1700	Seattle
28	(null) (null)	2000	Beijing
29	(null) (null)	3000	Bern
...			

35

This query retrieves all the rows in the `LOCATIONS` table, which is the right table, even if there is no match in the `COUNTRIES` table.

Full Outer Join

- A join between two tables that returns all matched rows, as well as the unmatched rows from both tables is called a FULL OUTER JOIN.
- Example:

```
SELECT e.last_name, d.department_id, d.manager_id,  
       d.department_name  
FROM   employees e FULL OUTER JOIN departments d  
ON     (e.manager_id = d.manager_id) ;
```

	LAST_NAME	DEPARTMENT_ID	MANAGER_ID	DEPARTMENT_NAME
1	King	(null)	(null)	(null)
2	Kochhar	90	100	Executive
3	De Haan	90	100	Executive
4	Hunold	(null)	(null)	(null)
...				
106	Higgins	(null)	(null)	(null)
107	Gietz	110	205	Accounting
108	(null)	270	(null)	Payroll
109	(null)	260	(null)	Recruiting

36

This query retrieves all the rows in the EMPLOYEES table, even if there is no match in the DEPARTMENTS table. It also retrieves all the rows in the DEPARTMENTS table, even if there is no match in the EMPLOYEES table.

Self-Join: Example

```
SELECT worker.last_name || ' works for '
       || manager.last_name
FROM   employees worker JOIN employees manager
ON worker.manager_id = manager.employee_id
ORDER BY worker.last_name;
```

	WORKER.LAST_NAME 'WORKSFOR' MANAGER.LAST_NAME
1	Abel works for Zlotkey
2	Davies works for Mourgos
3	De Haan works for King
4	Ernst works for Hunold
5	Fay works for Hartstein
6	Gietz works for Higgins
7	Grant works for Zlotkey
8	Hartstein works for King
9	Higgins works for Kochhar

...

37

Sometimes you need to join a table to itself. To find the name of each employee's manager, you need to join the `EMPLOYEES` table to itself, or perform a self-join. The example in the slide joins the `EMPLOYEES` table to itself. To simulate two tables in the `FROM` clause, there are two aliases, namely `worker` and `manager`, for the same table, `EMPLOYEES`.

In this example, the `WHERE` clause contains the join that means "where a worker's manager ID matches the employee ID for the manager."

Cross Join

- A `CROSS JOIN` is a `JOIN` operation that produces the Cartesian product of two tables.
- Example:

```
SELECT department_name, city
FROM department CROSS JOIN location;
```

	DEPARTMENT_NAME	CITY
1	Administration	Oxford
2	Administration	Seattle
3	Administration	South San Francisco
4	Administration	Southlake
5	Administration	Toronto
6	Marketing	Oxford
7	Marketing	Seattle
8	Marketing	South San Francisco
9	Marketing	Southlake
10	Marketing	Toronto

...

38

The `CROSS JOIN` syntax specifies the cross product. It is also known as a Cartesian product. A cross join produces the cross product of two relations, and is essentially the same as the comma-delimited Oracle Database notation.

You do not specify any `WHERE` condition between the two tables in the `CROSS JOIN`.

Summary

In this appendix, you should have learned how to use:

- The `SELECT` statement to retrieve rows from one or more tables
- DDL statements to alter the structure of objects
- DML statements to manipulate data in the existing schema objects
- Transaction control statements to manage the changes made by DML statements
- Joins to display data from multiple tables



39

There are many commonly used commands and statements in SQL. It includes the DDL statements, DML statements, transaction control statements, and joins.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

REF Cursors

Cursor Variables

- Cursor variables are like C or Pascal pointers, which hold the memory location (address) of an item instead of the item itself.
- In PL/SQL, a pointer is declared as `REF X`, where `REF` is short for `REFERENCE` and `X` stands for a class of objects.
- A cursor variable has the data type `REF CURSOR`.
- A cursor is static, but a cursor variable is dynamic.
- Cursor variables give you more flexibility.



2

Cursor variables are like C or Pascal pointers, which hold the memory location (address) of an item instead of the item itself. Thus, declaring a cursor variable creates a pointer, not an item. In PL/SQL, a pointer has the data type `REF X`, where `REF` is short for `REFERENCE` and `X` stands for a class of objects. A cursor variable has the `REF CURSOR` data type.

Like a cursor, a cursor variable points to the current row in the result set of a multirow query. However, cursors differ from cursor variables the way constants differ from variables. A cursor is static, but a cursor variable is dynamic because it is not tied to a specific query. You can open a cursor variable for any type-compatible query. This gives you more flexibility.

Cursor variables are available to every PL/SQL client. For example, you can declare a cursor variable in a PL/SQL host environment such as an OCI or Pro*C program, and then pass it as an input host variable (bind variable) to PL/SQL. Moreover, application development tools such as Oracle Forms and Oracle Reports, which have a PL/SQL engine, can use cursor variables entirely on the client side. The Oracle Server also has a PL/SQL engine. You can pass cursor variables back and forth between an application and server through remote procedure calls (RPCs).

Using Cursor Variables

- You can use cursor variables to pass query result sets between PL/SQL stored subprograms and various clients.
- PL/SQL can share a pointer to the query work area in which the result set is stored.
- You can pass the value of a cursor variable freely from one scope to another.
- You can reduce network traffic by having a PL/SQL block open (or close) several host cursor variables in a single roundtrip.

3

You use cursor variables to pass query result sets between PL/SQL stored subprograms and various clients. Neither PL/SQL nor any of its clients owns a result set; they simply share a pointer to the query work area in which the result set is stored. For example, an OCI client, an Oracle Forms application, and the Oracle Server can all refer to the same work area.

A query work area remains accessible as long as any cursor variable points to it. Therefore, you can pass the value of a cursor variable freely from one scope to another. For example, if you pass a host cursor variable to a PL/SQL block that is embedded in a Pro*C program, the work area to which the cursor variable points remains accessible after the block completes.

If you have a PL/SQL engine on the client side, calls from the client to the server impose no restrictions. For example, you can declare a cursor variable on the client side, open and fetch from it on the server side, and then continue to fetch from it back on the client side. Also, you can reduce network traffic by having a PL/SQL block open (or close) several host cursor variables in a single roundtrip.

A cursor variable holds a reference to the cursor work area in the Program Global Area (PGA) instead of addressing it with a static name. Because you address this area by a reference, you gain the flexibility of a variable.

Defining REF CURSOR Types

Define a REF CURSOR type:

```
Define a REF CURSOR type  
TYPE ref_type_name IS REF CURSOR [RETURN return_type];
```

Declare a cursor variable of that type:

```
ref_cv ref_type_name;
```

Example:

```
DECLARE  
TYPE DeptCurTyp IS REF CURSOR RETURN  
departments%ROWTYPE;  
dept_cv DeptCurTyp;
```

4

To define a REF CURSOR, you perform two steps. First, you define a REF CURSOR type, and then you declare cursor variables of that type. You can define REF CURSOR types in any PL/SQL block, subprogram, or package using the following syntax:

```
TYPE ref_type_name IS REF CURSOR [RETURN return_type];
```

where:

`ref_type_name` Is a type specifier used in subsequent declarations of cursor variables

`return_type` Represents a record or a row in a database table

In this example, you specify a return type that represents a row in the database table `DEPARTMENT`. REF CURSOR types can be strong (restrictive) or weak (nonrestrictive). As the next example shows, a strong REF CURSOR type definition specifies a return type, but a weak definition does not:

```
DECLARE
```

```
TYPE EmpCurTyp IS REF CURSOR RETURN employees%ROWTYPE;    -- strong
```

```
TYPE GenericCurTyp IS REF CURSOR;    -- weak
```

Strong `REF CURSOR` types are less error prone because the PL/SQL compiler lets you associate a strongly typed cursor variable only with type-compatible queries. However, weak `REF CURSOR` types are more flexible because the compiler lets you associate a weakly typed cursor variable with any query.

Declaring Cursor Variables

After you define a `REF CURSOR` type, you can declare cursor variables of that type in any PL/SQL block or subprogram. In the following example, you declare the cursor variable `DEPT_CV`:

```
DECLARE

    TYPE DeptCurTyp IS REF CURSOR RETURN departments%ROWTYPE;

    dept_cv DeptCurTyp;  -- declare cursor variable
```

Note: You cannot declare cursor variables in a package. Unlike packaged variables, cursor variables do not have persistent states. Remember, declaring a cursor variable creates a pointer, not an item. Cursor variables cannot be saved in the database; they follow the usual scoping and instantiation rules.

In the `RETURN` clause of a `REF CURSOR` type definition, you can use `%ROWTYPE` to specify a record type that represents a row returned by a strongly (not weakly) typed cursor variable, as follows:

```
DECLARE

    TYPE TmpCurTyp IS REF CURSOR RETURN employees%ROWTYPE;

    tmp_cv TmpCurTyp;  -- declare cursor variable

    TYPE EmpCurTyp IS REF CURSOR RETURN tmp_cv%ROWTYPE;

    emp_cv EmpCurTyp;  -- declare cursor variable
```

Similarly, you can use `%TYPE` to provide the data type of a record variable, as the following example shows:

```
DECLARE

    dept_rec departments%ROWTYPE;  -- declare record variable

    TYPE DeptCurTyp IS REF CURSOR RETURN dept_rec%TYPE;

    dept_cv DeptCurTyp;  -- declare cursor variable
```

In the final example, you specify a user-defined `RECORD` type in the `RETURN` clause:

```
DECLARE

    TYPE EmpRecTyp IS RECORD (

        empno NUMBER(4),

        ename VARCHAR2(10),

        sal    NUMBER(7,2));

    TYPE EmpCurTyp IS REF CURSOR RETURN EmpRecTyp;

    emp_cv EmpCurTyp;  -- declare cursor variable
```


Cursor Variables as Parameters

You can declare cursor variables as the formal parameters of functions and procedures. In the following example, you define the `REF CURSOR` type `EmpCurTyp`, and then declare a cursor variable of that type as the formal parameter of a procedure:

```
DECLARE

    TYPE EmpCurTyp IS REF CURSOR RETURN emp%ROWTYPE;

    PROCEDURE open_emp_cv (emp_cv IN OUT EmpCurTyp) IS ...
```

Using the OPEN-FOR, FETCH, and CLOSE Statements

- The `OPEN-FOR` statement associates a cursor variable with a multirow query, executes the query, identifies the result set, and positions the cursor to point to the first row of the result set.
- The `FETCH` statement returns a row from the result set of a multirow query, assigns the values of the select-list items to the corresponding variables or fields in the `INTO` clause, increments the count kept by `%ROWCOUNT`, and advances the cursor to the next row.
- The `CLOSE` statement disables a cursor variable.

7

You use three statements to process a dynamic multirow query: `OPEN-FOR`, `FETCH`, and `CLOSE`. First, you “open” a cursor variable “for” a multirow query. Then you “fetch” rows from the result set one at a time. When all the rows are processed, you “close” the cursor variable.

Opening the Cursor Variable

The `OPEN-FOR` statement associates a cursor variable with a multirow query, executes the query, identifies the result set, positions the cursor to point to the first row of the results set, and then sets the rows-processed count kept by `%ROWCOUNT` to zero. Unlike the static form of `OPEN-FOR`, the dynamic form has an optional `USING` clause. At run time, bind arguments in the `USING` clause replace corresponding placeholders in the dynamic `SELECT` statement. The syntax is:

```
OPEN {cursor_variable | :host_cursor_variable} FOR      dynamic_string
      [USING bind_argument[, bind_argument]...];
```

where `CURSOR_VARIABLE` is a weakly typed cursor variable (one without a return type), `HOST_CURSOR_VARIABLE` is a cursor variable declared in a PL/SQL host environment such as an OCI program, and `dynamic_string` is a string expression that represents a multirow query.

In the following example, the syntax declares a cursor variable, and then associates it with a dynamic `SELECT` statement that returns rows from the `EMPLOYEES` table:

```
DECLARE
  TYPE EmpCurTyp IS REF CURSOR;  -- define weak REF CURSOR      type
  emp_cv  EmpCurTyp;  -- declare cursor variable
  my_ename VARCHAR2(15);
  my_sal   NUMBER := 1000;
BEGIN
  OPEN emp_cv FOR  -- open cursor variable
    'SELECT last_name, salary FROM employees WHERE salary >
      :s'
    USING my_sal;
  ...
END;
```

Any bind arguments in the query are evaluated only when the cursor variable is opened. Thus, to fetch rows from the cursor using different bind values, you must reopen the cursor variable with the bind arguments set to their new values each time.

Fetching from the Cursor Variable

The `FETCH` statement returns a row from the result set of a multirow query, assigns the values of the select-list items to the corresponding variables or fields in the `INTO` clause, increments the count kept by `%ROWCOUNT`, and advances the cursor to the next row. Use the following syntax:

```
FETCH {cursor_variable | :host_cursor_variable}
  INTO {define_variable[, define_variable]... | record};
```

Continuing the example, fetch rows from the cursor variable `emp_cv` into the define variables `MY_ENAME` and `MY_SAL`:

```
LOOP
  FETCH emp_cv INTO my_ename, my_sal;  -- fetch next row
  EXIT WHEN emp_cv%NOTFOUND;  -- exit loop when last row is      fetched
  -- process row
END LOOP;
```

For each column value returned by the query associated with the cursor variable, there must be a corresponding, type-compatible variable or field in the `INTO` clause. You can use a different `INTO` clause on separate fetches with the same cursor variable. Each fetch retrieves another row from the same result set. If you try to fetch from a closed or never-opened cursor variable, PL/SQL raises the predefined exception `INVALID_CURSOR`.

Closing the Cursor Variable

The `CLOSE` statement disables a cursor variable. After that, the associated result set is undefined. Use the following syntax:

```
CLOSE {cursor_variable | :host_cursor_variable};
```

In this example, when the last row is processed, close the `emp_cv` cursor variable:

```
LOOP
    FETCH emp_cv INTO my_ename, my_sal;
    EXIT WHEN emp_cv%NOTFOUND;
    -- process row
END LOOP;
CLOSE emp_cv; -- close cursor variable
```

If you try to close an already-closed or never-opened cursor variable, PL/SQL raises `INVALID_CURSOR`.

Example of Fetching

```
DECLARE
  TYPE EmpCurTyp IS REF CURSOR;
  emp_cv   EmpCurTyp;
  emp_rec  employees%ROWTYPE;
  sql_stmt VARCHAR2(200);
  my_job   VARCHAR2(10) := 'ST_CLERK';
BEGIN
  sql_stmt := 'SELECT * FROM employees
              WHERE job_id = :j';
  OPEN emp_cv FOR sql_stmt USING
my_job;
  LOOP
    FETCH emp_cv INTO emp_rec;
    EXIT WHEN emp_cv%NOTFOUND;
    -- process record
  END LOOP;
  CLOSE emp_cv;
END;
/
```

10

The example in the slide shows that you can fetch rows from the result set of a dynamic multirow query into a record. You must first define a REF CURSOR type, `EmpCurTyp`. You then define a cursor variable `emp_cv`, of the type `EmpCurTyp`. In the executable section of the PL/SQL block, the `OPEN-FOR` statement associates the cursor variable `emp_cv` with the multirow query, `sql_stmt`. The `FETCH` statement returns a row from the result set of a multirow query and assigns the values of the select-list items to `EMP_REC` in the `INTO` clause. When the last row is processed, close the `emp_cv` cursor variable.

ORACLE

E

Managing PL/SQL Code

0

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this appendix, you should be able to do the following:

- Describe and use conditional compilation
- Hide PL/SQL source code by using dynamic obfuscation and the wrap utility



2

This appendix introduces the conditional compilation and obfuscating (or wrapping) PL/SQL code.

For Instructor Use Only.
This document should not be distributed.

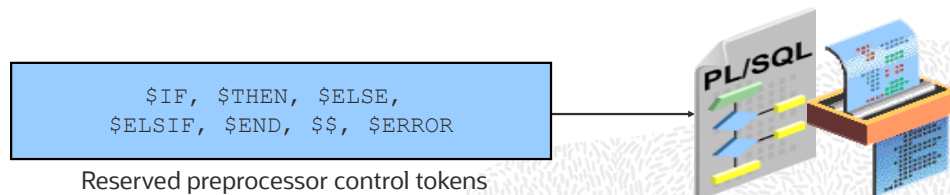
Agenda

- Using conditional compilation
- Obfuscating PL/SQL code

Conditional Compilation

Enables you to customize the functionality in a PL/SQL application without removing any source code:

- Use the latest functionality with the latest database release, or disable the new features to run the application against an older release of the database.
- Activate the debugging or tracing functionality in the development environment and hide that functionality in the application while it runs at a production site.



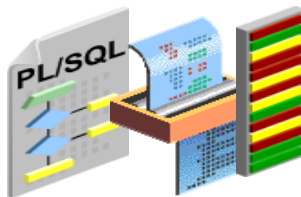
4

Conditional compilation enables you to selectively include code depending on the values of the conditions evaluated during compilation. For example, conditional compilation enables you to determine which PL/SQL features in a PL/SQL application are used for specific database releases. The latest PL/SQL features in an application can be run on a new database release; at the same time, those features can be conditional so that the same application is compatible with a previous database release. Conditional compilation is also useful when you want to execute debugging procedures in a development environment but want to turn off the debugging routines in a production environment.

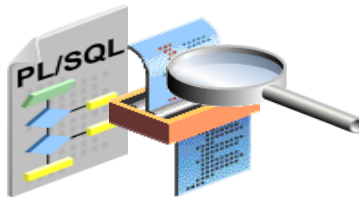
Benefits of Conditional Compilation

- Support for multiple versions of the same program in one source code
- Easy maintenance and debugging of code
- Easy migration of code to a different release of the database

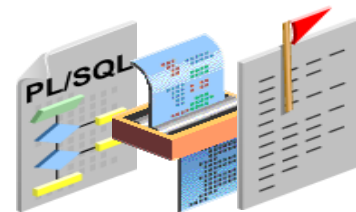
How Does Conditional Compilation Work?



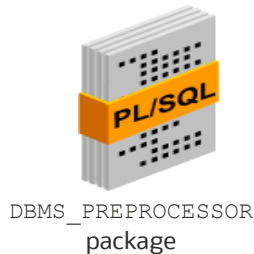
Selection directives:
Use the `$IF` token.



Inquiry directives:
Use the `$$` token.



Error directives:
Use the `$ERROR` token.



DBMS_PREPROCESSOR
package



DBMS_DB_VERSION
package

5

You can use conditional compilation by embedding directives in your PL/SQL source programs. When the PL/SQL program is submitted for compilation, a preprocessor evaluates these directives and selects parts of the program to be compiled. The selected program source is then handed off to the compiler for compilation.

Inquiry directives use the `$$` token to make inquiries about the compilation environment, such as the value of the PL/SQL compiler `PLSQL_CCFLAGS` or `PLSQL_OPTIMIZE_LEVEL` initialization parameters for the unit being compiled. This directive can be used in conjunction with the conditional selection directive to select the parts of the program to compile.

Selection directives can test inquiry directives or static package constants by using the `$IF` construct to branch sections of code for possible compilation if a condition is satisfied.

Error directives issue a compilation error if an unexpected condition is encountered during conditional compilation done by using the `$ERROR` token.

The `DBMS_DB_VERSION` package provides database version and release constants that can be used for conditional compilation.

The `DBMS_PREPROCESSOR` package provides subprograms for accessing the post-processed source text that is selected by conditional compilation directives in a PL/SQL unit.

Using Selection Directives

```
$IF <Boolean-expression> $THEN Text
$ELSIF <Boolean-expression> $THEN Text
. . .
$ELSE Text
$END
```

```
DECLARE
CURSOR cur IS SELECT employee_id FROM
employees WHERE
$IF myapp_tax_package.new_tax_code $THEN
    salary > 20000;
$ELSE
    salary > 50000;
$END
BEGIN
    OPEN cur;
. . .
END;
```

6

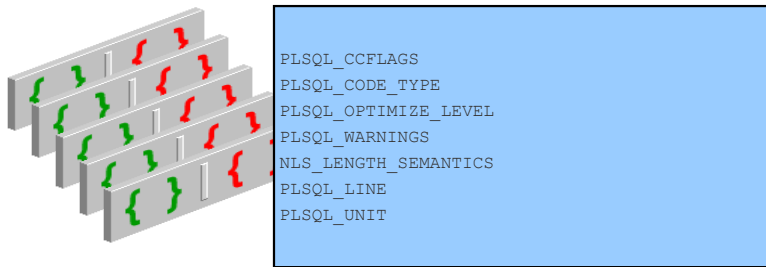
The conditional selection directive looks and operates like the `IF-THEN-ELSE` mechanism in PL/SQL proper. When the preprocessor encounters `$THEN`, it verifies that the text between `$IF` and `$THEN` is a static expression. If the check succeeds and the result of the evaluation is `TRUE`, the PL/SQL program text between `$THEN` and `$ELSE` (or `$ELSIF`) is selected for compilation.

The selection condition (the expression between `$IF` and `$THEN`) can be constructed by referring to constants defined in another package or an inquiry directive (or some combination of the two).

In the example in the slide, the conditional selection directive chooses between two versions of the cursor (`cur`) on the basis of the value of `MYAPP_TAX_PACKAGE.NEW_TAX_CODE`.

If the value is `TRUE`, employees with `salary > 20000` are selected; otherwise, employees with `salary > 50000` are selected.

Using Predefined and User-Defined Inquiry Directives



Predefined inquiry directives

```
PLSQL_CCFLAGS = 'plsql_ccflags:true,debug:true,debug:0';
```

User-defined inquiry directives

7

An inquiry directive can be predefined or user-defined. Following is the order of the processing flow when conditional compilation attempts to resolve an inquiry directive:

1. The ID is used as an inquiry directive in the form `$$id` for the search key.
2. The two-pass algorithm proceeds as follows:
 - a. The string in the `PLSQL_CCFLAGS` initialization parameter is scanned from right to left, searching by ID for a matching name (not case sensitive). Processing is done if an ID is found.
 - b. The predefined inquiry directives are searched. Processing is done if an ID is found.
3. If the `$$ID` cannot be resolved to a value, the `PLW-6003` warning message is reported if the source text is not wrapped. The literal `NULL` is substituted as the value for undefined inquiry directives.

Note: If the PL/SQL code is wrapped, the warning message is disabled so that the undefined inquiry directive is not revealed.

In the example in the slide, the value of `$$debug` is 0 and the value of `$$plsql_ccflags` is `TRUE`. Note that the value of `$$plsql_ccflags` resolves to the user-defined `plsql_ccflags` inside the value of the `PLSQL_CCFLAGS` compiler parameter. This occurs because a user-defined directive overrides the predefined one.

The PLSQL_CCFLAGS Parameter and the Inquiry Directive

Use the PLSQL_CCFLAGS parameter to control conditional compilation of each PL/SQL library unit independently.

```
PLSQL_CCFLAGS = '<v1>:<c1>,<v2>:<c2>,...,<vn>:<cn>'
```

```
ALTER SESSION SET  
PLSQL_CCFLAGS = 'plsql_ccflags:true, debug:true, debug:0';
```



8

Oracle Database 10g Release 2 introduced a new Oracle initialization parameter PLSQL_CCFLAGS for use with conditional compilation. This dynamic parameter enables you to set up name-value pairs. The names (called “flag names”) can then be referenced in inquiry directives. PLSQL_CCFLAGS provides a mechanism that enables PL/SQL programmers to control the conditional compilation of each PL/SQL library unit independently.

Values

- **vi:** Has the form of an unquoted PL/SQL identifier that is unrestricted and can be a reserved word or a keyword. The text is not case sensitive. Each is known as a flag or a flag name. Each **vi** can occur more than once in the string, each occurrence can have a different flag value, and the flag values can be of different kinds.
- **ci:** Can be any of the following:
 - A PL/SQL Boolean literal
 - A PLS_INTEGER literal
 - The literal NULL (default). The text is not case sensitive. Each is known as a flag value and corresponds to a flag name.

Displaying the PLSQL_CCFLAGS Initialization Parameter Setting

```
SELECT name, type, plsql_ccflags
FROM user_plsql_object_settings
```

Results:

	NAME	TYPE	PLSQL_CCFLAGS
1	DEPT_PKG	PACKAGE	(null)
2	DEPT_PKG	PACKAGE BODY	(null)
3	TAXES_PKG	PACKAGE	(null)
4	TAXES_PKG	PACKAGE BODY	(null)
5	EMP_PKG	PACKAGE	(null)
6	EMP_PKG	PACKAGE BODY	(null)
7	SECURE_DML	PROCEDURE	(null)
8	SECURE_EMPLOYEES	TRIGGER	(null)
9	ADD_JOB_HISTORY	PROCEDURE	plsql_ccflags:true, debug:true, debug:0
10	UPDATE_JOB_HISTORY	TRIGGER	(null)

...

0

Use the USER|ALL|DBA_PLSQL_OBJECT_SETTINGS data dictionary views to display the settings of a PL/SQL object.

You can define any allowable value for PLSQL_CCFLAGS. However, Oracle recommends that this parameter be used for controlling the conditional compilation of debugging or tracing code.

The flag names can be set to any identifier, including reserved words and keywords. The values must be the literals TRUE, FALSE, or NULL, or a PLS_INTEGER literal. The flag names and values are not case sensitive. The PLSQL_CCFLAGS parameter is a PL/SQL compiler parameter (like other compiler parameters) and is stored with the PL/SQL program unit. Consequently, if the PL/SQL program is recompiled later with the REUSE SETTINGS clause (example, ALTER PACKAGE ...REUSE SETTINGS), the same value of PLSQL_CCFLAGS is used for the recompilation. Because the PLSQL_CCFLAGS parameter can be set to a different value for each PL/SQL unit, it provides a convenient method for controlling conditional compilation on a per-unit basis.

The PLSQL_CCFLAGS Parameter and the Inquiry Directive: Example

```
ALTER SESSION SET PLSQL_CCFLAGS = 'Tracing:true';
CREATE OR REPLACE PROCEDURE P IS
BEGIN
    $IF $$tracing $THEN
        DBMS_OUTPUT.PUT_LINE ('TRACING');
    $END
END P;
```

```
ALTER SESSION SET succeeded.
PROCEDURE P Compiled.
```

```
SELECT name, plsql_ccflags
FROM USER_PLSQL_OBJECT_SETTINGS
WHERE name = 'P';
```

Results:		
	NAME	PLSQL_CCFLAGS
1	P	Tracing:true

In the example in the slide, the parameter is set and then the procedure is created. The setting is stored with each PL/SQL unit.

Using Conditional Compilation Error Directives to Raise User-Defined Errors

```
$ERROR varchar2_static_expression $END

ALTER SESSION SET Plsql_CCFlags = ' Trace_Level:3 '
/
CREATE PROCEDURE P IS
BEGIN
    $IF    $$Trace_Level = 0 $THEN ...;
    $ELSIF $$Trace_Level = 1 $THEN ...;
    $ELSIF $$Trace_Level = 2 $THEN ...;
    $else $error 'Bad: '||$$Trace_Level $END -- error
                                           -- directive
    $END -- selection directive ends
END P;
```

```
SHOW ERRORS
Errors for PROCEDURE P:
LINE/COL ERROR
-----
6/9      PLS-00179: $ERROR: Bad: 3
```

11

The \$ERROR error directive raises a user-defined error and is of the following form:

```
$ERROR varchar2_static_expression $END
```

Note: varchar2_static_expression must be a VARCHAR2 static expression.

Using Static Expressions with Conditional Compilation

- Boolean static expressions:
 - TRUE, FALSE, NULL, IS NULL, IS NOT NULL
 - >, <, >=, <=, =, <>, NOT, AND, OR
- PLS_INTEGER static expressions:
 - -2147483648 to 2147483647, NULL
- VARCHAR2 static expressions include:
 - ||, NULL, TO_CHAR
- Static constants:

```
static_constant CONSTANT datatype := static_expression;
```

12

As described earlier, a preprocessor processes conditional directives before proper compilation begins. Consequently, only those expressions that can be fully evaluated at compile time are permitted in conditional compilation directives. Expressions that contain references to variables or functions that require the execution of the PL/SQL are not available during compilation and cannot be evaluated.

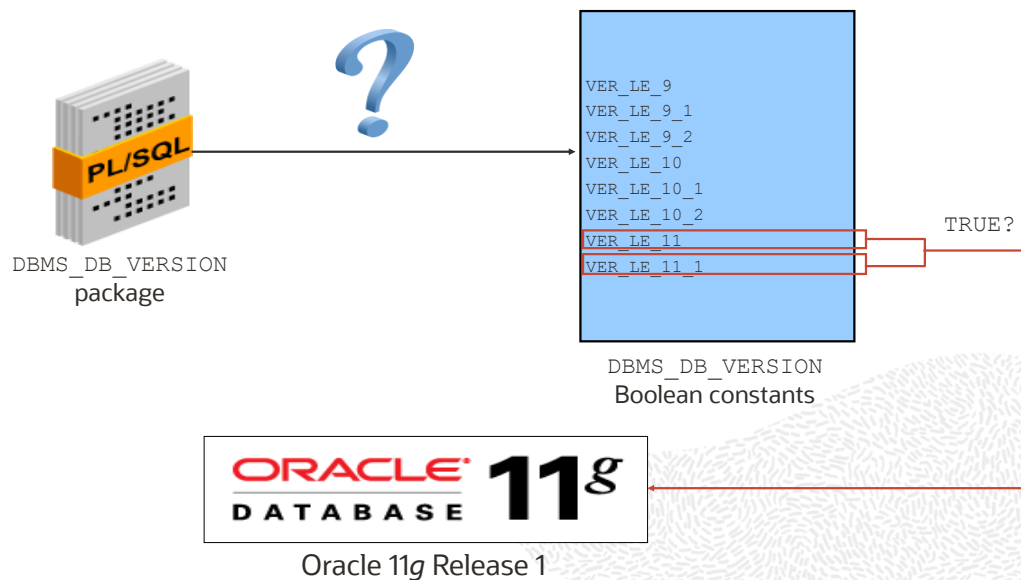
These PL/SQL expressions that are allowed in conditional compilation directives are referred to as “static expressions.” Static expressions are carefully defined to guarantee that if a unit is automatically recompiled without any changes to the values it depends on, the expressions evaluate in the same way and the same source is compiled.

Static expressions are typically composed of three sources:

- Inquiry directives marked with \$\$
- Constants defined in PL/SQL packages such as DBMS_DB_VERSION. These values can be combined and compared using the ordinary operations of PL/SQL.
- Literals such as TRUE, FALSE, 'CA', 123, and NULL

Static expressions can also contain operations that include comparisons, logical Boolean operations (such as OR and AND), or concatenations of static character expression.

DBMS_DB_VERSION Package: Boolean Constants



13

Oracle Database 10g Release 2 introduced the `DBMS_DB_VERSION` package. This package specifies the Oracle Database version and release numbers that are useful when making simple selections for conditional compilation.

The constants represent a Boolean condition that evaluates to less than or equal to the version and the release, if present.

Example

`VER_LE_11` indicates that the database version ≤ 11 . The values of the constants are either `TRUE` or `FALSE`. For example, in an Oracle Database 11g Release 1 database, `VER_LE_11` and `VER_LE_11_1` are `TRUE` and all other constants are `FALSE`.

DBMS_DB_VERSION Package Constants

Name	Description
VER_LE_9	Version <= 9.
VER_LE_9_1	Version <= 9 and release <= 1.
VER_LE_9_2	Version <= 9 and release <= 2.
VER_LE_10	Version <= 10.
VER_LE_10_1	Version <= 10 and release <= 1.
VER_LE_10_2	Version <=10 and release <= 2.
VER_LE_11	Version <= 11.
VER_LE_11_1	Version <=11 and release <= 1.

14

The package for the Oracle Database 11g Release 1 version is shown as follows:

```
PACKAGE DBMS_DB_VERSION IS
    VERSION CONSTANT PLS_INTEGER := 11; -- RDBMS version
    number
    RELEASE CONSTANT PLS_INTEGER := 1;  -- RDBMS release
    number
    ver_le_9_1      CONSTANT BOOLEAN := FALSE;
    ver_le_9_2      CONSTANT BOOLEAN := FALSE;
    ver_le_9        CONSTANT BOOLEAN := FALSE;
    ver_le_10_1     CONSTANT BOOLEAN := FALSE;
    ver_le_10_2     CONSTANT BOOLEAN := FALSE;
    ver_le_10       CONSTANT BOOLEAN := FALSE;
    ver_le_11_1     CONSTANT BOOLEAN := TRUE;
    ver_le_11       CONSTANT BOOLEAN := TRUE;
END DBMS_DB_VERSION;
```

The DBMS_DB_VERSION package contains different constants for different Oracle Database releases. The Oracle Database 11g Release 1 version of the DBMS_DB_VERSION package uses the constants shown in the slide.

Using Conditional Compilation with Database Versions: Example

```
ALTER SESSION SET PLSQL_CCFLAGS = 'my_debug:FALSE, my_tracing:FALSE';
CREATE PACKAGE my_pkg AS
  SUBTYPE my_real IS
    -- Check the database version, if >= 10g, use BINARY_DOUBLE data type,
    -- else use NUMBER data type
    $IF DBMS_DB_VERSION.VERSION < 10 $THEN    NUMBER;
    $ELSE    BINARY_DOUBLE;
    $END
  my_pi my_real; my_e my_real;
END my_pkg;
/
CREATE PACKAGE BODY my_pkg AS
BEGIN
  $IF DBMS_DB_VERSION.VERSION < 10 $THEN
    my_pi := 3.14016408289008292431940027343666863227;
    my_e  := 2.71828182845904523536028747135266249775;
  $ELSE
    my_pi := 3.14016408289008292431940027343666863227d;
    my_e  := 2.71828182845904523536028747135266249775d;
  $END
END my_pkg;
/
```

15

0

This example also shows the use of the `PLSQL_CCFLAGS` parameter. You first set the `PLSQL_CCFLAGS` parameter flag to display debugging code and tracing information.

In the example in the slide on this page and the next page, conditional compilation is used to specify code for database versions. Conditional compilation is used to determine whether the `BINARY_DOUBLE` data type can be used in the calculations for PL/SQL units in the database. The `BINARY_DOUBLE` data type can be used only in Oracle Database 10g or later. If you are using Oracle Database 10g, the data type for `my_real` is `BINARY_DOUBLE`; otherwise, the data type for `my_real` is `NUMBER`.

In the specification of the new package (`my_pkg`), conditional compilation is used to check for the database version. In the body definition of the package, conditional compilation is used again to set the values of `my_pi` and `my_e` for future calculations based on the database version.

The result of the example code in the slide is as follows:

```
PACKAGE my_pkg compiled
PACKAGE BODY my_pkg compiled
session SET altered.
```

Using Conditional Compilation with Database Versions: Example

```
CREATE OR REPLACE PROCEDURE circle_area(p_radius my_pkg.my_real) IS
  v_my_area my_pkg.my_real;
  v_my_datatype VARCHAR2(30);
BEGIN
  v_my_area := my_pkg.my_pi * p_radius * p_radius;
  DBMS_OUTPUT.PUT_LINE('Radius: ' || TO_CHAR(p_radius)
    || ' Area: ' || TO_CHAR(v_my_area));
  $IF $$my_debug $THEN -- if my_debug is TRUE, run some debugging code
    SELECT DATA_TYPE INTO v_my_datatype FROM USER_ARGUMENTS
      WHERE OBJECT_NAME = 'CIRCLE_AREA' AND ARGUMENT_NAME = 'P_RADIUS';
    DBMS_OUTPUT.PUT_LINE('Datatype of the RADIUS argument is: ' ||
      v_my_datatype);
  $END
END; /
```

```
PROCEDURE circle_area compiled
```

```
CALL circle_area(50); -- Using Oracle Database 11g Release 2
```

```
CALL circle_area(50) succeeded.
Radius: 5.0E+001 Area: 7.8504102072252062E+003
```

16

In the example in the slide, a new procedure called `circle_area` is defined. This procedure calculates the area of a circle based on the values of the variables in the `my_pkg` package defined on the previous page. The procedure has one informal parameter: `radius`.

The procedure declares two variables:

- `my_area`, which is the same data type as `my_real` in `my_pkg`
- `my_datatype`, which is a `VARCHAR2(30)`

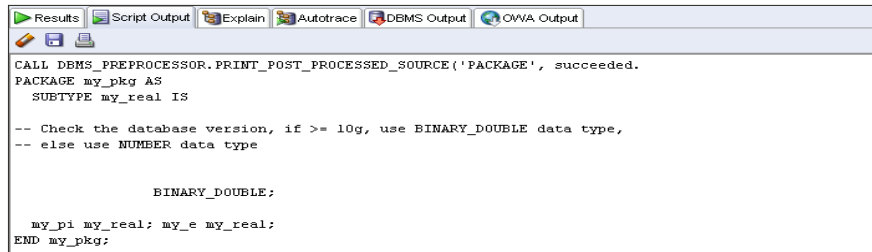
In the procedure's body, `my_area` becomes equal to the value of `my_pi` set in `my_pkg` multiplied by the value that is passed to the procedure as a radius. A message is printed displaying the radius and the area of the circle, as shown in the second code example in the slide.

Note: If you want to set `my_debug` to `TRUE`, you can make this change only for the `circle_area` procedure with the `REUSE SETTINGS` clause as follows:

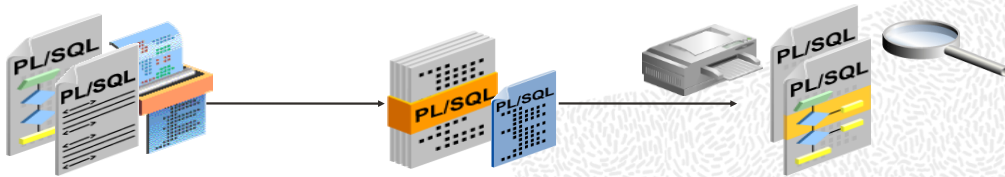
```
ALTER PROCEDURE circle_area COMPILE PLSQL_CCFLAGS = 'my_debug:TRUE' REUSE
SETTINGS;
```

Using DBMS_PREPROCESSOR Procedures to Print or Retrieve Source Text

```
CALL  
DBMS_PREPROCESSOR.PRINT_POST_PROCESSED_SOURCE('PACKAGE', 'ORA61', 'MY_PKG');
```



```
CALL DBMS_PREPROCESSOR.PRINT_POST_PROCESSED_SOURCE('PACKAGE', succeeded.  
PACKAGE my_pkg AS  
  SUBTYPE my_real IS  
  
  -- Check the database version, if >= 10g, use BINARY_DOUBLE data type,  
  -- else use NUMBER data type  
  
      BINARY_DOUBLE;  
  
  my_pi my_real; my_e my_real;  
END my_pkg;
```



17

DBMS_PREPROCESSOR subprograms print or retrieve the postprocessed source text of a PL/SQL unit after processing the conditional compilation directives. This postprocessed text is the actual source that is used to compile a valid PL/SQL unit. The example in the slide shows how to print the postprocessed form of `my_pkg` by using the `PRINT_POST_PROCESSED_SOURCE` procedure.

When `my_pkg` is compiled on an Oracle Database 10g release (or later) database using the HR account, the resulting output is shown in the slide.

The `PRINT_POST_PROCESSED_SOURCE` removes unselected text. The lines of code that are not included in the postprocessed text are removed. The arguments for the `PRINT_POST_PROCESSED_SOURCE` procedure are object type, schema name (using student account `ORA61`), and object name.

Note: For additional information about the `DBMS_PREPROCESSOR` package, refer to the *Oracle Database PL/SQL Packages and Types Reference*.

Agenda

- Using conditional compilation
- Obfuscating PL/SQL code

Obfuscation

- The obfuscation (or wrapping) of a PL/SQL unit is the process of hiding the PL/SQL source code.
- Wrapping can be done with the `wrap` utility and `DBMS_DDL` subprograms.
- The `wrap` utility is run from the command line. It processes an input SQL file, such as a SQL*Plus installation script.
- The `DBMS_DDL` subprograms wrap a single PL/SQL unit, such as a single `CREATE PROCEDURE` command, that has been generated dynamically.

19

Note: For additional information about obfuscation, refer to the *Oracle Database PL/SQL Language Reference*.

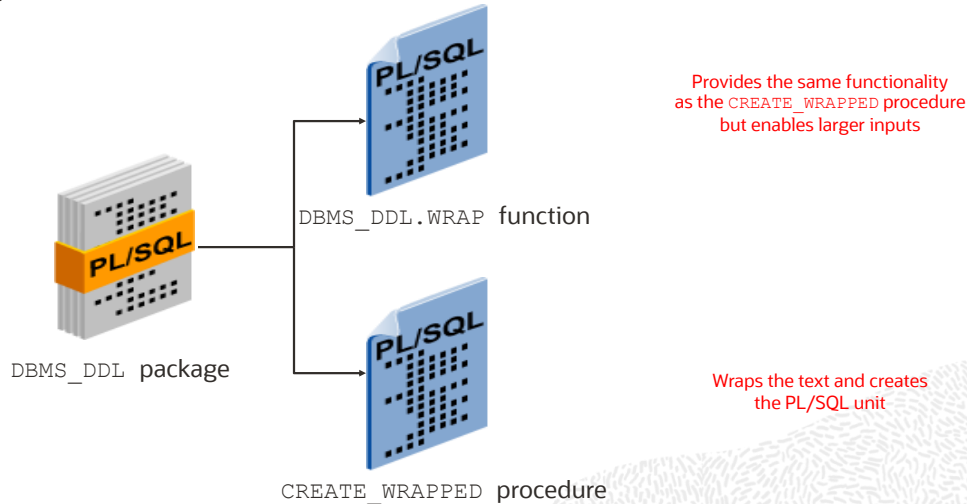
0

For Instructor Use Only.
This document should not be distributed.

Benefits of Obfuscating

- It prevents others from seeing your source code.
- Your source code is not visible through the `USER_SOURCE`, `ALL_SOURCE`, or `DBA_SOURCE` data dictionary views.
- SQL*Plus can process the obfuscated source files.
- The `Import` and `Export` utilities accept wrapped files.

What's New in Dynamic Obfuscating Since Oracle 10g?



21

`CREATE_WRAPPED` Procedure

Does the following:

1. Takes as input a single `CREATE` OR `REPLACE` statement that specifies creation of a PL/SQL package specification, package body, function, procedure, type specification, or type body
2. Generates a `CREATE` OR `REPLACE` statement in which the PL/SQL source text has been obfuscated
3. Executes the generated statement

`WRAP` Function

Does the following:

1. Takes as input a `CREATE` OR `REPLACE` statement that specifies the creation of a PL/SQL package specification, package body, function, procedure, type specification, or type body
2. Returns a `CREATE` OR `REPLACE` statement in which the text of the PL/SQL unit has been obfuscated

Nonobfuscated PL/SQL Code: Example

```
SET SERVEROUTPUT ON
BEGIN -- The ALL_SOURCE view family shows source code
EXECUTE IMMEDIATE '
  CREATE OR REPLACE PROCEDURE P1 IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE (''I am not wrapped'');
  END P1;
';
END;
/
CALL p1();
```

```
anonymous block completed
CALL p1() succeeded.
I'm not wrapped
```

```
SELECT text FROM user_source
WHERE name = 'P1' ORDER BY line;
```

	TEXT
1	PROCEDURE P1 IS
2	BEGIN
3	DBMS_OUTPUT.PUT_LINE ('I am not wrapped');
4	END P1;

22

In the first example in the slide, the `EXECUTE IMMEDIATE` statement is used to create the `P1` procedure. The code in the created procedure is not wrapped. The code is not hidden when you use any of the views from the `ALL_SOURCE` view family to display the procedure's code as shown in the slide.

Obfuscated PL/SQL Code: Example

```
BEGIN -- ALL_SOURCE view family obfuscates source code
  DBMS_DDL.CREATE_WRAPPED ( '
    CREATE OR REPLACE PROCEDURE P1 IS
    BEGIN
      DBMS_OUTPUT.PUT_LINE (''I am wrapped now'');
    END P1;
  ' );
END;
/
CALL p1();
```

```
anonymous block completed
p1 ) succeeded.
```

```
SELECT text FROM user_source
WHERE name = 'P1' ORDER BY line;
```

```
TEXT
-----
PROCEDURE P1 wrapped
a000000
b2
abcd
```

...

23

In the example in the slide, the `DBMS_DDL.CREATE_WRAPPED` package procedure is used to create the `P1` procedure. The code is obfuscated when you use any of the views from the `ALL_SOURCE` view family to display the procedure's code (as shown on the next page). When you check the `*_SOURCE` views, the source is wrapped, or hidden, so that others cannot view the code details shown in the output of the command in the slide.

Dynamic Obfuscation: Example

```
SET SERVEROUTPUT ON

DECLARE
c_code CONSTANT VARCHAR2(32767) :=
' CREATE OR REPLACE PROCEDURE new_proc AS
  v_VDATE  DATE;
BEGIN
  v_VDATE := SYSDATE;
  DBMS_OUTPUT.PUT_LINE(v_VDATE) ;
END; ' ;
BEGIN
  DBMS_DDL.CREATE_WRAPPED (c_CODE);
END;
/
```

24

The example in the slide displays the creation of a dynamically obfuscated procedure called NEW_PROC. To verify that the code for NEW_PROC is obfuscated, you can query from the DBA|ALL|USER_SOURCE dictionary views:

```
SELECT text FROM user_source
WHERE name = 'NEW_PROC';
```

TEXT

PROCEDURE new_proc wrapped
a0000000

7

71 9e

hBWMpGeSsd58b4jCP3/0d04rof0wg5nnm7+fMr2ywFyFDGLQ1haXriu4dCuPCWnnx1J0U1xp
pvc8nsr7Seq/riQvHRsXAQovdoh0K6ZvM1Kbskr+KLK957KzHQYwLK4k6rJLCS5EyJ7qJB/2
RDmm3j79Uw==

0

For Instructor Use Only.
This document should not be distributed.

PL/SQL Wrapper Utility

- The PL/SQL wrapper is a stand-alone utility that hides application internals by converting PL/SQL source code to portable object code.
- Wrapping has the following features:
 - Platform independence
 - Dynamic loading
 - Dynamic binding
 - Dependency checking
 - Normal importing and exporting when invoked

25

Using the PL/SQL Wrapper utility, you can deliver PL/SQL applications without exposing your source code, which may contain proprietary algorithms and data structures. The Wrapper utility converts the readable source code into unreadable code. By hiding application internals, it prevents misuse of your application.

Wrapped code (such as PL/SQL stored programs) has several features:

- It is platform independent, so you do not need to deliver multiple versions of the same compilation unit.
- It permits dynamic loading, so users do not need to shut down and restart to add a new feature.
- It permits dynamic binding, so external references are resolved at load time.
- It offers strict dependency checking, so that invalidated program units are recompiled automatically when they are invoked.
- It supports normal importing and exporting, so the import/export utility can process wrapped files.

Running the PL/SQL Wrapper Utility

```
WRAP INAME=input_file_name [ONAME=output_file_name]
```

- Do not use spaces around the equal signs.
- The INAME argument is required.
- The default extension for the input file is .sql, unless it is specified with the name.
- The ONAME argument is optional.
- The default extension for output file is .plb, unless specified with the ONAME argument.

Examples

```
WRAP INAME=demo_04_hello.sql  
WRAP INAME=demo_04_hello  
WRAP INAME=demo_04_hello.sql ONAME=demo_04_hello.plb
```

26

The PL/SQL Wrapper utility is an operating system executable called `WRAP`.

Note: This is 11g-only code.

To run the wrapper, enter the following command at your operating system prompt:

```
WRAP INAME=input_file_name [ONAME=output_file_name]
```

Each of the examples shown in the slide takes a file called `demo_04_hello.sql` as input and creates an output file called `demo_04_hello.plb`.

After the wrapped file is created, execute the `.plb` file from SQL*Plus to compile and store the wrapped version of the source code (as you would execute SQL script files).

Note

- Only the `INAME` argument is required. If the `ONAME` argument is not specified, the output file acquires the same name as the input file with an extension of `.plb`.
- The input file can have any extension, but the default is `.sql`.
- Case sensitivity of the `INAME` and `ONAME` values depends on the operating system.
- The output file is usually much larger than the input file.
- Do not put spaces around the equal signs in the `INAME` and `ONAME` arguments and values.

Results of Wrapping

-- Original PL/SQL source code in input file:

```
CREATE PACKAGE banking IS
  min_bal := 100;
  no_funds EXCEPTION;
  ...
END banking;
/
```

-- Wrapped code in output file:

```
CREATE PACKAGE banking
  wrapped
012abc463e ...
/
```

27

When it is wrapped, an object type, package, or subprogram has the following form: header, followed by the word `wrapped`, followed by the encrypted body.

The input file can contain any combination of SQL statements. However, the PL/SQL Wrapper utility wraps only the following `CREATE` statements:

- `CREATE [OR REPLACE] TYPE`
- `CREATE [OR REPLACE] TYPE BODY`
- `CREATE [OR REPLACE] PACKAGE`
- `CREATE [OR REPLACE] PACKAGE BODY`
- `CREATE [OR REPLACE] FUNCTION`
- `CREATE [OR REPLACE] PROCEDURE`

All other SQL `CREATE` statements are passed intact to the output file.

Guidelines for Wrapping

- You must wrap only the package body and not the package specification.
- The wrapper can detect syntactic errors but cannot detect semantic errors.
- The output file should not be edited. You maintain the original source code and wrap again as required.
- To ensure that all the important parts of your source code are obfuscated, view the wrapped file in a text editor before distributing it.

28

Guidelines include the following:

- When wrapping a package or object type, wrap only the body and not the specification. By doing this, you give other developers the information that they need to use the package without exposing its implementation.
- If your input file contains syntactic errors, the PL/SQL Wrapper utility detects and reports them. However, the wrapper cannot detect semantic errors because it does not resolve external references. For example, the wrapper does not report an error if the table or view `amp` does not exist:

```
CREATE PROCEDURE raise_salary (emp_id INTEGER, amount NUMBER) AS
BEGIN
    UPDATE amp  -- should be emp
    SET sal = sal + amount WHERE empno = emp_id;
END;
```

However, the PL/SQL compiler resolves external references. Therefore, semantic errors are reported when the wrapper output file (`.plb` file) is compiled.

- Because its contents are not readable, the output file should not be edited. To change a wrapped object, you must modify the original source code and wrap the code again.

DBMS_DDL Package Versus wrap Utility

Functionality	DBMS_DDL Package	wrap Utility
Code obfuscation	Yes	Yes
Dynamic Obfuscation	Yes	No
Obfuscate multiple programs at a time	No	Yes

29

Both the `wrap` utility and the `DBMS_DDL` package have distinct uses.

wrap Utility

The `wrap` utility is useful for obfuscating multiple programs with one execution of the utility. In essence, a complete application can be wrapped. However, the `wrap` utility cannot be used to obfuscate dynamically generated code at run time. The `wrap` utility processes an input SQL file and obfuscates only the PL/SQL units in the file, such as:

- Package specification and body
- Function and procedure
- Type specification and body

The `wrap` utility does not obfuscate PL/SQL content in:

- Anonymous blocks
- Triggers
- Non-PL/SQL code

DBMS_DDL Package

The `DBMS_DDL` package is intended to obfuscate a dynamically generated program unit from within another program unit. The `DBMS_DDL` package methods cannot obfuscate multiple program units at one execution. Each execution of these methods accepts only one `CREATE` OR `REPLACE` statement at a time as argument.

Summary

In this appendix, you should have learned how to:

- Describe and use conditional compilation
- Hide PL/SQL source code by using dynamic obfuscation and the `wrap` utility



For Instructor Use Only.
This document should not be distributed.

Implementing Triggers

Objectives

After completing this appendix, you should be able to do the following:

- Enhance database security with triggers
- Enforce data integrity with DML triggers
- Maintain referential integrity using triggers
- Use triggers to replicate data between tables
- Use triggers to automate computation of derived data
- Provide event-logging capabilities by using triggers



2

In this lesson, you learn to develop database triggers in order to enhance features that cannot otherwise be implemented by the Oracle server. In some cases, it may be sufficient to refrain from using triggers and accept the functionality provided by the Oracle server.

This lesson covers the following business application scenarios:

- Security
- Auditing
- Data integrity
- Referential integrity
- Table replication
- Computing derived data automatically
- Event logging

For Instructor Use Only.
This document should not be distributed.

Controlling Security Within the Server

Using database security with the GRANT statement

```
GRANT SELECT, INSERT, UPDATE, DELETE
ON    employees
TO    clerk;          -- database role
GRANT clerk TO scott;
```

3

Develop schemas and roles within the Oracle server to control the security of data operations on tables according to the identity of the user.

- Base privileges upon the username supplied when the user connects to the database.
- Determine access to tables, views, synonyms, and sequences.
- Determine query, data-manipulation, and data-definition privileges.

Controlling Security with a Database Trigger

```
CREATE OR REPLACE TRIGGER secure_emp
  BEFORE INSERT OR UPDATE OR DELETE ON employees
  DECLARE
    dummy PLS_INTEGER;
  BEGIN
    IF (TO_CHAR (SYSDATE, 'DY') IN ('SAT','SUN')) THEN
      RAISE_APPLICATION_ERROR(-20506,'You may only
        change data during normal business hours.');
```

```
    END IF;
    SELECT COUNT(*) INTO dummy FROM holiday
    WHERE holiday_date = TRUNC (SYSDATE);
    IF dummy > 0 THEN
      RAISE_APPLICATION_ERROR(-20507,
        'You may not change data on a holiday.');
```

```
    END IF;
  END;
/
```

4

Develop triggers to handle more complex security requirements.

- Base privileges on any database values, such as the time of day, the day of the week, and so on.
- Determine access to tables only.
- Determine data-manipulation privileges only.

Enforcing Data Integrity Within the Server

```
ALTER TABLE employees ADD  
CONSTRAINT ck_salary CHECK (salary >= 500);
```

5

You can enforce data integrity within the Oracle server and develop triggers to handle more complex data integrity rules.

The standard data integrity rules are not null, unique, primary key, and foreign key.

Use these rules to:

- Provide constant default values
- Enforce static constraints
- Enable and disable dynamically

Example

The code sample in the slide ensures that the salary is at least \$500.

0
For Instructor Use Only.
This document should not be distributed.

Protecting Data Integrity with a Trigger

```
CREATE OR REPLACE TRIGGER check_salary
BEFORE UPDATE OF salary ON employees
FOR EACH ROW
WHEN (NEW.salary < OLD.salary)
BEGIN
    RAISE_APPLICATION_ERROR (-20508,
        'Do not decrease salary.');
```

```
END;
/
```

6

Protect data integrity with a trigger and enforce nonstandard data integrity checks.

- Provide variable default values.
- Enforce dynamic constraints.
- Enable and disable dynamically.
- Incorporate declarative constraints within the definition of a table to protect data integrity.

Example

The code sample in the slide ensures that the salary is never decreased.

Enforcing Referential Integrity Within the Server

```
ALTER TABLE employees
  ADD CONSTRAINT emp_deptno_fk
    FOREIGN KEY (department_id)
      REFERENCES departments(department_id)
ON DELETE CASCADE;
```

7

Incorporate referential integrity constraints within the definition of a table to prevent data inconsistency and enforce referential integrity within the server.

- Restrict updates and deletes.
- Cascade deletes.
- Enable and disable dynamically.

Example

When a department is removed from the `DEPARTMENTS` parent table, cascade the deletion to the corresponding rows in the `EMPLOYEES` child table.

Protecting Referential Integrity with a Trigger

```
CREATE OR REPLACE TRIGGER cascade_updates
AFTER UPDATE OF department_id ON departments
FOR EACH ROW
BEGIN
  UPDATE employees
  SET employees.department_id=:NEW.department_id
  WHERE employees.department_id=:OLD.department_id;
  UPDATE job_history
  SET department_id=:NEW.department_id
  WHERE department_id=:OLD.department_id;
END;
/
```

8

The following referential integrity rules are not supported by declarative constraints:

- Cascade updates.
- Set to NULL for updates and deletions.
- Set to a default value on updates and deletions.
- Enforce referential integrity in a distributed system.
- Enable and disable dynamically.

You can develop triggers to implement these integrity rules.

Example

Enforce referential integrity with a trigger. When the value of `DEPARTMENT_ID` changes in the `DEPARTMENTS` parent table, cascade the update to the corresponding rows in the `EMPLOYEES` child table.

For a complete referential integrity solution using triggers, a single trigger is not enough.

Replicating a Table Within the Server

```
CREATE MATERIALIZED VIEW emp_copy  
NEXT sysdate + 7  
AS SELECT * FROM employees@ny;
```

9

Creating a Materialized View

Materialized views enable you to maintain copies of remote data on your local node for replication purposes. You can select data from a materialized view as you would from a normal database table or view. A materialized view is a database object that contains the results of a query, or a copy of some database on a query. The `FROM` clause of the query of a materialized view can name tables, views, and other materialized views.

When a materialized view is used, replication is performed implicitly by the Oracle server. This performs better than user-defined PL/SQL triggers for replication. Materialized views:

- Copy data from local and remote tables asynchronously, at user-defined intervals
- Can be based on multiple master tables
- Are read-only by default, unless using the Oracle Advanced Replication feature
- Improve the performance of data manipulation on the master table

Alternatively, you can replicate tables using triggers.

The example in the slide creates a copy of the remote `EMPLOYEES` table from New York. The `NEXT` clause specifies a date-time expression for the interval between automatic refreshes.

Replicating a Table with a Trigger

```
CREATE OR REPLACE TRIGGER emp_replica
BEFORE INSERT OR UPDATE ON employees FOR EACH ROW
BEGIN /* Proceed if user initiates data operation,
      NOT through the cascading trigger.*/
  IF INSERTING THEN
    IF :NEW.flag IS NULL THEN
      INSERT INTO employees@sf
        VALUES(:new.employee_id,...,'B');
      :NEW.flag := 'A';
    END IF;
  ELSE /* Updating. */
    IF :NEW.flag = :OLD.flag THEN
      UPDATE employees@sf
        SET ename=:NEW.last_name,...,flag=:NEW.flag
        WHERE employee_id = :NEW.employee_id;
    END IF;
    IF :OLD.flag = 'A' THEN :NEW.flag := 'B';
    ELSE :NEW.flag := 'A';
  END IF;
END IF;
END;
```

10

You can replicate a table with a trigger. By replicating a table, you can:

- Copy tables synchronously, in real time
- Base replicas on a single master table
- Read from replicas as well as write to them

Note: Excessive use of triggers can impair the performance of data manipulation on the master table, particularly if the network fails.

Example

In New York, replicate the local `EMPLOYEES` table to San Francisco.

Computing Derived Data Within the Server

```
UPDATE departments
SET total_sal=(SELECT SUM(salary)
                FROM employees
                WHERE employees.department_id =
                      departments.department_id);
```

11

By using the server, you can schedule batch jobs or use the database Scheduler for the following scenarios:

- Compute derived column values asynchronously, at user-defined intervals.
- Store derived values only within database tables.
- Modify data in one pass to the database and calculate derived data in a second pass.

Alternatively, you can use triggers to keep running computations of derived data.

Example

Keep the salary total for each department within a special `TOTAL_SALARY` column of the `DEPARTMENTS` table.

Computing Derived Values with a Trigger

```
CREATE PROCEDURE increment_salary
(id NUMBER, new_sal NUMBER) IS
BEGIN
  UPDATE departments
  SET   total_sal = NVL (total_sal, 0)+ new_sal
  WHERE department_id = id;
END increment_salary;
```

```
CREATE OR REPLACE TRIGGER compute_salary
AFTER INSERT OR UPDATE OF salary OR DELETE
ON employees FOR EACH ROW
BEGIN
  IF DELETING THEN      increment_salary(
    :OLD.department_id, (-1*OLD.salary));
  ELSIF UPDATING THEN   increment_salary(
    :NEW.department_id, (:NEW.salary-OLD.salary));
  ELSE                  increment_salary(
    :NEW.department_id, :NEW.salary); --INSERT
END IF;
END;
```

12

By using a trigger, you can perform the following tasks:

- Compute derived columns synchronously, in real time.
- Store derived values within database tables or within package global variables.
- Modify data and calculate derived data in a single pass to the database.

Example

Keep a running total of the salary for each department in the special `TOTAL_SALARY` column of the `DEPARTMENTS` table.

Logging Events with a Trigger

```
CREATE OR REPLACE TRIGGER notify_reorder_rep
BEFORE UPDATE OF quantity_on_hand, reorder_point
ON inventories FOR EACH ROW
DECLARE
  dsc product_descriptions.product_description%TYPE;
  msg_text VARCHAR2(2000);
BEGIN
  IF :NEW.quantity_on_hand <=
    :NEW.reorder_point THEN
    SELECT product_description INTO dsc
    FROM product_descriptions
    WHERE product_id = :NEW.product_id;
    _ msg_text := 'ALERT: INVENTORY LOW ORDER:' ||
      'Yours,' || CHR(10) || user || '.' || CHR(10);
  ELSIF :OLD.quantity_on_hand >=
    :NEW.quantity_on_hand THEN
    msg_text := 'Product #' || ... CHR(10);
  END IF;
  UTL_MAIL.SEND('inv@oracle.com', 'ord@oracle.com',
    message=>msg_text, subject=>'Inventory Notice');
END;
```

13

In the server, you can log events by querying data and performing operations manually. This sends an email message when the inventory for a particular product has fallen below the acceptable limit. This trigger uses the Oracle-supplied package `UTL_MAIL` to send the email message.

Logging Events Within the Server

1. Query data explicitly to determine whether an operation is necessary.
2. Perform the operation, such as sending a message.

Using Triggers to Log Events

1. Perform operations implicitly, such as firing off an automatic electronic memo.
2. Modify data and perform its dependent operation in a single step.
3. Log events automatically when data is changing.

Logging Events Transparently

In the trigger code:

- CHR(10) is a carriage return
- Reorder_point is not NULL
- Another transaction can receive and read the message in the pipe

Example

```
CREATE OR REPLACE TRIGGER notify_reorder_rep
BEFORE UPDATE OF amount_in_stock, reorder_point
ON inventory FOR EACH ROW
DECLARE
    dsc product.descrip%TYPE;
    msg_text VARCHAR2(2000);
BEGIN
    IF :NEW.amount_in_stock <= :NEW.reorder_point THEN
        SELECT descrip INTO dsc
        FROM PRODUCT WHERE prodid = :NEW.product_id;
        msg_text := 'ALERT: INVENTORY LOW ORDER: ' || CHR(10) ||
        'It has come to my personal attention that, due to recent '
        || CHR(10) || 'transactions, our inventory for product # ' ||
        TO_CHAR(:NEW.product_id) || '-- ' || dsc ||
        ' -- has fallen below acceptable levels.' || CHR(10) ||
        'Yours, ' || CHR(10) || user || '.' || CHR(10) || CHR(10);
    ELSIF :OLD.amount_in_stock >= :NEW.amount_in_stock THEN
        msg_text := 'Product #' || TO_CHAR(:NEW.product_id)
        || ' ordered. ' || CHR(10) || CHR(10);
    END IF;
    UTL_MAIL.SEND('inv@oracle.com', 'ord@oracle.com',
        message => msg_text, subject => 'Inventory Notice');
END;
```

Summary

In this appendix, you should have learned how to:

- Enhance database security with triggers
- Enforce data integrity with DML triggers
- Maintain referential integrity using triggers
- Use triggers to replicate data between tables
- Use triggers to automate computation of derived data
- Provide event-logging capabilities using triggers



15

This lesson provides some detailed comparison of using the Oracle database server functionality to implement security, auditing, data integrity, replication, and logging. The lesson also covers how database triggers can be used to implement the same features but go further to enhance the features that the database server provides. In some cases, you must use a trigger to perform some activities (such as computation of derived data) because the Oracle server cannot know how to implement this kind of business rule without some programming effort.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

ORACLE

G

Using the DBMS_SCHEDULER and HTP Packages

0

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this appendix, you should be able to do the following:

- Use the `HTP` package to generate a simple webpage
- Call the `DBMS_SCHEDULER` package to schedule PL/SQL code for execution



0

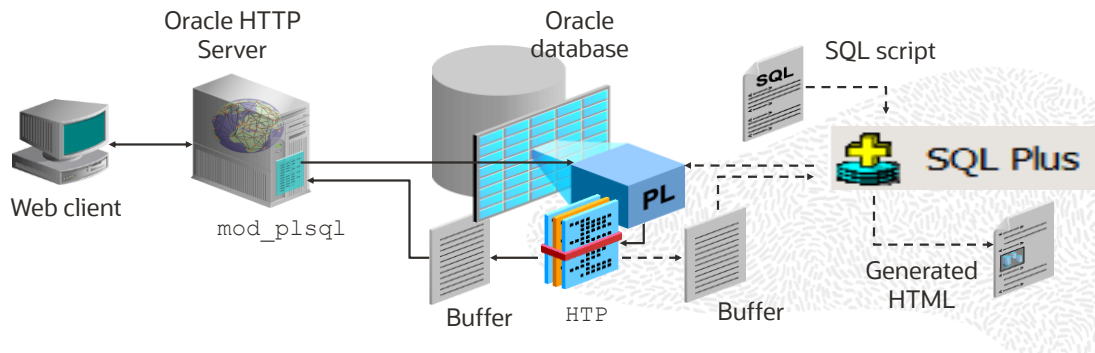
2

In this lesson, you learn how to use some of the Oracle-supplied packages and their capabilities. This lesson focuses on the packages that generate web-based output and the provided scheduling capabilities.

For Instructor Use Only.
This document should not be distributed.

Generating Webpages with the HTP Package

- The `HTP` package procedures generate HTML tags.
- The `HTP` package is used to generate HTML documents dynamically and can be invoked from:
 - A browser using Oracle HTTP Server and PL/SQL Gateway (`mod_plsql`) services
 - A SQL*Plus script to display HTML output



3

The `HTP` package contains procedures that are used to generate HTML tags. The HTML tags that are generated typically enclose the data provided as parameters to the various procedures. The slide illustrates two ways in which the `HTP` package can be used:

- Most likely your procedures are invoked by the PL/SQL Gateway services, via the `mod_plsql` component supplied with Oracle HTTP Server, which is part of the Oracle Application Server product (represented by solid lines in the graphic).
- Alternatively (as represented by dotted lines in the graphic), your procedure can be called from SQL*Plus that can display the generated HTML output, which can be copied and pasted to a file. This technique is used in this course because Oracle Application Server software is not installed as a part of the course environment.

Note: The `HTP` procedures output information to a session buffer held in the database server. In the Oracle HTTP Server context, when the procedure completes, the `mod_plsql` component automatically receives the buffer contents, which are then returned to the browser as the HTTP response. In SQL*Plus, you must manually execute:

- A `SET SERVEROUTPUT ON` command
- The procedure to generate the HTML into the buffer
- The `OWA_UTIL.SHOWPAGE` procedure to display the buffer contents

Using the HTP Package Procedures

- Generate one or more HTML tags. For example:

```
http.bold('Hello');           -- <B>Hello</B>
http.print('Hi <B>World</B>'); -- Hi <B>World</B>
```

- Are used to create a well-formed HTML document:

```
BEGIN                                -- Generates:
  http.htmlOpen;  ----->
  http.headOpen;  ----->
  http.title('Welcome'); -->
  http.headClose; ----->
  http.bodyOpen;  ----->
  http.print('My home page');
  http.bodyClose; ----->
  http.htmlClose; ----->
END;
```

```
<HTML>
<HEAD>
<TITLE>Welcome</TITLE>
</HEAD>
<BODY>
My home page
</BODY>
</HTML>
```

4

The `HTP` package is structured to provide a one-to-one mapping of a procedure to standard HTML tags. For example, to display bold text on a webpage, the text must be enclosed in the HTML tag pair `<B>` and `</B>`. The first code box in the slide shows how to generate the word `Hello` in HTML bold text by using the equivalent `HTP` package procedure, that is, `HTP.BOLD`. The `HTP.BOLD` procedure accepts a text parameter and ensures that it is enclosed in the appropriate HTML tags in the HTML output that is generated.

The `HTP.PRINT` procedure copies its text parameter to the buffer. The example in the slide shows how the parameter supplied to the `HTP.PRINT` procedure can contain HTML tags. This technique is recommended only if you need to use HTML tags that cannot be generated by using the set of procedures provided in the `HTP` package.

The second example in the slide provides a PL/SQL block that generates the basic form of an HTML document. The example serves to illustrate how each of the procedures generates the corresponding HTML line in the enclosed text box on the right.

The benefit of using the `HTP` package is that you create well-formed HTML documents, eliminating the need to manually enter the HTML tags around each piece of data.

Note: For information about all the `HTP` package procedures, refer to *PL/SQL Packages and Types Reference*.

Creating an HTML File with SQL*Plus

To create an HTML file with SQL*Plus, perform the following steps:

- Create a SQL script with the following commands:

```
SET SERVEROUTPUT ON
ACCEPT procname PROMPT "Procedure: "
EXECUTE &procname
EXECUTE owa_util.showpage
UNDEFINE proc
```

- Load and execute the script in SQL*Plus, supplying values for substitution variables.
- Select, copy, and paste the HTML text that is generated in the browser to an HTML file.
- Open the HTML file in a browser.

5

The example in the slide shows the steps for generating HTML by using any procedure and saving the output into an HTML file. You should perform the following steps:

1. Turn on server output with the `SET SERVEROUTPUT ON` command. Without this, you receive exception messages when running procedures that have calls to the `HTP` package.
2. Execute the procedure that contains calls to the `HTP` package.
Note: This does *not* produce output, unless the procedure has calls to the `DBMS_OUTPUT` package.
3. Execute the `OWA_UTIL.SHOWPAGE` procedure to display the text. This call actually displays the HTML content that is generated from the buffer.

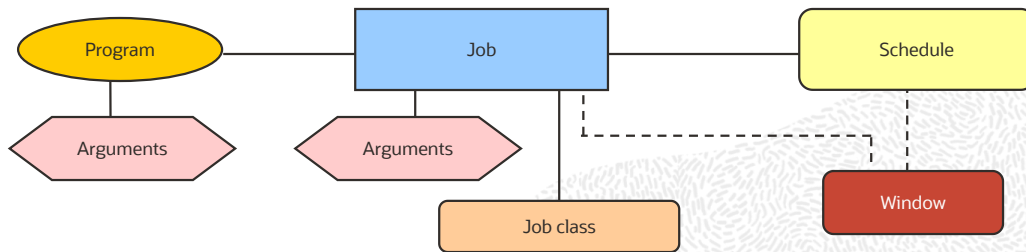
The `ACCEPT` command prompts for the name of the procedure to execute. The call to `OWA_UTIL.SHOWPAGE` displays the HTML tags in the browser window. You can then copy and paste the generated HTML tags from the browser window into an HTML file, typically with an `.htm` or `.html` extension.

Note: If you are using SQL*Plus, then you can use the `SPOOL` command to direct the HTML output directly to an HTML file.

The DBMS_SCHEDULER Package

The database Scheduler comprises several components to enable jobs to be run. Use the DBMS_SCHEDULER package to create each job with:

- A unique job name
- A program (“what” should be executed)
- A schedule (“when” it should run)



6

Oracle Database provides a collection of subprograms in the DBMS_SCHEDULER package to simplify management and to provide a rich set of functionality for complex scheduling tasks. Collectively, these subprograms are called the Scheduler and can be called from any PL/SQL program. The Scheduler enables database administrators and application developers to control when and where various tasks take place. By ensuring that many routine database tasks occur without manual intervention, you can lower operating costs, implement more reliable routines, and minimize human error.

The diagram shows the following architectural components of the Scheduler:

- A **job** is the combination of a program and a schedule. Arguments required by the program can be provided with the program or the job. All job names have the format [schema.] name. When you create a job, you specify the job name, a program, a schedule, and (optionally) job characteristics that can be provided through a **job class**.
- A **program** determines what should be run. Every automated job involves a particular executable, whether it is a PL/SQL block, a stored procedure, a native binary executable, or a shell script. A program provides metadata about a particular executable and may require a list of arguments.
- A **schedule** specifies when and how many times a job is executed.

- A **job class** defines a category of jobs that share common resource usage requirements and other characteristics. At any given time, each job can belong to only a single job class. A job class has the following attributes:
 - A database **service** name. The jobs in the job class will have an affinity to the particular service specified—that is, the jobs will run on the instances that cater to the specified service.
 - A **resource consumer group**, which classifies a set of user sessions that have common resource-processing requirements. At any given time, a user session or job class can belong to a single resource consumer group. The resource consumer group that the job class associates with determines the resources that are allocated to the job class.
- A **window** is represented by an interval of time with a well-defined beginning and end, and is used to activate different resource plans at different times.

The slide focuses on the job component as the primary entity. However, a program, a schedule, a window, and a job class are components that can be created as individual entities that can be associated with a job to be executed by the Scheduler. When a job is created, it may contain all the information needed inline, that is, in the call that creates the job. Alternatively, creating a job may reference a program or schedule component that was previously defined. Examples of this are discussed on the next few pages.

For more information about the Scheduler, see the online course titled *Oracle Database: Configure and Manage Jobs with the Scheduler*.

Creating a Job

- A job can be created in several ways by using a combination of inline parameters, named `Programs`, and named `Schedules`.
- You can create a job with the `CREATE_JOB` procedure by:
 - Using inline information with the “what” and the schedule specified as parameters
 - Using a named (saved) program and specifying the schedule inline
 - Specifying what should be done inline and using a named `Schedule`
 - Using named `Program` and `Schedule` components

8

The component that causes something to be executed at a specified time is called a **job**. Use the `DBMS_SCHEDULER.CREATE_JOB` procedure of the `DBMS_SCHEDULER` package to create a job, which is in a disabled state by default. A job becomes active and scheduled when it is explicitly enabled. To create a job, you:

- Provide a name in the format `[schema.]name`
- Need the `CREATE JOB` privilege

Note: A user with the `CREATE ANY JOB` privilege can create a job in any schema except the `SYS` schema. Associating a job with a particular class requires the `EXECUTE` privilege for that class.

In simple terms, a job can be created by specifying all the job details—the program to be executed (what) and its schedule (when)—in the arguments of the `CREATE_JOB` procedure. Alternatively, you can use predefined `Program` and `Schedule` components. If you have a named `Program` and `Schedule`, then these can be specified or combined with inline arguments for maximum flexibility in the way a job is created.

A simple logical check is performed on the schedule information (that is, checking the date parameters when a job is created). The database checks whether the end date is after the start date. If the start date refers to a time in the past, then the start date is changed to the current date.

Creating a Job with Inline Parameters

Specify the type of code, code, start time, and frequency of the job to be run in the arguments of the `CREATE_JOB` procedure.

```
-- Schedule a PL/SQL block every hour:

BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
    job_name => 'JOB_NAME',
    job_type => 'PLSQL_BLOCK',
    job_action => 'BEGIN ...; END;',
    start_date => SYSTIMESTAMP,
    repeat_interval=>'FREQUENCY=HOURLY; INTERVAL=1',
    enabled => TRUE);
END;
/
```

9

You can create a job to run a PL/SQL block, stored procedure, or external program by using the `DBMS_SCHEDULER.CREATE_JOB` procedure. The `CREATE_JOB` procedure can be used directly without requiring you to create Program or Schedule components.

The example in the slide shows how you can specify all the job details inline. The parameters of the `CREATE_JOB` procedure define “what” is to be executed, the schedule, and other job attributes. The following parameters define what is to be executed:

- The `job_type` parameter can be one of the following three values:
 - `PLSQL_BLOCK` for any PL/SQL block or SQL statement. This type of job cannot accept arguments.
 - `STORED_PROCEDURE` for any stored stand-alone or packaged procedure. The procedures can accept arguments that are supplied with the job.
 - `EXECUTABLE` for an executable command-line operating system application
- The schedule is specified by using the following parameters:
 - The `start_date` accepts a time stamp, and the `repeat_interval` is string-specified as a calendar or PL/SQL expression. An `end_date` can be specified.

Note: String expressions that are specified for `repeat_interval` are discussed later. The example specifies that the job should run every hour.

Creating a Job Using a Program

- Use `CREATE_PROGRAM` to create a program:

```
BEGIN
  DBMS_SCHEDULER.CREATE_PROGRAM(
    program_name => 'PROG_NAME',
    program_type => 'PLSQL_BLOCK',
    program_action => 'BEGIN ...; END;');
END;
```

- Use overloaded `CREATE_JOB` procedure with its `program_name` parameter:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
    program_name => 'PROG_NAME',
    start_date => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY',
    enabled => TRUE);
END;
```

10

The `DBMS_SCHEDULER.CREATE_PROGRAM` procedure defines a program that must be assigned a unique name. Creating the program separately for a job enables you to:

- Define the action once and then reuse this action within multiple jobs
- Change the schedule for a job without having to re-create the PL/SQL block
- Change the program executed without changing all the jobs

The program action string specifies a procedure, executable name, or PL/SQL block depending on the value of the `program_type` parameter, which can be:

- `PLSQL_BLOCK` to execute an anonymous block or SQL statement
- `STORED_PROCEDURE` to execute a stored procedure, such as PL/SQL, Java, or C
- `EXECUTABLE` to execute operating system command-line programs

The example shown in the slide demonstrates calling an anonymous PL/SQL block. You can also call an external procedure within a program, as in the following example:

```
DBMS_SCHEDULER.CREATE_PROGRAM(program_name => 'GET_DATE',
  program_action => '/usr/local/bin/date',
  program_type => 'EXECUTABLE');
```

To create a job with a program, specify the program name in the `program_name` argument in the call to the `DBMS_SCHEDULER.CREATE_JOB` procedure, as shown in the slide.

Creating a Job for a Program with Arguments

- Create a program:

```
DBMS_SCHEDULER.CREATE_PROGRAM(  
  program_name => 'PROG_NAME',  
  program_type => 'STORED PROCEDURE',  
  program_action => 'EMP_REPORT');
```

- Define an argument:

```
DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT(  
  program_name => 'PROG_NAME',  
  argument_name => 'DEPT_ID',  
  argument_position => 1, argument_type => 'NUMBER',  
  default_value => '50');
```

- Create a job specifying the number of arguments:

```
DBMS_SCHEDULER.CREATE_JOB('JOB_NAME', program_name  
=> 'PROG_NAME', start_date => SYSTIMESTAMP,  
repeat_interval => 'FREQ=DAILY',  
number_of_arguments => 1, enabled => TRUE);
```

11

Programs, such as PL/SQL or external procedures, may require input arguments. Using the `DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT` procedure, you can define an argument for an existing program. The `DEFINE_PROGRAM_ARGUMENT` procedure parameters include the following:

- `program_name` specifies an existing program that is to be altered.
- `argument_name` specifies a unique argument name for the program.
- `argument_position` specifies the position in which the argument is passed when the program is called.
- `argument_type` specifies the data type of the argument value that is passed to the called program.
- `default_value` specifies a default value that is supplied to the program if the job that schedules the program does not provide a value.

The slide shows how to create a job executing a program with one argument. The program argument default value is 50. To change the program argument value for a job, use:

```
DBMS_SCHEDULER.SET_JOB_ARGUMENT_VALUE(  
  
  job_name => 'JOB_NAME',  
  argument_name => 'DEPT_ID', argument_value => '80');
```

Creating a Job Using a Schedule

- Use `CREATE_SCHEDULE` to create a schedule:

```
BEGIN
  DBMS_SCHEDULER.CREATE_SCHEDULE('SCHED_NAME',
    start_date => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY',
    end_date => SYSTIMESTAMP +15);
END;
```

- Use `CREATE_JOB` by referencing the schedule in the `schedule_name` parameter:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
    schedule_name => 'SCHED_NAME',
    job_type => 'PLSQL_BLOCK',
    job_action => 'BEGIN ...; END;',
    enabled => TRUE);
END;
```

12

You can create a common schedule that can be applied to different jobs without having to specify the schedule details each time. The following are the benefits of creating a schedule:

- It is reusable and can be assigned to different jobs.
- Changing the schedule affects all the jobs that are using the schedule. The job schedules are changed once, not multiple times.

A schedule is precise to only the nearest second. Although the `TIMESTAMP` data type is more accurate, the Scheduler rounds off anything with a higher precision to the nearest second.

The start and end times for a schedule are specified by using the `TIMESTAMP` data type. The `end_date` for a saved schedule is the date after which the schedule is no longer valid. The schedule in the example is valid for 15 days after using it with a specified job.

The `repeat_interval` for a saved schedule must be created by using a calendaring expression. A `NULL` value for `repeat_interval` specifies that the job runs only once.

Note: You cannot use PL/SQL expressions to express the repeat interval for a saved schedule.

Setting the Repeat Interval for a Job

- Using a calendaring expression:

```
repeat_interval=> 'FREQ=HOURLY; INTERVAL=4'  
repeat_interval=> 'FREQ=DAILY'  
repeat_interval=> 'FREQ=MINUTELY; INTERVAL=15'  
repeat_interval=> 'FREQ=YEARLY;  
                  BYMONTH=MAR, JUN, SEP, DEC;  
                  BYMONTHDAY=15'
```

- Using a PL/SQL expression:

```
repeat_interval=> 'SYSDATE + 36/24'  
repeat_interval=> 'SYSDATE + 1'  
repeat_interval=> 'SYSDATE + 15/(24*60)'
```


Creating a Job Using a Named Program and Schedule

- Create a named program called `PROG_NAME` by using the `CREATE_PROGRAM` procedure.
- Create a named schedule called `SCHED_NAME` by using the `CREATE_SCHEDULE` procedure.
- Create a job referencing the named program and schedule:

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB('JOB_NAME',
    program_name => 'PROG_NAME',
    schedule_name => 'SCHED_NAME',
    enabled => TRUE);
END;
/
```

14

The example in the slide shows the final form for using the `DBMS_SCHEDULER.CREATE_JOB` procedure. In this example, the named program (`PROG_NAME`) and schedule (`SCHED_NAME`) are specified in their respective parameters in the call to the `DBMS_SCHEDULER.CREATE_JOB` procedure.

With this example, you can see how easy it is to create jobs by using a predefined program and schedule.

Some jobs and schedules can be too complex to cover in this course. For example, you can create windows for recurring time plans and associate a resource plan with a window. A resource plan defines attributes about the resources required during the period defined by execution window.

For more information, refer to the online course titled *Oracle Database: Configure and Manage Jobs with the Scheduler*.

Managing Jobs

- Run a job:

```
DBMS_SCHEDULER.RUN_JOB('SCHEMA.JOB_NAME');
```

- Stop a job:

```
DBMS_SCHEDULER.STOP_JOB('SCHEMA.JOB_NAME');
```

- Drop a job even if it is currently running:

```
DBMS_SCHEDULER.DROP_JOB('JOB_NAME', TRUE);
```

15

After a job has been created, you can:

- Run the job by calling the `RUN_JOB` procedure specifying the name of the job. The job is immediately executed in your current session.
- Stop the job by using the `STOP_JOB` procedure. If the job is running currently, it is stopped immediately. The `STOP_JOB` procedure has two arguments:
 - **job_name:** Is the name of the job to be stopped
 - **force:** Attempts to gracefully terminate a job. If this fails and `force` is set to `TRUE`, then the job slave is terminated. (Default value is `FALSE`.) To use `force`, you must have the `MANAGE SCHEDULER` system privilege.
- Drop the job with the `DROP_JOB` procedure. This procedure has two arguments:
 - **job_name:** Is the name of the job to be dropped
 - **force:** Indicates whether the job should be stopped and dropped if it is currently running (Default value is `FALSE`.)

If the `DROP_JOB` procedure is called and the job specified is currently running, then the command fails unless the `force` option is set to `TRUE`. If the `force` option is set to `TRUE`, then any instance of the job that is running is stopped and the job is dropped.

Note: To run, stop, or drop a job that belongs to another user, you need `ALTER` privileges on that job or the `CREATE ANY JOB` system privilege.

Data Dictionary Views

- [DBA | ALL | USER]_SCHEDULER_JOBS
- [DBA | ALL | USER]_SCHEDULER_RUNNING_JOBS
- [DBA | ALL]_SCHEDULER_JOB_CLASSES
- [DBA | ALL | USER]_SCHEDULER_JOB_LOG
- [DBA | ALL | USER]_SCHEDULER_JOB_RUN_DETAILS
- [DBA | ALL | USER]_SCHEDULER_PROGRAMS

16

The DBA_SCHEDULER_JOB_LOG view shows all completed job instances, both successful and failed.

To view the state of your jobs, use the following query:

```
SELECT job_name, program_name, job_type, state
FROM USER_SCHEDULER_JOBS;
```

To determine which instance a job is running on, use the following query:

```
SELECT owner, job_name, running_instance, resource_consumer_group
FROM DBA_SCHEDULER_RUNNING_JOBS;
```

To determine information about how a job ran, use the following query:

```
SELECT job_name, instance_id, req_start_date, actual_start_date, status
FROM ALL_SCHEDULER_JOB_RUN_DETAILS;
```

To determine the status of your jobs, use the following query:

```
SELECT job_name, status, error#, run_duration, cpu_used
FROM USER_SCHEDULER_JOB_RUN_DETAILS;
```

Summary

In this appendix, you should have learned how to:

- Use the `HTP` package to generate a simple webpage
- Call the `DBMS_SCHEDULER` package to schedule PL/SQL code for execution



17

This lesson covers a small subset of packages provided with the Oracle database. You have extensively used `DBMS_OUTPUT` for debugging purposes and displaying procedurally generated information on the screen in `SQL*Plus`.

In this lesson, you should have learned how to schedule PL/SQL and external code for execution with the `DBMS_SCHEDULER` package.

Note: For more information about all PL/SQL packages and types, refer to *PL/SQL Packages and Types Reference*.

For Instructor Use Only.
This document should not be distributed.